

Different Methods for Optimizing Quantum Computing

Table of Contents

Quantum circuit optimization with deep reinforcement learning	2
Wide Quantum Circuit Optimization with Topology Aware Synthesis	12
GA4QCO: Genetic Algorithm for Quantum Circuit Optimization	23
A Comprehensive Review of Quantum Circuit Optimization: Current Trends and Future Directions	32
Leveraging Machine Learning for Quantum Circuit Optimization	60
Improving Quantum Computation by Optimized Qubit Routing	96
Efficient compiler optimization by modeling passes dependence	127
Survey on Quantum Circuit Compilation for Noisy Intermediate-Scale Quantum Computers: Artificial Intelligence to Heuristics	145
END	157

Quantum circuit optimization with deep reinforcement learning

Thomas Fösel,^{1,2,3} Murphy Yuezhen Niu,⁴ Florian Marquardt,^{1,2} and Li Li (李力)³

¹*Max Planck Institute for the Science of Light, Staudtstr. 2, 91058 Erlangen, Germany*

²*Physics Department, University of Erlangen-Nuremberg, Staudtstr. 5, 91058 Erlangen, Germany*

³*Google Research, Mountain View, CA 94043, USA*

⁴*Google Research, Venice Beach, CA 90291, USA*

A central aspect for operating future quantum computers is quantum circuit optimization, i.e., the search for efficient realizations of quantum algorithms given the device capabilities. In recent years, powerful approaches have been developed which focus on optimizing the high-level circuit structure. However, these approaches do not consider and thus cannot optimize for the hardware details of the quantum architecture, which is especially important for near-term devices. To address this point, we present an approach to quantum circuit optimization based on reinforcement learning. We demonstrate how an agent, realized by a deep convolutional neural network, can autonomously learn generic strategies to optimize arbitrary circuits on a specific architecture, where the optimization target can be chosen freely by the user. We demonstrate the feasibility of this approach by training agents on 12-qubit random circuits, where we find on average a depth reduction by 27% and a gate count reduction by 15%. We examine the extrapolation to larger circuits than used for training, and envision how this approach can be utilized for near-term quantum devices.

I. INTRODUCTION

The long-term goal of fault-tolerant large-scale quantum computing promises disruptive progress in multiple important areas of science and technology [1], such as decryption [2], database search [3], quantum simulation [4], and for optimization problems [5]. However, the considerable overhead required for fault-tolerant operations implies a strong incentive in the near term to explore applications that can already work on “noisy intermediate-scale quantum” (NISQ [6]) devices. With the help of suitable algorithms [7, 8], such devices will be able to produce results that outperform classical computers, despite a moderately high rate of errors (“noise resilience”). Recently, a quantum advantage has been demonstrated experimentally for some first benchmark problems on NISQ devices [9, 10].

Research on NISQ applications aims to deliver useful results while keeping the overall resources (number of qubits, computation time) required at a manageable level. Quantum circuit optimization (QCO) constitutes an essential step in addressing this challenge. Starting from a given quantum circuit, i.e., a sequence of quantum operations (gates) acting on a set of qubits, the goal of QCO is to find a logically equivalent circuit that is likely to reduce the probability of errors, by having an overall shorter runtime and using fewer gates (possibly with emphasis on gates of a certain type). This can be achieved mainly by compression (via combination or cancellation) and efficient parallelization of the quantum gates.

There has been important progress in QCO over the years, giving rise to a set of approaches like the family of T-par [11, 12], TOpt [13] and T-Optimizer [14], and QCO based on ZX calculus [15]. These approaches are inspired by the requirements of large-scale fault-tolerant quantum computation and therefore work on scenarios which are quite removed from what will be encountered in NISQ hardware devices. In fact, they have been formulated in a

hardware-independent way and rather operate on a global level.

What is especially needed for QCO on NISQ devices is, however, an approach that is able to take into account the optimization opportunities afforded by a more detailed consideration of the actual hardware (connectivity, gate set, etc.) [16]. Finding the right strategy for this task is a challenging problem. The benefits of any change in the quantum circuit are very context-dependent, especially if the circuit structure is irregular, and a greedy strategy (to always aim for the immediate improvement) is often not ideal.

Optimizing circuits by hand is very time-consuming, requires revising the strategy for every new platform, and can be prone to human errors. Hard-coded algorithmic strategies require considerable human implementation effort specific to the given architecture. In situations with complex decision-making, hard-coded strategies are also prone to unconsidered special cases impeding their ability to generate high-quality results. Optimization techniques like genetic algorithms or simulated annealing improve on these aspects, but lack computational efficiency because they do not select actions in a deliberate way, and they are unable to transfer the experience from earlier solved problem instances.

Optimizing strategies for complex decision-making problems is the goal of reinforcement learning (RL) [17], an important subfield of machine learning that goes beyond straightforward supervised learning. Based on repeated attempts, strategies which have been successful in the past are reinforced over time (exploitation), while alternatives are still occasionally tried out in a trial-and-error fashion (exploration). Due to the generality of this approach, RL is being employed in a broad and growing spectrum of applications, such as robotics [18], video games [19, 20], board games like chess and Go [21, 22], chemistry [23–25], neural architecture search [26], and chip placement [27]. Because RL discovers strategies autonomously and

therefore does not rely on a teacher, it often achieves super-human performance in situations where such a comparison can be made.

In recent years, RL has also been proposed for several problems in the field of quantum computing. Examples include quantum phase estimation [28], the design of quantum experiments [29], quantum control [30–33], quantum error correction [34–37] (alongside other machine learning approaches [38–41]), and quantum metrology [42, 43].

In this work, we introduce deep reinforcement learning for quantum circuit optimization. Our approach enables the computer to autonomously discover strategies for reducing the depth and gate count of quantum circuits, for arbitrary gate sets and connectivity. It allows to choose the optimization target at will and permits extrapolation of the discovered strategy to larger circuits. Due to its flexibility and generality, the RL approach proposed here has the potential to become a valuable component of the toolbox needed to unlock the power of NISQ devices in the near future.

II. TECHNIQUE

A. Quantum circuit optimization as reinforcement learning problem

The goal of RL is to discover strategies for decision-making problems. This is described by an “agent” interacting with the rest of the world, the “environment”. In several rounds, the agent receives information from the environment and, in response to this observation, chooses an action which alters the state of the environment. The agent is supposed to adapt its strategy so as to maximize a success measure, the “reward”. More information is provided in Sec. II C.

In the spirit of previous RL applications to quantum problems [28, 30–34, 42, 43], the obvious approach seems to let the agent build a circuit gate by gate to implement a certain target operation. However, this would come with two central problems here. First, it is extremely unlikely to find a suitable circuit by chance, so an untrained agent would in practice probably never see a positive reward signal. This problem is exacerbated by the fact that the gate set is typically not discrete, but gates can depend on continuous parameters. Second, in the particularly interesting quantum supremacy regime where the circuit cannot be simulated on a classical computer, there is the problem that even if one had found a valid circuit, verifying its correctness would be very hard and computationally expensive. Note that some tools like ZX calculus promise to arrive at a statement in polynomial time, but with the two possible results being positive or inconclusive whether two circuits are equivalent.

Therefore, we follow a different strategy that appears more promising: In QCO, it is common to start from a complete and correct, but typically inefficient circuit, and to progressively optimize it by applying a sequence of

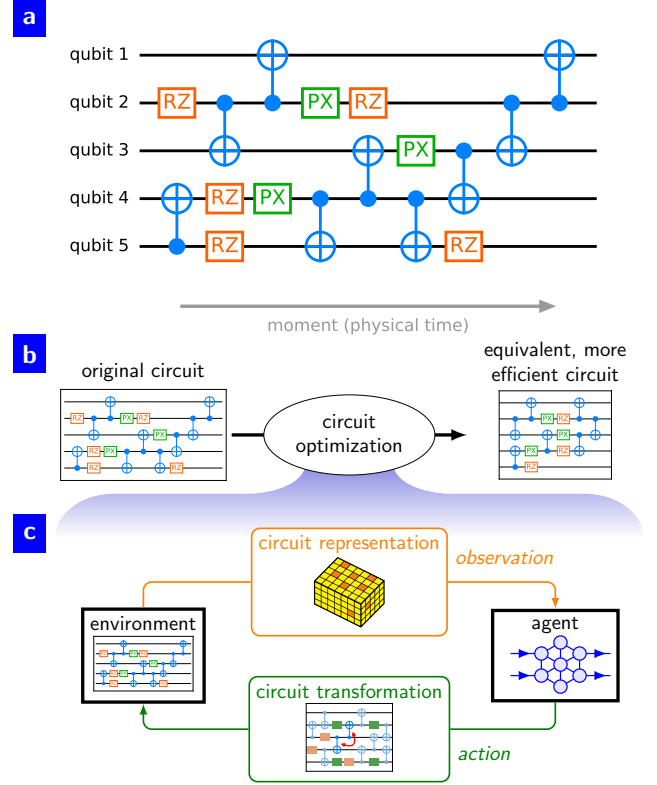


Figure 1. Overview. a) Diagram representation for quantum circuits. Each qubit is indicated with one line. The colored symbols represent operations (gates) on these qubits, with time increasing to the right. b) Quantum circuit optimization. For a given circuit, we aim to find a logically equivalent, but more efficient representation. c) Our reinforcement learning approach to quantum circuit optimization. Based on a diagram-like representation of the circuit, the agent, realized by a neural network, can choose between several circuit transformations to generate another, logically equivalent circuit; this process is repeated multiple times.

circuit transformations. However, it can be a formidable challenge to appropriately choose these transformations, and we make this decision the task of our agent. From the RL perspective, this means that *the states are the circuits* and *the actions are the circuit transformations*. By design, this approach immediately solves the challenge to finish with a correct, i.e., logically equivalent, circuit: we can preserve this property for the full process by allowing in each step only equivalence transformations. In addition, our approach is also scalable, i.e., it allows us to operate in the quantum supremacy regime: it is sufficient to verify equivalence for the few operations directly involved in an elementary circuit transformation, which is relatively cheap as long as all operations act only on a limited number of qubits.

Our general goal is to use RL to train a multi-purpose agent which afterwards will be able to optimize a wide class of circuits based on one given hardware architecture, without going through the RL procedure again in each

instance. Sometimes, though, it can be helpful to fall back to the more restricted approach, where an agent is trained to optimize one specific circuit only.

For our approach, we need to carefully distinguish between different notions of time: the physical time which refers to the execution order of the operations in the circuit, and the agent time in which the actions, here to modify the circuit, are applied. In contrast to comparable previous works on RL for quantum problems [28, 30–34, 42, 43], these two time notions are here independent. From the agent’s perspective, physical time is treated as an extra spatial dimension of the observation (cmp. Sec. II E). In addition, there is the training time along which the weights and thus the behavior of the agent changes.

B. Circuit transformations

Our goal is to optimize a circuit by applying a suitable sequence of circuit transformations. These transformations are based on local transformation rules. Such a rule could be to remove two subsequent operations that cancel each other; or, to reverse the order of two operations that commute (or anti-commute, as a global phase does not matter). Note that one given rule might be applicable to multiple locations in the circuit, yielding an independent transformation for each of them.

In our approach, we distinguish two kinds of transformation rules, which we refer to as “hard” and “soft” rules, and likewise call the transformations they induce hard and soft. Hard rules are those which are always advantageous, like the example of cancelling operations (if the goal is to shrink the circuit, cmp. Sec. II D). Soft rules, on the other hand, are those where the benefit of the transformation depends on the context. For example, for two commuting operations there might be a preferable order contingent on the surrounding operations, and dependent on the current configuration, it can make sense to exchange them or not. Also, it is not uncommon for soft transformations to be beneficial only in combination with several other hard or soft transformations.

In our approach, the decision of the agent is restricted to which soft transformation to apply next. This is always followed by a “pruning” step where automatically all permissible hard transformations are applied. In this sense, the decision of the agent can also be interpreted as not only selecting the soft, but implicitly also all ensuing hard transformations.

Currently, we determine and implement the small set of acceptable transformation rules by hand. The effort for this is manageable because only a moderate number of real-world quantum architectures exists, each with only a limited number of rules according to its specific gate set. As soon as we have arrived at a proper set of rules for one architecture, we can deal with any circuit on it.

C. Learning algorithm

The full RL training process is divided into several episodes. In our case, an episode corresponds to one optimization run for a specific circuit (which might change between episodes). Each episode itself is a sequence of T subsequent steps, in which the agent changes the state s_t with its action a_t into the new state s_{t+1} . For this, the agent receives a reward r_t ; in our case, the reward indicates the improvement of the circuit compared to the previous one. The objective in RL is to maximize the cumulative reward $\sum_{t=0}^{T-1} r_t$. This makes the return

$$R_t = \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}, \quad (1)$$

where γ is a discount rate, an important quantity: For $\gamma = 1$, maximizing $\mathbb{E}[\sum_{t=0}^{T-1} r_t]$ would be equivalent to greedily maximizing $\mathbb{E}[R_t]$ in each time step; in practice, however, a value $\gamma < 1$ often makes learning more stable and efficient.

The agent is typically implemented by a neural network, or in general by an arbitrary parameterized ansatz. The interpretation of its output is defined by the RL algorithm, which also specifies how to train the agent’s parameters θ . In this work, we use proximal policy optimization (PPO [44]), from the family of advantage actor-critic (AAC) methods. An AAC agent computes two quantities: the policy $\pi_\theta(a|s)$, according to which the actions are probabilistically chosen; and an estimator for the state value $V_\theta(s)$, that is the expectation value for the return R when starting from state s and following policy π_θ . In our case, the meaning of the state value $V_\theta(s)$ for the best policy π_* is the further potential for optimizations of the circuit representing state s .

AAC methods cooptimize policy π_θ and state value V_θ during training [17]: Actions a_t are reinforced according to their advantage $r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t)$, whose expectation value tells how much the average return $\mathbb{E}_{\pi_\theta}[R_t|s_t, a_t] = \mathbb{E}_{\pi_\theta}[r_t + \gamma V_\theta(s_{t+1})|s_t, a_t]$ for the chosen action a_t exceeds the average over all actions, that is $V_\theta(s_t)$. Simultaneously, the state-value estimator $V_\theta(s)$ is refined (and adjusted to changes in the policy π_θ) so as to satisfy the Bellman equation $V_\theta(s) = \mathbb{E}_{\pi_\theta}[r + \gamma V_\theta(s')|s]$. In the simplest case, this can be achieved by updating θ according to the learning gradient

$$g = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{T-1} (r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t)) \frac{\partial}{\partial \theta} \left[\ln \pi_\theta(a_t|s_t) - \lambda V_\theta(s_t) \right] \right] \quad (2)$$

where the coefficient λ weights the cost term for the state value. PPO extends this basic AAC scheme in several ways for more powerful and efficient learning.

D. Reward

The central motive behind QCO is often to decrease the probability for potential errors to occur. More generally speaking, we always want to improve some arbitrary desirable property of a circuit, which is defined by the user. The reward, which decides towards which behavior RL steers the agent, needs to be constructed according to this goal. Let $q(s)$ be a function which quantifies the desirable property for a given circuit s , and is to be minimized. A successful agent is expected to progressively improve the circuit over the course of an episode, so the goal is to minimize $q(s_T)$ at the final step T . We drive the agent towards this behavior with an immediate reward of the form

$$r_t = -(q(s_{t+1}) - q(s_t)). \quad (3)$$

This provides better time resolution than a final reward, such as $r_t \sim -q(s_T) \cdot \delta_{t,T-1}$ or $r_t \sim -(q(s_T) - q(s_0)) \cdot \delta_{t,T-1}$, and therefore promises more efficient learning.

Now, we will derive a simple form for $q(s)$ which estimates the error probability for circuit s , or – to be precise – is a monotonously increasing function thereof. The quite simplistic, but still reasonable model behind it is to assume that each qubit loses information with a certain rate Γ while being idle, and that every gate induces additional errors. Let u_k indicate how much the k -th gate underperforms compared to decay with rate Γ during idle times. Then, the probability to execute a circuit error-free is given by

$$P_{\text{success}} = e^{-m\Gamma T} \cdot \prod_{k=1}^n u_k \quad (4)$$

where m is the number of qubits, n is the number of gates in the circuit, and T denotes the total runtime. Our goal to maximize P_{success} is equivalent to minimizing $q = -\ln P_{\text{success}} = m\Gamma T - \sum_{k=1}^n \ln u_k$.

During application of our RL approach, we will have to make concrete choices for the parameters, but we want to remain as general as possible. The least special assumption we can make is to attribute unit cost to all operations, i.e., an equal execution time $\alpha/(m\Gamma)$ and an equal underperformance factor $u_k = e^{-\beta}$. In this case, q can be expressed via the circuit depth d and gate count n :

$$q(s) = \alpha d(s) + \beta n(s). \quad (5)$$

Explicitly, we will set $\alpha = 1$ and $\beta = 0.2$ in Sec. III. However, we emphasize that $q(s)$ in Eq. (5) can be replaced with any function of a circuit, and because we employ model-free RL, nothing in the learning algorithm would need to be changed.

E. Representation of observations and actions

With perfect information about the environment state, i.e., the circuit, the agent can make the best decisions.

Therefore, we provide a complete description of the circuit as observation to the agent. In quantum computing, circuit diagrams are widely used to visualize circuits for humans. Our format is inspired by them, but adjusted to the conditions of a neural network: We reserve one input neuron for each possible operation in the circuit, and activate neurons depending on whether the corresponding operation is actually present. These input neurons are arranged on a 3D grid, whose axes correspond to qubit index, moment (time step) and gate class. For gate types with a discrete number of instances, each of them forms its own gate class. For gate types which depend on (continuous) control parameters, we distinguish separate classes for special parameter values (e.g., for Z rotations by $\pi/2$, π and $3\pi/2$), and group all remaining instances of this gate type into one class. This allows to represent quantum circuits for arbitrary choices of their continuous parameters, but to distinguish gates for which additional transformation rules apply. To characterize multi-qubit gates, in principle all qubit indices would have to be indicated. In a 1D chain of qubits where gates are allowed only between nearest neighbors, as in our examples in Sec. III, we can characterize two-qubit gates simply by their lowest-index qubit, and specify by the gate class which roles the qubits play (e.g., control vs. target qubit for a CNOT gate).

A particular challenge in this project was to design the output format to represent the policy of the agent. The basic problem is that the actions, i.e., the transformations of the circuit, are defined by the underlying transformation rule and all the gates affected by this transformation (which are characterized by their location in the circuit). Due to combinatorial explosion, the usual approach for discrete action spaces, to reserve one output neuron for every possible action, is not feasible here. One alternative would be to sort the transformations in a certain way and assign them consecutively to a queue of output neurons. However, this approach is not very promising either since the meaning for each output neuron would change for every circuit, potentially depending on aspects of very distant locations of the circuit, and therefore be very hard for the network to understand. Another alternative would be Q-learning where the agent, instead of computing the action value $Q(s, a)$ for state s and action a , computes $Q(s, s')$ for input state s and output state s' [24]; while conceptually this should work, this RL variant is less widely used and therefore not as well explored, and is restricted to Q-learning.

Instead, inspired by the representation of chess moves in [22], we follow a different approach where we associate transformations with certain locations in the circuit, in a way that uniquely characterizes which gates they affect. In our case, we use the following mapping: for a transformation involving a single gate, we indicate the moment and the smallest qubit index of the gate; for a transformation involving two gates, we indicate the smallest qubit index shared between both gates, and the moment of the first one. By providing multiple neurons per location to

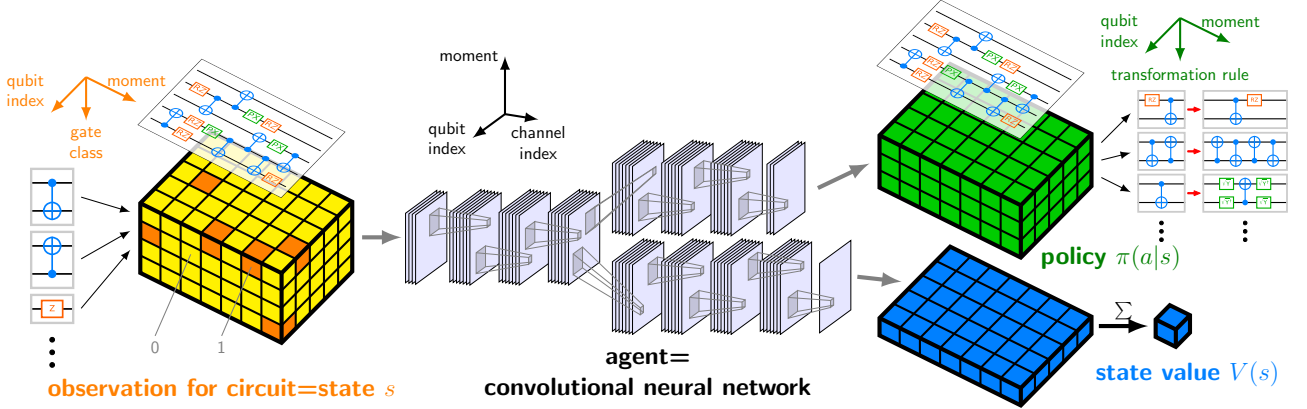


Figure 2. Deep convolutional network architecture of our RL agent. As observation, the agent receives a complete description of the state s , i.e., the quantum circuit. The input neurons are arranged on a 3D grid, whose axes correspond to qubit index, moment and gate class. This information is processed through a stack of multiple convolutional layers, where qubit index and moment are treated as spatial dimensions and the gate classes as input color channels. For the output, the agent computes two quantities: (i) The policy $\pi(s|a)$, according to which the actions a in state s are probabilistically chosen. Every action, i.e., circuit transformation, is mapped uniquely to one policy output neuron; the remaining neurons are disabled with an action mask. And (ii), the state value $V(s)$, which helps to update the policy $\pi(s|a)$ more efficiently during training. For us, $V(s)$ has the meaning of the optimization potential for the circuit.

indicate the underlying rule, we can achieve an injective mapping from transformations to output neurons (for the policy). Therefore, also these neurons are arranged on a 3D grid, whose axes correspond to qubit index, moment and transformation rule. There can be neurons to which no transformation is associated; we disable them with an action mask, whose value changes with the input circuit.

Besides solving the problem to keep the total number of output neurons at a moderate level, another central advantage of this format is that we can exclusively use convolutional layers [45] to process the observation into the policy, treating qubit and moment as spatial dimensions, and the remaining grid axis (gate class and transformation rule, respectively) as input “color” channels. Also to compute the state value, we use convolutional layers (with one output channel), and eventually average over the spatial dimensions. Fig. 2 illustrates the architecture of our deep convolutional network. The weight sharing in the convolutional layers contributes to efficient and robust learning, and a *fully* convolutional architecture will allow us to directly extrapolate to different circuit sizes (see Sec. III B).

F. RL problem classification

The RL problem in this article can be classified as a Markov decision process (MDP) with perfect information (since the circuit representation, which is given to the agent as its input, completely describes the state of the environment). The set of all circuits comprises the state space. The set of possible circuit transformations repre-

sents the action space, which is therefore discrete, and its size depends on the state, i.e., the circuit. The environment is deterministic: a fixed transformation (action) on a fixed circuit (state) always leads to the same outcome. Because the goal is to optimize a property which can be evaluated for any single given circuit, we can construct an immediate reward scheme (as opposed to situations where the reward is given only at the end of an episode).

III. RESULTS

In exploring the power of our RL approach, we need to select both a specific architecture (available gate set, processor layout, qubit connectivity) as well as the family of quantum circuits on whose optimization we want to focus.

Gate set For our simulations, we consider the gate set consisting of Z-Rotation, Phased-X and Controlled-Not (CNOT) gates. Together, they form a universal gate set. Whereas Z-Rotation and Phased-X are actually gate classes parameterized by 1 and 2 continuous variables, respectively, the CNOT is one fixed gate.

For our purposes, Z-Rotation, Phased-X and CNOT is a decent gate set because on the one hand, it induces a rich set of relatively simple transformation rules. On the other hand, they are quite similar to current real-world quantum hardware, such as Google’s Bristlecone (Z-Rotation, Phased-X and Controlled-Z gate [46]) and Sycamore (Z-Rotation, Phased-X and fermionic simulation gate [9, 47, 48]) processor: The CNOT gate differs from the Controlled-Z gate only by local gates (e.g., Hadamard or

Sqrt-Y) on the target qubit, and the fermionic simulation gate is a generalization of the Controlled-Z gate.

A. Training on random circuits

Dataset While a quite large number of quantum circuits for various applications and architectures can be found in publications of the recent years, to the knowledge of the authors a joint dataset of such circuits does not exist, and collecting them by hand would have been very tedious. At the stage of developing the technique and verifying its feasibility, we therefore decided to employ randomly generated circuits. Although the majority of them is likely not to be useful for actual quantum applications, they have the big advantage that they can be easily generated in large numbers. Due to the irregular structure of such circuits, the benefits of transformations are strongly context-dependent, and therefore random circuits constitute a case where formulating a generally applicable strategy would be enormously complex.

We now briefly describe our method to generate these random circuits. We consider 12 qubits arranged in a 1D line; two-qubit gates, here CNOTs, are only allowed between nearest neighbors. First, we build a sequence of 150 gates whose type (either CNOT or local) and the qubits it acts on (constrained by the connectivity) are chosen randomly. Because local gates might be decomposed into a Z-Rotation and a Phased-X gate, we obtain a slightly higher value for the mean number of hardware gates, $\langle n_1 \rangle \approx 159$, and for the mean depth we find $\langle d_1 \rangle \approx 48$. For many of these circuits, some hard rules (cmp. Sec. II B) can immediately be applied, i.e., there are some “trivial” optimizations; for example, two subsequent CNOTs on the same qubits directly cancel. To apply these transformations, which we refer to as pruning, reduces the mean depth to $\langle d_2 \rangle = 37.15 \pm 0.01$ and the mean gate count to $\langle n_2 \rangle = 115.27 \pm 0.03$. From here, we continue by applying random transformations (based on soft rules, each time followed by a pruning step), which turns out to make the circuit much larger and hence very inefficient. After 500 of these transformations, we find a mean depth of $\langle d_3 \rangle \approx 248$ and a mean gate count of $\langle n_3 \rangle \approx 508$.

We use these randomly expanded circuits as the training tasks for the RL agent, where each of them defines the starting point for one episode, i.e., for one optimization run. The random expansion step enhances the ability of the fully trained agent to generalize, by demanding it to deal well with a broad spectrum of both inefficient and efficient circuits, which will be encountered during every successful optimization run.

Simulated annealing As a benchmark for the results by the RL agent, we consider simulated annealing as an alternative approach, using an adapted Metropolis rule and an exponential cooling schedule. We apply simulated annealing to the randomly expanded circuits, i.e., on the same dataset as later for the agent. The results depend

strongly on the number of steps: Within 10000 steps, simulated annealing is able to merely reach the level after pruning (on average). Within 200000 steps, out of which roughly 15...20 % are actually accepted, a mean depth $\langle d \rangle = 27.35 \pm 0.08$ and gate count $\langle n \rangle = 105.15 \pm 0.32$ can be achieved.

RL Now, we apply the RL approach as described in Sec. II on our dataset of expanded random circuits. The hyperparameters have been optimized in a systematic search. One full learning process takes 6 to 7 days, on a node with 32 CPU cores.

We find that the learning process can be divided roughly into two phases (cmp. Fig. 3c): In the first ca. 50 epochs, the agent rapidly improves in minimizing the mean depth and gate count, to a level of $\langle d \rangle \approx 29$ and $\langle n \rangle \approx 100$ which is already considerably better than after pruning. In the remaining course of the training, the agent can slowly, but steadily improve towards $\langle d \rangle = 27.20 \pm 0.07$ and $\langle n \rangle \approx 97.86 \pm 0.33$ at around epoch 1000. Hence, the agent can clearly beat the results from simulated annealing, especially regarding the gate count. Because we use a new circuit in each training episode, no difference is to be expected between the performance on training and validation samples.

In Fig. 3d, we examine closer how a trained agent proceeds in optimizing a circuit. For this purpose, we choose 3200 circuits and plot how d and n evolve with every transformation. As Fig. 3d shows, the agent can rapidly improve the circuits during the first 150 to 200 steps. Afterwards, the curve is mostly flat, except for two effects: First, there are occasional excursions into lower-quality states, but the agent usually returns quickly to the previous level. Second, the agent sometimes manages to further optimize the circuit after some time, partially via passing some intermediate lower-quality states, i.e., to cross a reward barrier. Note that we did not give the agent the ability to terminate an episode.

Compared to the pruning level, we have seen that the agent achieves much better values for $\langle d \rangle$ and $\langle n \rangle$ on average. How effective the agent has actually become can be seen by comparing d and n for individual circuits between the agent and after pruning. In Fig. 3e, we plot this comparison for epochs 900 – 1000, i.e., for 3200 circuits in total. The agent achieves a reduction of both d and n for the vast majority of circuits, partially up to even more than 50 %. Only for 12 out of 3200 circuits, i.e., in around 0.4 % of the cases, the agent fails to achieve a better $q(s)$ than after pruning.

B. Extrapolation to larger random circuits

In Sec. III A, the agent has been trained and evaluated on 12-qubit random circuits. However, due to its fully convolutional architecture, this very same agent can – at least in principle – be directly used to optimize also circuits of a different size, in particular much larger ones. Now, we will test how well the agent can actually generalize its

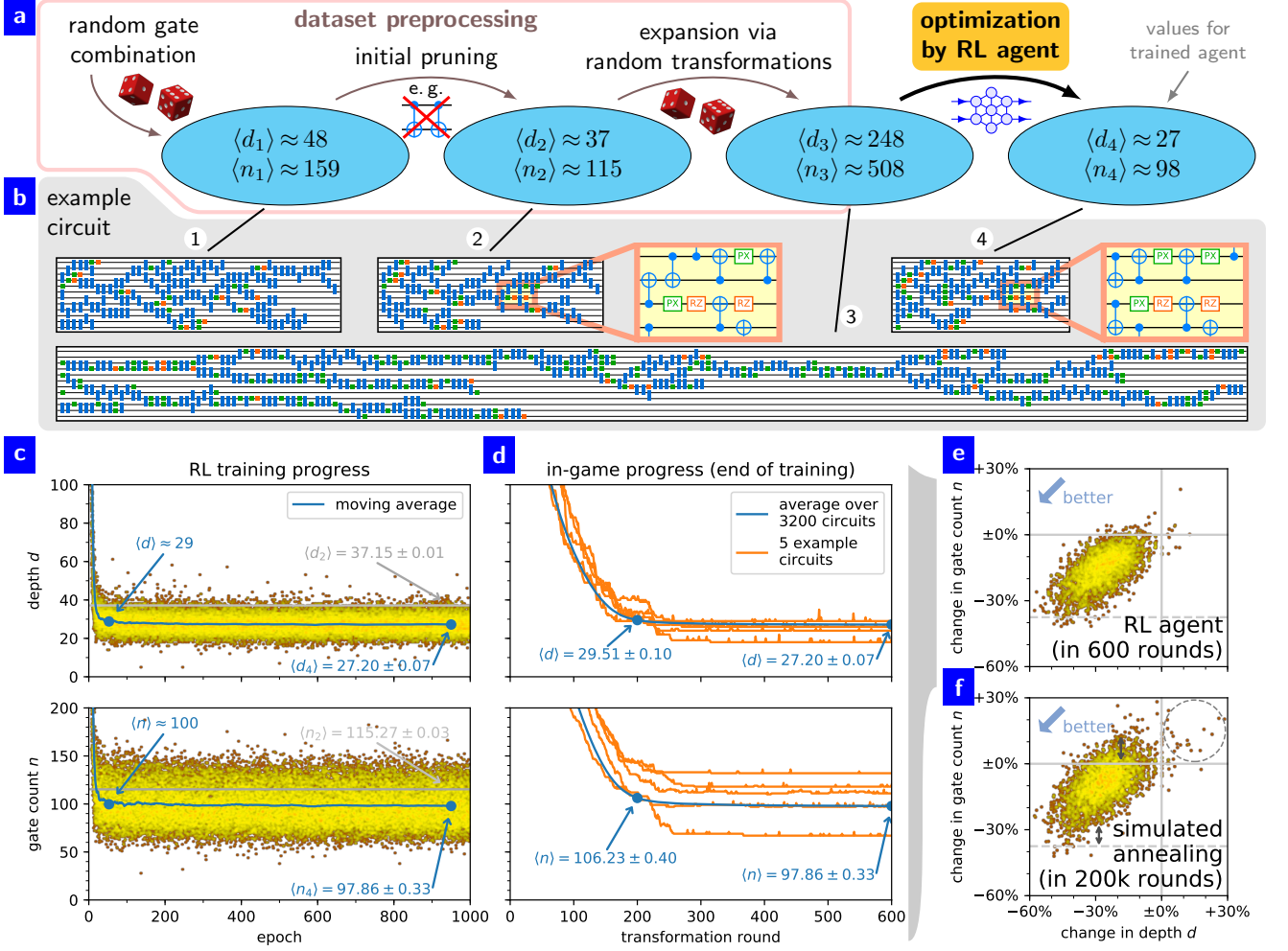


Figure 3. Training on random circuits. a) Circuit processing pipeline (see Sec. III A for details). After choosing an initial circuit by randomly combining gates, a pruning step follows where all “trivial” optimizations are applied. Afterwards, 500 random transformations are performed on this circuit, which turns out to significantly increase their depth d and gate count n . These expanded circuits are then used as the starting point of the episodes to train and evaluate the RL agent. b) Diagrams illustrating the evolution of one example circuit through this pipeline. c) Learning progress during training, demonstrating how the agent improves in reducing both the depth d (top) and the gate count n (bottom) of the circuits. The point cloud indicates, for all episodes during training, the corresponding quantity in the final time step. The blue curve shows the moving average over the latest 10% of epochs. For comparison, the gray line indicates the corresponding averages after pruning; already early in the training, the agent falls below this level for both quantities. d) In-game progress at the end of the learning process, showing for 5 episodes during the last epoch (orange) how the agent progressively optimizes (d, n) during an episode. The blue curve indicates the average over all episodes in the last 100 epochs of training. e) Relative improvement achieved by the RL agent, in reference to the corresponding circuit size after pruning. Each point corresponds to one episode during the last 100 epochs. f) Comparison with circuit optimization by simulated annealing (see Sec. III A for details). The graphical depiction and the considered circuits are equivalent to (e), which makes them directly comparable.

knowledge in this situation.

For this purpose, we reuse the scheme to generate random circuits as described in Sec. III A, except for changing two parameters: we increase the number of qubits from 12 to 50, and the number of initial gates from 150 to 2500. We find $\langle d \rangle = 199.25 \pm 0.08$ and $\langle n \rangle = 2655.3 \pm 1.2$ before pruning, and $\langle d \rangle = 156.67 \pm 0.07$ and $\langle n \rangle = 1940.2 \pm 1.6$ after pruning. Because we will not

use the circuits here to train the agent, we can skip the step to expand them by random transformations, whose purpose it was to feed to the agent also very inefficient circuits during training. Instead, the optimization by the agent starts here directly from the pruned circuits. As shown in Fig. 4b, the agent achieves to reduce $\langle d \rangle$ to 110.84 ± 0.07 and $\langle n \rangle$ to 1616.3 ± 2.0 within 2500 transformations. Remarkably, the reduction ratio in these

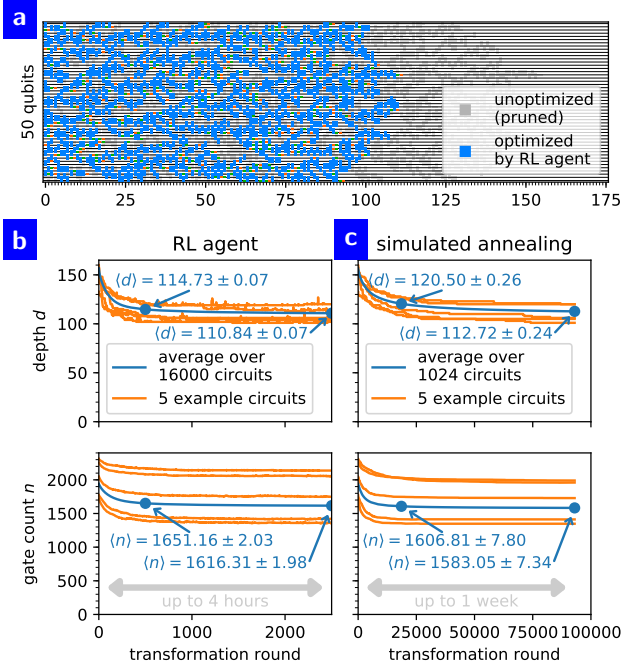


Figure 4. Extrapolation to 50-qubit random circuits. The agent has been trained on 12-qubit circuits (cmp. Fig. 3), no further learning updates are performed here. (a) shows the comparison between an unoptimized example circuit (after pruning) and the result of the optimization by the RL agent. (b) shows the progress of the agent in reducing depth d and gate count n over the course of 2500 transformations. (c) shows the corresponding curves for simulated annealing, which requires almost 100000 transformations to achieve a comparable degree of optimization (the computation was terminated after 1 week, at transformation 93000).

two quantities is comparable to the smaller circuits it has been trained on (cmp. Fig. 3).

Simulated annealing arrives at similar values, $\langle d \rangle = 112.72 \pm 0.24$ and $\langle n \rangle = 1583.0 \pm 7.3$, within 93000 transformations. These are much fewer transformations than required to optimize the smaller random circuits in Sec. III A, probably because here the random expansion step has been skipped. Nevertheless, 93000 transformations for each larger random circuits here have already taken one week (our termination criterion), which is comparable to the time needed to train an RL agent. Afterwards, this agent can optimize arbitrary circuits, in a relatively short time (3...5 h in this case).

Our results show that an agent can actually extrapolate its knowledge to larger circuits. More generally, they demonstrate that our approach, both with RL and simulated annealing, works deep in the quantum supremacy regime. Furthermore, this also highlights a situation where optimizing even a single circuit with simulated annealing needs already a runtime comparable to the full training of an RL agent and subsequently optimizing the particular circuit.

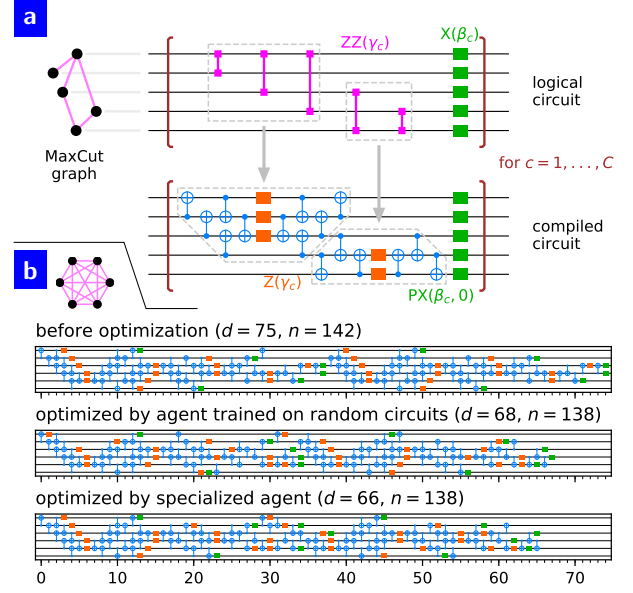


Figure 5. Optimization of QAOA-MaxCut circuits. (a) indicates how to translate the MaxCut problem for a graph into a quantum circuit following QAOA, and how to efficiently compile this logical circuit into our gate set. We display one of M cycles which form the full circuit, each with a different set of parameters (γ_c, β_c) whose values are refined during the QAOA algorithm. (b) shows the compiled circuit for $C = 2$ cycles and an all-to-all-connected graph with 6 nodes, which has depth $d = 75$ and gate count $n = 142$ (top). Using a generic agent trained on random circuits as in Fig. 3, we find (by postselection) improved circuits with $d = 68$ and $n = 138$ (middle). A specialized agent trained on this particular circuit can further optimize it to $d = 66$ and $n = 138$ (bottom).

C. QAOA-MaxCut circuit

As an example for a real-world quantum algorithm, we now consider the MaxCut problem. The goal is to arrange the nodes of an undirected, non-weighted graph into two groups such that the amount of cut edges is maximized. Finding the exact solution is an NP-hard problem. Following the quantum approximate optimization algorithm (QAOA [7]), approximate solutions can be found with the help of a quantum circuit consisting of repeated cycles of ZZ gates and local X rotations with variable angles [48] (cmp. Fig. 5a). We consider the same gate set as in the examples above, such that we can reuse the previously trained agent. Also, this covers the realistic situation where the native gates of the quantum algorithm do not necessarily match the native gates of the hardware. Fig. 5a shows an efficient compilation of this circuit into our gate set, where ZZ gates need to be decomposed into CNOTs and local Z rotations (local X rotations are a special case of Phased- X gates). Note that the variable angles of the gates do not affect the optimization strategy, as long as we assume these angles to be generic (i.e., not set to special values which would

allow additional optimizations).

Every graph corresponds to a different circuit. We restrict our analysis here to all-to-all connected graphs. While any such graph has trivial MaxCut solutions (nodes evenly distributed), the corresponding circuit is still useful for us to consider because the circuit for any other graph layout can be derived from it by removing some of the ZZ gates; therefore, it also provides a worst-case estimate. In Fig. 5b, we specifically consider a graph with 6 nodes and 2 cycles in the QAOA circuit.

First, we try to optimize this QAOA circuit with a generic agent trained on 12-qubit *random* circuits, as in Fig. 3. We find that on average, the agent manages to slightly reduce the gate count, although the depth is slightly increased. Even though this agent is, thus, not able to reliably optimize the circuit, it is still able to sometimes achieve an improvement: By postselecting over all time steps in multiple optimization runs, we find that the best circuit reduces the depth from 75 to 68 and, simultaneously, the gate count from 142 to 138 (cmp. Fig. 5b); note that different runs deviate because the policy is to some degree stochastic, and that the circuit quality fluctuates also with time. We observe that the optimizations happen at the interface between subsequent cycles, and at the end of the last cycle.

If we, however, train an agent via RL directly on this specific circuit, it does not only learn to reliably improve the circuit, but it also finds two further optimizations, each reducing the depth by 1. These optimizations are very hard to discover, as they require a lookahead by 7 and 9 transformations, respectively, where the search breadth is around 350. The resulting circuit, as shown in Fig. 5b, has a depth of 66 and a gate count of 138. To our knowledge, this is the most efficient realization on the gate set we provided (when including additional gates, one can arrive at an even shorter circuit [48]).

For these QAOA-MaxCut circuits, we start from an already quite efficient compilation, so the optimization potential is not very large here. Nevertheless, the agent is still able to find optimizations, including some which require complex transformation sequences and would thus be very hard to find by hand. On the general level, this example demonstrates that our approach can also be used to optimize real-world circuits, where the optimization potential might very well be larger than here. However, we have got reliable improvement and the best optimization quality only for agents trained on this circuit, which emphasizes the importance of a proper training dataset.

During early simulations, we made a noteworthy observation: Sometimes, the agent trained on these MaxCut circuits achieved optimizations far below a depth of 66 and a gate count of 138. It turned out that this happened only if the rotation angle for one cycle of ZZ gates, which was chosen at random, was relatively close to a multiple of π . In this case, the agent found a creative way to exploit that the closeness criterion for one of the transformation rules was too loose, which it used to partially or completely remove this cycle from the circuit. Although in

this particular case these transformations were not valid, the agent’s behavior shows that it would readily exploit the simplification potential in other complex circuits.

D. Discussion

Dataset We anticipate our RL agents to be considerably more powerful and reliable on real-world quantum circuits when trained on a dataset with similar properties, which is also indicated by our results in Sec. III C. In contrast to currently existing collections of quantum circuits, like [49], we need for our purposes first the circuits being compiled for specific hardware architectures, and second, also a significantly larger dataset size to fully train agents on them (based on Sec. III A, this can require in the order of 10000 to 100000 episodes).

We think of our future dataset to be generated as follows. We consider a set of various quantum algorithms, like QAOA [7], variational quantum eigensolver [8], Shor’s algorithm [2] (for small integers), and many more [50]. Many of these algorithms imply a large number of different circuits; for example, QAOA can be applied to multiple combinatorial optimization problems, and for each of them, the circuit changes with the concrete problem instance (e.g., graph layout) and the number of cycles. These circuits are described in a high-level, hardware-independent way. For every specific architecture (gate set, layout, connectivity), we would then generate an own dataset by compiling the circuits for them, which can be done using existing quantum libraries like Cirq [51].

Eventually, each of our agents will be trained on one of these architecture-specific datasets. If the dataset size is not yet sufficient for this, we can use data augmentation techniques (like mirroring the circuit in space and/or time), and to some degree also repeat circuits during training.

RL vs. simulated annealing In Sec. III A, simulated annealing achieved $98.48 \pm 1.06\%$ of the reduction in gate count n by the RL agent, but only $58.06 \pm 2.15\%$ in the depth d ; Sec. III B is more ambiguous, as simulated annealing achieved $109.67 \pm 2.44\%$ of the reduction in n by the RL agent, but only $95.88 \pm 0.57\%$ in d (all numbers in reference to the pruning level). Therefore, in these scenarios, RL tends to give overall slightly better results than simulated annealing.

Another aspect to consider in this comparison is runtime. After having been trained once on a certain class of circuits, an agent is considerably faster than simulated annealing in optimizing arbitrary circuits of this class (ca. 2 min vs. 1...3 d in Sec. III A, 3...5 h vs. 7 d in Sec. III B). Of course, for a fair comparison of the computational effort, it is necessary to also take into account the time needed for training the agent in the first place (up to 1 week), and eventually also the time to optimize the hyperparameters for both approaches. Therefore, if we take all those aspects into account, the question which method is less computationally expensive cannot be an-

swered in general, but depends on the class of circuits and previously solved problem instances – the only safe statement is that for a sufficient number of circuits to be optimized, the RL approach will be computationally cheaper.

Symbolic parameters In our current implementation, gates are represented numerically, which requires to consider fixed choices for the gate parameters. This is not directly a property of the RL approach presented in this work, but of our underlying quantum circuit framework [52]. With manageable effort, this framework can be extended to allow also gates parameterized by symbolic variables. This would simplify the optimization of entire classes of parameterized circuits, like for QAOA with its varying angles β_c and γ_c . So far, this is only indirectly possible by repeating the same sequence of transformations for differently parameterized circuits.

IV. CONCLUSION

In this article, we have introduced RL as a powerful and flexible approach to QCO. In particular, we have focused on the optimization of the local circuit structure to the available hardware resources, which is especially relevant for the NISQ era. The main challenge here is to discover suitable sequences of circuit transformations; the size of the search space and the lack of simple strategies in many situations, especially for circuits with an irregular structure, make this a difficult optimization problem. RL is well-suited to problems of this kind, and promises to simultaneously fulfill the following key features: *hardware-efficient*, i.e., that it can find the optimal solution given the available resources; *cross-platform*, i.e., that the approach works for different quantum architectures (gate sets, chip layouts, connectivities, etc.), without large migration costs; *autonomous*, i.e., that the only human effort is to specify the problem, and the optimization is done by the computer completely on its own, without requiring further interaction; and *reliable*, i.e., that the results are always correct and for a wide spectrum of circuits (close-to-)optimum.

For our approach to be successful, we had to address several key challenges: First, we have formulated QCO as a problem which can be efficiently addressed using RL (cmp. Secs. II A and II B). Second, we have developed efficient schemes to represent circuits for the network input and transformations for the network output (cmp. Sec. II E). Third, as prerequisite for applying techniques like RL or simulated annealing, we have implemented a Python framework to automate the process of identifying

all transformations within a circuit [52].

Both RL and simulated annealing turn out to be valid optimization techniques for the QCO problems considered in this work. In terms of optimization quality, the difference between both approaches in reducing depth and gate count were not larger than 10% (in reference to the pruning level), except for Sec. III A where simulated annealing achieved only $58.06 \pm 2.15\%$ of the depth reduction by the RL agent (cmp. Sec. III D). In terms of computational effort, our RL agents required up to 1 week of training time, but were afterwards considerably faster than simulated annealing in optimizing arbitrary circuits from the class they have been trained on (ca. 2 min vs. 1...3 d in Sec. III A, 3...5 h vs. 7 d in Sec. III B). We expect that RL can deal better with situations where decision-making becomes even harder than here. Therefore, RL seems to be, especially in the long run, the more promising approach.

So far, we have mostly considered randomly generated circuits to circumvent the lack of a sufficient dataset of realistic circuits; one central future direction will be to collect such a dataset. Several direct extensions of our work are straightforward to implement, yet potentially very fruitful for future applications, such as implementations for different gate sets, 2D geometries, etc. A (semi-)automated procedure to identify transformation rules, where machine learning could be beneficial, would further reduce the effort for including new gate sets. The optimization of parameterized circuit classes can be simplified by implementing symbolic variables as gate parameters in the underlying quantum circuit framework. Furthermore, the existing complementary QCO techniques which focus on optimizing the global circuit structure [11–15] could be combined in a two-stage process with our technique to optimize the local circuit structure.

We expect our approach to soon become a standard tool for optimizing circuits in NISQ experiments. Central practical advantages are the negligible additional effort to consider non-ideal circumstances, like inhomogeneous error rates or defect qubits, and the ability to customize the optimization criterion.

Data and code availability The code is publicly available under [52]. Other findings of this study are available from the corresponding author on reasonable request.

Acknowledgements T.F. thanks Google Research for hosting during his internship.

Competing interests The authors declare no competing interests.

Corresponding author Correspondence should be addressed to T.F.

-
- [1] Michael A Nielsen and Isaac Chuang. Quantum computation and quantum information, 2002.
 [2] Peter W Shor. Polynomial-time algorithms for prime fac-

- torization and discrete logarithms on a quantum computer. *SIAM review*, 41(2):303–332, 1999.
 [3] Lov K Grover. Quantum mechanics helps in searching for

Wide Quantum Circuit Optimization with Topology Aware Synthesis

Mathias Weiden¹, Justin Kalloor¹, John Kubiatowicz¹, Ed Younis², and Costin Iancu²

¹Department of Electrical Engineering and Computer Science, University of California, Berkeley
`{mtweiden, jkalloor3, kubitron}@cs.berkeley.edu`

²Computational Research Division, Lawrence Berkeley National Laboratory
`{edyounis, cciancu}@lbl.gov`

Abstract—Unitary synthesis is an optimization technique that can achieve optimal multi-qubit gate counts while mapping quantum circuits to restrictive qubit topologies. Because synthesis algorithms are limited in scalability by their exponentially growing run time and memory requirements, application to circuits wider than 5 qubits requires divide-and-conquer partitioning of circuits into smaller components. In this work, we will explore methods to reduce the depth (program run time) and multi-qubit gate instruction count of wide (16-100 qubit) mapped quantum circuits optimized with synthesis. Reducing circuit depth and gate count directly impacts program performance and the likelihood of successful execution for quantum circuits on parallel quantum machines.

We present TopAS, a topology aware synthesis tool built with the *BQSKit* framework that preconditions quantum circuits before mapping. Partitioned subcircuits are optimized and fitted to sparse qubit subtopologies in a way that balances the often opposing demands of synthesis and mapping algorithms. This technique can be used to reduce the depth and gate count of wide quantum circuits mapped to the sparse qubit topologies of Google and IBM. Compared to large scale synthesis algorithms which focus on optimizing quantum circuits after mapping, TopAS is able to reduce depth by an average of 35.2% and CNOT gate count an average of 11.5% when targeting a 2D mesh topology. When compared with traditional quantum compilers using peephole optimization and mapping algorithms from the Qiskit or *t|ket* toolkits, our approach is able to provide significant improvements in performance, reducing CNOT counts by 30.3% and depth by 38.2% on average.

I. INTRODUCTION

Modern quantum machines are subject to high levels of environmental noise, are difficult to control, and consist of only tens of qubits. Quantum error correction and mitigation are widely regarded as vital components in the development of practical quantum computers [16]. However, because quantum error correction is not yet realistically implementable, it is essential that quantum programs be optimized so that run times and operation counts are minimized, as each operation adds additional error into the system and longer run times increase the probability that qubits will decohere. Operations that involve multiple qubits, such as the *CNOT*, *iSWAP*, *Toffoli*, and *CZ* gates, are typically far more error prone and expensive to implement compared to single qubit rotation gates [6]. For this reason, multi-qubit gate counts can be used as a metric for the error introduced into quantum programs.

Optimization techniques that aim to decrease the multi-qubit gate count of quantum circuits are becoming increasingly used and studied [12], [18], [25]. Unitary synthesis is a method of decomposing a quantum circuit into a simpler quantum circuit that fits some specified gate set. Synthesis seeks to produce shorter circuits with reduced depth and gate count in order to improve the likelihood that circuits are executed correctly.

The runtime of unitary synthesis algorithms scales exponentially with the number of qubits n . For current state of the art unitary synthesis algorithms such as the QSearch/LEAP algorithm, only circuits up to 5 qubits are realistically decomposable. Larger circuits must thus be partitioned into smaller subcircuits. After synthesizing each subcircuit independently, subcircuits in the original circuit are replaced with their synthesized versions.

Other than sensitivity to environmental noise, machines in the Noisy Intermediate Scale Quantum (NISQ) era are defined by their limited sizes and connectivities (which physical qubits are allowed to interact). The connectivity of qubits in machines using superconducting qubits is especially restrictive. Examples of realistic and popular superconducting qubit physical topologies are illustrated in Figure 1.

Each vertex in a physical qubit topology graph represents a hardware qubit. The set of edges between these hardware qubits is the set of supported multi-qubit interactions for that quantum machine. Quantum algorithms are typically designed assuming that each qubit is able to interact directly with all other qubits: logical circuits assume all to all connectivity. In order for a quantum processor to execute these densely connected logical circuits, a qubit mapping algorithm must insert SWAP gates routing all interactions along edges in the physical topology.

As quantum circuits become wider and physical topologies remain sparse, the number of routing gates inserted by these mapping algorithms grows quickly. It is therefore desirable to approximate and transform wide quantum circuits using unitary synthesis so that the total number of multi-qubit gates and depth of quantum circuits is reduced.

In this paper, we present TopAS, a qubit topology aware synthesis tool that applies unitary synthesis to wide quantum circuits to reduce multi-qubit gate counts and circuit depth. TopAS first partitions a logical quantum circuit, synthesizes

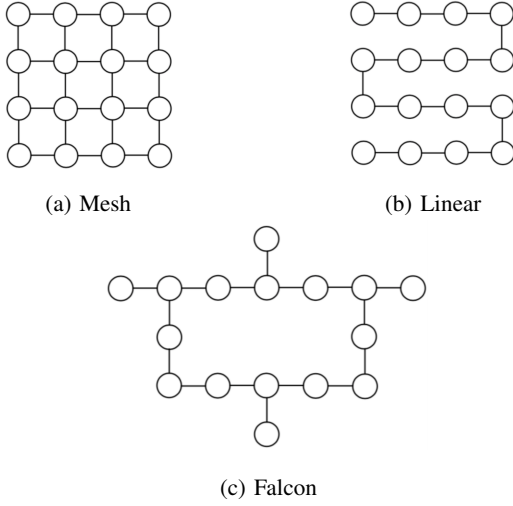


Fig. 1: Example Physical Qubit Topologies. The 2D mesh topology is based off quantum machines provided by Google, while the falcon topology follows IBM’s heavy hexagonal connectivity scheme.

each partitioned subcircuit independently, reassembles the optimized logical circuit, then uses a mapping algorithm to ensure that all operations are legal according to some specified qubit topology. When targeting the mesh physical topology, TopAS is able to produce circuits with an average of 30.1% fewer CNOT gates than *Qiskit* [4] and *t|ket* [20], and 11.5% fewer CNOT gates than other large scale synthesis techniques such as QGo [23]. TopAS improves upon the QGo algorithm by optimizing before mapping and by preconditioning circuit partitions so that the fully mapped results are less deep and require fewer multi-qubit gates.

The remainder of this paper is structured as follows: Section II discusses the general processes of unitary optimization, quantum circuit partitioning, and compares tools that apply synthesis before and after mapping. Section III presents the design choices made for the TopAS tool. Section IV compares the TopAS tool to other optimization and mapping techniques. Finally, Section V provides commentary and discussion about advancements that could further improve the performance of wide quantum circuit synthesis tools.

II. BACKGROUND

A. Quantum Computing Basics

The fundamental unit of information in a quantum computer is the qubit, which can be represented as a vector of the form

$$|\psi\rangle = \alpha \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \beta \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

where α and β are complex numbers such that $|\alpha|^2 + |\beta|^2 = 1$ and $[1 \ 0]^T$ and $[0 \ 1]^T$ are orthonormal basis vectors representing two distinct quantum states. The state of a quantum system with n qubits in states $|\psi_1\rangle, |\psi_2\rangle, \dots, |\psi_n\rangle$ lies in a $2^n \times 2^n$ Hilbert space, and can be represented by the tensor product

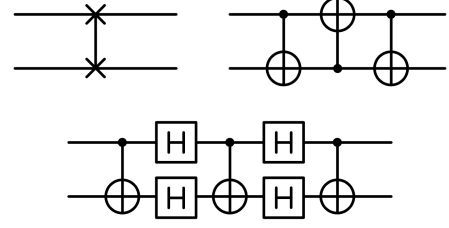


Fig. 2: How to implement a SWAP operation using a SWAP gate, 3 CNOTs, and 3 CNOTs and 4 Hadamards.

$|\psi_1\rangle \otimes |\psi_2\rangle \otimes \dots \otimes |\psi_n\rangle$. This quantum state $|\psi\rangle$ can be evolved by use of $2^n \times 2^n$ unitary operators [14].

The quantum circuit model represents this unitary as a series of quantum gates [3]. Single qubit and multi-qubit gates act on qubits which are drawn as horizontal wires (see Figure 3). The number of wires or qubits n is called the circuit width, while the critical path length or depth of the circuit is T . The depth is typically drawn as the x-axis of the circuit. For the scope of this paper, a universal gate set of $\{U3, CNOT\}$ is assumed. Note that SWAP gates can trivially be decomposed into 3 CNOTs as shown in Figure 2.

B. Quantum Circuit Synthesis

Given a target $2^n \times 2^n$ unitary U and an error threshold ϵ , a unitary synthesis algorithm builds a new circuit whose unitary U_S satisfies the inequality $\|U - U_S\| \leq \epsilon$ [25]. Most recent synthesis tools base their distance metric off the Hilbert-Schmidt inner product as it is computationally inexpensive [2], [9], [11]:

$$\|U - U_S\|_{HS} = \text{Tr}(U^\dagger U_S)$$

There are broadly two types of synthesis, top-down and bottom-up. Top-down synthesis techniques are rule-based, and aim to break down large unitaries into smaller ones. Usually, these algorithms are quick, but the resultant circuit depth grows exponentially, which limits their effectiveness. Bottom-up synthesis starts with an empty circuit and gradually adds gate until a solution is found. Techniques such as QSearch/LEAP use an A* heuristic search to find an optimal depth approximation for the overall unitary [21].

Furthermore, synthesis algorithms such as QSearch also accept a coupling graph as input so that the connectivity between qubits may be specified. The coupling graph provided restricts the synthesis algorithm by requiring that it only place multi-qubit gates that obey this connectivity. Doing so means that the resulting circuit is fully mapped to the specified coupling graph. Previous work [21] has demonstrated that for small circuits, synthesis algorithms are able to map circuits to restrictive qubit topologies using fewer gates than other compiler tools such as *Qiskit* [4] and *t|ket* [20]. Thus when provided a coupling graph and a small width circuit, synthesis is able to completely remove the need for a mapping algorithm. Synthesis algorithms are ultimately limited by the size of the

solution search trees, and the size of the unitary matrices for which the Hilbert-Schmidt distance must be calculated.

C. Quantum Circuit Fidelity and Performance

In the NISQ era, the probability that a quantum circuit is successfully executed depends heavily on the number of multi-qubit gates and the execution time of circuits. This is because multi-qubit gates have a high probability of introducing noise and because quantum states tend to decohere after a certain amount of time [8]. Producing quantum circuit implementations with both fewer multi-qubit gates and lower depth is therefore desirable, and signals that an implementation has a higher chance of executing properly. For N partitions and a synthesis threshold $\epsilon = 10^{-10}$ per partition, the total circuit approximation error $N\epsilon$ is typically between 10^{-8} - 10^{-7} in Hilbert-Schmidt distance for our selection of benchmarks. This means although synthesis introduces approximation errors into circuit implementations, the error threshold is sufficiently low that gate and decoherence errors will dominate overall error. For this reason, we present both CNOT gate counts and circuit depth to evaluate circuit implementations (see Section IV).

Reducing depth is analogous to reducing the run time of a classical program, and thus also acts as a direct measure of circuit performance assuming a quantum machine is capable of executing gates in parallel. Mitigating other sources of error, such as superconducting crosstalk [17], remains an important area of research but is not considered in our evaluation.

D. Mapping Quantum Circuits

Quantum circuits are typically designed with the assumption that the underlying hardware that may run the circuit can support interactions between any pair of qubits. However, due to technological restrictions modern quantum machines typically have very sparse inter-physical qubit connectivity. Several examples of realistic sparse physical qubit topologies are illustrated in Figure 1.

In order to ensure that a quantum circuit can be run on a quantum computer, a mapping algorithm is used to transform the circuit so that it conforms to some restricted qubit connectivity. The process of mapping quantum circuits to physical topologies happens in two stages: placement or layout and routing. During the placement phase, an assignment of logical qubits in the quantum circuit to physical qubits in the qubit topology is created. In the next phase, the routing phase, quantum SWAP operations are inserted to ensure that all multi-qubit interactions specified in the quantum circuit may take place along edges in the physical topology [12], [18]. Throughout both phases of circuit mapping, the goal is to minimize the number of SWAP operations inserted into the final circuit. This is because for machines using the {CNOT, U3} gate set, a SWAP is implemented using 3 CNOT gates (Figure 2). As quantum circuits become wider and more densely connected, the overhead in CNOT count due to routing SWAP gates quickly rises. However, minimizing the number of SWAP operations is known to be NP-Hard [19], so

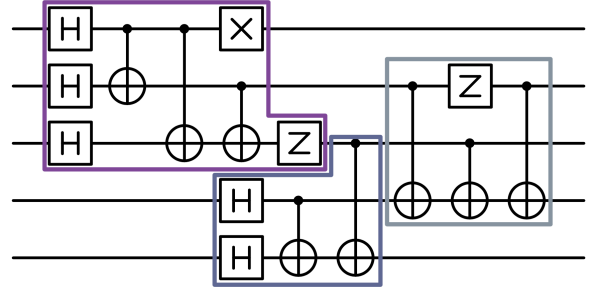


Fig. 3: Example quantum circuit with 5 qubits. Each qubit is represented by a horizontal line, and the timing of the circuit is described by the x-axis. The circuit has been partitioned into 3 subcircuits.

most algorithms use heuristic-based approaches that are sub-optimal, especially for wider circuits.

E. Quantum Circuit Partitioning

Given a quantum circuit and a positive integer k (called the partition width or block size), a partitioning algorithm divides the circuit into subcircuits (also called blocks or partitions) of width at most k . Valid partitions consist only of gates that act on those specific qubits in the partition. As soon as a gate crosses a partition boundary, the partition must be cut vertically along the violating wire. A new partition can then be created, or more gates can be added so long as they act only on qubits still within the partition. An example circuit with width 3 partitions is shown in Figure 3.

For the purposes of unitary synthesis, the goal of a partitioning algorithm is to form as few partitions as possible, and thus ensure that each of these partitions is as large as possible. Our experiments show that these large partitions tend to see a larger reduction in CNOTs as compared to smaller partitions. This effect is illustrated in Figure 5.

F. Post-Mapping vs. Logical Circuit Synthesis

Previous tools to apply unitary synthesis optimization to wide quantum circuits, such as QGo [23], follow a *post-mapping* synthesis flow as illustrated in Figure 4a. After preliminary circuit optimizations, the logical circuit is mapped to a specified physical qubit topology. This mapped quantum circuit is then partitioned into subcircuits, each of which are independently synthesized. The synthesized subcircuits are then reassembled into the complete optimized and mapped circuit.

Although post-mapping synthesis schemes are able to reduce depth and multi-qubit gate count, there are several pitfalls that limit their effectiveness. First, because mapped circuits typically contain more gates than their unmapped counterparts, partitioning algorithms tend to form far more partitions on mapped circuits. Because each of these partitions must be individually synthesized, this approach is typically more time consuming than partitioning and synthesizing the

logical, unmapped, circuit. Second, these methods are very sensitive to the quality of the mapping algorithm used, as they can only reduce inefficiencies introduced into mapped circuits. Doing the same on the logical circuit instead allows for mapping to be done on an (often much) shorter depth and gate count approximate circuit. Logically synthesized circuits typically have fewer gates, which means fewer SWAP gates are typically needed during routing.

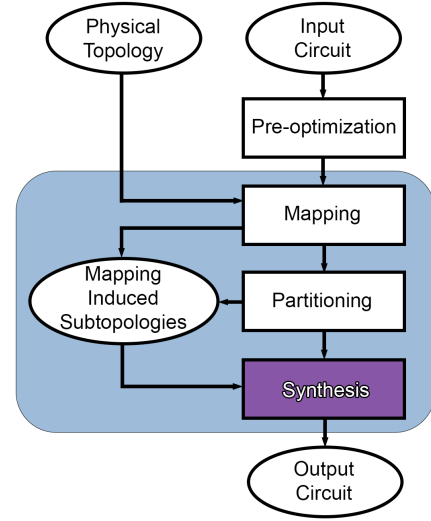
These points motivate the reasoning behind adopting a pre-mapping, or logical circuit synthesis flow. The process of logical circuit synthesis for wide quantum circuits is illustrated in Figure 4b. After some initial quick circuit optimizations, the logical circuit is partitioned into synthesizable subcircuits. Each of these subcircuits is paired with a synthesis subtopology to which the subcircuits are mapped. After each subcircuit is synthesized, the circuit is reconstructed and finally fully mapped to the specified physical qubit topology.

G. Synthesis Subtopology Selection

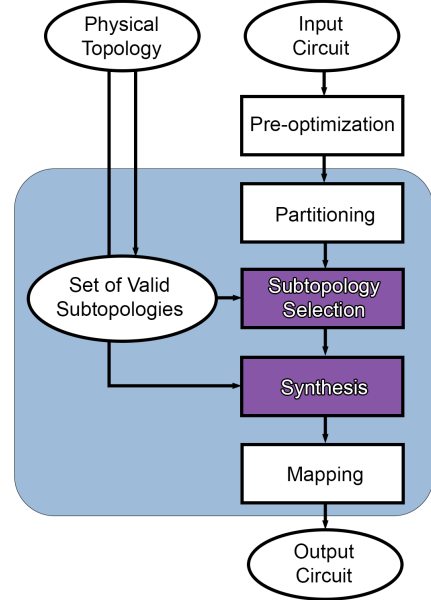
Each partitioned subcircuit must be assigned a graph $G_S = (V, E_S)$ that specifies the connectivity between qubits in the partition. The synthesis algorithm chooses multi-qubit gates that correspond to edges in the set E_S , mapping the subcircuit to this *synthesis subtopology* G_S . The synthesis subtopology selection process in logical circuit synthesis tools diverges from that in post-mapping synthesis tools such as QGo. Since post-mapping tools assume circuits are already routed, the interactions between the qubits in a partition are guaranteed to fit the physical topology. Therefore, we can choose G_S to be the subgraph induced by the physical qubits in a partition.

Logical synthesis affords more choice here as the partitioned circuit is not yet routed to obey the restrictions of the physical hardware. Each synthesis subtopology is a graph of order equal to the number of qubits in that partition. The only other requirement for synthesis subtopologies is that they are connected. This is because if two qubits interact in the partitioned circuit, but there is no path between them in the synthesis subtopology, the synthesis algorithm will be unable to find a circuit whose unitary approximates the original subcircuit's unitary. As synthesis algorithms are mostly limited to operating in the 3-5 qubit partition width ranges, the number of possible synthesis subtopologies is relatively limited.

Our experiments show that the choice of synthesis subtopology can have drastic effects on the number of multi-qubit gates in the optimized subcircuit, as well as on the performance of the routing algorithm during the final mapping process. The goal of the synthesis subtopology selection process is thus to choose a graph G_S for each partition such that the synthesis algorithm is able to produce low gate count subcircuits and the mapping algorithm is able to more efficiently place and route the subcircuit. The structure of the subtopology G_S also heavily impacts the depth of synthesized subcircuits. Depth is heavily dependent on the degrees of vertices in the subtopology G_S . Subtopology selection thus also must carefully consider the connectivity within a partition to ensure that the depth of the optimized circuits is minimized. Section



(a) Post-mapping synthesis optimization.



(b) Pre-mapping or logical circuit synthesis optimization.

Fig. 4: Program flow diagrams for wide circuit optimization tools using unitary synthesis post- and pre-mapping.

III-C further details the choices made for the subtopology selection algorithm for the TopAS tool.

III. TOPOLOGY AWARE SYNTHESIS

We present TopAS, a physical qubit topology aware synthesis tool. By partitioning and synthesizing subcircuits from the logical circuit, the number of multi-qubit operations in a quantum circuit is reduced before mapping begins. The choice to synthesize logical quantum circuits enables TopAS to produce mapped circuits with fewer total CNOT gates than many other optimization tools. TopAS selects synthesis subtopologies for

partitioned subcircuits that are sparse and easily embedded within the underlying physical qubit topology on which the circuit will be run. Synthesis subtopology selection is done in such a way that reduces the number of operations needed for both computation and mapping.

The TopAS tool uses a *scan* partitioning strategy introduced by the authors of QGo [23]. The synthesis algorithm used is the QSearch/LEAP algorithm [21]. Working versions of the partitioning and synthesis algorithms are provided by the BQSKit tool [24]. A flowchart of the TopAS tool’s execution is displayed in Figure 4b.

A. Logical Quantum Circuit Partitioning

The partitioning algorithm views the input circuit as a two dimensional grid, where qubits are represented by rows, and time steps in the circuit are represented by columns. Each element of the grid either contains a quantum operation or is empty. The width of this grid is n , the number of qubits in the circuit. The length is T , the depth or critical path length of the circuit. The *scan* partitioner algorithm sequentially *scans* through all as yet unpartitioned gates in a circuit, examines each possible grouping of k or fewer qubits, and picks the one that lends the largest partition. The *scan* partitioner tends to form few partitions, and often forms partitions with a high average number of multi-qubit gates. As mentioned in Section II-E and illustrated in Figure 5, subcircuits with a large number of multi-qubit gates tend to yield the most reduction by synthesis. The distribution of partitions produced by the *scan* partitioning algorithm for a subset of benchmarks is also shown in Figure 5.

Note that partitions consist only of subcircuits with interacting qubits. A circuit logical connectivity graph is used to enforce this policy. This graph has a vertex for each qubit in the circuit. The presence of an edge (u, v) indicates that there is a multi-qubit gate between qubits u and v at some point in the circuit. Given a partition width of k , candidate partitions consist of all connected subgraphs in this circuit logical connectivity graph of order at most k . In the worst case, a circuit where all qubits interact with each other, there are $O(\binom{n}{k})$ candidate partitions.

B. Synthesizing Logical Quantum Circuits

As logical circuits do not contain gates needed to conform to some restrictive physical topology, they tend to be shorter than their mapped counterparts. Shorter circuits tend to yield fewer partitioned subcircuits, as there are fewer total gates to partition. Partitioning circuits before mapping thus conserves the number of subcircuits produced by the partitioning algorithm. As fewer subcircuits must be independently synthesized in this case, less time is often required to optimize circuits that are partitioned before mapping. Circuits with fewer partitions also tend to have more accurate output circuits. Each partitioned circuit is synthesized independently, so each synthesis procedure will produce a mapped circuit whose unitary representation will be within some Hilbert-Schmidt norm distance ϵ away from the target unitary. The

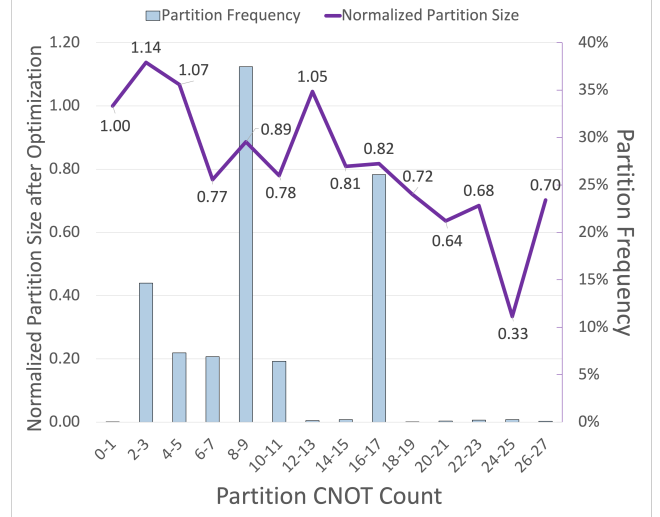


Fig. 5: Partition frequency and mean normalized partition size after optimization as a function of CNOT count. Normalized partition size is calculated using the number of CNOTs in the synthesized subcircuit divided by CNOTs in the original partition. A value less than 1 indicates that synthesis reduces those partitions. The partitioner predominately forms subcircuits with 6-11 CNOTs, but is also able to form many subcircuits with 16-17 CNOTs.

authors of [15] showed that the total error in a circuit that consists of synthesized subcircuits is bounded by the sum of each subcircuit’s distance. If a circuit is partitioned into N subcircuits, because each partition is synthesized to within a distance ϵ , the total distance of the partitioned and synthesized circuit is bounded by $N\epsilon$. Decreasing N , as is done in the logical partitioning case, thus improves the accuracy of the synthesized circuit. Results comparing the upper bound on total circuit errors for several synthesized benchmarks are shown in Table II.

C. Topology Aware Subtopology Selection

The main contribution of this work is a subtopology selection strategy and compilation workflow built around it. It allows for partitioned subcircuits to be preconditioned in such a way that balances the opposing demands of synthesis and mapping. As discussed in Section II-G, the goal of the subtopology selection process is to assign a graph G_S to each partitioned subcircuit. The vertices in G_S represent qubits in the subcircuit, while edges describe the allowed interactions between qubits in the output synthesized subcircuit.

Ideally, a quantum circuit is synthesized so that it uses only edges present in the physical topology for multi-qubit interactions. Before mapping, the placement and thus allowed interactions between qubits is not known. However, synthesizing densely connected subcircuits to sparse graph structures can decrease the need for SWAP operations during the mapping phase. Limiting the set of synthesis subtopologies and carefully selecting sparse subtopologies for each subcircuit

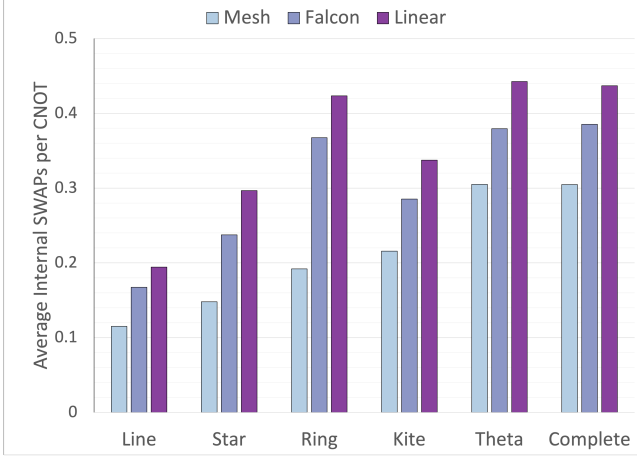


Fig. 6: The number of SWAP operations within partitions synthesized to different synthesis subtopologies for the mesh, falcon, and linear physical topologies. Simpler subtopologies result in fewer SWAP, but may produce circuits with more CNOTs.

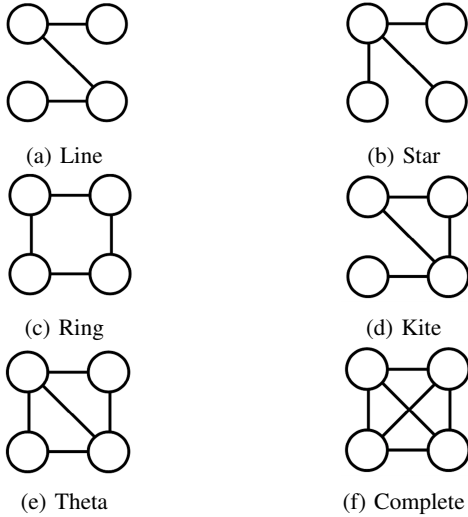


Fig. 7: Possible order 4 synthesis subtopologies. The graphs a, b, and c are embedded within the mesh physical topology. Graphs d, e, and f are not. Only a and b are embedded in the falcon topology, while a is the only embedded subgraph of the linear topology.

allows us to balance the opposing demands of the routing and synthesis algorithms, leading to fewer total CNOT gates. Figure 6 shows the average number of internal SWAP gates per CNOT for a variety of synthesis subtopologies targeting three physical topologies. Internal SWAP gates are routing operations that are inserted between the first and last gate of a partition. If each subcircuit is synthesized to an embedded subgraph of the physical topology and is executed atomically, there is a solution to the mapping problem that requires zero SWAP operations on the subcircuit’s qubits during its

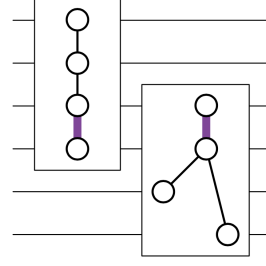


Fig. 8: Neighbor aware subtopology selection encourages the reuse of edges between sequentially executing partitions.

execution. However, because routing often disrupts the execution of subcircuits, it is unlikely for there to be exactly zero internal SWAP operations even for subtopologies that are embedded within the physical topology. The number of internal SWAP gates observed for a partition is normalized by the number of CNOTs in the partition. This allows us to more effectively compare subcircuits of various sizes. Typically, sparser subtopologies require more gates for computation than denser subtopologies, but they require fewer SWAP gates during mapping.

The TopAS tool restricts the set of graphs from which to choose synthesis subtopologies to the set of connected graphs that are embedded in the target physical topology. TopAS uses a partition width of 4 qubits. The order 4 candidate subtopologies are illustrated in Figure 7.

QSearch, the synthesis algorithm at the core of the TopAS optimization flow, aims to synthesize circuits using as few multi-qubit gates as possible. Often times, circuit depth is minimized as a consequence of this goal but is not the primary metric of success. Subtopology selection is the only variable that TopAS uses to reduce circuit depth directly. For certain subtopologies choices, circuits produced tend to contain many gates that must be executed sequentially. The most significant contributor to this property is the possible graph matchings present in a given subtopology. In topologies such as the star (Figure 7b), any choice of a single edge is a maximum matching. This means that for the star subtopology, no two CNOTs can be executed in parallel. Other subtopologies like the line and ring (Figure 7a and 7c) have larger maximum matchings, and thus allow more parallelism within synthesized subcircuits. Although they typically require more routing gates, subtopologies containing more edges tend to allow for lower depth circuits.

The qubit interactions within a partitioned subcircuit can be described using a weighted undirected graph $G_L = (V, E_L)$. For each multi-qubit interaction that occurs between qubits $k_1, k_2 \in [K]$ in the subcircuit, there is an edge $(k_1, k_2, w) \in E_L$, where the weight w corresponds to the number of times that interaction occurs. Synthesis algorithms such as QSearch allow for the connectivity between qubits to be specified by an unweighted undirected graph $G_S = (V, E_S)$. For a maximum partition size of k , $V = [k]$ and $E_S \subseteq V \times V$.

Our experiments indicate that synthesis is best able to reduce the multi-qubit gate count of subcircuits when the logical connectivities and synthesis subtopologies match. A kernel function [7] is used to quantify the similarity between the graphs. The scoring function $K : G_L \times G_S \rightarrow [0, 1]$ examines each edge in the synthesis subtopology G_S and logical connectivity G_L . From the edges, two vectors $v_L, v_P \in R^{k(k-1)/2}$ are constructed. The vector v_P is simply an indicator vector, with a 1 at each element that corresponds to a present edge in G_S . Element i of v_L contains the weight w associated with edge i in G_L . The normalized inner product

$$\text{similarity}(v_P, v_L) = \frac{v_P^T v_L}{\sum_i v_L(i)}$$

is then returned to quantify the similarity between the graphs. For each partition, all $k!$ permutations of qubit labels are evaluated for each of the candidate synthesis subtopologies. The permuted subtopology with the highest kernel function score is considered the best candidate subtopology for the purposes of producing the smallest output subcircuit. The effectiveness of the similarity kernel function was evaluated by synthesizing partitions of width 4 to all order 4 subgraphs embedded within the mesh physical topology (Figures 7a, 7b, and 7c). In this scenario, the similarity function was able to identify the subtopology that would produce the synthesized subcircuits with the fewest CNOT gates 89% of the time.

If multiple subtopologies have the same similarity score, that with the fewest edges is preferred. In order to favor sparse subtopologies, each similarity is multiplied by a bias factor. The biases that produced the best results for the mesh topology were 1.0 (line), 1.0 (star), and 0.8 (ring).

When selecting subtopologies, TopAS also considers the impact of subtopology choice for the immediately preceding and succeeding partitions. Although partitions are synthesized separately, they are not executed in isolation. If a partition assumes a physical edge exists between two qubits, and the partition immediately following also assumes this physical edge exists, fewer total SWAPs may be needed between the execution of these two partitions. The TopAS tool therefore checks each subcircuit's neighboring partitions for qubits that are shared. If interactions from the neighboring partitions occur between shared qubits, they are added as edges to the current partition's logical connectivity graph G_L . Interactions from neighboring partitions carry the same weight as interactions within a partition, even if the qubits in the neighbor partitions do not interact directly within the partition itself. This policy was chosen as larger amounts of edge sharing between partitions greatly reduced the final CNOT count of optimized circuits. TopAS uses this *neighbor aware* version of the logical connectivity graph in the computation of the similarity function. An example of two sequentially executing partitions with subtopologies that have been selected to reuse an edge is shown in Figure 8.

Algorithm 1 Subtopology Selection

Input: graph G_L , set of graphs $G_{neighbors}$, *bias* function
Output: graph G_S

```

for all graphs  $G_n$  in  $G_{neighbors}$  do
  if shared_edge in  $G_n$  and in  $G_L$  then
     $G_{overlap} \leftarrow \text{shared\_edge}$ 
  end if
end for
 $v_L \leftarrow$  edge weight vector for  $G_{overlap} \cup G_L$ 
 $candidates \leftarrow$  order  $\leq 4$  subgraphs in physical topology
for all graphs  $G_P$  in  $candidates$  do
   $v_P \leftarrow$  indicator vector for  $G_P$ 
   $score = \text{similarity}(v_P, v_L)$ 
  if  $\text{bias}(G_P) \times score > \text{high\_score}$  then
     $\text{high\_score} \leftarrow \text{bias}(G_P) \times score$ 
     $G_S \leftarrow G_P$ 
  end if
end for
return  $G_S$ 

```

D. Partition Replacement and Mapping

As indicated in Figure 5, it is sometimes the case that synthesized subcircuits grow instead of reduce in size. When this happens, TopAS considers replacing the synthesized subcircuit with the original subcircuit. Always choosing the logical subcircuit when it has fewer CNOTs can be problematic in the final routing pass, as the original subcircuit may be more difficult to route than the synthesized version. Thus, TopAS considers differences in both the number of CNOTs and logical connectivities of the two subcircuits. TopAS adopts a policy of replacing the synthesized subcircuit with the original if it shows more than 30% (empirically determined best threshold) fewer CNOTs or has a more routable logical connectivity. The routability of a subcircuit's logical connectivity is determined by the average number of internal SWAP operations per CNOT for a given physical topology (see Figure 6).

After subtopology selection and synthesis take place, TopAS maps the synthesized quantum circuit to a physical topology. For placement and routing, the SABRE Layout and SABRE Swap [12] mapping algorithms are used.

IV. RESULTS

The performance of TopAS is compared to other optimization and mapping tools by measuring the depth and total CNOT operation counts of optimized and mapped circuits. Here, we compare TopAS (as described in Section III) to the *Qiskit*, *t|ket*, and QGo tools. In all cases, circuits are optimized using *t|ket*'s full peephole optimization. Afterwards, either *Qiskit* or *t|ket* are used to map the quantum circuit to the specified physical topology. The QGo tool was used to optimize either the *Qiskit* or *t|ket* mapped circuits, whichever contained fewer CNOT operations. The synthesis tool used is the QSearch/LEAP algorithm [21]. The original QGo algorithm was re-implemented using the partitioning and synthesis algorithms provided by the *BQSKit* toolkit [24].

	Mapping	Topology	Total Error	CNOTs per Partition
mult_16	QGo	Falcon	1.38E-08	15.3
	QGo	Mesh	1.51E-08	12.9
	TopAS	-	7.20E-09	15.4
mult_32	QGo	Falcon	1.17E-07	12.5
	QGo	Mesh	1.15E-07	11.7
	TopAS	-	4.96E-08	15.1
qft_64	QGo	Falcon	5.83E-08	7.3
	QGo	Mesh	5.18E-08	7.8
	TopAS	-	2.32E-08	8.1
qft_100	QGo	Falcon	8.09E-08	8.1
	QGo	Mesh	8.72E-08	7.4
	TopAS	-	3.85E-08	8.0
add_65	QGo	Falcon	9.09E-08	6.5
	QGo	Mesh	7.02E-08	7.3
	TopAS	-	3.53E-08	7.0
add_101	QGo	Falcon	1.26E-07	11.8
	QGo	Mesh	1.23E-07	9.4
	TopAS	-	6.18E-08	7.0
tfim_40	QGo	Falcon	1.42E-07	7.61
	QGo	Mesh	1.57E-07	6.3
	TopAS	-	1.05E-07	8.1
tfim_100	QGo	Falcon	5.17E-07	8.4
	QGo	Mesh	4.71E-07	7.7
	TopAS	-	2.70E-07	8.0
hubbard_18	QGo	Falcon	1.83E-07	8.0
	QGo	Mesh	1.60E-07	6.5
	TopAS	-	9.56E-08	3.7
shor_26	QGo	Falcon	3.86E-07	10.3
	QGo	Mesh	3.32E-07	11.6
	TopAS	-	1.52E-07	13.9

TABLE I: Upper bound of total circuit error (sum of Hilbert-Schmidt distances per partition) and average CNOTs per partition for each benchmark. A synthesis threshold of $\epsilon = 10^{-10}$ was used for both QGo and TopAS.

Performance was evaluated using a variety of wide quantum circuit benchmarks. The SupermarQ benchmark suite [22] proposed a *volume* metric to describe a benchmark suite’s diversity. Our selection of benchmarks has a SupermarQ volume of 7.46×10^{-6} . This small selection of circuits outperforms many other benchmark suites in SupermarQ volume due to its focus on real algorithms. Our suite’s SupermarQ volume score is limited by the fact that none of our benchmarks include intermediate measurements. This decision was made as the primary focus in the evaluation of synthesis based optimization tools is purely depth and multi-qubit gate count reduction. Each benchmark assumes qubits are measured after the final program gate is executed.

The *qft* and *shor* circuits were generated using *Qiskit*. The *add* and *mult* circuits were generated with [10]. The *hubbard* benchmark was generated with help from [13]. TFIM circuits were generated using the ArQtic tool [1]. Benchmarks are labeled so that the number following the benchmark name indicates the number of qubits in the circuit. These benchmarks were selected as they were found to be relatively easily

generated, and represent a variety of wide circuits that may soon be executable on quantum machines. Results for the QGo and TopAS tools were collected using NERSC’s *Perlmutter* supercomputer.

Figures 9 and 10 show the relative CNOT gate count and depth of circuits optimized and mapped to the mesh and falcon physical topologies illustrated in Figure 1. The CNOT gate count and depth are normalized by the results obtained for circuits optimized then mapped using *Qiskit*’s SABRE mapping algorithms. A synthesis threshold distance of $\epsilon = 1 \times 10^{-10}$ and a partition width of $k = 4$ was used for both QGo and TopAS.

On average, the TopAS tool is able to reduce CNOT count compared to *Qiskit* and *t|ket* by 35.9% and 24.3% respectively when targeting a mesh physical topology. For the falcon physical topology, TopAS outperforms *Qiskit* and *t|ket* by an average of 33.4% and 19.1% respectively. For the *tfim_100* circuit, TopAS outperforms *Qiskit* by 62.1% and *t|ket* by 53.1%. For the *shor_26* benchmark, TopAS outperforms *Qiskit* by 38.7% and *t|ket* by 30.8%. Reductions in circuit depth are even greater, averaging 38.6% and 37.6% lower compared to *Qiskit* and *t|ket* across all benchmarks.

In most cases, the TopAS tool is able to produce circuits with fewer CNOTs and much lower depth than the QGo tool, especially for the mesh physical topology. On average, TopAS outperforms QGo by 11.5% in CNOT count and 34.3% in depth when targeting the mesh physical topology. For the falcon physical topology, TopAS only outperforms QGo by an average of 0.8% in CNOT count, but maintains a 37.3% average depth reduction.

QGo optimized circuits always reduce CNOT count compared to unsynthesized input circuits, but sometimes lead to deeper circuits. This is because when synthesizing pure SWAP gates, the QSearch algorithm tends to find the more expensive depth 5 implementation illustrated in Figure 2. TopAS is able to avoid this pitfall by synthesizing partitions that do not contain SWAP gates, thus resulting in much reduced circuit depths.

As discussed in Section III-B, working on the logical, unmapped, quantum circuit also allows TopAS to form fewer partitions compared to QGo. This results lower upper bounds on the amount of synthesis error for each benchmark. Synthesis error upper bounds are given in Table 1. This table also lists the average number of CNOTs per partition for all benchmarks.

TopAS is able to outperform QGo in CNOT count for all benchmarks when targeting the mesh physical topology except for the *hubbard_18* circuit benchmark. The discrepancy in performance for this benchmark is primarily explained by the average size of partitions formed. Table 1 shows that for this circuit, TopAS is only able to form partitions with an average of 3.7 CNOT gates. Figure 5 illustrates that typically, partitions with this number of CNOTs tend not to reduce. Partitions formed by QGo tend to include SWAP gates, which greatly increases the number of CNOTs per partition (6.5 CNOT gates on average for *hubbard_18*). For the *hubbard_18* circuit,

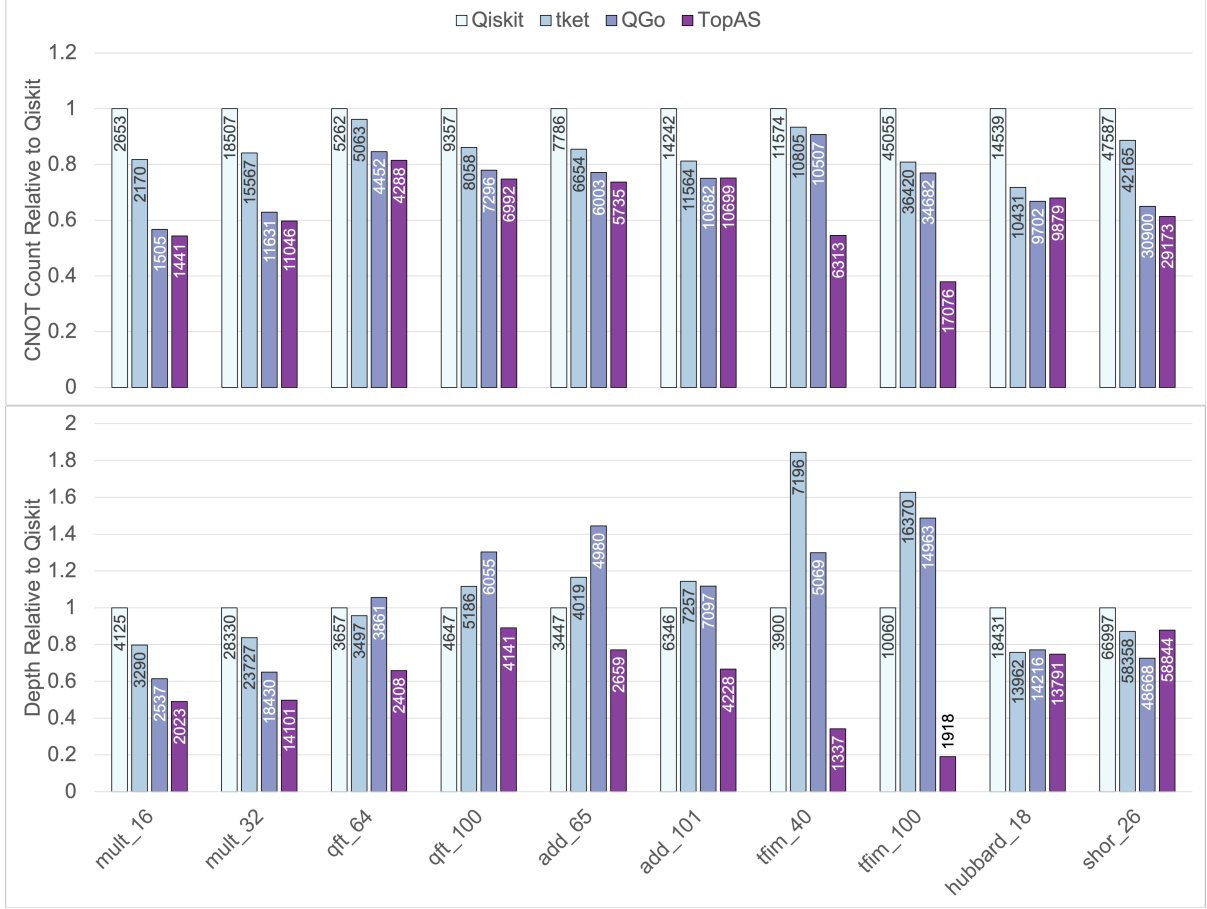


Fig. 9: Comparison of relative CNOT count (top) and depth (bottom) for circuits mapped to the Google style mesh physical topology. CNOT count and depth is shown relative to optimized circuits mapped using *Qiskit*'s SABRE Swap algorithm. Lower depth corresponds directly to better program runtimes on machines with parallel gate execution, lower depth and CNOT count correspond to improved execution probability on noisy machines.

TopAS's logical partitioning was simply not able to form few enough partitions to compensate for the small average number of CNOT gates per partition. Using a larger partition width, or a partitioning algorithm that is able to produce larger partitions on average would likely improve the performance of TopAS for the *hubbard* benchmark. This point also partially explains the drop in TopAS' performance compared to QGo for benchmarks targeting the falcon physical topology.

The scalability of the TopAS tool compared to *Qiskit*, *tket*, and QGo is illustrated in Figure 11. Each tool was used to optimize and map QFT and TFIM circuits with widths of 10-100 qubits. Despite performing poorly in comparison at small circuit widths, TopAS maintains an advantage compared to other optimization tools as the circuit width increases. In the case of certain circuits such as the TFIM circuits, TopAS' advantage increases with circuit width.

V. DISCUSSION

TopAS is able to produce circuits with fewer multi-qubit gates and lower depth by partitioning logical quantum circuits

and matching subcircuits with sparse qubit subtopologies in such a way that balances the demands of synthesis and mapping algorithms. Partitioning and using unitary synthesis to optimize the logical quantum circuit allows for smaller circuits to be passed to mapping algorithms. In the case of the *tfim_40* and *tfim_100* benchmarks, the logical connectivity graphs are simply linear path graphs of order 40 and 100 respectively. These circuits have implementations that do not require any SWAP operations, as they are directly embeddable within the mesh physical topology. However, mapping algorithms implemented in *Qiskit* and *tket* are unable to find these placements, likely due to the large widths and depths of these circuits. By optimizing the logical quantum circuits, TopAS is able shorten circuits in such a way that allows for these mapping algorithms to find better placements. This advantage in mapping performance is maintained in the falcon topology case, where the TFIM circuits are no longer directly embeddable in the physical topology.

Using synthesis to map partitions to restrictive subtopologies also improves the performance of the mapping algorithm.

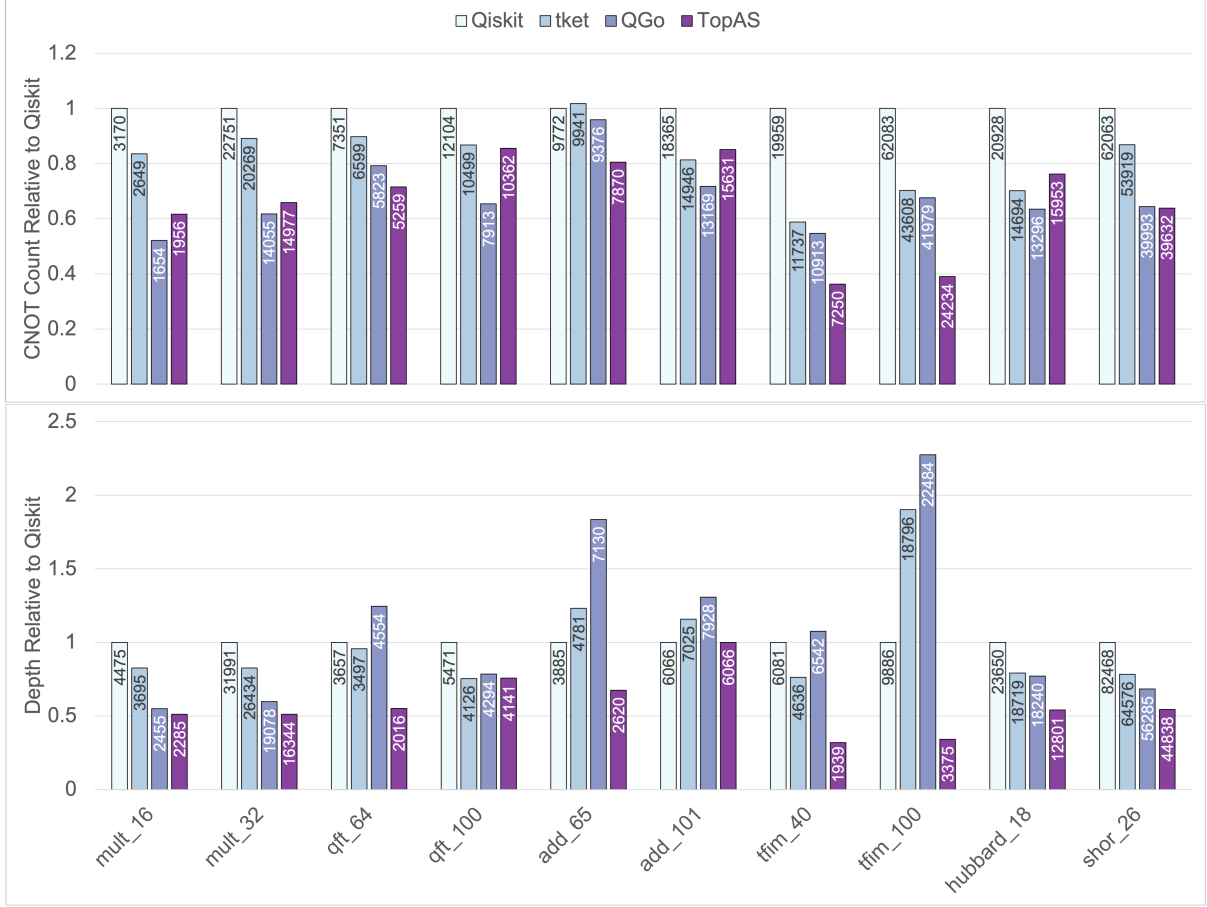


Fig. 10: Comparison of relative CNOT count (top) and depth (bottom) for circuits mapped to the IBM style falcon physical topology. CNOT count and depth is shown relative to optimized circuits mapped using *Qiskit*'s SABRE Swap algorithm. Lower depth corresponds directly to better program runtimes on machines with parallel gate execution, lower depth and CNOT count correspond to improved execution probability on noisy machines.

Mapping to subtopologies that are easily embedded within the physical topology means that fewer SWAPs are needed within the execution time of partitions. This effect is illustrated in Figure 6. The *neighbor aware* subtopology selection mechanism illustrated in Figure 8 helps to maximize the reuse of edges in the physical topology between the execution of partitions. The combination of these design choices allows for TopAS to outperform other optimization and mapping tools.

Section IV demonstrates how TopAS is able to reduce both CNOT count and circuit depth significantly compared to other tools. These metrics both play a major role in the likelihood that a circuit will be executed correctly. The amount of error due to synthesis (shown in Table II) is far lower than that introduced by gate noise and decoherence. In total, optimizing wide circuits with TopAS therefore greatly increases circuit fidelity compared to other tools.

Because mapping is done after partitioning and synthesis in the TopAS program flow, it is not limited to using a single mapping algorithm. We observed that although TopAS synthesizes partitions to subtopologies embedded within the

physical topology, it is often the case that the mapping algorithm disrupts the execution of synthesized subcircuits. When partitions are executed atomically, there is a solution to the routing problem such that no SWAP gates are needed during the partition's execution. A mapping algorithm that is aware of higher level structures than the primitive gate set may therefore further improve the performance of TopAS. Such a tool was put forth by the authors of [5], but it is only able to consider Toffoli gates instead of arbitrary k qubit unitary operations. A full *partition aware* mapping algorithm is therefore likely necessary to reap the full benefits of this strategy.

Although TopAS' set of valid synthesis subtopologies only includes graphs that are embedded within the physical topology, it does not consider whether a set of subtopologies can be packed into the physical topology simultaneously. This fact explains the discrepancy in TopAS' performance between the mesh and the falcon physical topologies. For example, although the star subtopology is embedded within the falcon physical topology, far fewer star graphs can be packed into

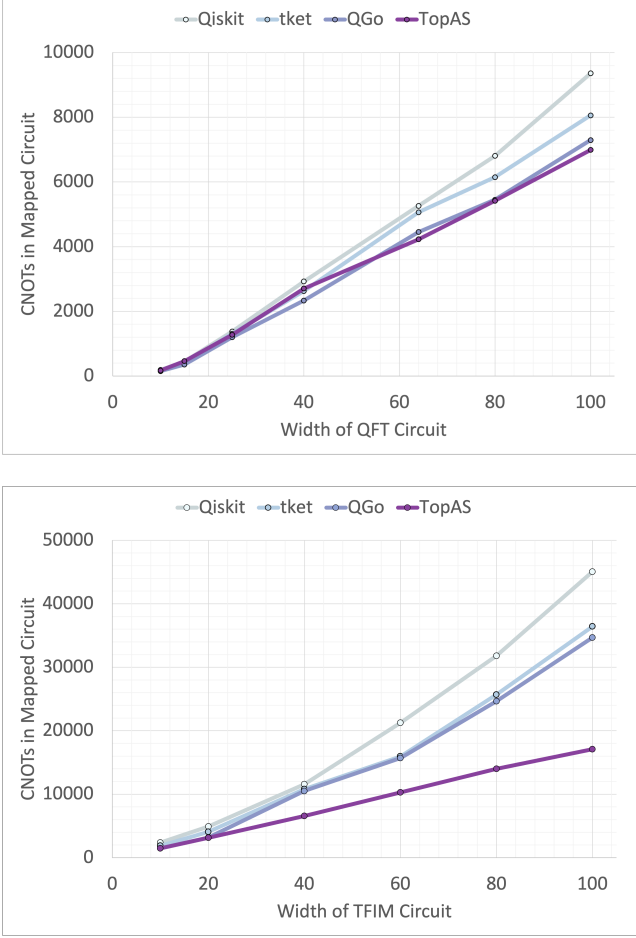


Fig. 11: Number of CNOTs in QFT (top) and TFIM (bottom) circuits mapped to the 2D mesh physical topology as a function of circuit width.

the falcon topology than the mesh topology. A subtopology selection process that more precisely weighs the frequency with which subgraphs appear in a physical topology would thus likely further improve the performance of TopAS.

The two primary factors that limit the scalability of TopAS are the runtime of the partitioning algorithm used and the runtime of the synthesis algorithm used. In the case where qubits in the logical circuit interact with all other qubits, the runtime of the *scan* partitioner is $O\left(\binom{n}{k}\right)$, where n is the circuit width and k is the partition width. This poor scaling limited the width of benchmark circuits tested to 100 qubits. Other partitioning schemes capable of forming large partitions on average are therefore needed to optimize larger quantum circuits with TopAS. Algorithms such as QFAST [25] effectively increase the width of circuits that can be optimized, but in our experience tend not to perform as well as QSearch/LEAP in the range of 1-25 CNOT gates. Increasing the partition width tends to increase the number of CNOTs per partition, which may provide more opportunity for improvement. Increasing

the partition width also has the effect of adding more graphs to the set of valid synthesis subtopologies. Supposing that the optimization algorithm scales, this may further improve the performance of TopAS by allowing for a greater amount of mapping to be handled by the optimization algorithm.

As shown in Figure 11, the number of CNOTs in QFT circuits optimized with the TopAS tool grows roughly linearly with the number of qubits in the range shown. As the QFT is a common component of more complex quantum algorithms, the scaling of this circuit is of particular importance. With a width of 100 qubits, the TopAS optimized QFT circuit contains 23.9% and 11.7% fewer CNOTs than the *Qiskit* and *tket* optimized and mapped circuits. Notably, the rate of increase in CNOTs per qubit in the QFT circuit seems to grow more slowly for TopAS than for *Qiskit* and *tket*. Because QGo only optimizes circuits that have already been mapped by these tools, we expect that TopAS' performance advantage over QGo can improve at larger (100-1000 qubit) circuit widths. This effect can clearly be seen in the TFIM circuit scaling results.

Although out of the scope for the current paper, an obvious next question would be to see what happens when TopAS is combined with QGo. We have clearly shown that a single application of TopAS' logical circuit synthesis almost always outperforms a single round of QGo optimization. When applying both TopAS and QGo to the *hubbard_18* benchmark mapped to the 2D mesh physical topology (a benchmark which TopAS fails to reduce CNOT count more than QGo for the mesh topology), CNOT count was reduced by approximately 5% compared to QGo alone. Thus, combining multiple levels of optimization has the potential to further improve the performance of quantum circuits.

VI. CONCLUSION

In this work, we have presented TopAS, a topology aware synthesis tool that optimizes wide quantum circuits. By optimizing quantum circuits using unitary synthesis before they are mapped to restrictive qubit topologies, TopAS preconditions circuits so that they are minimized by synthesis and made easier to route by mapping algorithms. TopAS is able to reduce CNOT count and circuit depth, and thus improves circuit performance, compared to other state of the art synthesis based optimization tools targeting wide circuit optimization. TopAS also outperforms the optimization and mapping frameworks provided by *Qiskit* and *tket*. Because CNOT count and circuit depth are significantly reduced, the likelihood that circuits optimized with TopAS execute successfully on noisy, realistic, near term qubit topologies is greatly increased. Further reduction is possible on TopAS optimized circuits by applying successive rounds of post-mapping circuit optimization.

ACKNOWLEDGEMENTS

This work was supported by the DOE under contract DE-5AC02-05CH11231 through the Office of Advanced Scientific Computing Research (ASCR) Quantum Algorithms Team and Accelerated Research in Quantum Computing programs, and

GA4QCO: Genetic Algorithm for Quantum Circuit Optimization

Leo Sünkel ^{*1}, Darya Martyniuk ^{†2}, Denny Mattern ^{‡2}, Johannes Jung ^{§2, 3}, and Adrian Paschke ^{2, 3}

¹Institute for Informatics, LMU Munich, Germany

²Data Analytics Center, Fraunhofer FOKUS, Berlin, Germany

³Institute for Computer Science, Freie Universität Berlin, Germany

Abstract

The design of quantum circuits is often still done manually, for instance by following certain patterns or rule of thumb. While this approach may work well for some problems, it can be a tedious task and present quite the challenge in other situations. Designing the architecture of a circuit for a simple classification problem may be relatively straightforward task, however, creating circuits for more complex problems or that are resilient to certain known problems (e.g. barren plateaus, trainability, etc.) is a different issue. Moreover, efficient state preparation or circuits with low depth are important for virtually most algorithms. In attempts to automate the process of designing circuits, different approaches have been suggested over the years, including genetic algorithms and reinforcement learning. We propose our GA4QCO framework that applies a genetic algorithm to automatically search for quantum circuits that exhibit user-defined properties. With our framework, the user specifies through a fitness function what type of circuit should be created, for instance circuits that prepare a specific target state while keeping depth at a minimum and maximizing fidelity. Our framework is designed in such a way that the user can easily integrate a custom designed fitness function. In this paper, we introduce our framework and run experiments to show the validity of the approach.

This work has been submitted to the IEEE for possible publication. Copyright may be transferred without notice, after which this version may no longer be accessible.

*leo.suenkel@ifi.lmu.de

†darya.martyniuk@fokus.fraunhofer.de

‡denny.mattern@fokus-extern.fraunhofer.de

§johannes.jung@fokus.fraunhofer.de

1 Introduction

While quantum computing has been gaining momentum in recent years, the capabilities of quantum computers of the current so-called Noisy-Intermediate-Scale-Quantum (NISQ) [15] era are still severely limited. That is, current quantum computers contain relatively few qubits that are error-prone, circuits are restricted to a low depth and algorithms are run without any error-correction. To overcome these limitations, hybrid algorithms, i.e., algorithms that partially run on classical and quantum computers, are often used instead of purely quantum algorithms [12, 18]. Although quantum supremacy was claimed in 2019 [2], it is yet to be seen whether a practical advantage can be produced in the NISQ-era.

Putting aside whether NISQ-algorithms will produce the highly anticipated quantum advantage, other fundamental questions must also be addressed. For instance, how does one construct or design a quantum circuit for a specific task? What constitutes a good circuit? Why choose one ansatz over another? Quantum circuits are often hand-crafted, i.e., one manually comes up with a design of a particular circuit. This can for example be done by following standard practices. However, designing efficient and powerful circuits for NISQ-devices is a non-trivial task. Naturally the question arises whether this process can be automated, that is, could an algorithm design novel quantum circuits for a given task? This question, as we will see, is not new and different approaches in this regard have been investigated by the research community.

In this paper, we introduce our GA4QCO framework, a framework that uses a genetic algorithm to design completely new circuits and to optimize existing ones. More importantly, our proposed framework allows its users to define a custom fitness function to automatically search for quantum circuits matching

a user-defined criteria. We start by briefly discussing related work and different approaches that have been proposed for the task of searching for quantum circuits by the research community. In Section 3 we give a short recap of the fundamentals of genetic algorithms and quantum computing required to understand the core components of our proposed method. We introduce our framework in Section 4. We apply our framework to different settings relevant in the NISQ-era and discuss our experimental setup and results in Section 5. We give an outlook on future work in Section 6 and a conclusion in Section 7.

2 Related Work

The idea of automatically searching and designing quantum circuits has been around for a while, and a prominent approach in this area is to apply genetic algorithms. Williams and Gray use a genetic programming algorithm to design quantum circuits in [20]. In their approach, the algorithm is given a target unitary matrix and the goal is to find circuits whose unitary matrix is minimally different than the target. They apply their algorithm to the problem of creating circuits for quantum teleportation. Rubinstein proposes a genetic programming approach for evolving circuits that result in maximally entangled qubits in [17]. Lukac and Perkowski propose a genetic algorithm with the goal to evolve circuits that correspond to a specified target unitary matrix [11]. More recently, Ding and Spector proposed a genetic algorithm for quantum architecture search for parameterized quantum circuits in the domain of reinforcement learning [4] and Rattew et al. propose their evolutionary variational eigensolver in [16]. Zhang and Zhao propose an evolutionary algorithm for the task of circuit architecture search and apply it to classification [22].

Other algorithms and methods have also been proposed for this task. Du et al. introduced their quantum architecture search (QAS) algorithm in [5], which is an algorithm to automatically design quantum circuits with improved learning behavior for variational quantum algorithms. In [10] Lu et al. propose their Markovian quantum neuralevolution (MQNE)

algorithm, which searches for quantum circuits in the context of quantum machine learning. Pirhooshyaran and Terlaky investigate the use of three different approaches, i.e., random search, reinforcement learning and Bayesian optimization to search for quantum circuits. In their approach, the trainability of the circuits is taken into account during the search [14]. Wu et al. introduced a compilation and circuit optimization framework named QGo in [21]. QGo can produce an optimized circuit for a target device. Chivilikhin et al. [3] propose a technique that, among other things, optimizes the topology of a quantum circuit for a hardware-efficient variational quantum eigensolver through the use of a multi-objective genetic algorithm. The genetic algorithm aims to minimize the number of CNOT gates in a circuit as well as the energy of the system. Kuo et al. employ deep reinforcement learning to find quantum circuits that generate a specified target state from an initial state [9].

3 Background

In this section, we will briefly recap the fundamentals of genetic algorithms (GAs) and quantum computing (QC) that are required to understand the core concepts of our proposed framework, which we will introduce in Section 4. For more comprehensive introductions to GAs we refer to [7] and to QC to [13].

3.1 Genetic Algorithms

GAs are meta-heuristic optimization algorithms that incorporate many methods and concepts inspired by biological Darwinian evolution. That is, they are algorithms inspired by nature to tackle a wide range of problems that may appear in a plethora of domains. A GA is made of a number of building blocks and we will introduce the essential ones next.

Individual: A single solution is represented by an individual. An individual may consist of several properties, however, we can think of it as an representation or encoding of a single solution for the problem at hand.

Population: Every GA consists of a population, which contains a number of individuals. It can be

thought of as a list of individuals.

Crossover: Throughout the evolutionary process new individuals (i.e. solutions) are created through the use of crossover operations. While there exist a variety of different crossover operations, their common purpose is to create a new individual, often referred to as the child, by combining the properties of two existing individuals, often referred to as the parents. This recombination does not always produce better individuals, however, through the evolutionary process, which is inspired by the idea of survival of the fittest, bad solutions and traits should be neglected and better solutions should ultimately be preferred and thrive.

Mutation: Random mutations are applied to individuals resulting in a slightly different solution. A mutation could for example be a simple bit-flip, however, more elaborate mutations may be applied.

Generation: A GA runs for a number of generations. In each generation, new solutions are created via crossover and mutation. At the end of each generation, some individuals are discarded while the rest survive and take part in the next generation.

Selection: Selection pressure is applied in two occasions. For one, there is the parent selection, i.e., the process of determining what individuals are allowed to take part in the crossover operations in a particular generation. Secondly, there is the survivor selection, i.e., the process of determining which individuals are going to be part of the next generation and which are discarded.

Fitness: Each individual has a fitness value, which determines how good a particular solution is. It drives the evolutionary process and allows the ranking of individuals.

Evolutionary process: A GA starts with a population of random individuals and then runs for a number of generations with the goal to find individuals with the highest fitness value. In each generation, new individuals (children) are created by applying crossover operations on existing individuals (parents). Mutations are applied randomly to newly created solutions. Individuals are ranked by fitness determined by a problem specific fitness function. At the end of each generation, weaker individuals are replaced by children thus creating the population for

the next generation. This process repeats until the specified number of generations is reached. As randomness is part of the algorithm, convergence is not guaranteed, and each run may return different solutions. [7, 6]

These are the basic building blocks of a GA and we will see how we construct our algorithm for the search for quantum circuits in Section 4. However, we will first introduce the basics of QC. [4]

3.2 Quantum Computing

Quantum computing is a concept in quantum information theory that utilizes quantum properties to gain an advantage over classical computing for some problems. Instead of working with classical bits, that can be either zero or one, quantum computers work with so called *qubits*. Qubits can be in a superposition of zero and one, which we will denote as $|0\rangle$ and $|1\rangle$. A qubit state $|\phi\rangle$ can be seen as an unit vector in a two dimensional complex Hilbert space with basis states $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ (which we will call the *computational basis*), such that

$$|\phi\rangle = \alpha_0 |0\rangle + \alpha_1 |1\rangle = \alpha_0 \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \alpha_1 \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad (1)$$

where $\alpha_0, \alpha_1 \in \mathbb{C}$ and correspond to the probabilities of collapsing to the states $|0\rangle$ or $|1\rangle$ respectively. Note that $|\alpha_0|^2 + |\alpha_1|^2 = 1$ holds.

A system of multiple qubits can be described as a product space of the single qubit spaces. For example, if we have two qubits, a general state $|\varphi\rangle \in \mathcal{H} \times \mathcal{H}$ can be described as $|\varphi\rangle = \alpha_{00} |00\rangle + \alpha_{01} |01\rangle + \alpha_{10} |10\rangle + \alpha_{11} |11\rangle$, where now $|\alpha_{00}|^2$ is the probability that the first and second qubit is measured in the $|0\rangle$ state, $|\alpha_{01}|^2$ is the probability that the first state is measured in the $|1\rangle$ state and the second qubit in the $|0\rangle$ state and so on.

A quantum computer can act via *gates* to map an initial quantum state to another state. Since quantum states are unit vectors in the 2^n -dimensional Hilbert space $\mathcal{H}$, the gates must ensure that the resulting state is always a unit vector. That is the reason why gates of a quantum computer are always

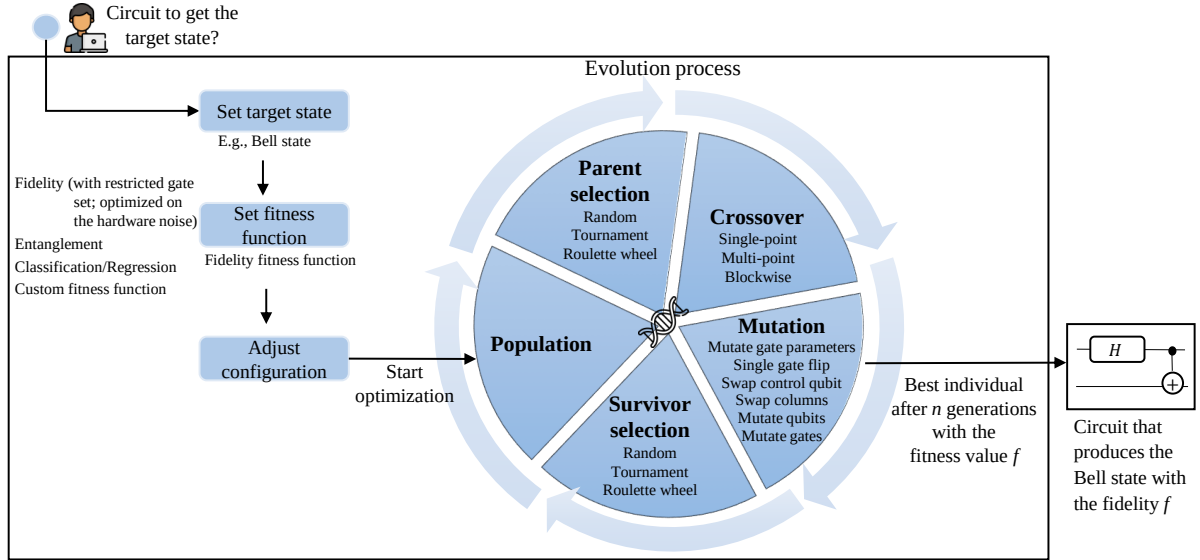


Figure 1: Optimization of quantum circuits in the GA4QCO framework.

represented by unitary matrices. A matrix is unitary iff $U^\dagger U = U U^\dagger = I$, where I is the identity matrix. U has the property that it maps a unit vector $|\varphi\rangle \in \mathcal{H}$ to another unit vector $U|\varphi\rangle \in \mathcal{H}$. In theory, any unitary matrix can be applied to the system, but in practice only local gates (gates that act on a subsystem of only a few qubits) are implemented on a quantum computer. This is due to the physical limitations (e.g. noise).

Entanglement is another important aspect in QC. Loosely speaking, two qubits are entangled if their states depend on each other. For example, the bell state $|\varphi\rangle = \frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|11\rangle$ is an entangled state: If the first qubit is measured in the $|0\rangle$ state the second qubit also have to be in the $|0\rangle$ state and the same for the $|1\rangle$ state. Formally, a two-qubit state $|\varphi\rangle \in \mathcal{H} \times \mathcal{H}$ is in an entangled state if there are no $\varphi_1, \varphi_2 \in \mathcal{H}$ such that $|\varphi\rangle = |\varphi_1\rangle \otimes |\varphi_2\rangle$.

The existence of entangled states on a quantum computer is a fundamental difference to classical computers and a main reason why quantum computing can yield an advantage. Such an advantage have be proven for various quantum algorithms theoretically.

For example, the Grover algorithm [8] for finding elements in an unsorted database is quadratically faster than any classical algorithm and Shors algorithm [19] for prime factorization leads even to an exponential speed up. □

4 The GA4QCO-Framework

Our GA based framework optimizes quantum circuits while enforcing specific properties with respect to given constraints, e.g. searches for circuits with minimal depth while still achieving high fidelity. The framework is designed as a toolbox, allowing different components to be combined. Furthermore, custom, i.e., user-defined fitness functions are easily integrated, allowing users to apply the framework to a wide range of problems and domains.

The framework simulates the evolution process similar to the illustration in Figure 1. The user can control the main properties of the evolution via a property file, e.g. number of individuals in the population, number of generations, probability of the mu-

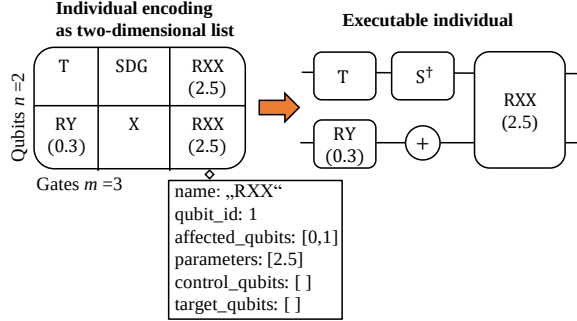


Figure 2: Individual representation: Quantum circuit on the left side with $n = 2$ qubits and $m = 3$ gates is stored as 2×3 python list. Gates have predefined attributes, which gather metadata needed to generate the executable circuit on the right side.

tation and crossover operations, the depth and width of the circuits, type of the fitness function and others.

4.1 Components

Individual Representation: A population P consist of N individuals, which represent quantum circuits. Each individual I_k , $0 < k < N$, with n qubits and m gates per qubit is encoded as a two-dimensional list with n rows and m columns. Note that n and m correspond the circuit *width* and *depth* respectively.

Each gate g_j , $0 < j < m$, acting on the qubit q_i , $0 < i < n$, has following characteristics:

- *name* chosen from a predefined gate set,
- *qubit id* to identify on which qubit the gate is applied,
- *affected qubits* to represent multiple-qubit gates,
- *rotation angles* θ for parametrized gates, e.g. $RX(\theta)$; if no constant value is set for the parameters, they are initialized randomly in the range $[-\pi; \pi]$,
- *control and target qubits* for controlled gates.

Figure 2 depicts the described individual representation.

□

Fitness Functions: The fitness function defines the objective of the optimization and assigns a scalar value to each solution / individual reflecting its quality with respect to the optimization goal. A fitness function can take multiple properties of the individual into account, for example the number of gates, the quantum state it produces or its behavior in a variational training setting. It is up to the researchers to define how the different properties shall be combined and weighted to calculate the fitness. We’ve implemented several example fitness functions using our proposed framework, and we describe a selection of these next.

Fidelity fitness function: The aim of the fidelity fitness function is to find a quantum circuit that maximizes the fidelity to a given target state. Additional constraints can be added to the fitness function in order to find circuits matching a specific criteria, e.g. depth can be included to prefer shallow circuits, the set of quantum gates used for individual generation can be restricted, or the noise model of a specific quantum hardware can be considered.

Entanglement fitness function: Using this fitness function, the algorithm seeks a state with maximum entanglement, i.e., a state equivalent to the Bell state for the two qubit case. Note that here the entanglement is measured with the von Neumann entropy.

Machine learning fitness function (Classification/Regression): In this use case, training accuracy on a given dataset serves as fitness. Here, we distinguish between evolution process and learning from data points, which takes place within the evolution process. As an individual learns from the dataset, parameters in parameterized gates are changed with the aim to increase training accuracy. After a certain number of learning steps, training accuracy is determined and stored as fitness value. Thus, using this fitness function, the user can discover a beneficial circuit ansatz for the given classification/regression problem.

Selection Methods: The framework supports three selection methods, each one can be used for the parent and survivor selection.

Random selection: Individuals are selected ran-

domly from the population. Here, the fitness value is not taken into account. This selection should be used when a high genetic diversity is required.

Tournament selection: Selects n individuals from the population. Therefore, the algorithm creates n tournaments with a given number of individuals. In each tournament the individuals are compared with respect to their fitness. The winner of each tournament is then returned.

Roulette wheel selection: Returns a population according to the roulette wheel selection approach used in genetic algorithms. The selection of an individual is a random decision, where fitness of an individual increases the probability to get selected.

Crossover Methods: We provide three crossover methods (see visualization on the Figure 3), differing in complexity and resulting diversity of the genetic information.

All following crossover methods assume that the parent individuals can differ in their width and depth. Hence, the parent circuits will be padded with identity gates to meet the same size before applying the crossover logic. It is configurable whether the crossover returns all produced children or only one of them.

Single-point crossover: Is the simplest crossover method. It chooses at the column level a random point to split each parent at and recombines the four resulting parts into two children.

Multi-point crossover: Differs from the first method only in the number of splitting points (more than one). Due to more randomly selected splitting points, the recombination results in more diverse mix of the parental genetic information increasing the diversity and explorative capabilities of the whole evolution process. On the other hand, this method might destroy qubit sequences, which already perform well with respect to the optimization goal.

Blockwise crossover: Splits the parent circuits not only column-wise like the two methods before, but it chooses two-dimensional splitting points with row-wise and a column-wise dimension. While the first two crossover methods preserve all genetic information of the parents distributed across the two chil-

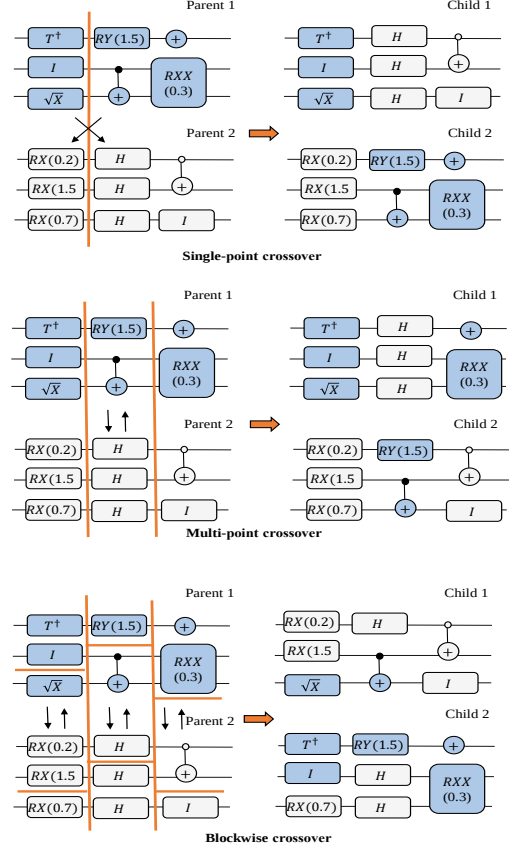


Figure 3: Crossover methods currently available in the GA4QCO framework.

dren, blockwise crossover will certainly destroy some multi-qubit gates due to its two-dimensional splitting. The algorithm will reconstruct broken multi-qubit gates occurring in child-solutions. That way, one parental multi-qubit gate distributed (and such broken) on the two children will result in two valid multi-qubit gates in the children after reconstruction, i.e., the all-over genetic information of the parents was changed in summa. However, this behavior is not considered negative since the evolution process works with mutating genetic information.

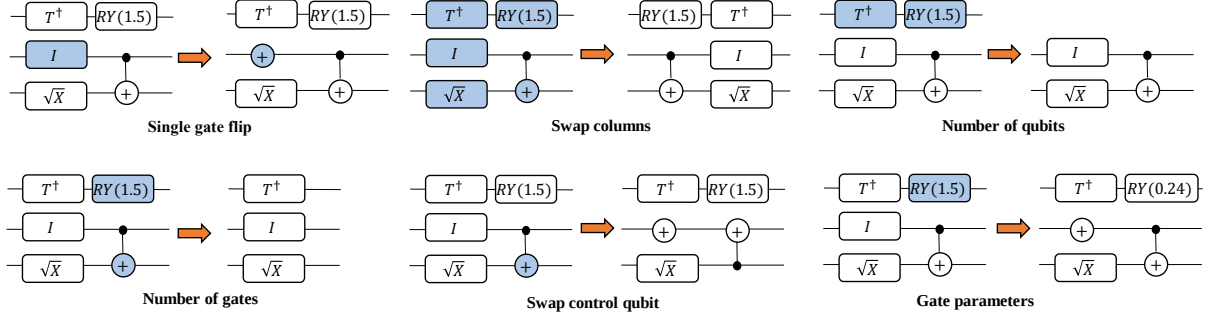


Figure 4: Mutation methods currently available in the GA4QCO framework.

Mutation Methods: New individuals produced by a crossover operation are mutated with a specific probability. This maintains genetic diversity and can result in children that are better than either parent wrt. the fitness function. Following mutations are provided in our framework (see Figure 4):

Single gate flip: Randomly selects a gate and replaces it with a random gate. If the selected gate is a multi-qubit gate, all affected gates are replaced with a random gate as well.

Swap control qubit: Randomly searches for a controlled-gate and swaps control and target qubit.

Mutate qubits: Adjusts the number of qubits randomly by adding or removing a qubit.

Mutate gates: Adjusts the number of gates randomly by adding random gates or removing gates on each qubit.

Swap columns: Exchanges all gates from two randomly chosen columns of the circuit.

Mutate gate parameters: Randomly selects a parameterized gate and adjusts its parameter (if such a gate is found).

4.2 Availability

We will be making our GA4QCO framework available to the research community on GitHub¹ soon, including documentation and a number of tutorials. All experiments included in this paper are also available

on GitHub. As described above, a core feature of the framework is its easy integration of custom fitness functions, i.e., the functionality that allows users to define their own fitness functions that will guide the evolution process to search for specific circuits. This allows the framework to be applied to a wide range of domains. We plan to adapt and extend the framework to integrate new features and fitness functions.

5 Experiments

In this section, we show results of preliminary experiments from our framework to illustrate the validity of our approach. We performed experiments using the fidelity fitness function in 2 different ways: Maximizing fidelity without any additional constraints and maximizing fidelity using a restricted gate set. All experiments were implemented in Python 3 using Qiskit² and are available on the public GitHub repository, including the parameters used. For each experiment, we show the best fitness per generation, average fitness per generation as well as a random baseline. Note that the results are averaged over all experiments. The random baseline shows the best randomly created individual per generation, in each generation an equal amount of solutions is created as with the GA.

¹<https://github.com/PlanQK/GA4QCO>

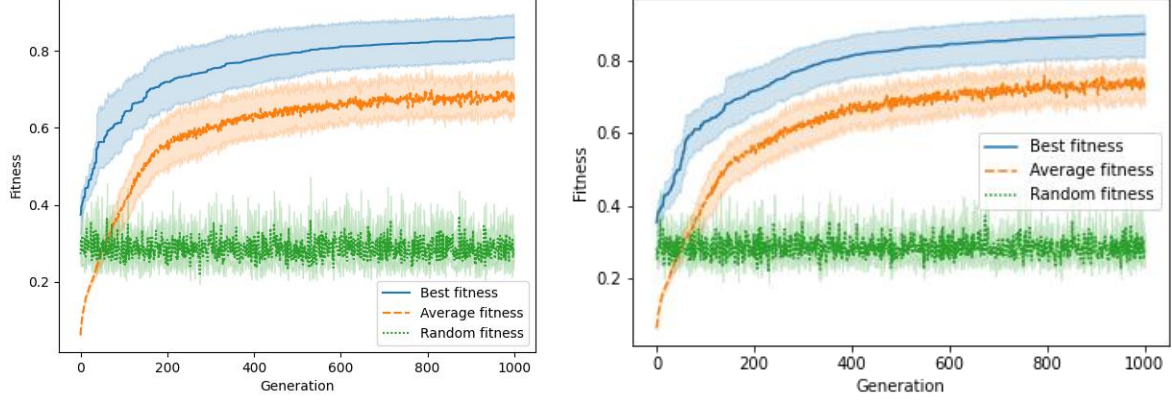


Figure 5: The best and average fitness (i.e. fidelity) compared to randomly created solutions aggregated over 10 experiments with different targets to show the mean and 95% confidence interval. The plot on the left depicts the results without restricting the set of gates, and the plot on the right shows the results from running the same experiment with only five gates.

Fidelity: Recall that the aim of the fidelity fitness function is to maximize the fidelity between a given target and the state prepared by a circuit created by the GA4QCO framework. For our experiments, we used Qiskit [1] to create random circuits whose statevectors serve as the target states. Each circuit has 4 qubits and a depth of 20. We created 10 such circuits randomly and ran the algorithm for each with the same parameters. The number of generations was set to 1000 and a population of 200 individuals was used. A full set of parameters used is available in the GitHub repository.

The results are shown in Figure 5. The fitness (i.e. fidelity) continuously improves, approaching a fidelity of 0.8. Running the algorithm longer with a larger population may even yield better results.

Fidelity with the restricted gate set: As in the previous case, the framework searches for a circuit that prepares a quantum state close to the specified target state. But this time, we restrict the gates that can be used in individuals. This experiment has a practical background, e.g. when the user knows the properties of the quantum computation device and aspire to use only the gates physically implemented on this device.

In our experiment, we restrict gates to the following set: $\{\text{'id'}, \text{'rz'}, \text{'sx'}, \text{'x'}, \text{'cx'}\}$ and run the evolution with the same parameters as in the first example.

The results shown in Figure 5 demonstrate that even with the restricted gate set our framework is able to find a much better solution than a random guessing. The highest fitness of the best individuals returned after 1000 generations in 10 runs we observed was 0.994 from maximum 1.0 (in contrast to random baseline with the highest fitness 0.79), the lowest one was 0.67, which, nevertheless, is still significantly better than the average fitness of the random baseline (0.29).

Figure 6 demonstrates the random circuit used to prepare target statevector and the best individual as well as target and output statevectors in comparison.

6 Outlook

The aim of the experiments from the previous section was, among other things, to show the validity of our approach, i.e., to demonstrate how GA can be applied to search for quantum circuits that satisfy some constraint or achieve some objective. These experiments could also be modified and extended. For



Figure 6: Results from restricted gate set experiment for a single seed. Top: A randomly created circuit (seed 10) vs. the circuit of the individual with the highest fitness 0.994 (fidelity). Bottom: A comparison of the target-statevector and the statevector prepared by the best individual.

instance, a further constraint for the fidelity fitness function could be the optimization under noise, i.e., inclusion of a noise model and noisy gates. Furthermore, the GAs used in our experiments ran for 1000 generations with a population of 200 individuals, increasing the number of generations or population size may improve the results even further. We limited the circuits to 4 qubits and a depth of 20. Investigating how the algorithm performs on larger circuits is a crucial question, especially in comparison to other approaches.

Moreover, as the framework is designed as a toolbox, we plan to extend and improve it and also integrate more advanced fitness functions.

7 Conclusion

The design of quantum circuits is still mostly performed manually, a task which is often tedious and not intuitive. However, different approaches to automate this have been explored by the research com-

munity over the years, most notably applying genetic algorithms or reinforcement learning. In this paper, we proposed our GA4QCO framework, which uses a genetic algorithm to search, i.e., construct, the architecture of quantum circuits with specific characteristics. Our framework allows the easy integration of custom fitness functions, a feature allowing its users to apply the framework to a wide range of domains.

We furthermore ran experiments on different problem settings to demonstrate the validity of our approach. We define target circuits and run the algorithm to search for circuits that maximize the fidelity between the constructed and target states. We also optimize the circuits using a restricted gate set. While our framework can be successfully applied to basic problems, we plan to further extend it and evaluate it on more advanced problems as there are still many open questions in the field of circuit construction.

A Comprehensive Review of Quantum Circuit Optimization: Current Trends and Future Directions

Krishnageetha Karuppasamy *, Varun Puram, Stevens Johnson and Johnson P. Thomas

Department of Computer Science, Oklahoma State University, Stillwater, OK 74078, USA;
vpuram@okstate.edu (V.P.); stevens.johnson@okstate.edu (S.J.); johnson.thomas@okstate.edu (J.P.T.)

* Correspondence: kkarupp@okstate.edu

Abstract: Optimizing quantum circuits is critical for enhancing computational speed and mitigating errors caused by quantum noise. Effective optimization must be achieved without compromising the correctness of the computations. This survey explores recent advancements in quantum circuit optimization, encompassing both hardware-independent and hardware-dependent techniques. It reviews state-of-the-art approaches, including analytical algorithms, heuristic strategies, machine learning-based methods, and hybrid quantum-classical frameworks. The paper highlights the strengths and limitations of each method, along with the challenges they pose. Furthermore, it identifies potential research opportunities in this evolving field, offering insights into the future directions of quantum circuit optimization.

Keywords: quantum computing; quantum circuit optimization; quantum machine learning; hybrid algorithm; quantum circuit layouts

1. Introduction

Quantum computing has the potential to change the landscape of computing. Tech behemoths like Google, Microsoft, and IBM have invested millions of dollars in quantum research. Quantum information science is now at the same stage of development as a classical computer was in the 1950s. Quantum computing adopts quantum mechanics concepts such as superposition, entanglement, and interference to improve the performance of solving hard problems that are difficult to solve by even the most powerful classical supercomputers.

The quantum factorization algorithm [1] developed by Peter Shor takes polynomial time compared to exponential time to execute in a classical computer. Searching an element in unstructured databases takes at least $O(n)$ in a classical algorithm, but the Grover's quantum search algorithm [2] takes only $O(\sqrt{N})$. In recent years, numerous applications have emerged leveraging the quantum framework, including quantum optimization for 6G wireless communications [3] and significant advancements in machine learning [4–7].

The emergence of advanced quantum devices, including Google's 73-qubit Sycamore [8] and 1121-qubit IBM Condor [9], along with the availability of quantum computing cloud services from providers such as IBM Q Experience [10], Microsoft Azure Quantum [11] and Origin Quantum demonstrate significant advancements in quantum hardware and platforms. This trend in improved hardware utilities is expected to continue.

At the heart of quantum computation lies the quantum circuit, a fundamental construct composed of quantum gates that manipulate qubits, the quantum analog of classical bits. However, harnessing the full potential inherent in quantum circuits presents a formidable challenge. Quantum circuits are highly susceptible to errors and inefficiencies due to quantum noise and the limited capabilities of existing quantum hardware. Present-day quantum processors, which serve as the workhorses for quantum circuit execution, have not yet reached the level of sophistication required for fault-tolerant quantum

computing or achieving quantum supremacy. These devices are inherently sensitive to their environment, exhibiting noise, and are susceptible to quantum decoherence. Furthermore, they lack the capacity for continuous quantum error correction. This class of quantum hardware is referred to as Noisy Intermediate Scale Quantum (NISQ) hardware [12]. The confluence of high error rates in the hardware, architectural constraints, limited qubit count, and gate errors due to decoherence collectively stand as a substantial impediment to realizing the full potential of quantum algorithms.

It is well known that quantum computers are highly susceptible to decoherence, which is qubit loss of quantum information over a period of time. Each gate operation introduces a certain amount of noise and can cause qubits to lose their quantum properties. In a large circuit with many gates, errors can propagate through the circuit, which means that if an error occurs early in the circuit, subsequent gates may operate on a corrupted quantum state, leading to a cascade of errors, which can be difficult to mitigate. Hence, the noise level is directly proportional to the quantum circuit size. By reducing the size of the quantum circuit both computational speedup and reduced gate count are achieved. Therefore, the effects of quantum decoherence although not eliminated, are mitigated to some extent. Therefore, optimization by minimizing the number of quantum gates is crucial to the overall reliability and efficiency of quantum computations. Since 2003, when Alfred and Krysta [13] initially proposed addressing the challenge of quantum computing by quantum circuit optimization, this field has evolved into a prominent and actively researched domain in recent years.

Over the past two decades, quantum circuit optimization has undergone significant development. Researchers have devised innovative techniques and algorithms to minimize the number of quantum gates, reduce the quantum circuit's depth, and improve the overall performance of quantum algorithms. These advancements are pivotal in addressing some of the key challenges in the field of quantum computing, such as mitigating quantum hardware constraints and enhancing the practicality of quantum algorithms for real-world problems. This paper broadly discusses circuit optimization based on gate, depth, circuit, and gate fidelity. Gate level optimization refers to reducing the count of quantum gates in the circuit. Depth level optimization refers to increasing parallel computation in the circuit. Circuit level optimization refers to finding the equivalent optimized circuit/sub-circuit. Gate fidelity is the measure of how close the output of a gate is to the expected value of that gate. Optimization techniques to address these challenges may be classified as follows:

1. **Heuristic:** In quantum circuit optimization, this is a method or technique designed to find effective solutions or approximations to a problem using practical and intuitive approaches. The goal is to find the best solution rather than exhaustively searching for all possible ones. Iteratively applying heuristics or rules that improve a circuit's performance can be used to find efficient quantum circuits when used in quantum circuit optimization. In these heuristics, gates are swapped to reduce the circuit depth, adjacent gates are merged to reduce gates, and the gate number is reduced by decomposing multi-qubit gates. Quantum circuit optimization is an NP-hard problem, which makes heuristic algorithms much more practical when applied to large-scale circuits than exact algorithms that promise optimal solutions.
2. **Machine Learning:** Machine learning has shown promising results in several quantum applications, including quantum error correction, quantum control systems, and quantum chemistry simulations. Machine learning model is trained on a set of input-output pairs of quantum circuits, and then the model is used to predict the optimal circuit for a given task.

3. **Unitary Synthesis:** This method focuses on efficiently constructing unitary matrices that represent quantum operations. The factorization process of unitary matrices leads to enhanced performance and a reduction in gate counts, thereby contributing to improved computational efficiency.
4. **Algorithmic approaches:** Where quantum circuit optimization involves systematic methods and procedures designed to enhance the efficiency and performance of quantum circuits. These approaches aim to address specific challenges in quantum computation, such as minimizing gate counts, optimizing resource utilization, and improving overall circuit fidelity.

This paper focuses on the domain of quantum circuit optimization with a primary focus on the methodologies and techniques currently used to enhance the efficiency or speed of quantum computations. This survey will examine various aspects of quantum circuit optimization, including gate-level optimizations, circuit depth reduction, resource-efficient mapping for error mitigation, and their implications for quantum algorithm performance.

The remainder of the paper is structured as follows: Section 2 provides an overview of quantum computing fundamentals, and Section 3 outlines quantum algorithm flows. Section 4 focuses on low-level quantum circuit gate-level optimization classified as level I optimization, and reviews techniques to enhance the efficiency of quantum circuits. Section 5 looks at scale level I optimization of large circuits. Section 6 elaborates on quantum circuit layout optimization, which is classified as level II optimization, emphasizing strategies to improve the spatial arrangement of quantum components. The paper concludes by summarizing key findings and insights drawn from the preceding sections.

2. Quantum Computing Basics

The following provides an overview of fundamental concepts in quantum computation that are pertinent to this study. In-depth exploration is given in [14].

In classical computing, the basic unit of information is a bit, which exists in one of two states, 0 or 1. Quantum computing, however, uses quantum bits, or qubits, which can simultaneously represent 0 and 1 due to a phenomenon called superposition. A qubit's state, represented as $|\varphi\rangle$, is a vector in Hilbert space. One qubit is spanned by two possible classical states represented by $|0\rangle$ and $|1\rangle$, where a qubit is a linear combination of $|0\rangle$ and $|1\rangle$, with complex coefficients a and b . $|\varphi\rangle = a|0\rangle + b|1\rangle$ where $a, b \in \mathbb{C}$ and $\mathbb{C}$ is the set of complex numbers. The measurement of a qubit collapses its state to either $|0\rangle$ or $|1\rangle$ with probabilities $|a|^2$ and $|b|^2$, respectively.

For a system of n qubits, there are 2^n possible basis states. These basis states can be represented with binary values from 0 to $2^n - 1$. In a classical computer, when you have an n -bit input register, it can only exist in one specific state at any given time. It can only be in one state at a time whereas a quantum system in superposition can exist in all possible states simultaneously. This property enables quantum parallelism, allowing a quantum computer to perform a single computation on all possible inputs at once, vastly increasing computational efficiency for specific tasks.

In quantum computing, entanglement is a property that allows qubits to become interconnected, meaning the state of one qubit is dependent on the state of another, regardless of the distance between them. For a two-qubit system (qubits A and B), the Hilbert space is the tensor product of the individual qubit states, with basis states $|0\rangle_A |0\rangle_B$, $|0\rangle_A |1\rangle_B$, $|1\rangle_A |0\rangle_B$, and $|1\rangle_A |1\rangle_B$, typically written as $|00\rangle$, $|01\rangle$, $|10\rangle$, and $|11\rangle$. Consider a superposition state of two basis states as shown below.

$|\psi\rangle_{AB} = \frac{1}{\sqrt{2}} (|0\rangle_A |0\rangle_B + |1\rangle_A |1\rangle_B)$ represents an entangled state. This specific entangled state is known as a Bell state or EPR (Einstein–Podolsky–Rosen) pair, where the measurement outcome of one qubit A determines the outcome of qubit B, demonstrating perfect correlation. The state $|\psi\rangle_{AB}$ cannot be factorized into the form $|x\rangle_A |y\rangle_B$ for any choice of $|x\rangle_A$ and $|y\rangle_B$. Since $|\psi\rangle_{AB}$ is an entangled state, meaning qubits A and B are not independent; their quantum states are correlated. Thus, $|\psi\rangle_{AB}$ cannot be separated into independent qubit states.

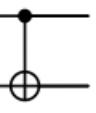
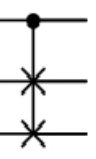
In general, consider a linear combination of n qubit quantum states such as $|\chi\rangle = \sum_{i=0}^{N-1} a_i |i\rangle$; $N = 2^n$. Accessing information about $|\chi\rangle$ is limited, especially since we cannot directly observe the complex amplitudes a_i . When measuring this n qubit system in the state $|\chi\rangle$, there are 2^n possible outcomes, represented by binary integers from 0 to $2^n - 1$. A measurement in the standard basis yields the outcome of $|i\rangle^{\text{th}}$ state with a probability equal to $|a_i|^2$.

Quantum Computing Gates

Similar to how a classical computer is constructed using an electrical circuit with wires and computations are performed using logic gates, a quantum computer is constructed using a quantum circuit composed of qubit wires and elementary quantum gates. These quantum gates enable the transportation and manipulation of quantum information within the system. A few elementary gates and their unitary representations are shown in Table 1. Quantum computation is reversible, and each quantum gate can be mathematically represented by a unitary matrix, which describes the transformation the gate applies to quantum states. From another perspective, a quantum circuit can be viewed as a composite unitary matrix. This composite matrix can be decomposed into a series of smaller unitary matrices, representing the individual gates applied in the circuit. The computations involved in obtaining a composite unitary matrix for the circuit include matrix multiplications and tensor products (also known as Kronecker products) of smaller unitary matrices.

Table 1. Quantum elementary gates.

Gate Name	# Qubits	Matrix Representation	Mathematical Properties	Quantum Circuit Representation
NOT (X)	1	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$	$X 0\rangle = 1\rangle$ $X 1\rangle = 0\rangle$	$ q_0\rangle \text{ --- } \boxed{X} \text{ ---}$
Hadamard	1	$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$	$H 0\rangle = \frac{1}{\sqrt{2}}(0\rangle + 1\rangle)$ $H 1\rangle = \frac{1}{\sqrt{2}}(0\rangle - 1\rangle)$	$ q_0\rangle \text{ --- } \boxed{H} \text{ ---}$
Z-gate	1	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$	$Z 0\rangle = 0\rangle$ $Z 1\rangle = - 1\rangle$	$ q_0\rangle \text{ --- } \boxed{Z} \text{ ---}$
S-gate	1	$\begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$	$S 0\rangle = 0\rangle$ $S 1\rangle = i 1\rangle$	$ q_0\rangle \text{ --- } \boxed{S} \text{ ---}$

T-gate	1	$\begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$	$T 0\rangle = 0\rangle$ $T 1\rangle = e^{i\pi/4} 1\rangle$	$ q_0\rangle$ 
CNOT(CX)	2	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$	$CX 00\rangle = 00\rangle$ $CX 01\rangle = 01\rangle$ $CX 10\rangle = 11\rangle$ $CX 11\rangle = 10\rangle$	$ q_0\rangle$  $ q_1\rangle$
Controlled Z (CZ)	2	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}$	$CZ 00\rangle = 00\rangle$ $CZ 01\rangle = 01\rangle$ $CZ 10\rangle = 10\rangle$ $CZ 11\rangle = - 11\rangle$	$ q_0\rangle$  $ q_1\rangle$
SWAP	2	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	$Swap 00\rangle = 00\rangle$ $Swap 01\rangle = 10\rangle$ $Swap 10\rangle = 01\rangle$ $Swap 11\rangle = 11\rangle$	$ q_0\rangle$  $ q_1\rangle$
C-SWAP	3	$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$	$CSwap 000\rangle = 000\rangle$ $CSwap 001\rangle = 001\rangle$ $CSwap 010\rangle = 010\rangle$ $CSwap 011\rangle = 011\rangle$ $CSwap 100\rangle = 100\rangle$ $CSwap 101\rangle = 110\rangle$ $CSwap 110\rangle = 101\rangle$ $CSwap 111\rangle = 111\rangle$	$ q_0\rangle$  $ q_1\rangle$ $ q_2\rangle$
CCX	3	$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$	$CCX 000\rangle = 000\rangle$ $CCX 001\rangle = 001\rangle$ $CCX 010\rangle = 010\rangle$ $CCX 011\rangle = 011\rangle$ $CCX 100\rangle = 100\rangle$ $CCX 101\rangle = 101\rangle$ $CCX 110\rangle = 111\rangle$ $CCX 111\rangle = 110\rangle$	$ q_0\rangle$  $ q_1\rangle$ $ q_2\rangle$

3. Algorithm Flow in Quantum Computing

Quantum computing can solve problems that were previously considered unsolvable or computationally infeasible for classical computers. Quantum computing takes advantage of the unique principles of quantum mechanics, such as superposition, interference, and entanglement, to achieve a level of parallelism and computational power that surpasses classical computing capabilities [15]. When designing a quantum computing algorithm, the first crucial step is defining a well-structured problem statement. Once the problem is defined, the next step is designing a quantum algorithm that can potentially solve that problem. A quantum algorithm is a set of instructions, or a computational procedure designed to be executed on a quantum computer to solve specific problems more efficiently than classical algorithms. It encapsulates input codes, quantum circuits, and quantum oracles. The quantum oracle is a black box function that is used as input to another algorithm. For instance, in a Grover search, the oracle identifies which values match the search and which values do not match the search. After designing a quantum algorithm for the given problem, a quantum circuit is built. A quantum circuit is a gate

representation of a quantum algorithm. It consists of sequences of quantum gates and input qubits. Each gate performs a specific operation on qubits. The circuit represents the sequence of operations required to execute the quantum algorithm and the qubits are initialized and manipulated according to the algorithm's design. Figure 1 illustrates the process of executing a quantum algorithm.

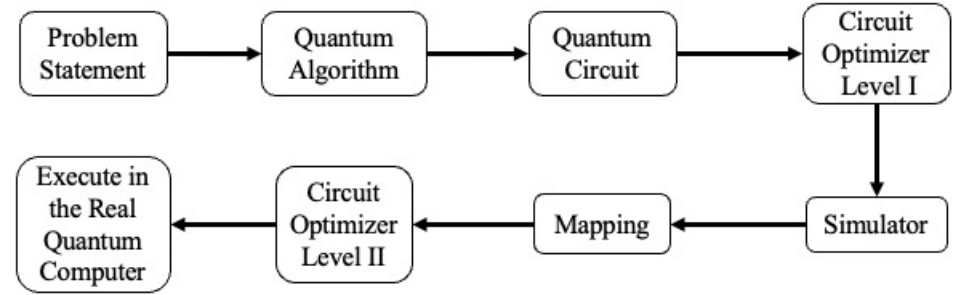


Figure 1. Execution pipeline of quantum algorithms.

Depending on the problems and algorithms proposed, quantum circuits can be complex, and circuit optimization helps to leverage computational speed and reliability. Quantum circuit optimization encompasses a suite of strategies, including gate cancellation, gate fusion, and gate reordering, aimed at minimizing the overall circuit depth. The primary objective of these techniques is to decrease both the gate count and depth of quantum circuits, consequently enhancing their efficiency.

Before running the quantum algorithm on a real quantum computer, it is usual to simulate its behavior using a quantum simulator, such as Qiskit [16], Cirq [17], Braket [18], Qutip [19], etc. Quantum simulators are software tools that mimic the behavior of quantum computers on classical hardware. They help to validate the correctness of the quantum algorithm and understand its behaviors without the constraints and noise associated with actual quantum hardware. **Optimization level I** refers to when these optimization procedures are applied prior to simulating a quantum circuit. We will discuss this concept later.

Currently, quantum algorithms are implemented on NISQ architectures. Depending on the hardware platform, there would be a number of constraints, such as geometric limitations (e.g., lack of full connectivity among qubits), quantum gate constraints (e.g., insufficient number of qubits), decoherence limitations (e.g., where qubit loses their information due to external noise or interactions), error rates varying in time and number of qubits. To overcome these limitations and after simulations, the quantum circuit is mapped to the specific hardware environment. This process is referred to as quantum circuit compilation (QCC) or quantum mapping [20]. During the mapping process, the qubit order in the circuit is rearranged according to the underlying hardware to avoid long-time connection. Qubits interact with each other via quantum gates, such as CNOT gates. These interactions are more error-prone if qubits are physically far apart (or have long-time connection) because the error rates increase with the distance over which quantum information must travel.

In physical qubit mapping, qubits that frequently interact are optimally physically placed close to each other, such that decoherence and gate errors are reduced. This is because shorter distances between interacting qubits reduce the time and the noise during operations. To accomplish this task, additional gates may be included to preserve the computation. We will discuss these constraints in more detail in Section 6. Optimization that considers specific hardware qubit mapping constraints and characteristics of the quantum

computer is referred to as **optimization level II**. Finally, the optimized quantum circuit or algorithm is run on a quantum computer.

3.1. Quantum Circuit Optimization

The complex nature of quantum circuits necessitates meticulous optimization to ensure their efficiency and adaptability for execution on quantum computing hardware. Quantum circuit optimization techniques play a central role in harvesting the full potential of quantum computing, facilitating the efficient solution of real-world problems. In addition to that, circuit optimization helps to mitigate the errors inherent in quantum hardware. In essence, quantum circuit optimization is pivotal in bridging the gap between theoretical quantum algorithms and practical quantum computing, empowering quantum computers to tackle real-world challenges with greater efficiency.

3.2. Quantum Circuit Optimization Metrics

Quantum circuit optimization aims to optimize quantum circuits, improve the execution time, require less resource usage, and affect a variety of other factors. In [21], several metrics are used to evaluate the quality and efficiency of optimized quantum circuits. Some of the key metrics used to evaluate quantum circuit optimization are:

Circuit Depth: The number of quantum gates executed sequentially in a quantum circuit. Reducing circuit depth is a primary goal of optimization because it can lead to faster execution on quantum hardware.

Gate Count: Total number of quantum gates used in a circuit. Minimizing the gate count leads to optimizing resource usage and minimizing the potential for gate errors.

Qubit Count: Number of qubits required for a quantum circuit. Reducing the number of qubits is important for optimizing resource usage and for mapping circuits onto quantum hardware with limited qubit availability.

Two-Qubit Gate Count: Two-qubit gates are often more resource-intensive and have a higher error probability compared with single-qubit gates. Reducing the number of two-qubit gates in a circuit can significantly impact its efficiency.

Gate Fidelity: Gate fidelity measures how closely quantum gates on a quantum computer match the ideal gates. Higher gate fidelity indicates more reliable gate operations, which can impact the overall accuracy of a quantum computation.

Parallelism: Parallelism measures the degree to which quantum gates can be executed concurrently in a quantum circuit. Maximizing parallelism can lead to faster quantum computation.

This paper focuses on both optimization level I and optimization level II with two main objectives: 1. Enhancing computational speed through the minimization of resource utilization by specifically focusing on reducing the use of gates and qubits, while also addressing error reduction, called *circuit simplification*. 2. Elevating computational performance by rearranging the order of execution and adjusting qubit mapping while preserving the correctness of the circuit, called *circuit layout optimization*.

4. Optimization Approaches

Hardware-independent quantum circuit optimization refers to a set of techniques and methodologies aimed at enhancing the efficiency and performance of quantum circuits without relying on specific details of the underlying quantum hardware. This approach focuses on developing algorithms and strategies that can be applied across different quantum computing architectures, irrespective of their physical implementations. This is classified as Level I Optimization. The emphasis is on overarching methodologies, such as pattern matching and unitary synthesis, which are supported by specific examples

of gate cancellations, commutation, and depth reduction. These methodologies provide a unified framework for simplifying quantum circuits and enhancing their efficiency.

4.1. Core Methodologies for Circuit Simplification

Pattern Matching: This technique involves recognizing and replacing common sub-structures in quantum circuits with optimized equivalents. For instance, adjacent gates that cancel or combine into simpler operations are identified and replaced, reducing gate count and depth. Directed Acyclic Graphs (DAGs) are often used to model circuits, representing gates as vertices and dependencies as edges. Gate cancellations and fusions are systematically derived by analyzing the circuit structure.

Unitary Synthesis: This strategy focuses on decomposing complex unitary operations into smaller, optimized components. Then, the simplification techniques are applied using machine learning and an algorithmic approach to optimize the circuit.

4.2. Simplification Techniques

This section provides a detailed discussion of circuit simplification rules. These rules are essential for optimizing quantum circuits by reducing the gate count, circuit depth, and resource requirements while preserving the intended functionality of the quantum computation.

4.2.1. Commuting Quantum Gates

Two quantum gates G_1 , G_2 are commuting if, and only if, for any states $|\phi\rangle$, $U_1 U_2 |\phi\rangle = U_2 U_1 |\phi\rangle$ holds, where U_1 and U_2 are unitary matrices corresponding to gates G_1 and G_2 , respectively. $|\phi\rangle$ is also a column vector of the state. Irrespective of the execution orders of gates G_1 and G_2 , the outcome will be the same. The underlying idea is that rearranging the order of execution while preserving the correctness can increase the computation speed through parallelism as well as find the possibility of any gate cancellations. More than two quantum gates can be commuted; for instance, three gates A, B, and C commute if they satisfy $ABC = ACB = BCA = BAC = CAB = CBA$. Figure 2 demonstrates basic examples of commuting gates, with additional examples in [21].

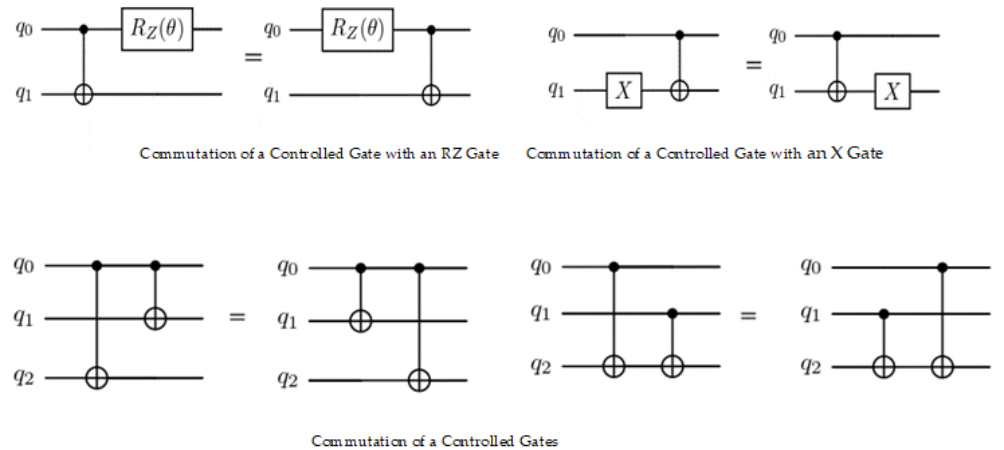


Figure 2. Illustration of gate commutations in quantum circuits.

4.2.2. Optimization by Gate Cancellation Rule

Any symmetric matrix U with real entries, is termed a real unitary matrix. All the entries a_{ij} of the matrix are real numbers ($\forall a_{ij} \in \mathbb{R}$). Then, $U = (U^*)^T$, where T is the transpose of the matrix and $*$ represents the conjugate inverse. If U is a real unitary matrix,

then it satisfies $\mathbf{U} \cdot \mathbf{U} = \mathbf{I}$, where $\mathbf{I}$ is the identity matrix. When a unitary matrix multiplies by itself, the result will be unitary. Some examples of real unitary matrices are as follows:

$$\text{NOT gate} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

As $\mathbf{X} \cdot \mathbf{X} = \mathbf{I}$, the NOT gate is a real unitary matrix. That is,

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \mathbf{I}.$$

Other gates, such as the Z gate

$$\mathbf{Z} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix},$$

the Hadamard gate $\mathbf{H} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$, CNOT, SWAP, CCNOT, CSWAP, etc. all have real unitary matrices. The T gate $= \begin{bmatrix} 0 & 1 \\ 1 & e^{i\frac{\pi}{4}} \end{bmatrix}$, and S gate $= \begin{bmatrix} 0 & 1 \\ 1 & i \end{bmatrix}$ have complex numbers in matrix entries ($\forall \alpha_{ij} \notin \mathbb{R}$). Hence, these are not real unitary matrices and $\mathbf{U} \neq (\mathbf{U}^*)^T$ for these gates. In any quantum circuit, when two identical gates are placed next to each other and they have real unitary matrices, the elimination of those gates does not affect the outcome. Refs. [21–25] explore possible gate cancellations for different circuits.

In quantum circuits, adjacent identical gates with real unitary matrices can cancel without altering the circuit's outcome. Refs. [21–23] have examined gate cancellation scenarios for various circuits by using Directed Acyclic Graphs (DAGs) to model quantum circuits, identifying gate dependencies that facilitate cancellations. Similarly, refs. [24,25] use a DAG's canonical form, representing each gate as a vertex and dependencies as edges, allowing analysis of computation flow in the circuit. The DAG framework, applied in quantum circuit optimization, helps visualize gate dependencies, leading to gate cancellation for adjacent, identical gates. Figure 3 shows a quantum circuit C and its corresponding DAG representation.

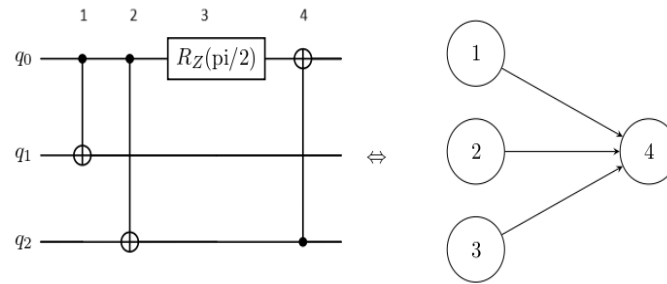


Figure 3. Sample circuit C and its associated DAG representation.

In DAG representation, if two adjacent nodes represent the same real unitary gate, they cancel out, yielding the identity matrix. For commuting gates, swapping nodes can lead to cancellations. In certain cases [22], introducing a modification gate enables gate exchange or merging, which can lead to further reductions.

4.2.3. Optimization by Hadamard Gate Reduction

In certain quantum algorithms, the sequence of gates can be optimized by recognizing that you only need specific combinations of Clifford gates to achieve your desired transformation. By simplifying the gate sequence, you can reduce the number of Hadamard (H) gates required. Figure 4 illustrates a few examples where the Hadamard gate count is reduced. Refs. [21–25] discuss more Hadamard reduction scenarios.

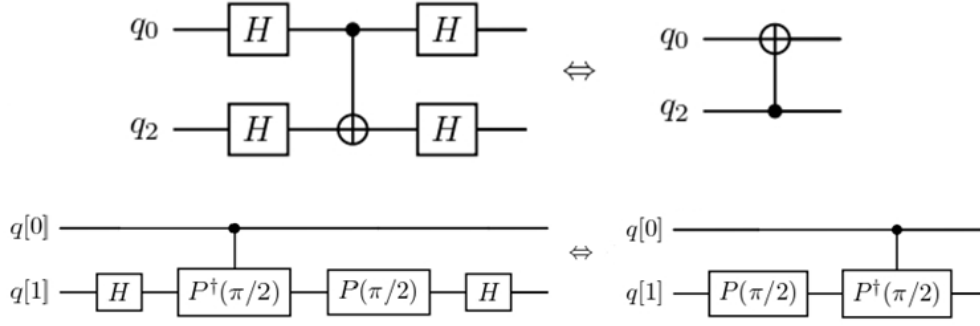


Figure 4. Illustrations of Hadamard gate reduction.

In [25], the authors propose a comprehensive 3-level strategy aimed at optimizing circuits through reversible mapping and quantum levels to reduce the H-count. At the reversible level, the logic gates and their configurations are optimized to ensure minimal quantum gate usage and prepare for effective quantum-level mapping. The mapping level focuses on the efficient allocation of quantum gates to physical qubits in the quantum processor. This step is crucial to minimize latency and the overall gate count, further enhancing the circuit's performance. At the quantum level, the operational parameters of the quantum gates are fine-tuned, and their coherence and control are optimized to achieve the best possible H-depth outcomes. H-depth refers to the depth of a quantum circuit when only considering the Hadamard (H) gates.

4.2.4. Merging Rz Gates Using Phase Polynomial Estimation

This strategy focuses on decomposing complex unitary operations into smaller, optimized components. Specifically, phase polynomial estimation is employed to merge rotation gates (Rz gates) when the circuit comprises only CNOT, NOT, and Rz gates. This approach leverages phase dependencies, effectively reducing both the circuit depth and gate count, as proposed in [26].

Imagine adjusting multiple radio dials to tune into specific frequencies. If two dials are set to the same frequency, you do not need both—you can combine them into one dial with the sum of their adjustments. Similarly, in a quantum circuit, when two rotation gates share the same pattern, they can be merged into a single gate with a combined rotation angle. In other words,

- Every RZ gate contributes a rotation angle to the quantum state.
 - Each gate's contribution depends on how the input qubits interact with the rotation operation.
 - These interactions follow specific patterns or dependencies based on the input state.
- If two rotation gates produce the same phase pattern, they can be merged into one gate with a single, combined rotation angle. This merging reduces the number of gates without changing the circuit's final behavior.

4.3. Depth Reduction

The depth of a quantum circuit is defined as the number of sequential layers or time steps required to execute a given quantum computation. It is a crucial parameter in measuring the overall performance of a quantum algorithm. So far, we have been discussing how to reduce computations by gate cancellations and gate fusions. Rearranging the order of gates, while preserving the correctness of the algorithm at the same time, can also lead to minimizing the number of computations.

In [27], Zhu et al., proposed depth optimization for linear reversible circuits. Circuits which implement the linear building blocks are called linear reversible circuits, which only consist of CNOT gates. e.g., stabilizer circuits. Any CNOT gate controlled by the j th qubit, acting on the i th qubit ($i \neq j$), can be written as $E_{ij} = I + e_{ij}$. Where I is the identity matrix and e_{ij} is a matrix with all entries equal to 0, except for entry (i, j) , which is one. It is well known that any invertible matrix can be decomposed into a combination of elementary matrices. $U = P \prod_{i=k}^1 E(c_i, t_i)$, where P is a permutation matrix.

Consider the decomposition sequence SEQ of a unitary operator U as a finite sequence of elementary matrices with a particular order. $SEQ = \{E(c_1, t_2), E(c_2, t_4), E(c_3, t_1), \dots, E(c_k, t_k)\}$, where $E(c_i, t_i)$ is the elementary matrix representation of a CNOT with control qubit c_i , target as qubit t_i . The elementary matrix/operator (CNOT) can be commutable if $\cap_i = \{c_i, t_j\} = \emptyset$ for any i and j in SEQ, which means for any two adjacent gates $E(c_k, t_k)$ and $E(c_{k+1}, t_{k+1})$ in SEQ, the order of the two gates can be swapped if, and only if, $c_k \neq t_{k+1}$ and $c_{k+1} \neq t_k$.

For example, in Figure 5, the given quantum circuit (Top left) has a depth of 7. $SEQ = \{E(0, 1), E(1, 3), E(2, 3), E(2, 0), E(0, 3), E(0, 2), E(2, 1)\}$. According to the rule described above, the adjacent gates $E(1, 3)$ and $E(2, 3)$ can be swapped since their control and target qubits do not overlap in a way that creates a dependency, meaning the control qubit of one gate is not the target qubit of the other ($c_k \neq t_{k+1}$ and $c_{k+1} \neq t_k$). Thus, the depth becomes 4 as shown in Figure 5 (bottom).

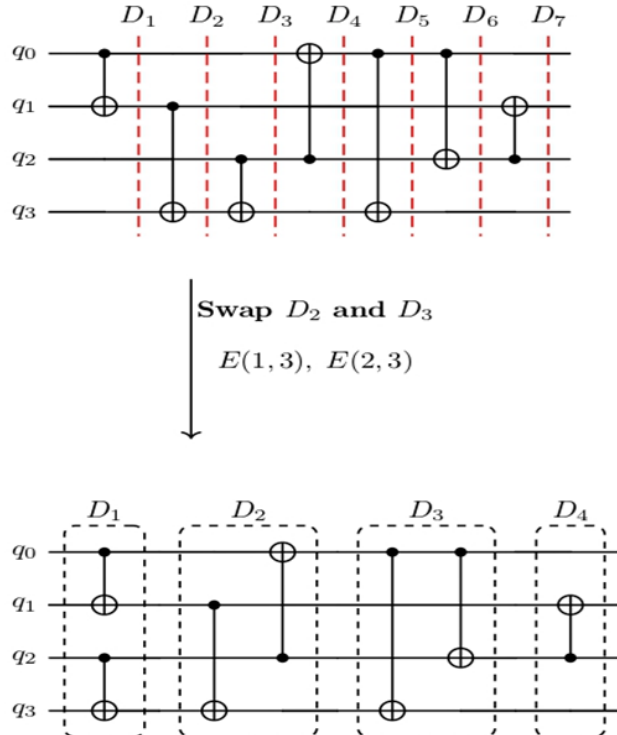


Figure 5. Illustration of depth reduction by linear arrangements.

In [28], proposed that depth reduction is achieved primarily through the parallel execution of gates by leveraging commutation relationships within the cost operator, which often consists of independent terms acting on subsets of qubits. This allows gates to be executed simultaneously, reducing sequential layers. However, the circuit depth grows linearly with the number of QAOA layers (p), which limits scalability for near-term devices. The method's effectiveness depends heavily on the problem structure; for dense graphs or highly connected cost operators, fewer gates can commute. In [29], Babbush et al. found sparsity in the Hamiltonian allows gates corresponding to non-interacting terms to be executed in parallel. In fermionic systems (fermionic systems refer to physical systems that are governed by fermions, which are particles that obey the Pauli exclusion principle. Examples of fermions include electrons, protons, and neutrons), the Hamiltonian often contains local terms that act on disjoint qubit pairs, and because these terms commute (i.e., they do not interfere with each other), the corresponding operations can be applied simultaneously. This parallel execution reduces the number of sequential steps in the quantum circuit. Additionally, techniques like Givens rotations and fermionic swap networks are employed to reorder gates efficiently and to further decrease them. But in systems with dense interactions, where many terms overlap, fewer operations can be parallelized, limiting the benefits. In [30], Quantum Fourier Transform (QFT) circuits, controlled-phase gates, are applied sequentially, leading to the risk of depth increment. By leveraging ancilla qubits, these gates can be precomputed and stored, allowing phase shifts to be applied simultaneously across multiple qubits. This eliminates sequential dependencies and reduces the depth of controlled-phase operations. Furthermore, the recursive structure of QFT benefits from ancilla qubits, as the circuit can be broken into smaller, independent subproblems that execute in parallel. The overall circuit depth is thereby reduced from $O(n^2)$ to $O(n \log n)$. In [31], Draper et al. use a similar technique to store carry in quantum arithmetic operations to reduce the depth. Ref. [9] utilizes this technique to optimize modular arithmetic operations in Shor's algorithm, improving depth efficiency by leveraging ancilla qubits for intermediate computations and carry propagation. However, this improvement comes at the cost of increased qubit usage, as more resources are required to manage the ancilla qubits. More qubits increase susceptibility to noise and decoherence, particularly in NISQ devices.

In [32], Kun Zhang proposed methods to reduce the circuit depth of Grover's algorithm, addressing its limitations for near-term quantum devices. The authors introduce a critical ratio, the ratio between the depths of the oracle and the global diffusion operator, as a threshold parameter to determine when the global diffusion operator can be replaced with a local diffusion operator. The global diffusion operator performs a reflection of all quantum states around their mean amplitude, requiring operations across the entire Hilbert space, which leads to significant circuit depth. In contrast, the local diffusion operator performs a similar reflection but acts only on specific, smaller subsets of the state space, reducing computational dependencies and enabling parallel execution of gates. By identifying when the critical ratio is less than one, the authors demonstrate that replacing the global operator with local versions significantly reduces depth while maintaining Grover's quadratic query complexity.

The replacement of the global diffusion operator with the local one not only reduces depth but also allows for further optimization. It enables commutation between consecutive oracle operators, creating opportunities for gate cancellation and gate fusion (combining multiple gates into single operations). Furthermore, the authors introduce a multi-stage quantum search strategy, where the search process is recursively divided into

smaller, rescaled subsets of the database. This approach reduces the overall search space and enhances depth efficiency without increasing the number of oracle calls. The proposed optimizations maximize parallelism, making the algorithm more practical for near-term quantum hardware with limited coherence times. However, the local diffusion operator reduces depth; it introduces additional complexity in identifying and implementing the appropriate local operations, requiring extra pre-processing overhead.

In summary, depth reduction is essential for improving the performance of quantum circuits, particularly for near-term devices. Techniques such as gate rearrangement, ancilla-assisted optimizations, and specific algorithmic strategies (e.g., for QAOA or Hamiltonian simulation) play a vital role in achieving shallower, more efficient circuits. This example of optimizing CNOT sequences demonstrates how careful analysis, and reordering can significantly reduce circuit depth while preserving correctness

4.4. Challenges

While these optimization techniques yield significant benefits, they come with challenges, including the following: Techniques like ancilla-assisted optimizations require additional qubits, which may not be feasible for near-term devices. Advanced optimizations, such as unitary synthesis or pattern matching, introduce computational overhead for identifying patterns and dependencies; Noise Sensitivity. Reducing depth often increases qubit usage, leading to higher susceptibility to noise and decoherence on NISQ devices.

5. At-Scale Optimization

Implementation of optimization level I at a large scale, by using the heuristic simplification rules discussed in the previous section, such as gate fusion, gate cancellation, and rearranging the execution order for gate/depth reductions, can be extended to automate quantum circuit simplifications effectively. Three primary methodologies are widely adopted for large-scale optimization:

1. Artificial intelligence (AI)-based simplification
2. Synthesizing unitary matrices
3. Algorithmic approaches

These methodologies address different aspects of circuit simplification and are summarized below, with detailed explanations.

5.1. Optimization Using Artificial Intelligence-Based Approaches

Since the early 1990s, AI has become an essential tool in optimization, offering flexible and innovative ways to solve complex problems in a variety of fields [33]. The main advantage of automation in optimization is that it can handle large, complex quantum circuits that would be difficult to process using traditional optimization methods. AI algorithms can learn from data to make predictions and decisions about optimizing a given objective function [33]. These techniques can adapt to changes in the optimization problem, such as changes in the input data or constraints, without requiring manual updates to the optimization strategy. In this section, we discuss machine learning approaches currently being used in quantum circuit optimization.

5.1.1. Optimization Using Reinforcement Learning

Reinforcement learning (RL) [34–37] is a type of machine learning model that focuses on training agents to learn from experience and make optimal decisions in each environment. RL has been applied to a wide range of problems, including game playing, robotics, and natural language processing.

In RL, an “agent” interacts with the rest of the world or the “environment”. In several iterations, the agent receives information from the environment and, in response to this observation, chooses an action which alters the state of the environment. The agent adapts its strategy to maximize a success measure, the “reward”. Figure 6 illustrates a reinforcement learning process where an agent iteratively optimizes a quantum circuit. At time t_1 , the agent extracts a subcircuit from a larger circuit, referred to as the environment A_t . The agent then searches for the best match from the Q-table. Based on this match, the agent takes actions $AE(S_{t+1})$ to modify the circuit. It receives feedback in the form of a reward R_{t+1} , which is used to refine its actions over time, leading to continuous improvement.

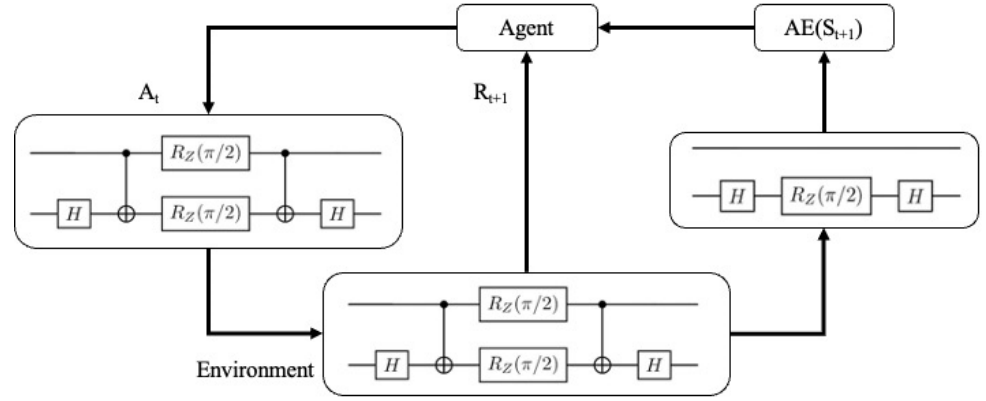


Figure 6. Flow diagram for quantum circuit optimization using RL.

In [38], Fasel et al. propose a novel reinforcement learning (RL) framework to automate this process, enabling the systematic reduction of gate counts and circuit depths. This method leverages the inherent flexibility of RL to identify and apply a sequence of circuit transformations, improving performance while preserving the intended functionality of the circuit.

At the heart of the framework are two types of transformation rules. **Soft rules** involve local modifications, such as gate fusion and rearrangement, which reorganize the sequence of gates for improved efficiency. On the other hand, **hard rules** focus on removing consecutive quantum gates that cancel each other, leading to a direct reduction in gate count. Each transformation is applied independently at different locations in the circuit, and the RL agent learns to prioritize these rules through a trial-and-error process, guided by a reward policy that incentivizes effective optimization.

A key innovation of this approach is its representation of quantum circuits in a 3D grid format, designed to capture the intricate structure of quantum operations. The three axes of the grid represent the quantum circuit index, which identifies the circuit being optimized; the moment (**timestamp**), which corresponds to the parallel execution of gates; and the gate class, which categorizes gates. Each timestamp is treated as a subcircuit, encapsulating gates that can be executed simultaneously. This structured representation allows the RL agent to systematically explore equivalent circuits within the action space, searching for transformations that minimize circuit complexity.

The RL framework relies on a robust mapping of states and actions. In traditional reinforcement learning, this mapping is stored in a Q-Table, which grows as the agent explores new configurations. The Q-Table initially starts small but expands rapidly during the exploratory phase, before stabilizing as the agent begins to exploit its accumulated knowledge. Each entry in the Q-Table represents a subcircuit and its equivalent optimized form, encoding the agent’s experience. However, in deep reinforcement learning, the Q-Table is replaced by a machine learning model that dynamically learns the mapping

between states and actions. This enhancement allows the model to scale more effectively, handling larger circuits and more complex optimization tasks.

The optimization process begins with the RL agent interacting with the circuit environment. At each step, the agent evaluates potential transformations, applying either soft or hard rules to improve the circuit. Based on a predefined reward policy, the agent selects the transformation that offers the greatest benefit, iteratively refining its strategy. Over time, the agent develops a comprehensive understanding of the optimization space, identifying the most efficient configurations for the circuit. The final optimized circuit is then decoded from the 3D representation, yielding a design that is both functionally equivalent and resource efficient.

In conclusion, the reinforcement learning framework introduced by Fosel et al. represents a significant advancement in quantum circuit optimization. By combining robust circuit representation, flexible transformation rules, and dynamic learning, this method provides a powerful tool for improving the efficiency of quantum circuits. As quantum computing continues to evolve, such automated optimization techniques will play an essential role in bridging the gap between theoretical designs and practical implementations. Although the RL-based approach to quantum circuit optimization is a promising step forward, its practical implementation is hindered by computational costs, scaling issues, and sensitivity to noise and hardware constraints.

In [39], the authors propose a reinforcement learning (RL) framework to optimize the design and architecture of variational quantum circuits (VQCs). The methodology leverages a policy-based RL agent to iteratively refine VQC structures by dynamically selecting gates, qubits, and circuit layouts to maximize performance. The agent explores the space of possible circuit architectures and is trained using reward functions based on the quality of the quantum output, such as energy estimation accuracy for tasks like solving the Variational Quantum Eigensolver (VQE) problem. The approach is distinct in its focus on optimizing the circuit architecture itself, rather than just parameter tuning, which allows for more efficient and tailored circuit designs for specific tasks. However, the exploration phase can be computationally expensive, as evaluating the performance of each candidate circuit requires multiple quantum simulations or hardware executions. Additionally, the reliance on predefined reward functions and training data limits the method's ability to generalize to entirely new quantum tasks or architectures.

Moro et al. [40] used a deep reinforcement learning (DRL) framework to optimize quantum circuit compilation. The methodology employs an RL agent to decompose target unitary operations into sequences of quantum gates tailored to specific hardware constraints. The agent interacts with a simulated environment representing the quantum circuit and learns to generate optimal gate sequences through trial and error. The framework incorporates a policy network that predicts gate sequences and a value network to estimate future rewards, trained using Proximal Policy Optimization (PPO). The reward function incentivizes the agent to minimize the cost of the compiled circuit, balancing gate count, depth, and fidelity. This approach demonstrates significant improvements in scalability and efficiency over traditional heuristic-based compilation methods, especially for complex quantum circuits. However, the framework has some limitations. Training the DRL model requires extensive computational resources and simulation time, making it challenging for larger systems.

Similarly, Salehi et al. [41] propose an optimization framework aimed at improving the efficiency of quantum circuits used in quantum machine learning (QML). The methodology involves a rule-based approach where gate elimination and circuit simplifications are applied iteratively to reduce the resource utilization (e.g., gate count and depth) while maintaining the performance of the quantum model. The framework leverages heuristic

rules to identify redundancies and symmetries within the circuit, enabling transformations that preserve the functionality of the QML model. The optimization process also incorporates a fidelity measure to ensure that the modified circuits closely approximate the original computational outcomes. This method is particularly advantageous for QML tasks, as it minimizes computational overhead and improves scalability for larger datasets. However, the approach has limitations. The reliance on heuristic rules means the optimization process may not be universally applicable to all QML architectures, particularly those with unconventional or highly entangled structures.

In [42], Riu et al. introduced a reinforcement learning (RL) framework for quantum circuit optimization that leverages the ZX-calculus, a graphical language for quantum mechanics. The methodology employs Graph Neural Networks (GNNs) to represent and process the intricate dependencies within quantum circuits, effectively capturing their graph-like structures. The RL agent, trained using the Proximal Policy Optimization (PPO) algorithm, applies ZX-calculus simplification rules to iteratively reduce circuit complexity. This approach enables the agent to identify and execute optimizations that minimize gate counts and circuit depth, enhancing overall efficiency. However, the computational intensity associated with training the GNN-based RL agent is significant, particularly for large-scale circuits. The complexity of the ZX-calculus rules and the vast action space can lead to extended training times and increased resource consumption. Additionally, the effectiveness of the optimization heavily depends on the quality and comprehensiveness of the training data, which may limit the agent's performance when encountering novel or highly complex circuit configurations.

5.1.2. Optimization Generative Adversarial Network

Ref. [43] proposed a Quantum Generative Adversarial Network (QuGAN) framework to generate efficient quantum circuit approximations with a focus on minimizing the gate count and depth while maintaining high fidelity. The methodology leverages quantum state fidelity as a training metric, ensuring that the generated circuits closely approximate the target quantum states. The QuGAN architecture consists of a quantum generator and a quantum discriminator, both of which are iteratively trained; the generator creates candidate circuits, and the discriminator evaluates their fidelity against the target states. By optimizing the fidelity measure, the framework identifies circuit configurations with reduced complexity. This approach demonstrates the potential for resource-efficient quantum circuit synthesis, particularly for applications like state preparation in quantum chemistry. However, a key drawback of QuGAN is its reliance on fidelity-based training, which becomes computationally expensive for large-scale systems due to the need for repeated state evaluations.

5.2. Optimization by Synthesizing Unitary Matrices

Another method for automating quantum circuit simplifications is by synthesizing unitary matrices. This approach involves using mathematical techniques to analyze and manipulate quantum circuits at a more fundamental level by decomposing quantum gates into their underlying unitary matrices [44–46]. Techniques like matrix factorization and decomposition algorithms help to find simpler representations of quantum gates or find equivalent sequences of gates that achieve the same quantum operation but with fewer resources.

Quanto, which is the first quantum circuit optimizer that automatically generates circuit identities (equivalent optimized circuit) is proposed in [47]. Quanto can optimize a wide range of quantum circuits, including circuits with non-Clifford gates, multi-qubit gates, and controlled gates. To demonstrate Quanto's effectiveness, the authors have used

it to optimize quantum circuits, such as quantum error correction circuits, surface code circuits, and quantum circuits for solving optimization problems. In all cases, Quanto significantly reduced the number of gates and improved the overall performance of the circuits. Pointing et al. [45] developed a novel approach to optimizing quantum circuits by exploiting circuit symmetries. For the development of practical quantum computers, the Quanto technique shows promising results and can enhance the performance and scalability of quantum circuits significantly. Quanto proposes compiling a quantum circuit to an optimized executable circuit for target hardware using synthesis. Quantum circuit synthesis is defined by decomposing a quantum circuit into smaller sequences of unitary matrices. This approach proposes a hierarchical step to optimize the given quantum circuit. It has been proven that more than 30% of CNOT gates are reduced by this method.

In [48], Mingkuan Xu et al. (2022) present an automated optimization framework for quantum circuits that combines equivalence checking, super optimization, and backtracking techniques. Quartz first generates Equivalence Circuit Classes (ECCs) by iteratively identifying alternative representations of subcircuits, allowing for transformations such as gate cancellations, merging Rz gates, and depth reduction. Using equivalence rules and pruning redundant operations, Quartz employs a backtracking search to systematically explore possible optimizations, ensuring global circuit minimization. The optimized circuits are then converted into symbolic representations for compatibility with hardware-specific transformations. This methodology achieves significant gate and depth reductions, particularly for circuits with non-Clifford gates or high connectivity constraints. However, a notable drawback is the computational overhead of the equivalence-checking and backtracking process, which can scale poorly for large circuits. Additionally, the reliance on pre-defined equivalence rules limits adaptability to novel circuit structures or hardware-specific optimizations. Despite these challenges, Quartz demonstrates substantial improvements in circuit efficiency, particularly for NISQ-era devices.

In [49], Wu et al. propose an optimization framework that leverages automated synthesis techniques to reduce the depth and gate count of quantum circuits while maintaining scalability for large systems. QGo employs a divide-and-conquer strategy, breaking down circuits into smaller subcircuits that can be optimized independently using rewrite rules and equivalence transformations. By automating the identification and application of these optimizations, QGo integrates hardware-specific constraints, ensuring compatibility with quantum processors. The framework iteratively refines circuits by exploring alternative configurations by balancing the circuit complexity with execution efficiency. QGo achieves scalability by partitioning large circuits and optimizing them in parallel, leveraging a dynamic programming approach to merge subcircuit optimizations. However, its reliance on heuristic-based synthesis can lead to suboptimal results for certain circuit configurations, as the framework may overlook deeper global optimizations. Additionally, the partitioning process can introduce overhead and dependency constraints, limiting the efficiency of parallel optimization in circuits with high connectivity.

Quantum circuit synthesis is defined by decomposing a quantum circuit into smaller sequences of unitary matrices. This approach proposes a hierarchical step to optimize the given quantum circuit. It has been proven that more than 30% of CNOT gates are reduced by this method. According to Shende, Markov, and Bullock [50], universal n -qubit circuits must contain at least $\frac{1}{4}(4^n - 3n - 1)$ CNOT gates. Michal Sedláč et al., ref. [51], propose an approach to find, for an arbitrary given unitary operator, a quantum circuit containing the lowest possible number of CNOT gates using the singular value decomposition (SVD) method.

In [52], Zhang et al. showed, by utilizing the properties of decomposition of diagonal invertible matrices which can be commutable, that any quantum diagonal unitary

operator can be decomposed as Rz and CNOT gates. Commuting the execution order of the gates helps to cancel the gates and possible parallelism on execution which results in depth optimization.

In [53], Markov and Shi (2008) outline procedures that leverage tensor network representations to enhance the efficiency of quantum circuit simulations, with implications for circuit optimization. The key strategy involves representing quantum operations and states as tensors, enabling sparse representations that reduce computational overhead by avoiding redundant operations. By optimizing the order of tensor contraction, the computational cost of simulating large circuits is minimized, which parallels gate reordering and depth reduction in circuit synthesis. The authors further exploit the structure of quantum circuits by decomposing large tensors into smaller, manageable substructures, facilitating parallel execution and resource efficiency. Graph-theoretic methods, such as minimizing the treewidth of the tensor network graph, ensure that intermediate tensor sizes are kept minimal, correlating to reduced ancillary resources and depth in circuits. Additionally, handling high-dimensional systems via factorization of unitary matrices allows the identification and removal of redundant components, directly translating to gate reduction. While the focus is on simulation, these techniques provide a robust framework for optimizing quantum circuits, to reduce gate count, depth, and resource utilization. Despite their strengths, while tensor networks effectively reduce computational overhead for many quantum circuits, their efficiency depends on the structure and sparsity of the circuit. For highly entangled systems or dense circuits, the tensor contraction process can become computationally expensive due to the rapid growth of intermediate tensor sizes. For circuits with complex connectivity or large treewidth, the optimization benefits diminish, and the contraction order becomes computationally intensive to determine. Identifying the optimal contraction order and decomposing tensors requires significant pre-processing. This overhead can be a bottleneck.

The methodologies discussed in this section highlight the significant advancements in quantum circuit synthesis and optimization, focusing on techniques to simplify circuits, reduce gate counts, and minimize depth. Approaches such as automated synthesis frameworks (e.g., Quanto, Quartz, and QGo) and mathematical decomposition methods (e.g., SVD and tensor networks) provide robust tools for tackling the challenges of resource-intensive quantum circuits. These techniques demonstrate remarkable efficiency in reducing gates like CNOTs, merging rotation gates, and leveraging equivalence rules for global circuit minimization. Furthermore, methods like tensor network representations and hierarchical decompositions enable scalable solutions for high-dimensional systems, while incorporating hardware-specific constraints ensures compatibility with current quantum devices. However, these approaches are not without limitations, including computational overheads, dependency on pre-defined equivalence rules, and challenges in scalability for highly entangled or densely connected circuits. Despite these drawbacks, the continuous evolution of quantum circuit synthesis techniques, particularly for NISQ-era devices, underscores their critical role in advancing the practical implementation of quantum algorithms and enhancing the overall performance of quantum systems.

5.3. Optimization Using Algorithmic Approaches

An algorithmic approach [54,55] involves the systematic and methodical application of algorithms, which are sets of precise instructions and procedures, to solve complex problems, streamline processes, or accomplish specific goals. By breaking down intricate tasks into manageable steps, this structured methodology ensures efficiency and accuracy in problem-solving, decision-making, and data processing.

Quantum circuit optimization can be automated by variational algorithms. In [56–58], the underlying principle for quantum classical fusion is the variational principle. This principle can be compared to a radio tuner. The quantum circuit is analogous to the electronic radio circuit. The quantum circuit consists of an encoding layer and a trainable layer. The encoding layer consists of rotation gates where the angles of the rotation gates become the parameters that can be tuned by the classical component. This parametrized circuit is called the ansatz. The classical component is analogous to the tuner/capacitor in the radio. The tuner is used to tune in order to match the frequency of the radio station. Likewise, the classical layer is used to tune the parameters of the quantum circuit until the cost function associated with the quantum circuit is optimized. The Variational Quantum Eigen solver reduces the quantum resources or quantum gates that would be required in a quantum circuit.

Ref. [59] proposes Variational Gate Circuit Optimization (VQGO). VQGO uses a Variational Quantum Eigensolver (VQE). The multi-qubit quantum circuit is initially parametrized into single-qubit gates and source gates. This decomposition into elementary gates is essential as the input multi-qubit gate circuit may not be compatible with running on specific quantum hardware, whereas elementary gates, such as source gates or single qubit, would be available on the hardware. VQGO uses these single-qubit gates with parametrized rotation angles along with a drive to create multi-qubit gates. The optimization is performed by iterating through the rotation angles of the single-qubit gates that are based on the cost function's measured values. Figure 7 illustrates the VQGO method. The cost function is the Average Gate Infidelity (AGI). AGI compares two quantum channels and checks how far on average they are from the pure states.

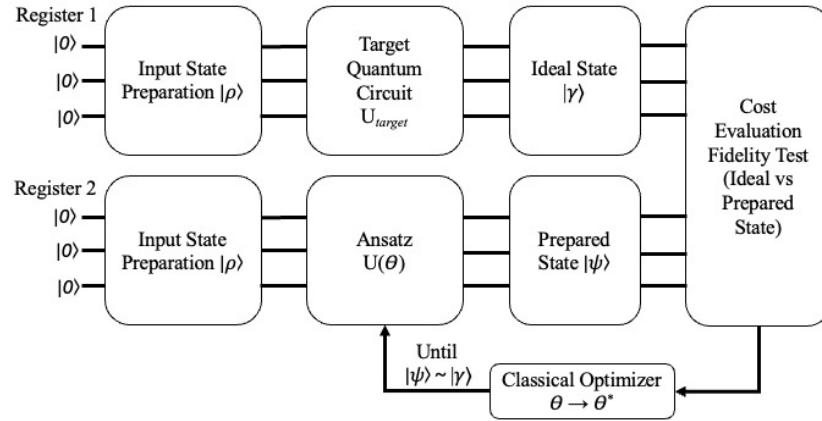


Figure 7. Illustration of hybrid quantum circuit optimization.

The cost function depends on the target circuit U_{target} parameters and the ansatz circuit $U(\theta)$ parameters. The objective is to align the target circuit to the ansatz circuit. Initially, a high-cost function would be obtained because of the difference between the two quantum circuits. With every iteration, the parameters of the target circuit are adjusted to reflect the pure state parameters. The target circuit is optimized when the AGI value associated with the two quantum channels is minimal. VQGO can achieve a higher fidelity compared with conventional methods even in cross-resonance environments. Cross-resonance is interference or unwanted interaction between qubits, which can lead to errors in quantum operations. In a CNOT gate, cross-resonance can result in the control qubit interacting with other qubits in the circuit, which results in the target qubit flipping or changing its state even when it was not intended. The comparative table, Table 2,

highlights the distinctions between classical machine learning (ML) approaches and hybrid quantum-classical methods for quantum circuit optimization.

Table 2. Comparison between classical optimization model with hybrid approach.

Feature	Classical Optimization Models	Hybrid Quantum-Classical Optimization Models
Core Principle	Fully reliant on classical computational resources and algorithms.	Combine quantum computing for specific tasks with classical optimization layers.
Problem-Solving Scope	Suitable for well-defined, small to medium-sized optimization problems.	Designed for problems where quantum advantages can be leveraged (e.g., VQE, QAOA).
Algorithm Examples	RL, GNN, GAN	Variational Quantum Eigensolver (VQE), Quantum Approximate Optimization Algorithm (QAOA).
Scalability	Efficient for many large-scale problems, though constrained by classical limits.	Scale better for specific quantum problems but limited by quantum hardware capabilities.
Optimization Speed	Fast due to mature classical computing technologies.	Slower due to iterative communication between quantum and classical layers.
Cost Function Dependency	Classical cost functions with no quantum measurements.	Hybrid cost functions often incorporate quantum measurements for updates.

In [60], gate-free implementations of the encoding circuit of VQEs are also shown. Control VQEs showed significant improvement in the speed of the state preparation handled by hardware-controlled pulses as compared to the encoding circuits of VQEs. Stenger et al. [61] proposed a pulse-scaling method that translated the area of the CR and rotary pulses to the 3-dimensional space with $RzX(\theta)$ rotations. Their approach shows an increase in gate fidelity without any added calibrations. The above work is extended into even arbitrary gates to construct a circuit transpiration framework that decomposes two-qubit gates into the RzX rather than the traditional C-NOT transpiration.

In [62–65], genetic algorithms (GAs) mimic nature’s process; that is, individuals better adapted to the environment have a greater chance of surviving and passing on their genetic traits to their offspring. In the context of GAs, a candidate solution is represented as a chromosome, and a group of these solutions forms a population. The encoding of chromosomes can vary depending on the specific problem and it may utilize binary strings, direct value representations, and tree structures. Ref. [66] proposed a genetic algorithmic method for optimizing quantum circuits. Circuit optimization is formalized as follows: N quantum circuits form the initial population. Each circuit’s fitness is determined by its output state vector. From the population, one circuit is selected and a cross-over made with the one that needs to be optimized. The crossover would be swapping a single time step or multiple steps of the circuit. The mutation is also performed by flipping a single-qubit gate, interchanging the control and target of control gates, and tuning the parameter for the rotation gates for exploring new circuits. The newly generated circuit is added to the populations depending on the fitness score. The above process is repeated until the desired optimum is reached. In [67], Wei et al. generated the population by using a tree-structured method, where they incorporated the equivalent circuit for each leaf as well as each subtree. This helps to reduce the time complexity of the algorithm. Peng et al., in [68], successfully optimized the quantum teleportation circuit using a genetic algorithm.

The methodologies discussed here, provide a comprehensive framework for large-scale quantum circuit optimization. Table 3 describes the comparative analysis in detail. AI-based approaches excel in adaptability and iterative learning, while unitary synthesis enables resource-efficient transformations through mathematical techniques. Algorithmic approaches, on the other hand, offer systematic strategies tailored for specific goals, such as depth reduction or fidelity improvement. Each method has distinct advantages and drawbacks, making their combination a promising direction for further research in NISQ-era quantum computing.

Table 3. Comparative analysis of optimization methodologies.

Methodology	Key Techniques	Advantages	Drawbacks
AI-Based Approaches	Reinforcement Learning, Deep RL, GAN	Adaptive, scalable for complex circuits	High computational demands
Unitary Synthesis	Matrix Decomposition	Gate and depth reductions	Computational overhead, matrix structure dependency
Algorithmic Approaches	Variational Algorithms, Genetic Algorithms	Hardware-aware, systematic optimization	Time-intensive, computational complexity

6. Quantum Circuit Layout Optimization

Quantum circuit layout optimization is a level II optimization. The practical realization of theoretical quantum circuits on quantum hardware is a complex process [69,70]. Each hardware has specific physical layouts of qubits. In a superconducting qubit computer, every qubit in the layout cannot be connected with every other qubit. But theoretical quantum computational models assume that direct interactions between any two physical qubits are always possible. However, in actual physical implementations, establishing direct interactions between distinct qubits can be extremely challenging—sometimes, it may not be possible. For example, in the IBM QX4 architecture [12] shown in Figure 8, Q_1 can directly interact with Q_0 only. Q_2 can directly interact with Q_0 and Q_1 . There is no direct interaction possible between Q_3 and Q_1 , Q_0 to Q_1 , and so on.

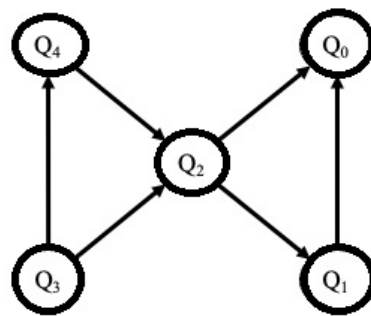


Figure 8. IBM QX4 architecture.

In addition, certain qubit pairs may have interaction frequencies as low as 0.2 Hz; for example, carbon qubit pairs C2 and C6 have interaction frequencies, while decoherence processes can take around one second. This means that the interaction between such qubits would generate undesirable noise. The connectivity between qubits determines how efficiently these algorithms can be implemented. If qubits are not directly connected,

additional operations may be needed to enable their interaction, which can increase the complexity and runtime of the algorithm. The fewer additional gate operations a quantum computation needs, the faster it can execute and the less chance there is for errors to creep in. The connectivity varies greatly between various quantum hardware.

Mapping the simulated circuit to a specific hardware is a crucial step in the process of implementing an algorithm on a quantum computer. It becomes necessary to introduce additional bridge gates, such as SWAP, H, and CNOT gates, into the theoretical or simulated circuits. This mapping process is called quantum circuit placement (QCP). It incurs overhead in terms of computational resources and time and harms the fidelity of the execution. To address this issue, several approaches have been proposed in the literature to minimize or optimize these bridge gates.

Quantum circuit placement ensures that the logical qubits of an algorithm are mapped efficiently onto the available physical qubits while adhering to the device's connectivity constraints. Figure 9 shows an example of QCP. The initial circuit (Figure 9a) has multiple layers of gates applied across four qubits. For instance, Hadamard (H) gates and CNOT gates are placed based on the logical operations required by the algorithm and mapped directly to the available qubits. The optimized circuit in Figure 9b shows fewer gates and a more compact arrangement. For instance, CNOT gates are now placed in a way that reduces the need for intermediary swaps, and Hadamard gates are applied more efficiently. The qubits q_3 , q_2 , q_0 , and q_1 are reordered for more efficient gate count reductions.

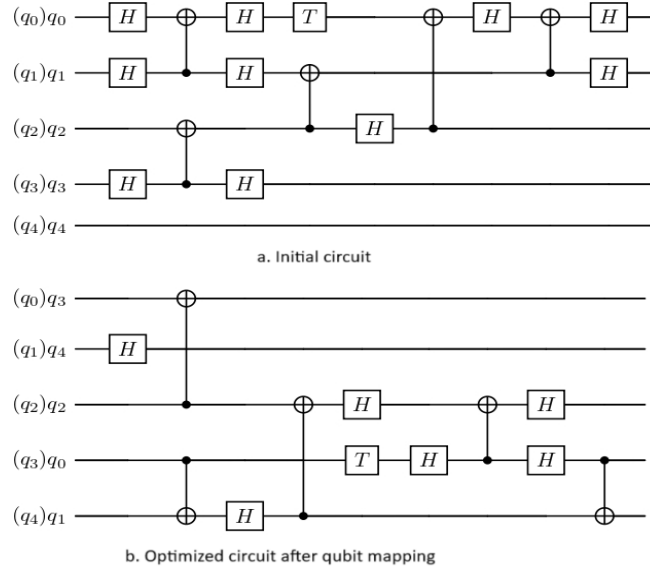


Figure 9. Quantum circuit placement illustration.

Optimizations might include merging gates, eliminating redundant gates, and reducing gate operations on each qubit to reduce error rates and execution time on quantum hardware. Layout optimization is an NP hard problem, which preserves the theoretical circuit functionality while aiming to increase computational speed. This is called **optimization level II**, which can be defined as optimization by adding additional bridge gates to satisfy the mapping constraints; that is, the mapping of logical or simulated circuits to a physical layout. In this section of the paper, we will briefly delve into the current trends and approaches aimed at tackling this intricate problem.

Ref. [71] formulates the QCP as a search problem and successfully demonstrates the work of on two IBMQ 20-qubit systems, named Tokyo and Poughkeepsie with fidelity as the benchmark. The graph structure of quantum circuits is used to predict their expected

fidelities for certain devices considering their respective noise characteristics. An idea related to the one proposed in this work, which follows a more coarse-grained approach, is demonstrated in [72], where a supervised machine learning model is used to predict good combinations of devices and existing compilation flows. RL is applied to learn a strategy for applying individual gate transformation rules to optimize quantum circuits.

Ref. [73] proposes avoiding long interactions by considering the linear nearest neighbor computational architecture where all qubits are lined up in a chain and only the nearest neighbors are allowed to interact. Mathematically, for a system with n qubits $q_1, q_2, \dots, q_n$, the 2-qubit gates are allowed on qubits whose subscript values differ by one. However, this is not necessarily possible in real quantum circuits. Any circuit with gates spanning more than two qubits is translated into a circuit with a single qubit, and two qubit gates. Next, permutations are generated of possible connections between qubits using graph theory, where each qubit can be represented as a node in a graph, the connections between them can be depicted as edges, and the coupling strength between qubits as the weight of the edges. Using a dynamic programming approach the optimum topology map is selected for the required circuit. Wille et. al. formulates the QCP as a symbolic optimization problem [74]. Only multi-qubit gates are impacted by connectivity constraints [75]. Hence, before mapping the logical circuit into the physical qubits, the coupling map of a device is checked, and the optimum mapping solution is chosen using a Boolean satisfiability solver to minimize the Hadamard (H) and SWAP operations on each timestamp.

Bhattacharjee et. al. [76] used a multi-tier method to address the QCP problem, which includes the selection of topology, resolving swap gate constraints, and choosing the QC map which yields the highest fidelity. Itoko et al. [77] proposed an algorithmic approach for compiler optimization using gate transformation and commutation to reduce swap and bridge gates. Besides circuit optimization at the logical level, using the assertion method, Haner et al. [78] show that entanglement description can also improve circuit optimization for a specific target architecture.

Fan et. al. [79] proposed an algorithm that entails training a reinforcement learning (RL) agent to discover an optimal swap strategy for randomly generated mappings. The agent's objective is to select the best strategy from a pool of possible combinations, where each combination consists of a random mapping paired with a specific swap strategy. They formulate QCP as a bi-level optimization problem. At level I, the framework finds the optimum placement mapping for a given quantum circuit. At level II, the framework focuses on reducing the swap gate cost. To achieve this, they use a deep learning RL model. The input to the model is a state matrix S and the initial mapping of qubits. The i^{th} row of the state matrix represents the i^{th} physical qubit and its associated logical qubit. Each column of the matrix state represents a separate time step (level). The element in the i^{th} row and the j^{th} column denotes the position of the qubit connected to the i^{th} qubit, which is specified by its corresponding quantum gate at time step j . The value is set to be -2 if no quantum gates are associated with the qubit at a certain time step. The underlying algorithm uses all the SWAP insertions allowed by the target quantum devices as the action space. The reward policy is described as follows: (i) gate reward, which equals the number of quantum gates executed given the current action, the updated state, and hardware constraints; (ii) done reward, which is applied when all the quantum gates have been executed; (iii) SWAP penalty, when a SWAP gate is inserted; and (iv) non-execution penalty, when there are no executable gates after the SWAP gate is inserted.

In [80], Paler et al. proposed a mapping technique that helps to reduce bridge gates. They experimented with both heuristic and machine learning approaches. They formulated the mapping problem as a weighted random search (WRS) problem. The optimum

mapping is selected by assigning weights to parameters such as the maximum depth of the tree (number of bridge gates required), the maximum number of children (qubits), and the configuration of the search space. After that, the algorithm is run iteratively to find the best mapping techniques based on reducing the overall weight as much as possible. This approach is called QXX. Once the initial solution is obtained, this will be used as learning data for the machine learning approach QXX-MLP; next, the parameters weights are tuned to improve the QXX solutions. Figure 10 illustrates the process of the Paler et.al mapping method.

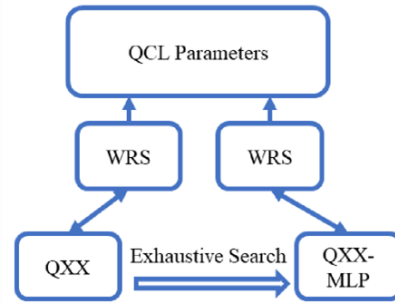


Figure 10. Continuous learning QCP.

Ref. [81] presents a methodology integrating reinforcement learning (RL) to optimize quantum circuit synthesis and routing. In circuit synthesis, the task is modeled as a sequential decision-making process where an RL agent iteratively selects gates to transform an initial operator into the identity operator, effectively constructing the desired circuit. During training, the agent follows a curriculum learning approach, gradually tackling more complex target operators, and is rewarded for successful synthesis while penalized for excessive gate usage to encourage efficiency. After training, the agent synthesizes circuits by applying its learned policy, ensuring near-optimal solutions that align with device-specific constraints. Similarly, in circuit routing, RL is employed to optimize SWAP operations, which adapt circuits to hardware coupling maps. The agent evaluates potential SWAPs to minimize the qubit distances for forthcoming two-qubit operations, reducing the circuit's depth and gate count. Together, these RL-based approaches streamline circuit optimization for diverse quantum hardware environments. However, challenges related to scalability, training overhead, adaptability, and reward function design need to be addressed to fully realize its potential in practical quantum computing workflows.

In [82], Siraichi et al. (2018) address the challenge of mapping logical qubits in quantum programs to physical qubits on hardware with restricted connectivity. They propose a qubit allocation framework that optimizes the placement of logical qubits to minimize the need for additional SWAP gates which are required to satisfy connectivity constraints. The methodology involves modeling the allocation problem as a graph optimization task, where the connectivity of physical qubits is represented as a graph, and logical qubit operations are mapped to nodes. The framework uses a cost model to evaluate different mappings based on metrics such as the gate fidelity, latency, and the SWAP gate overhead, selecting the optimal allocation through search algorithms. Techniques like lookahead heuristics and backtracking further refine the mapping to reduce runtime overhead. While effective, the framework has drawbacks: the optimization process can become computationally expensive for circuits with high qubit counts or dense connectivity requirements, limiting scalability. Additionally, the reliance on static cost models may not fully account for dynamic noise and hardware variability, potentially leading to suboptimal allocations for specific devices.

In [83], Zulehner et al. propose an optimization framework to map quantum circuits onto IBM QX devices which have limited qubit connectivity. The methodology involves transforming the logical qubit interactions in a quantum circuit to conform to the hardware's physical constraints while minimizing the insertion of SWAP gates. The mapping process is formulated as a search problem, where the authors employ A* search algorithms to explore optimal solutions by considering both the immediate and future costs of qubit placements. A key innovation is the use of lookahead mechanisms to evaluate the long-term impact of SWAP gate insertions, leading to more globally optimized solutions. Additionally, the framework introduces heuristics for dynamically selecting SWAP operations that minimize the gate count and circuit depth. While this approach significantly improves the efficiency of mapping circuits to IBM QX architectures, its main drawback is the computational overhead associated with the A* search, which scales poorly for large circuits. Furthermore, the reliance on static heuristics can lead to suboptimal performance when hardware noise and variability are not accounted for.

Ref. [84] introduced a novel approach to compiler optimization known as "Relaxed Peephole Optimization" (RPO). This approach operates under the assumption that certain qubit states in quantum gates can be determined during compilation. Leveraging this assumption, RPO offers the advantage of replacing quantum operations with more cost-effective structures (such as gates or layers) while preserving their equivalence. One notable advantage of this approach, in contrast to earlier techniques as documented, is its inherent capacity to dynamically alter the unitary matrix during runtime without affecting the outcome. In their work, the authors introduced a relaxed peephole optimization for specific quantum gates and well-defined input states like basis and pure states.

To distinguish between quantum states, the author introduced two optimization passes, namely, the Quantum Basis State Optimization Pass and the Quantum Pure State Optimization Pass. In their paper, they defined $|0\rangle$, $|1\rangle$, $|+\rangle$, $|-\rangle$, $|L\rangle$, and $|R\rangle$ as basis states.

$$|+\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} ; |-\rangle = \frac{|0\rangle-|1\rangle}{\sqrt{2}} ; |L\rangle = \frac{|0\rangle+i|1\rangle}{\sqrt{2}} ; |R\rangle = \frac{|0\rangle-i|1\rangle}{\sqrt{2}}$$

In each of the passes, a partial state automaton was performed to show how they can analyze the inputs (quantum gates) acting on specific states (basis or pure states). The implementation of the Quantum Basis State Optimization Pass (QBO) and the Quantum Pure State Optimization Pass (QPO) was carried out within the open-source quantum computing framework Qiskit.

Ref. [85] demonstrated an automated method to recompile a quantum circuit A into a target circuit B, with the aim of achieving identical results for a specific input, as expressed by the equation $B |in\rangle = A |in\rangle$. The process is initiated with the user defining B as an empty template consisting of parameterized unitary gates set to the identity matrix. Subsequently, the compilation process is carried out utilizing quantum hardware to execute an isomorphic energy-minimization task. An isomorphic energy-minimization task refers to the process of finding an alternative representation or layout of a quantum circuit while minimizing the energy or cost associated with the quantum gates and operations used in the circuit. To overcome potential slow convergence in variational eigen-solving techniques, the approach explored involves using a sequence of intermediate targets that act as "lures" to guide and accelerate the optimization process toward the final target. Below is a comparative analysis in Table 4. Summarizing content in this section.

Table 4. Comparative table for layout optimization.

Methodology	Key Innovations	Advantages	Drawbacks
-------------	-----------------	------------	-----------

Bhattacharjee et al. [76]	Multi-tier method involving topology selection and swap constraint resolution.	High fidelity through tiered optimization.	Computationally expensive for dense connectivity requirements.
Itoko et al. [77]	Algorithmic gate transformation and commutation optimization to reduce SWAP and bridge gates.	Logical-level optimization and improved circuit efficiency.	Limited focus on physical hardware constraints.
Fan et al. [79]	RL-based framework using deep learning for QCP as a bi-level optimization problem.	Adaptive learning of SWAP strategies; effective handling of dynamic mapping problems.	Training overhead; scalability issues; challenges in reward function design.
Paler et al. (QXX and QXX-MLP) [80]	Weighted random search for mapping optimization and machine learning-based parameter tuning.	Iterative improvement of mapping techniques; reduced overall gate weight.	Limited scalability for large circuits.
Siraichi et al. [82]	Graph optimization task for qubit allocation with cost models and lookahead heuristics.	Minimized SWAP gate overhead, improved fidelity and latency.	Computationally expensive for high qubit counts; static cost models limit adaptability to dynamic noise.
Zulehner et al. [83]	Formulated QCP as a search problem using A* search algorithms and lookahead mechanisms to minimize SWAP gates.	Globally optimized solutions with reduced gate count and circuit depth.	Computational overhead; poor scalability for large circuits; static heuristics may not handle hardware variability effectively.
Relaxed Peephole Optimization (RPO) [84]	Replacing quantum operations dynamically during runtime.	Cost-effective gate replacements with runtime adaptability.	Applicability limited to specific input states and gates.
Automated Circuit Recompilation [85]	Energy minimization task for recompiling circuit A into target circuit B using intermediate targets.	Accelerated convergence towards optimized circuit representation.	Potential for slow convergence in certain cases; dependency on intermediate “lure” targets.

7. Review and Future Directions

While quantum computing represents a revolutionary technological advancement, the development of real-time quantum applications remains a big challenge. This challenge is primarily attributed to the Noisy Intermediate-Scale Quantum (NISQ) nature of current quantum devices, which are inherently susceptible to noise and errors. Though there are ongoing efforts to improve error correction and mitigation technologies, quantum circuit optimization is one of the promising approaches to do so. These techniques are instrumental in economically mitigating errors and enhancing the overall performance and reliability of quantum computations.

The phase polynomial method discussed in [21] encounters limitations when applied to quantum circuits restricted to certain gate sets, such as CNOT, NOT, and Rz gates. Among these, the Hadamard (H) gate holds particular significance in quantum computing, as it facilitates the creation of superposition states and entanglement, which are foundational to quantum algorithms. Quantum circuits can employ a wide range of gates, including SWAP, CSWAP, T, and P gates, each contributing uniquely to the circuit’s overall functionality. Efficiently merging Rz gates with these different gate types is crucial for

optimizing circuit performance. However, as the number of gates to be merged increases, challenges arise in maintaining the accuracy of phase polynomial estimations.

The phase polynomial approach mathematically represents the circuit's operations and interactions, particularly focusing on Rz gates. When dealing with many gates, inaccuracies in this representation can lead to suboptimal gate merging. Such inaccuracies can adversely affect the circuit's overall computational efficiency and fidelity, potentially undermining its ability to execute tasks with precision.

Optimization based on reinforcement learning (RL) considers both physical and logical level quantum circuit optimization achieved by learning from past experiences and optimizing itself over time. Ref. [37] demonstrates optimizing certain types of quantum gates, such as X, Z, and CNOT gates. To generalize this algorithm for all elementary gates by adding more circuit patterns to help the agent learn, it may be able to generalize its knowledge to new circuits with similar patterns. However, adding additional circuit patterns to the training data can also cause a problem called Qtable exploitation. This happens when the agent focuses on many specific models of the training data and does not explore other possible solutions.

In summary, quantum circuit optimization begins with encoding the quantum circuit into a structured representation, such as tuples, tree structures, symbolic forms, or graphs. The optimization problem is then defined with goals like minimizing the gate count, circuit depth, and improving fidelity, while tools like machine learning (ML), algorithms, and variational principles are employed to address the problem. The process involves iterative refinement to decode the optimized circuit structure, resulting in a more efficient quantum circuit ready for implementation. Figure 11. emphasizes systematic problem-solving and tool-assisted optimization to achieve high-performance circuits.

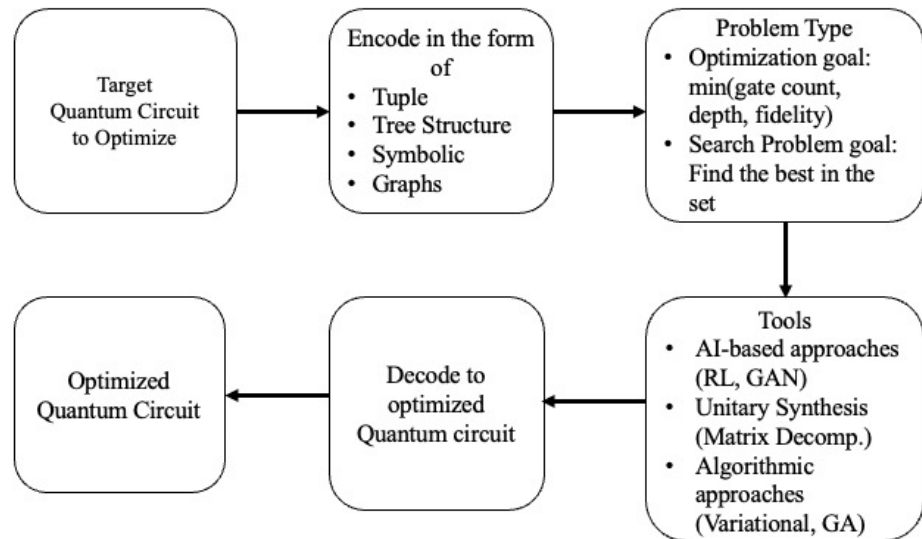


Figure 11. Summary of quantum circuit optimization methods.

Quantum circuit optimization has made significant advancements, yet several challenges remain, particularly in the Noisy Intermediate-Scale Quantum (NISQ) era. Noise and decoherence in NISQ devices, stemming from qubit susceptibility to gate errors and environmental factors, continue to degrade computation fidelity as circuit depth and gate count increase. Addressing this requires noise-aware optimization frameworks that integrate error-resilient gates and dynamically adapt to hardware error characteristics. Scalability is another critical issue, as many methods falter when applied to large circuits with

thousands of gates or qubits, necessitating hybrid classical-quantum approaches, hierarchical frameworks, and adaptive strategies to balance accuracy and efficiency. The long-term goal of achieving fault-tolerant quantum computing depends on breakthroughs in error correction and optimization techniques to enable large-scale, high-fidelity computations. Additionally, universal optimization frameworks that combine hybrid, heuristic, and AI-based methods are vital for standardizing processes across diverse hardware platforms and algorithms, ensuring compatibility and scalability. By addressing these challenges, researchers can advance scalable, reliable, and efficient quantum systems, paving the way for practical quantum computing applications.

This paper provides a comprehensive literature review on the topic of quantum circuit optimization, highlighting the strategies and guidelines for enhancing circuit efficiency, as well as the implementation techniques for scaling up quantum circuits. The paper also touches on the distinction between logical quantum circuits and physical quantum circuits. To achieve efficiency goals in quantum computing, the current quantum era needs to use a combination of both technology-independent and technology-dependent optimization techniques. However, given the current limitations of scalable quantum computer technology, machine-independent optimization techniques are receiving more attention.

Funding: This research received no external funding.

Data Availability Statement: No new data were created or analyzed in this study.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Shor, P.W. Algorithms for quantum computation: Discrete logarithms and factoring. In Proceedings of the 35th Annual Symposium on Foundation of Computer Science, Washington, DC, USA, 20–22 November 1994; pp. 124–134.
2. Grover, L.K. A fast quantum Mechanical algorithm for database search. *arXiv* **1996**, arXiv:quant-ph/9605043.
3. Kim, M.; Kasi, S.; Lott, P.A.; Venturelli, D.; Kaewell, J.; Jamieson, K. Heuristic Quantum Optimization for 6G Wireless Communications. *IEEE Netw.* **2021**, *35*, 8–15. <https://doi.org/10.1109/MNET.012.2000770>.
4. Building a Quantum Nim Game. Available online: <https://medium.com/qiskit/building-a-quantum-nim-game-8c7872b6ce0d> (accessed on 29 April 2023).
5. Puram, V.; Kang, D.; George, K.M.; Thomas, J.P. Algorithm to build quantum circuit from classical description of DFSM. In Proceedings of the 2022 IEEE International Conference on Quantum Computing and Engineering (QCE), Broomfield, CO, USA, 18–23 September 2022; pp. 745–748. <https://doi.org/10.1109/QCE53715.2022.00104>.
6. Pastorello, D.; Blanzieri, E. A Quantum Binary Classifier based on Cosine Similarity. In Proceedings of the 2021 IEEE International Conference on Quantum Computing and Engineering (QCE), Broomfield, CO, USA, 17–22 October 2021; pp. 477–478. <https://doi.org/10.1109/QCE52317.2021.00086>.
7. Zhang, X.D.; Zhang, X.M.; Xue, Z.Y. Quantum hyperparallel algorithm for matrix multiplication. *Sci. Rep.* **2016**, *6*, 24910. <https://doi.org/10.1038/srep24910>.
8. Available online: <https://thequantuminsider.com/2022/07/14/google-sycamore/> (accessed on 26 November 2024).
9. Available online: <https://newsroom.ibm.com/2022-11-09-IBM-Unveils-400-Qubit-Plus-Quantum-Processor-and-Next-Generation-IBM-Quantum-System-Two> (accessed on 26 November 2024).
10. Devitt, S.J. Performing quantum computing experiments in the cloud. *Phys. Rev. A* **2016**, *94*, 032329.
11. Amazon Braket. Azure Quantum—Quantum Service: Microsoft Azure. Available online: <https://azure.microsoft.com/en-us/services/quantum/> (accessed on 1 May 2022).
12. Preskill, J. Quantum Computing in the NISQ era and beyond. *arXiv* **2019**, arXiv:1801.00862v3.
13. Alfred, A.; Krysta, S. Compiling Quantum Circuits using the Palindrome Transform. *arXiv* **2003**, arXiv:quant-ph/0311008.
14. Nielsen, M.A.; Chuang, I.; Grover, L.K. Quantum computation and quantum information. *Am. J. Phys.* **2002**, *70*, 558–559.

Leveraging Machine Learning for Quantum Circuit Optimization

*Mathias Weiden
John D. Kubitowicz, Ed.*

Electrical Engineering and Computer Sciences
University of California, Berkeley

Technical Report No. UCB/EECS-2023-84

<http://www2.eecs.berkeley.edu/Pubs/TechRpts/2023/EECS-2023-84.html>

May 10, 2023



Abstract

In the era of noisy, non-fault tolerant, quantum hardware, reducing the number of operations in quantum programs is essential to ensure computations produce meaningful results. Unitary synthesis is a particularly promising technique which uses non-linear numerical optimization tools paired with heuristic-guided tree search algorithms to produce resource efficient implementations of quantum algorithms. This report illustrates how unitary synthesis can be used to optimize quantum circuits with many qubits. The TopAS algorithm demonstrates how minimizing both operation count and interaction complexity before mapping can produce better circuit implementations than the state of the art. This report also presents several discoveries that enable the use of machine learning in accelerating unitary synthesis. Namely, a canonical representation of unitary matrices, and an argument for the learnability of mappings between a specific class of unitary matrices and parameterized quantum circuits are presented. The QSeed algorithm shows how machine learning can be leveraged to accelerate the optimization of wide quantum circuits without sacrificing solution quality.

Leveraging Machine Learning for Quantum Circuit Optimization

Mathias Weiden

Spring 2023

1 Introduction

The idea to employ the seemingly unnatural powers of quantum mechanics for computation originated in Richard Feynman's 1981 lecture titled "Simulating Physics with Computers" [11]. Sixteen years later, in 1997, Simon's problem demonstrated the first oracle separation between quantum computers and probabilistic classical computers [25]. In that same year, Peter Shor showed that quantum computers can achieve super-polynomial speedups over classical computers for the prime number factoring problem [23]. Yet, the execution of these algorithms remains out of reach for modern quantum machines. The current Noisy Intermediate Scale Quantum (NISQ) era is defined by limited hardware that is extremely susceptible to environmental noise and decoherence [21]. Quantum error correction promises to elevate the field into an era of fault tolerance, but questions relating to the scaling of hardware capable of carrying out these complex error correction schemes remain unanswered [1]. Error mitigation, an ever widening suite of techniques used to reduce the effects of environment interference, serve as a NISQ era aid to these problems. Google's quantum supremacy experiments have confirmed the accuracy of a simple digital noise model, meaning just reducing the number of operations in quantum programs can dramatically improve the likelihood that computations are correct [3].

In this report, I will discuss common techniques used to optimize the operation count and depth of quantum circuits. I will focus on the use of unitary synthesis, a promising optimization technique. I will also present research efforts I have made in applying and improving unitary synthesis algorithms with machine learning.

This report is divided as follows: Section 2 surveys common techniques used for quantum circuit optimization and introduces unitary synthesis. Section 3 details the efforts I have made in applying machine learning to circuit optimization. Finally, Section 4 discusses promising future directions for the field of machine learning in circuit optimization.

2 Quantum Circuit Optimization

In this section, I will introduce preliminary concepts in quantum computing and the circuit optimization problem. I will also detail several widely used optimization techniques, and algorithms used for a specific style of optimization called unitary synthesis.

2.1 Preliminaries

Definition 2.1 (Qubits). A qubit is the base unit of quantum information. The qubit model assumes that a quantum system can be in any arbitrary superposition of two orthogonal states $|0\rangle$ and $|1\rangle$, so long as it satisfies

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \quad |\alpha|^2 + |\beta|^2 = 1$$

where $\alpha, \beta \in \mathbb{C}$. Given a system of n qubits and defining $N = 2^n$, the quantum state lies in a Hilbert space of size $\mathbb{C}^{N \times N}$. The composite state $|\phi\rangle$ of this n qubit system can be found by taking the n -fold tensor product

$$|\phi\rangle = \bigotimes_{i=1}^n |\psi_i\rangle.$$

Definition 2.2 (Quantum Gates). Operations on qubits must preserve the unit L2 norm of the quantum state. For this reason, operations on qubits must be unitary. Gates may act on one or multiple qubits, and they can be functions of real parameters. Two noteworthy quantum gates are the single qubit U3 gate

$$U3(a, b, c) = \begin{bmatrix} \cos(\frac{a}{2}) & -e^{ic} \sin(\frac{a}{2}) \\ e^{ib} \sin(\frac{a}{2}) & e^{i(b+c)} \cos(\frac{a}{2}) \end{bmatrix}$$

and the two qubit CNOT or CX gate

$$CNOT = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

The gate set of $\{CNOT, U3\}$ is both universal for quantum computation and capable of creating any unitary matrix [19].

Quantum gates can be composed in parallel or sequentially. Parallel composition corresponds to a tensor product while sequential composition corresponds to a matrix multiplication.

Definition 2.3 (Quantum Circuits). A quantum circuit is a data structure that describes a quantum program. Qubits are represented as horizontal wires. Quantum gates are composed in sequences where time is assumed to flow from left to right. The single-qubit U3 gate appears as a squares, the multi-qubit CNOT gate spans the qubits on which it is applied. The width of a circuit

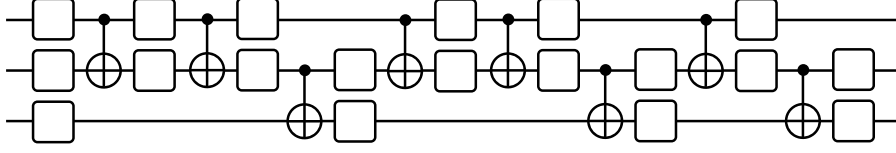


Figure 1: An example of a quantum circuit with U3 and CNOT gates.

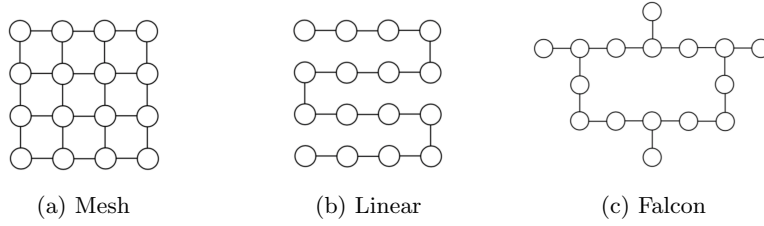


Figure 2: Examples of three common qubit topologies. The heavy hexagonal “falcon” style of topology is used in IBM’s superconducting machines. Meshes and lines are also common topologies.

refers to the number of qubits in the circuit, the depth refers to the number of time steps along the circuit’s critical path. Circuits that contain one or more parameterized gates (such as the U3 gate) are called Parameterized Quantum Circuits (PQCs). The unitary operation implemented by quantum circuit C is U_C . If C is a PQC parameterized by a vector $\theta \in \mathbb{R}^m$, then the unitary associated with C is denoted $U_C(\theta)$.

Although quantum circuits allow interactions between any pair of qubits, hardware is typically more limited. To account for this, a *qubit topology* can be used to describe more restrictive connectivities.

Definition 2.4 (Qubit Topology). A qubit topology is a graph $G = (V, E)$ where V is a set of vertices and E is a set of edges. Vertices $v \in V$ represent qubits, while edges $(u, v) \in E$ represent (hardware) allowed interactions between qubits.

The goal of quantum circuit optimization is to reduce the operation count and depth of quantum programs. This is necessary as NISQ era qubits are extremely sensitive to noise and are not error corrected. Certain quantum gates are more likely to introduce error than others. Multi-qubit gates, such as the CNOT, take far more time to execute and are far more error prone than single-qubit gates. Depending on the specific machine, some single-qubit gates may be more error prone and more difficult to calibrate than others [7]. Critically, the optimized version of a circuit must implement the same functionality as the original circuit.

2.2 Rule Based Circuit Optimization

Rule based circuit optimizations are a class of optimization techniques that make local gate transformations based on a pre-compiled series of transformation rules. Often, these sorts of algorithms are implemented with pattern recognition and replacement schemes [6]. Peephole optimization techniques (which are heavily used in classical compilation), where a window scans across the quantum circuit seeking opportunities for pattern replacement, are commonly used in practice [26, 2, 13].

This sort of optimization is fast, and is often very effective at reducing both circuit depth and gate counts, but, it has two inherent limitations. First, patterns for replacement must be identified beforehand, a process which is typically carried out by human observation. The space of possible circuit pattern-replacement pairs is vast, and the greatest opportunities for optimization have almost certainly not been fully explored. These replacement rules are human constructed and guided by heuristic and intuition. This restriction of peephole optimization has also been pointed out in classical compilers [4]. In response, tools such as CompilerGym [8] have been developed to use Reinforcement Learning (RL) to automate compiler optimization flows in classical compilers. The use of RL in peephole optimization for quantum computers has also been proposed.

Second, many of these rule based optimization techniques only consider very small width windows for optimization. Although fast, much opportunity for optimization is passed by because a more global perspective is not taken. Ultimately, the greatest issue with this particular aspect of rule based optimization is that typically small tweaks to imperfectly programmed algorithms are made. It is often the case that human written algorithm implementations are written in a way that is more understandable to humans, but this often results in non-optimal code. In these cases, it may instead be advantageous to rewrite instructions completely rather than make small changes to the existing code.

Both of these concerns are addressed in unitary synthesis optimization.

2.3 Unitary Synthesis

Unitary synthesis is a bottom-up method of implementing unitaries as quantum circuits. More formally,

Definition 2.5 (Unitary Synthesis). Given a unitary matrix $U \in \mathbb{C}^{N \times N}$, a unitary synthesis algorithm constructs a quantum circuit C that implements a unitary U_C satisfying

$$\|U - U_C\|_{HS} \leq \epsilon \quad (1)$$

where $\|\cdot\|_{HS}$ is the Hilbert-Schmidt norm defined as

$$\|A\|_{HS} = \text{Tr}(A^\dagger A). \quad (2)$$

Unitary synthesis describes the process of implementing a unitary as a quantum circuit in a way that allows for approximation. In the NISQ era, this is

a particularly interesting feature. Increasing the amount of approximation (or increasing the constant ϵ) typically reduces the number of gates in a circuit. So one can find a balance between error introduced by approximation, and error “saved” by gate reduction.

The Hilbert-Schmidt norm is one of the many metrics that can be used for this task, the most natural of which is the diamond norm [5]. However, the diamond norm is typically not used because of its extreme computational demands.

Synthesis can be performed with both parameterized and non-parameterized gate sets. This fact is known due to the Solovay-Kitaev theorem, which proves that discrete non-parameterized gate sets are capable of implementing any unitary to within some approximation factor [18]. For practicality, we typically assume a parameterized gate set such as $\{CNOT, U3\}$. Circuits expressed in this gate set are typically far less deep. Given a PQC, the process of solving for parameters to approximate a unitary is called *instantiation*. Expressed concretely,

Definition 2.6 (Instantiation). Given a target unitary U and a unitary $U(\theta)$ parameterized by $\theta \in \mathbb{R}^m$, instantiation solves the problem

$$\arg \min_{\theta} \|U - U(\theta)\|_{HS}. \quad (3)$$

The instantiation problem becomes exponentially more difficult as qubits are added to the circuit. This is because the size of the unitaries grow with $O(2^n)$. The parameterized unitary $U(\theta)$ is typically implemented with a PQC. Adding more gates to this circuit increases the number of parameters θ , which also adds difficulty.

2.3.1 QSearch

QSearch was among the first unitary synthesis algorithms [10]. QSearch approaches unitary synthesis by decomposing the problem into two operations: synthesis tree search and instantiation.

As input, QSearch accepts a qubit topology $G = (V, E)$ and a target unitary U . The edges $(u, v) \in E$ describe allowed operations in the synthesized circuit. Synthesis begins with a base circuit containing only U3 gates applied to each qubit. The circuit is instantiated to determine if it is a suitable approximation of the target unitary. At this point, the synthesis tree search process begins. A unit of gates is appended to the base circuit, one along each edge in the qubit topology. Circuits derived this way can be arranged in a tree, called the *synthesis circuit tree*. A node in the tree acts as a circuit prefix to its children; its children will only differ by gates appended to the end of the circuit. The process of synthesis can then be considered a search through the synthesis circuit tree. An example of synthesis circuit tree is depicted in Figure 3.

QSearch uses an A^* heuristic that considers the Hilbert-Schmidt distance to the target unitary as well as the current circuit’s gate count to guide the

synthesis tree search. Given a node in the tree, QSearch calls instantiation on each of the node’s children, and ranks them according to the heuristic.

Synthesis begins at the root node, then traverses through the synthesis circuit tree until a target circuit is reached. The number of instantiation calls made is at least as large as the number of circuits visited in this traversal. Other than wall-clock time, one can also measure the time needed to synthesize a circuit by measuring this path distance in the synthesis tree search.

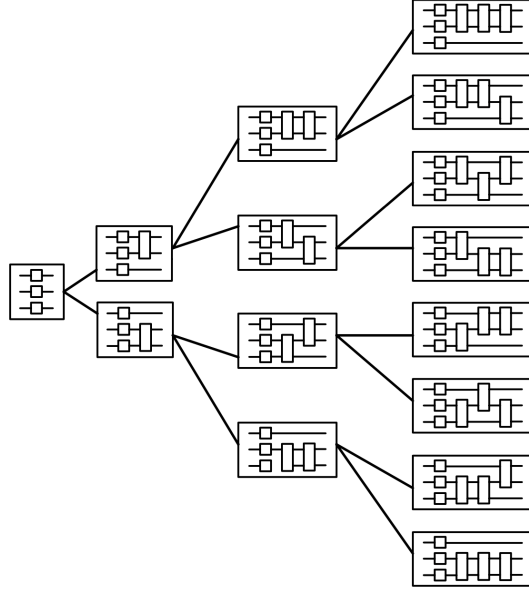


Figure 3: The synthesis circuit tree for a three qubit circuit with linear connectivity.

The details of QSearch’s CNOT gate count reduction compared to other rule-based methods can be found in [10]. QSearch’s bottom-up approach and allowed tunable approximation threshold result in unmatched gate reductions. This degree of optimization is possible because QSearch is capable of taking a broader view of the unitary being implemented. Redundancies and inefficiencies present in human written circuits are eschewed, and a globally efficient path to the target unitary is found. This can lead to rather non-intuitive results: Transverse Field Ising Model (TFIM) circuits, which simulate a material’s magnetization, can be compiled into constant depth circuits. These circuits iteratively append gates that compute the magnetization for new time steps in the simulation, but synthesis reliably finds a circuit with the same structure that summarizes the entire simulation.

Because it only applies gates along allowed edges in some qubit topology, circuits synthesized by QSearch can also be mapped to restrictive hardware.

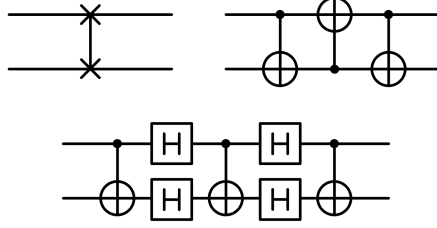


Figure 4: Three equivalent implementations of a quantum SWAP gate, which can be decomposed into three CNOT gates.

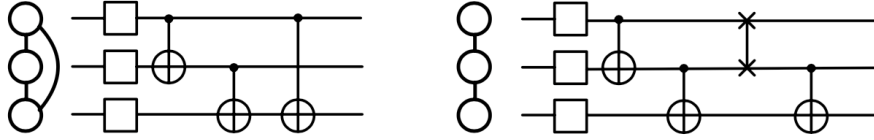


Figure 5: A quantum circuit that has been mapped to a restrictive qubit topology

Definition 2.7 (Mapped Quantum Circuits). A quantum circuit C is said to be mapped to a qubit topology $G = (V, E)$ if multi-qubit gates in C only occur along edges in E . A quantum circuit can be mapped to a qubit topology by the use of SWAP operations, which exchange the quantum state held by two qubits. The process of injecting SWAP gates into a circuit is called *routing*.

2.3.2 Synthesis Time Scaling

The run time of unitary synthesis algorithms such as QSearch grows exponentially as the number of qubits increases. Both stages, instantiation and synthesis tree search, contribute to this scaling. The time complexity of instantiation grows as $O(2^n)$ because the width of the target and parameterized unitaries grows exponentially. The run time of the tree search component also scales exponentially, as the minimum number of CNOT gates needed to implement an arbitrary unitary is $\frac{1}{4}[4^n - 3n - 1]$ [22]. Both components of QSearch’s approach to synthesis result in synthesis times quickly becoming untenable.

2.3.3 Circuit Partitioning

In order to handle the run time scaling issues of bottom-up synthesis, wide quantum circuits must be broken into smaller subcircuits. This process is referred to as *circuit partitioning*.

Definition 2.8 (Circuit Partitioning). A partitioned quantum circuit is a quantum circuit C along with a sequence of sets S . Each set $B \in S$ is a set of gates

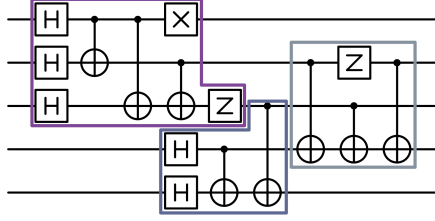


Figure 6: A five qubit quantum circuit that has been partitioned into three qubit subcircuits.

which appear in C . B is called a block, partition, or subcircuit. Blocks satisfy the properties

$$\forall B \in S \quad \bigcap B = 0 \quad \text{and} \quad \bigcup B = C \quad (4)$$

meaning each gate is a member of only one block, and all gates are a member of a block. To be a valid circuit partitioning, gates in block B must precede or succeed other gates in B . Valid partitionings only contain gates that act on interacting qubits. This means that the subgraph induced by some set of gates in B is connected. Typically, the width of partitions are limited to a value $k \in \mathbb{Z}$ called the *partition or block width*.

There are several strategies for partitioning quantum circuits. Their run times vary from growing with $\binom{n}{k}$, to linear in the number of gates. Should partitioning algorithms form blocks containing a similar number of gates? Should blocks greedily be formed so there are a few blocks with many gates? It remains unclear what the goal of a partitioning algorithm should be. The criteria for circuit partitioning is likely dependent on what optimization will be run, but this requires further investigation. Typically, it is the case that synthesis performs better when there are more CNOT gates in the circuit being synthesized. However, the time required to partition circuits is in practice far more important than this, so simple partitioning strategies are preferred.

To apply synthesis optimization to circuits wider than four (possibly five) qubits, circuit partitioning must be used. In a quantum compilation flow, the end goal is typically a series of instructions that can be run on real hardware. Optimized circuits must therefore be mapped to hardware. But, should optimization, such as synthesis, be applied before or after mapping?

2.3.4 QGo

QGo was the first wide quantum circuit synthesis algorithm [28]. QGo optimizes quantum circuits that have already been mapped to restrictive qubit topologies. This style of optimization is called post-mapping optimization, and the programmatic flow of QGo is illustrated in Figure 8. The guiding philosophy behind QGo is: use synthesis to combine redundant operations introduced by mapping with operations necessary for computation.

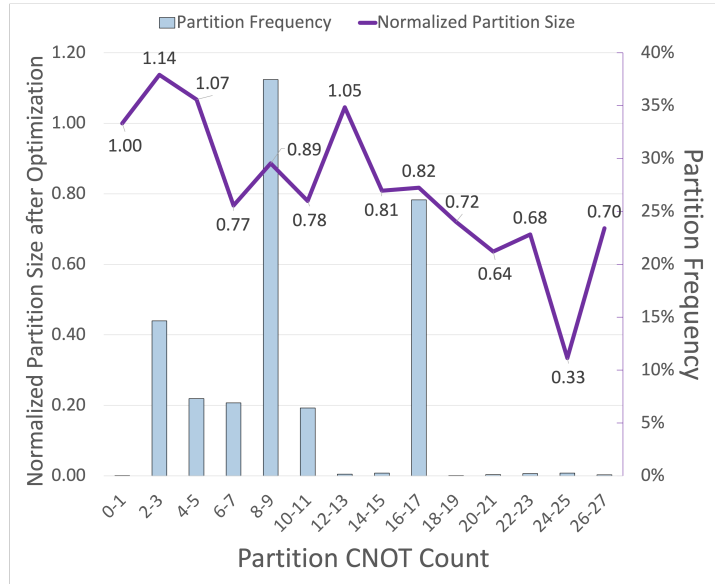


Figure 7: Synthesis performance vs. partition CNOT count. Also pictured is the distribution or frequency of blocks with that number of CNOTs in a set of partitioned benchmark circuits. A partition size of less than one indicates optimization occurred, lower is better.

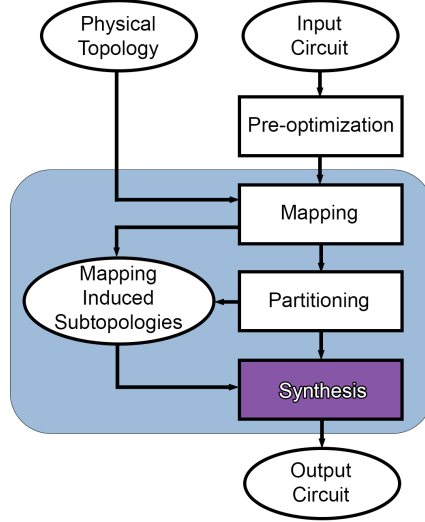


Figure 8: The program flow of a post-mapping synthesis algorithm such as QGo.

2.3.5 TopAS

The program flow of QGo begs the question, can further reductions in depth and gate count be achieved by applying synthesis before mapping? This “pre-mapping” style of optimization is illustrated in Figure 9. TopAS, a Topology Aware Synthesis algorithm for wide circuit optimization, is a pre-mapping synthesis algorithm [27].

The TopAS algorithm has one guiding principle: use synthesis to make the circuit mapping problem easier. Synthesis can be used to achieve this in two ways. First, reducing the number of operations in a circuit before mapping means fewer gates must be mapped, which typically results in better mapping algorithm performance. Second, restricting the complexity of operations in a circuit before mapping helps mapping performance, as simpler communication patterns require fewer mapping operations.

The first observation, that mapping algorithms do better when there are fewer gates, came by observing the performance of Qiskit’s implementation of the SABRE Swap algorithm on TFIM circuits mapped to mesh style topologies. The interactions in TFIM circuits are linear, meaning the circuit can be mapped to a linear qubit topology without the use of any SWAP operations. Because linear graphs are embedded within meshes, this implies that TFIM circuits can also be mapped to mesh topologies without the use of SWAP operations. However, our experiments showed that the SABRE mapping algorithm was unable to find a zero SWAP mapping of TFIM circuits, unless synthesis was performed beforehand.

The second observation, that mapping prefers simpler interactions, was

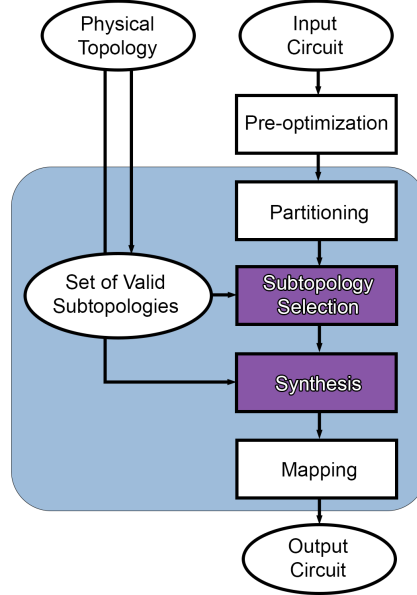


Figure 9: The program flow of a pre-mapping synthesis algorithm such as TopAS.

guided by a shift in perspective regarding mapping algorithms. Once partitions are formed, SWAPs can be divided into two categories: intra-partition SWAPs that exchange state within a partition, and inter-partition SWAPs that exchange qubits between partitions. Inter-partitions SWAPs are unavoidable. They describe movements of quantum information that are necessary for the correct sequence of interactions to take place. Intra-partitions SWAPs, however, describe a partition’s suitability for execution on that region of the qubit topology. If that partition were instead mapped to the region of the topology (called a subtopology) on which it is being executed, then no intra-partition SWAPs would be needed.

As mentioned in Section 2.3.1, QSearch accepts as input both a target unitary and a graph $G = (V, E)$ which describes the allowed interactions between qubits. This graph G is often denoted G_S and is called a *synthesis subtopology*. Subcircuits synthesized given a synthesis subtopology G_S are also mapped to that synthesis subtopology. Synthesis subtopologies differ from physical subtopologies in that physical subtopologies describe connected subgraphs in a qubit topology, whereas synthesis subtopologies need not have any connection to a real machine. Superconducting qubit machines are rarely fully connected, but synthesis subtopologies can be complete graphs. All order four synthesis subtopologies are illustrated in Figure 10.

The multi-qubit gates in a circuit also define a graph, called the *logical*

connectivity. Each edge in the logical connectivity corresponds to a multi-qubit gate between two qubits in a circuit. The weight of an edge in the logical connectivity corresponds to the number of times that interaction occurs. Logical connectivity graphs are often given the notation $G_L = (V, E)$, where E consists of weighted edges (w, u, v) for $w \in \mathbb{Z}$ and $u, v \in V$.

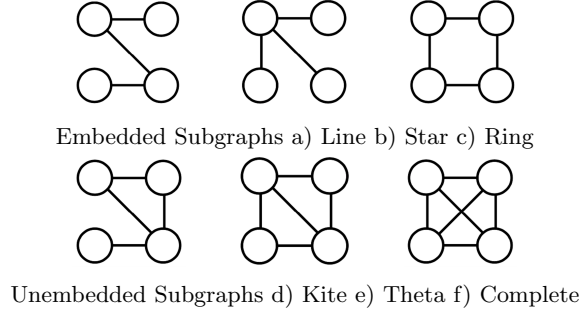


Figure 10: Possible order 4 synthesis subtopologies. The graphs a, b, and c are embedded within the mesh physical topology. Graphs d, e, and f are not. Only a and b are embedded in the falcon topology, while a is the only embedded subgraph of the linear topology.

When optimization is carried out before mapping, there is no a priori way of determining what synthesis subtopology each partition should be mapped to. Instead, TopAS uses a heuristic approach to select subtopologies for each partition. We observed that partitions synthesized to subtopologies that were similar to their unoptimized logical connectivity graphs typically saw greater reductions in gate counts. In order to quantify the similarity between a partition and a subtopology, we devised a graph similarity function inspired by machine learning kernel functions [15].

Definition 2.9 (Similarity Score). The similarity score function examines each edge in a synthesis subtopology G_S a logical connectivity G_L . From the edges, two vectors $v_L, v_S \in R^{k(k-1)/2}$ are computed. The vector v_S has a 1 at each location that corresponds to a present edge in G_S . Element i of v_L contains the weight w associated with edge i in G_L . The normalized inner product

$$\text{similarity}(v_S, v_L) = \frac{v_S^T v_L}{\sum_i v_L(i)} \quad (5)$$

quantifies the similarity between graphs G_S and G_L .

We also observed that using more restrictive synthesis subtopologies typically resulted in fewer routing operations in the final mapped and optimized circuits. Furthermore, selecting subtopologies embedded in the underlying qubit topologies also resulted in fewer total routing operations. These results are illustrated in Figure 11.

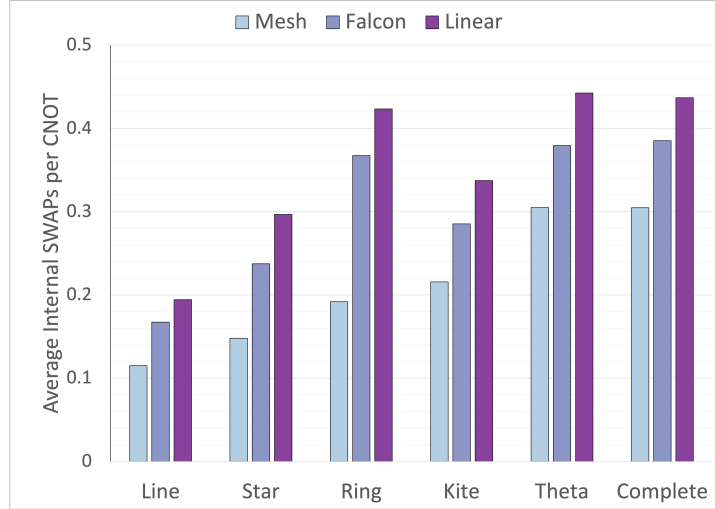


Figure 11: Simpler synthesis subtopologies result in fewer routing operations. Synthesis subtopologies embedded in the underlying qubit topology also result in fewer average routing operations. Internal SWAPs are intra-partition SWAPs.

So, the process of synthesis favors more densely connected synthesis subtopologies, as they are more similar to the logical connectivities of the circuits being synthesized, but the mapping algorithm performs better when more restrictive subtopologies are used. We devised a subtopology selection algorithm that combines and balances these two opposing demands. First, the set of allowed synthesis subtopologies is limited to those that are embedded in the underlying qubit topology. Then, the most restrictive of these subtopologies with the highest similarity score is used.

This subtopology selection policy allowed for TopAS to reach gate count reductions similar too, or just below those of QGo. This policy was able to reduce the number of intra-partition SWAPs, but something had to be done to reduce the number of inter-partition SWAP operations. The development of a *neighbor-aware* subtopology selection scheme was devised for this reason. Although partitioned subcircuits are synthesized independently, they are not executed in isolation. If a given partition can peek into a neighboring block to see what edges it uses, then the likelihood that a SWAP is inserted between blocks can be decreased by using edges that appear in both blocks. To accommodate this change, the weighted logical connectivity graph for each partition was modified to include shared edges in neighboring partitions. The inclusion of neighbor-awareness was the last piece of TopAS’ pre-mapping synthesis flow (see Figure 12). The subtopology selection algorithm is detailed in Algorithm 1.

Figures 13 and 14 show the CNOT gate count reduction and depth reduction of TopAS optimized circuits compared to Qiskit, Pytket, and QGo. Benchmarks

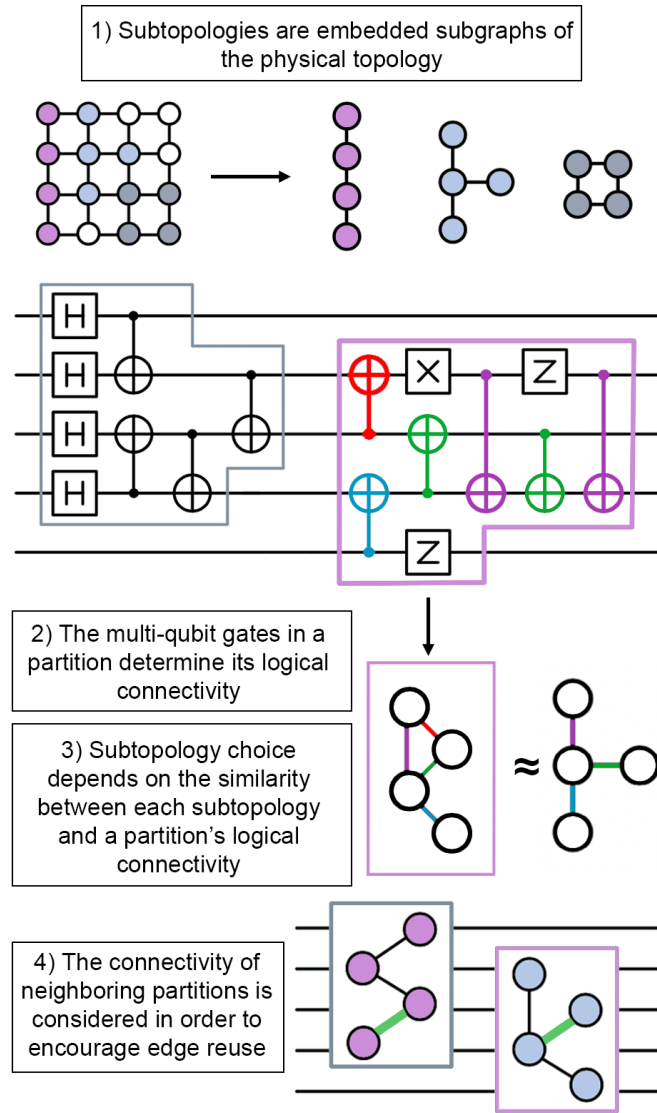


Figure 12: A high level view of the TopAS style implementation of pre-mapping synthesis optimization.

Algorithm 1 Subtopology Selection

Input: graph G_L , set of graphs $G_{neighbors}$, $bias$ function

Output: graph G_S

```
for all graphs  $G_n$  in  $G_{neighbors}$  do
  if  $shared\_edge$  in  $G_n$  and in  $G_L$  then
     $G_{overlap} \leftarrow shared\_edge$ 
  end if
end for
 $v_L \leftarrow$  edge weight vector for  $G_{overlap} \cup G_L$ 
 $candidates \leftarrow$  order  $\leq 4$  subgraphs in physical topology
for all graphs  $G_P$  in  $candidates$  do
   $v_P \leftarrow$  indicator vector for  $G_P$ 
   $score = \text{similarity}(v_P, v_L)$ 
  if  $bias(G_P) \times score > high\_score$  then
     $high\_score \leftarrow bias(G_P) \times score$ 
     $G_S \leftarrow G_P$ 
  end if
end for
return  $G_S$ 
```

are labeled so that the number following the benchmark name indicates the number of qubits in the circuit. The CNOT gate count and depth results reported are normalized by the values obtained for circuits optimized then mapped with Qiskit’s SABRE SWAP mapping algorithm.

TopAS is able to, on average, reduce CNOT count compared to Qiskit by 35.9% and Pytket by 24.3% when targeting a mesh qubit topology. For the falcon/heavy hexagonal qubit topology, TopAS outperforms Qiskit by 33.4% and Pytket by 19.1% on average. TopAS shows even greater reductions in circuit depth, averaging 38.6% and 37.6% shallower circuit compared to Qiskit and Pytket across all benchmarks.

TopAS is able to produce circuits with lower CNOT counts and depth than QGo. On average, TopAS outperforms QGo by 11.5% in CNOT count and 34.3% in depth when compiling to a mesh qubit topology. For the heavy hexagonal topology, TopAS does as well as QGo (only an average of 0.8% better) in CNOT count, but maintains an advantage of 37.3% in average depth reduction.

The TopAS project highlights an interesting fact: unitary synthesis is still able to outperform rule based optimization flows such as Qiskit and Pytket, even when a global view of the unitary being optimized is too large to do direct synthesis on. Synthesizing partitioned circuits may not produce the same results as direct, complete, synthesis, but by operating in a bottom-up manner, results are still impressive.

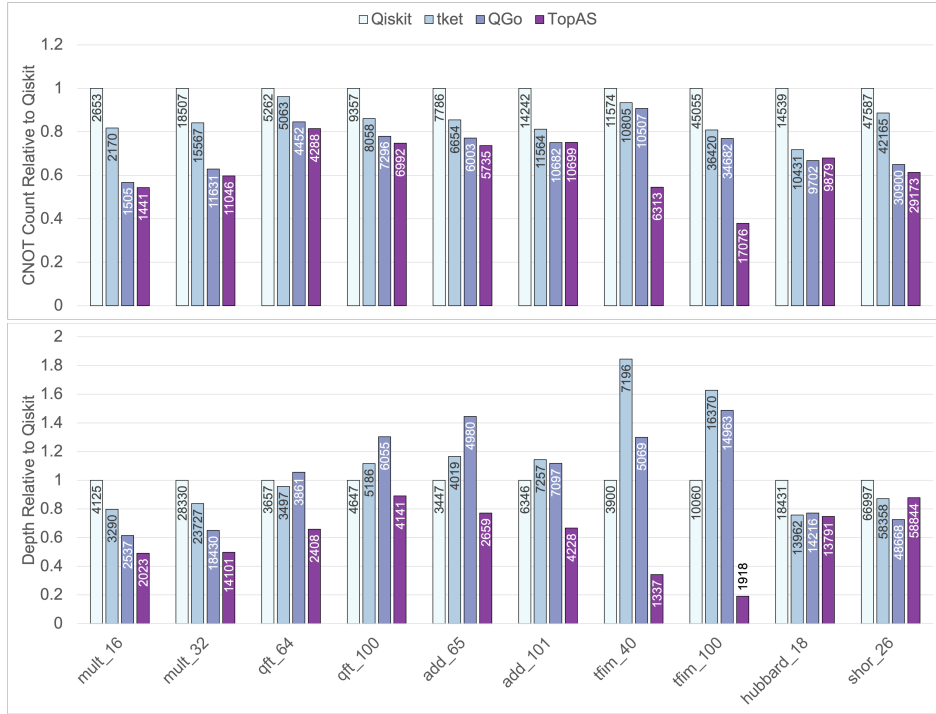


Figure 13: Comparison of relative CNOT count (top) and depth (bottom) for circuits mapped to the mesh qubit topology. CNOT count and depth is shown relative to optimized circuits mapped using Qiskit’s SABRE Swap algorithm. Both lower depth and CNOT count correspond to improved output fidelity on noisy machines.



Figure 14: Comparison of relative CNOT count (top) and depth (bottom) for circuits mapped to the IBM style falcon/heavy hexagonal qubit topology. CNOT count and depth is shown relative to optimized circuits mapped using Qiskit’s SABRE Swap algorithm. Both lower depth and CNOT count correspond to improved output fidelity on noisy machines.

3 Using Machine Learning for Circuit Optimization

In this section, I will justify the use of machine learning in circuit optimization and present recent work in this field. I will discuss my findings relating to the dimensionality of unitary datasets of interest. Finally, I will discuss QSeed, a seeded synthesis algorithm that leverages machine learning to accelerate unitary synthesis optimization.

3.1 Related Work in Reinforcement Learning

Reinforcement Learning (RL), is a machine learning paradigm where agents learn policies based off rewards received from interacting with an environment. In classical compiling, RL is becoming a popular technique for discovering optimal policies for applying peephole optimizations. The CompilerGym and CompilerZoo tools both provide methods of training RL agents to guide compilation flows in order to minimize the number of machine instructions in programs [8]. RL gyms for quantum compilation do not exist, but they almost certainly will in the future, as this style of optimization has proven to be very powerful in the classical compilation space.

The work of [12] demonstrated how RL can be used to learn good peephole style replacement rules for randomly generated circuits. To my knowledge, this is the first project to have applied machine learning to quantum circuit optimization. I believe that RL policies trained to perform peephole optimization may ultimately become the de facto method of performing circuit optimization. However, I believe a weakness of this particular project is that these agents were trained to discover optimizations in random circuits. As I will show, random circuits, especially random unitaries, do not necessarily correspond to cases that we can or desire to optimize.

3.2 Dimensionality of Unitary Datasets

The manifold hypothesis is a conjecture of deep learning that posits that neural networks operate by learning low dimensional manifolds in high dimensional spaces. Although far from the first to propose the manifold hypothesis, the authors of [24] showed how a digit that is rotated, translated, or stretched still lies along the same manifold. Actually learning the shape of these manifolds is hard, but we can begin to identify some of their properties by looking at high level linear approximations of data. The Principle Component Analysis (PCA) specifically is a widely used technique for doing this [17]. PCA changes the basis of a set of data so that the new directions, or Principle Components (PCs), point in the directions of maximum variance in the dataset. When the dataset can be largely explained by a few PCs, neural networks typically perform well, as patterns of interest in the data are easier to learn in these cases [14].

For a fixed number of qubits, the space of unitaries is large. To present support for this claim, we can conduct an experiment to measure the dimensionality

of two sets of unitaries. First we prepare a set of random unitaries generated by PQC. These unitaries are generated by taking one of twelve candidate PQCs with parameters $\theta \in \mathbb{R}^m$, and uniformly sampling parameters $\theta_i \sim U[-\pi, \pi]$. Each of the twelve PQCs generate 10,000 unitaries. Next, we prepare a dataset that consists of about 203,000 unitaries taken from partitioned subcircuits. For each unitary, random and partitioned, we extract a *Pauli coefficient vector*.

Definition 3.1 (Pauli Coefficient Vector). Given a unitary $U \in \mathbb{C}^{2^n \times 2^n}$, there exists a Hermitian matrix $H \in \mathbb{C}^{2^n \times 2^n}$ such that

$$U = e^{iH}. \quad (6)$$

This matrix H can be decomposed into the linear combinations of all 4^n Pauli matrices. Concretely, we can construct a vector of Pauli matrices

$$\sigma_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \sigma_2 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \sigma_3 = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \sigma_4 = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

$$\vec{\sigma} = [\sigma_1 \otimes \sigma_1 \cdots \otimes \sigma_1 \quad \sigma_2 \otimes \sigma_1 \cdots \otimes \sigma_1 \quad \dots \quad \sigma_4 \otimes \sigma_4 \cdots \otimes \sigma_4]$$

such that,

$$H = \alpha \cdot \vec{\sigma} \quad (7)$$

where $\alpha \in \mathbb{R}^{4^n}$ and $\alpha_i \in (-\pi, \pi]$. The vector α is called the Pauli coefficient vector. The Pauli coefficient vector is a convenient way to represent matrices as representing a unitary matrix requires 4^n *complex* values, while the Pauli coefficient vector requires only 4^n *real* values.

After the Pauli coefficient vectors have been collected for each unitary, we can do two PCAs; one for the random and one for the partitioned unitaries. Figure 15 shows the cumulative variance explained as a function of the number of principle components considered. Because a small number of PCs describe the majority of the variance in the partitioned dataset (the first 16 explain 72.2%, the first 32 explain 90.7%), we expect patterns of interest to be learnable by a neural network. The random dataset's variance is spread even across its PCs. We would not expect that learning patterns of interest in this dataset is easy to do.

A natural way to think of what a PCA shows is how compressible a dataset is. For the partitioned unitary case, we could project our dataset into the space spanned by the first 32 principle components, then reconstruct the original dataset with 90.7% accuracy. To achieve this same level of accuracy on the random unitary dataset, the first 57 principle component directions must be kept. In this sense, the random unitary data is much higher dimensional than the partitioned unitary case.

This analysis shines a light on an important insight, random unitaries are very different from unitaries that come from circuits that do real computation. Random circuits (that implement random unitaries) are not representative of what we expect to see in real benchmarks.

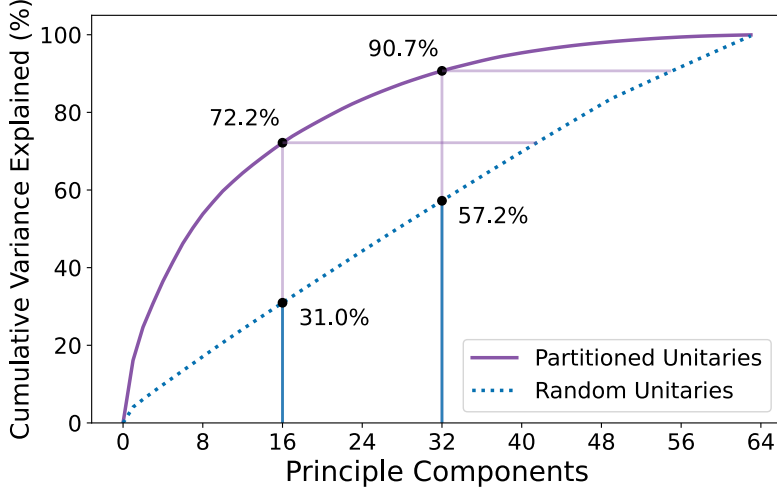


Figure 15: Cumulative variance explained by Principle Components of two datasets of three qubit unitaries. One is randomly generated, the other is made of partitioned unitaries from circuits in Table 1. The PCA of the partitioned unitaries implies these unitaries lie on a low dimensional manifold; there are patterns that can be identified in this dataset. The random unitaries do not exhibit identifiable low dimensional patterns. Learning on random unitaries is difficult, learning on partitioned unitaries is easier.

3.3 Learning a Mapping from Unitaries to Quantum Circuits

Now that I have established that we expect to be able to learn patterns of interest in partitioned unitaries, what can we do with this information?

Imagine that we have a series of n qubits, and a set of unitaries that these n qubits can implement $\mathcal{U}(2^n)$. We can also imagine a set $\mathcal{PQC}(n, K)$, which represents all PQCs with n qubits and K or fewer CNOT gates. We want to find a mapping

$$\phi : \mathcal{U}(2^n) \longrightarrow \mathcal{PQC}(n, K) \quad (8)$$

such that all $U \in \mathcal{U}(2^n)$ are mapped to the $PQC \in \mathcal{PQC}(n, K)$ that implements U with the fewest CNOTs. We assume that the set $\mathcal{PQC}(n, K)$ is large enough so that some PQC implements U with error at most ϵ .

Why might learning this mapping be useful? Because if we can, then we can replace the expensive synthesis tree search process described in Section 2.3.1 with much less expensive ML inference! Before this is possible, there are several things that must be addressed.

3.3.1 Expressivity of Parameterized Quantum Circuits

Much work has been done to quantify the expressivity of parameterized quantum circuits [16]. Many unitaries can be implemented by any given PQC, and many PQCs can implement any given unitary. It is important to note this because it makes the problem of finding the mapping $\phi : \mathcal{U}(2^n) \rightarrow \mathcal{PQC}(n, K)$ difficult. This explains why the PCA of random unitaries (Figure 15) shows that the variance is spread evenly across the principle components.

Another added difficulty of working with such expressive circuits is that it makes the problem of instantiation (Equation (3)) more difficult. To explain how we know this, we can again return to the example of the random unitary dataset. Here, we directly generate unitaries from a given PQC by randomly sampling parameters. But, if we have the unitary and the PQC, asking the instantiation function to solve for the parameters of that circuit that yield that unitary often fails. This means that even when we know the correct mapping of random unitaries to the PQCs that generated them, it is difficult to verify that this mapping is correct.

3.3.2 Canonical Representation of Unitaries

If two unitaries differ only by a global phase, states transformed by the two unitaries will always yield the same distribution of measurement results [19]. This means that for all intents and purposes, unitaries that differ only by a global phase are the same. However, if we want to apply numerical methods to unitaries, it is important that there is a common, canonical way of representing unitaries. Formally, we need a function f such that for unitaries U, U^* ,

$$\forall \theta \in \mathbb{R} \quad U = e^{i\theta} U^* \implies f(U) = U^*.$$

If $f(U) = U^*$ then the matrix U^* is the canonical representation of U . The algorithm for computing U^* is given by Algorithm 2. Performing these steps

Algorithm 2 Canonical Unitary Transformation

Input: unitary $U \in \mathbb{C}^{N \times N}$,

Output: unitary $U^* \in \mathbb{C}^{N \times N}$

Find $e^{i\theta} = |U|$

Compute $V = e^{i\theta} U = e^{i2\pi k/N} U^* \in \mathbb{SU}(N)$

$e^{i2\pi k/N}$ is the complex phase of first non-zero element in V

$U^* = e^{-i2\pi k/N} V$

return U^*

ensures that each unitary is mapped to a canonical special unitary. To my knowledge, this is the first algorithm that finds a canonical form for unitaries.

Previous works stop at the special unitary conversion step. Just doing this part is not enough to build global phase resilience into methods that do numerical computation on unitaries. I noticed by looking at many random examples of

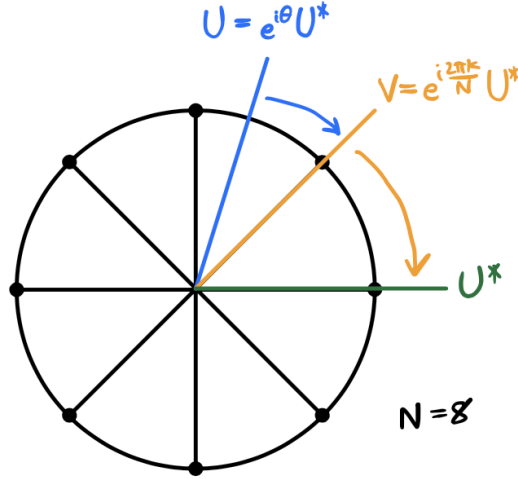


Figure 16: An illustration of the two steps in the canonical unitary transformation process.

this process, that for an $N \times N$ unitary, there were N different special unitaries that appeared. This lead me to notice that converting to a special unitary only removes the small phase differences between the different roots of unity (the blue arrow in Figure 16). In order to find a real canonical form, we need to always go to the same root of unity. A unitary will always have at least one non-zero element in each row and column. If we ensure that the first non-zero element of the first row has zero complex phase, we will always arrive at the same root of unity, no matter which root the special unitary conversion step took us to (orange arrow in Figure 16).

Even though this process is simple, I believe that it is a pretty significant tool to have. Unitary synthesis algorithms, and other numerical methods that work on unitary matrices, must choose cost functions that are resilient to global phase, otherwise intricacies of the numerical optimization process may skip over valid solutions to problems. Having a canonical representation is therefore significant because it broadens the scope of what can be done with unitary matrices. Cost functions need not be phase resilient anymore.

3.4 QSeed

Equipped with a canonical representation of unitary matrices and the knowledge that patterns of interest could be learned for partitioned unitary matrices, I began work on using machine learning to learn a mapping between unitaries and the PQCs that implement them. This work culminated in an algorithm called QSeed, a seeded synthesis algorithm that uses ML to predict seed circuits from which to begin synthesis. QSeed is able to achieve dramatic speedups compared

to QSearch because it decreases the time spent searching the synthesis circuit tree.

3.4.1 Why use ML for this task?

Before detailing the development behind QSeed, first I want to address why using ML might be advantageous for this task. If we want to learn a mapping from unitaries to PQCs, could we instead just observe many instances of unitaries that can implement PQCs and memorize those results? This approach would construct a database of previously seen unitary-PQC pairs that we could quickly interrogate. The problem with this approach is that it only allows us to do this mapping for unitary-PQC pairs that have already been observed.

Why might ML work any better than this database? A learned mapping is more robust than this database approach because the point of the learned mapping is that it can generalize to cases that have not yet been seen. This is useful because the number of possible unitaries we can see is vast (even when limiting ourselves to unitaries from partitioned circuits), and surely we cannot capture everything we might expect to see in a training dataset.

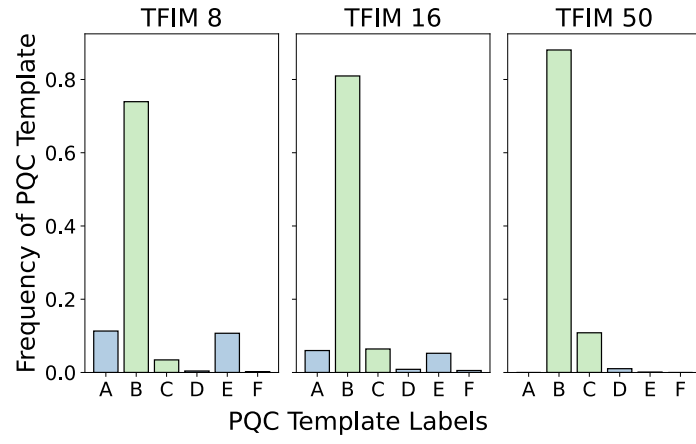
Why might we expect that learning this mapping is possible? We expect this is possible because there are common patterns that appear in similar quantum circuits. When synthesizing generated circuits from the same “family” but of different widths, the distribution of PQC templates that appear has similarities. This is illustrated in Figure 17. This figure shows how synthesized TFIM circuits of various widths all have similar looking distributions of used PQC templates, which are also illustrated.

Furthermore, circuits not from the same family, but that are still in some sense related, share common structure as well. Figure 18 shows how both QFT and Shor style modular exponentiation circuits have distributions of selected PQCs that look similar. The modular exponentiation (Shor) distribution look like the QFT distribution, with a extra mode around the template labeled “P”. The existence of these commonalities means that there is in fact some underlying structure or pattern to the mapping of unitary to PQCs. Now we can rest assured that there are in fact patterns of interest in our data that we can train a model to recognize.

3.4.2 Seeded Synthesis

If this mapping between unitaries and PQCs is learned, then the synthesis circuit tree search process could be almost entirely replaced. In actuality, it is unlikely that this mapping will be learned perfectly, so instead of immediately using the first PQC proposed, we simply begin synthesis from that circuit.

Definition 3.2 (Seeded Synthesis). Seeded synthesis accepts three inputs, a target unitary U , a qubit topology $G = (V, E)$, and a seed circuit C_{seed} . Instead of beginning the synthesis tree search process at the root node of the tree (which contains only single-qubit gates), synthesis begins at the circuit C_{seed} . CNOT



(a) PQC templates that appear in synthesized TFIM circuits of various widths. Circuits of different widths share characteristic modes around PQC's B and C.

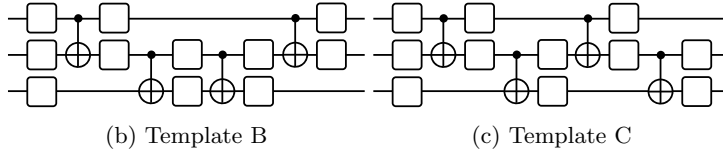
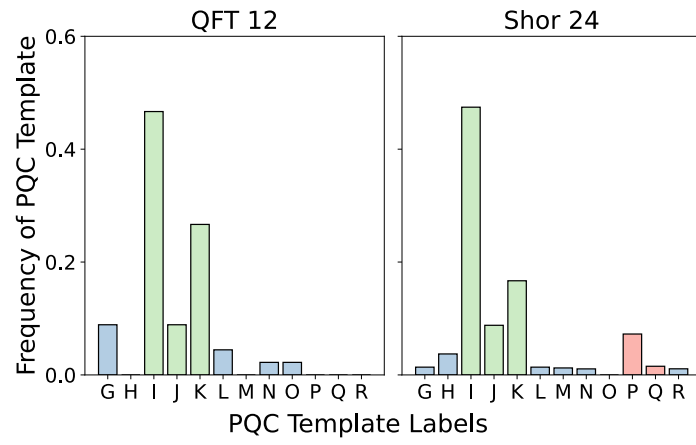
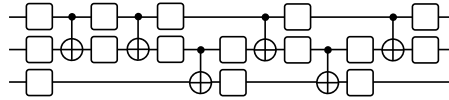


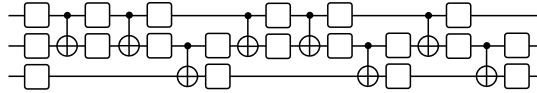
Figure 17: Histograms and examples of commonly repeated PQC templates that appear in optimized TFIM circuits.



(a) PQC templates that appear in synthesized QFT and Shor modular exponentiation circuits.



(b) Template I



(c) Template P

Figure 18: Histograms and examples of commonly repeated PQC templates that appear in optimized QFT and modular exponentiation circuits.

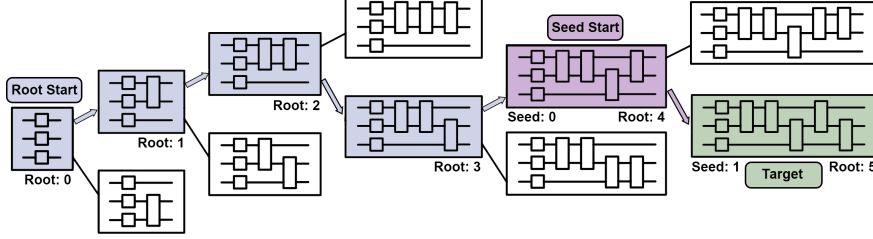


Figure 19: Seeded synthesis allows for the synthesis tree search process to begin at circuits closer to the target circuit node.

gates can now be both added and removed, which means that the tree search progresses both towards leaf nodes and the root node.

The purpose of the learned mapping between unitaries and PQCs is now simply to propose seed circuits from which the synthesis process can begin. Figure 19 shows why seeded synthesis can provide speedups compared to normal synthesis. QSearch always begins synthesis at the node marked “Root Start.” In this case, the path between the root and the “Target” has distance five. This means if the shortest path is taken, then at least six instantiation calls must be made. If we instead begin synthesis at the node marked “Seed Start,” because the path to the target has distance one, only two instantiation calls must be made. This example illustrates why we might expect to see large speedups in a seeded synthesis algorithm compared to QSearch.

3.4.3 Dataset Preparation

I took a supervised learning approach to learn the mapping in Equation (8). After collecting unitaries, computing their canonical forms (Section 3.3.2), and extracting their Pauli coefficient vectors (Definition 3.1), these datapoints still lacked labels. I approached this problem by enumerating all PQC templates with three qubits and up to eight CNOTs, then synthesizing each unitary. Each unitary was then paired with the integer label that corresponded to the PQC that implemented it. Because some unitaries appeared multiple times and some PQCs were more frequent than others, I normalized this dataset by removing repeated unitaries and randomly discarding examples from over represented PQCs. This ensured that whatever model I trained would not simply learn that some PQCs were more prevalent than others, but actually learn to propose PQCs based off the unitaries themselves.

The process of producing these labels is synthesis, and therefore labeling these unitaries is expensive.

3.4.4 The Recommender Model

A recommender is any function (learned or not) that proposes seed circuits for seeded synthesis. I chose to implement a recommender model with a neural network. More specifically, I chose an autoencoder style architecture. The results of the PCA of partitioned unitaries in Figure 15 motivated this choice. A purely linear projection of only half the total dimensionality of the original space captured the vast majority of the variance in the partitioned unitary dataset. I therefore believed that the underlying structure of these unitaries was simple, and that compressing their representation could add a degree of robustness to noise [14]. As the process of labeling the data for this task was expensive, I also wanted to ensure I could use unsupervised techniques to pre-train the model. Autoencoders can easily be pre-trained on the task of input reconstruction, so the recommender model was pre-trained to reconstruct input unitaries.

The output of the recommender model is a large vector where element i corresponds to how likely the model believes the PQC with label i is able to implement the input unitary. There were a total of 1199 possible output PQCs to select. To state this concretely, the output of the model is a vector $y \in \mathbb{R}^{1199}$, where

$$\text{softmax}(y_i) \in [0, 1] \quad (9)$$

represents the model’s confidence that PQC i can implement the input.

As this was meant primarily as a proof of concept, I elected to limit the width of unitaries and circuits to three qubits. I began by training a model which took as input the Pauli coefficient vector associated with each unitary, which is a vector in $\mathbb{R}^{64}$. Later on, I discovered that the process of converting unitaries into Pauli coefficient vectors was extremely expensive, and so the architecture was changed to instead operate directly on the canonical form of unitaries. As elements of the unitaries are complex, each element of the input unitary was split into real and imaginary parts. These new vectors $x \in \mathbb{R}^{128}$ were used as the new input.

The training and test datasets consisted of circuits from twelve different families of generated circuits. Each family contained multiple instances of that algorithm implemented at various circuit widths. The training and test data splits are shown in Table 1. Note that no Grover’s algorithm circuits of any size appear in the training dataset. The whole flow of the QSeed algorithm is depicted in Figure 20.

3.4.5 Comparing QSeed to QSearch

Recall that the goal of the QSeed algorithm is to accelerate unitary synthesis by avoiding spending too much time traversing the synthesis circuit tree. We can measure this by directly measuring the time it takes to synthesize circuits with both QSearch and QSeed. Importantly, we do not want to degrade the quality of our results for the sake of speed. Therefore, we must also ensure that

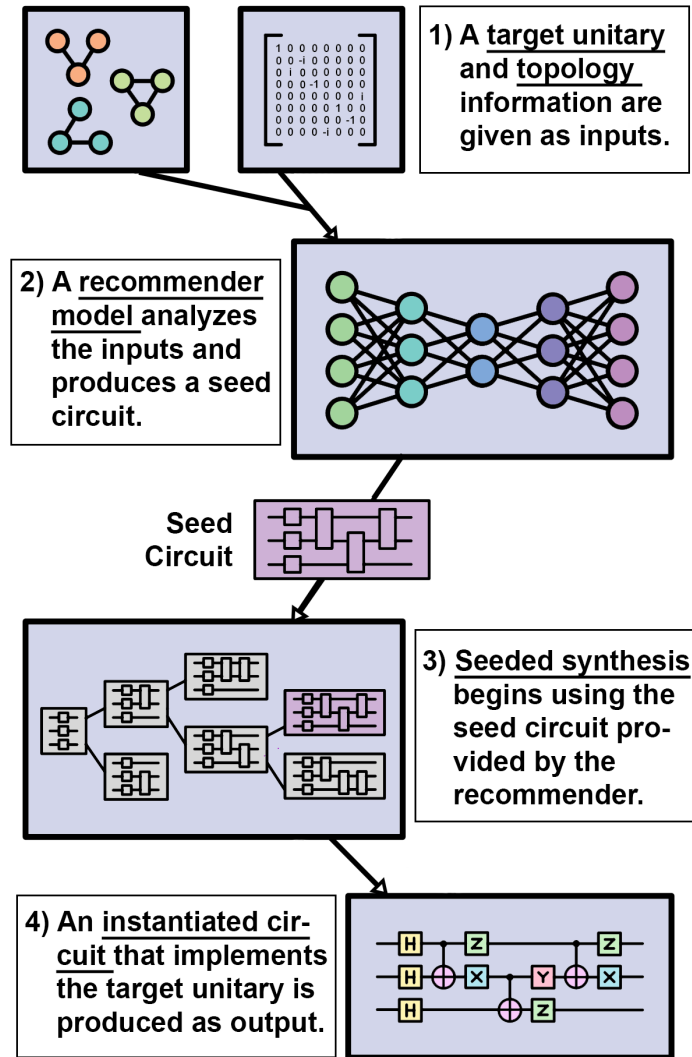


Figure 20: A high level depiction of the QSeed algorithm.

Circuit Name	Training Widths	Test Widths
add	17, 65	41
grover	-	10
heisenberg	4, 6, 7, 8, 16, 32, 64	5
hhl	8	6
hubbard	4, 18, 50	8
mult	8, 32, 64	16
qae	11, 33, 101	65
qft	3, 4, 8, 16, 32	64
qml	4, 25, 60, 108	128
qpe	6, 10, 14	18
shor	16, 32	64
tfim	3, 4, 5, 6, 7, 8, 16, 32,	64
vqe	12, 14	18

Table 1: Split of circuits withheld from recommender training for testing.

the number of CNOTs in the QSeed synthesized circuits is close to the number of CNOTs in the QSearch synthesized circuits.

Figure 21 summarizes the results of experiments run to compare the time to- and quality of- solutions for circuits optimized with QSearch and QSeed. Each of these circuits is first mapped with Qiskit’s SABRE SWAP mapping algorithm, then optimized with QGo style post-mapping synthesis (Figure 8). QSeed achieves an average speedup of $2.4\times$ compared to QSearch, while only suffering an average of 1.2% increase in CNOT count. Note that these test circuits contain unitaries not included in the training of the recommender model. The 64 qubit modular exponentiation circuit, a core component of Shor’s algorithm, has a speedup of $3.7\times$ (nearly 8000 seconds). The 18 qubit VQE circuit achieves a speedup of $2.3\times$. This particular benchmark is worth noting as classical-quantum hybrid algorithms have strict compilation time demands [20]. It is also worth noting that QSeed’s synthesis times include the time required to do inference/make seed recommendations.

Most impressive are the results for the compilation of the 10 qubit Grover’s algorithm circuit. No unitaries for the family of Grover’s algorithm circuits were included in training, so this benchmark is entirely alien to the recommender model. Still, a speedup of $3.2\times$ is achieved, and there are actually fewer CNOTs than in the QSearch optimized version.

3.4.6 Evaluating the Recommender Model

The results presented in Figure 21 demonstrate that using the recommender model and seeded synthesis provides substantial speedups over QSearch. However, one can imagine other recommender schemes. To ensure that our recommender model is performing as expected, we can do a deeper analysis of what is happening each time a partition is synthesized. To do this, we can observe the

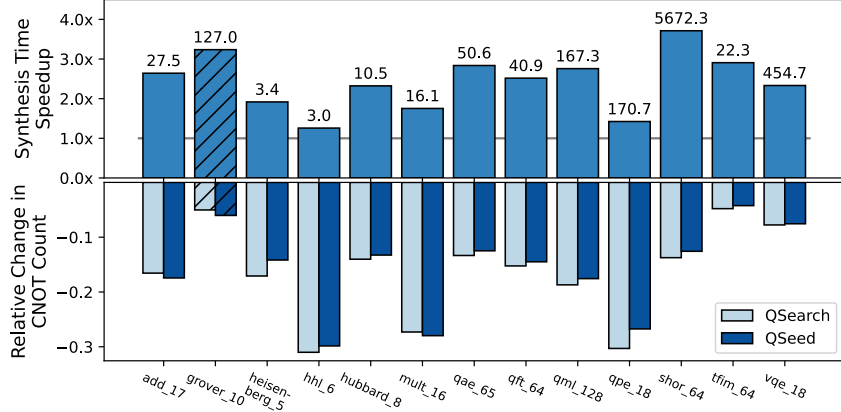


Figure 21: Comparing QSeed and QSearch. The top graph wall-clock synthesis time speedups. Larger speedups are better. The bottom graph shows relative changes in CNOT gate count relative to Qiskit level 3 optimized circuits. Lower relative change in CNOT counts are better.

number of instantiation calls made and the number of CNOTs in the optimized version of each block.

Recall that the number of instantiation calls is a proxy of how long synthesis will take, because each circuit along a path through the synthesis circuit tree must be instantiated. Figure 22 shows the distribution of the number of instantiation calls for the QSeed, randomly seeded, and QSearch approaches. The randomly seeded scheme simply randomly selects one of the 1199 possible seeds. QSearch acts like a seeded synthesis algorithm that always proposes the root as a seed. QSeed’s recommender scheme is able to propose seeds that require only a single instantiation call almost 90% of the time. The randomly seed case also typically requires very few instantiation calls. By measuring the expected number of instantiation calls, we can see that on average, QSeed and the randomly seeded scheme require almost $6\times$ fewer instantiations on average. This is why QSeed is able to synthesize unitaries faster than QSearch.

Figure 23 shows the number of CNOTs in the optimized blocks divided by the number of CNOTs in the original blocks. A value of 1 here indicates that no reduction in CNOTs has occurred, whereas a value less than 1 indicates the optimized subcircuit has fewer CNOTs. Lower values are better. Values higher than 1 indicate growth, which is undesirable. The expected reduction for QSeed closely matches that of QSearch. This is why we can expect that QSeed produces circuits with a similar number of CNOTs as QSearch. The randomly seeded scheme may offer speedups, but it averages an increase of 19% in CNOTs.

The randomly seeded results for these two experiments indicate that a randomly proposed PQC is likely to be capable of implementing a unitary, but

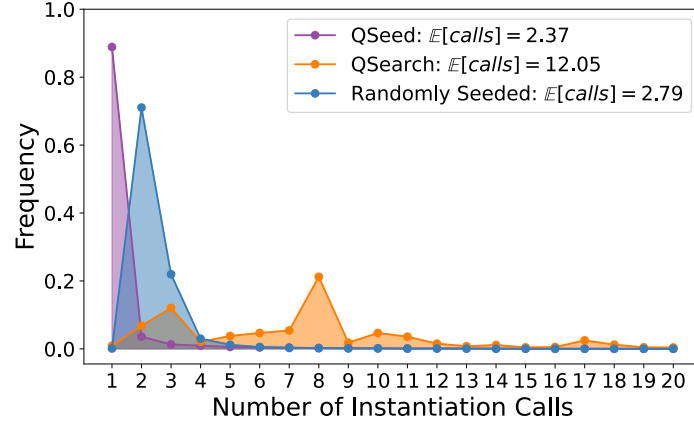


Figure 22: Distribution of instantiation calls made for various recommender schemes. QSeed uses the autoencoder style recommender model while the Randomly Seeded strategy randomly picks a seed circuit. Both are faster than QSearch as seen by the lower expected number of instantiation calls. The X-axis only shows up to 20 instantiation calls, but QSearch requires as many as 1704.

it is likely to be far deeper than is necessary. These experiments also show why QSeed is able to achieve speedups over QSearch while maintaining solution quality.

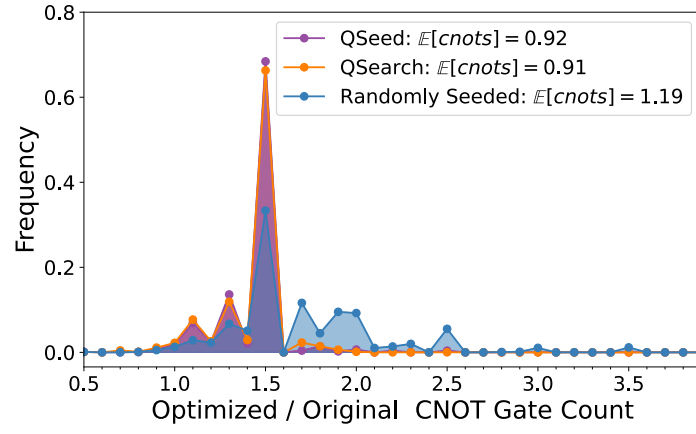


Figure 23: Distribution of relative CNOT gate counts for various recommender schemes. A value less than one corresponds to there being fewer gates in optimized block compared to the original block. Lower is better. The relative CNOT counts of QSeed closely match those of QSearch. The Randomly Seeded scheme results in circuit growth, as seen by the expected value being larger than 1.

4 Discussion and Future Directions

In this section, I will offer further discussion and describe future research directions that could further leverage machine learning for quantum circuit optimization.

4.1 Increasing Unitary Widths

Although promising, the results presented in Section 3 are limited to three qubit unitaries. A PCA of partitioned unitaries with four and five qubit widths show similar structure; therefore we expect that learning is still possible in these cases. Increasing the width of the unitaries operated on by the QSeed algorithm is limited by the lack of a labeled dataset of larger unitaries. Compiling such a dataset for the three qubit case was possible, but synthesis is much slower in these wider cases, so labeling becomes much more difficult.

When the width of the unitaries in the QSeed algorithm increase, so does the number of possible output PQC. This is a problem because it means the enumeration process used to exhaustively label PQCs is no longer viable. Luckily, it appears that not all PQCs are equally important. In the dataset of 203,000 partitioned unitaries. Only 415 appear more than 10 times. Of these, only 220 appear 100 or more times. I therefore expect that the number of PQCs that are widely used in the larger width cases to be similarly small.

The results presented in Figure 22 show that proposing any seed in the three qubit case often works, though not optimally. This is because the number of circuits capable of implementing a given unitary is high; there are many possible solution circuits. It seems that as widths increase, the complexity of both unitaries and circuits also increases substantially. This means that solution circuits become more sparse, and this random seed strategy is expected to perform far worse. This also means that a good recommender model is likely capable of demonstrating even larger speedups in these wider unitary cases.

4.2 Network Architectures for Learning on Unitaries

The selection of an autoencoder style neural network for the recommender model was motivated by the PCA results of Figure 15. There are likely architectures that are better suited to this task. This is especially true for cases where wider unitaries are considered, as the number of input parameters grows by 4 with the addition of each new qubit.

Convolutional Neural Networks (CNNs) are popular in computer vision tasks as they are translationally equivariant and particularly adept at working with data which exhibits spatial locality. CNNs are likely not well suited to the task of learning on unitaries, as the placement of a gate on the first of last qubit is in some sense very similar, but results in very differently structured unitaries.

Graph Neural Networks (GNNs) are a generalization of CNNs that have gained much popularity recently. It could be possible that using a GNN to learn a dense representation of a quantum circuit expressed as a Directed Acyclic

Graph could address the issues associated with the number of input parameters scaling with $O(4^n)$. Because unitaries of interest (non-random unitaries) are lower dimensional than their general counterparts (which use all 4^n parameters), it is worth investigating if one can learn a dense representation directly from a quantum circuit, without explicitly computing the full unitary.

4.3 Further Exploring the use of Reinforcement Learning

The work presented in [12] show that reinforcement learning can be used to learn good peephole style optimization techniques. I am interested to see whether these results can be further improved by exposing the agent exclusively to non-randomly generated circuits. I expect that this was not done originally because the number of non-random quantum circuits that do useful computation is rather limited.

I also wonder if a reinforcement learning agent can be trained to learn good synthesis tree search strategies. Paired with seeded synthesis, this could provide additional benefits over QSearch’s static A^* heuristic approach.

4.4 Learning to Incorporate Noise Resilience

How machine learning can be used to incorporate noise resilience is a research direction that also requires more attention. In [9], ML is used to learn a noise model for a machine. By executing a circuit consisting of only Clifford gates, and simulating this same circuit (which can be done efficiently as all gates are Cliffords), deviations in the expected and actual outcomes offer the necessary signals to train a neural network. I believe it is worth exploring other methods of predicting the amount of noise in or output fidelity of circuits executed on hardware.

I believe the role of machine learning in quantum circuit optimization, and quantum compilation as a whole, will surely grow over time.

References

- [1] D. Aharonov and M. Ben-Or, “Fault-tolerant quantum computation with constant error rate,” 1999.
- [2] G. Aleksandrowicz, T. Alexander, P. Barkoutsos, L. Bello, Y. Ben-Haim, D. Bucher, F. Cabrera-Hernández, J. Carballo-Franquis, A. Chen, C. Chen *et al.*, “Qiskit: An open-source framework for quantum computing,” *Accessed on: Mar*, vol. 16, 2019.
- [3] F. Arute, K. Arya, R. Babbush, D. Bacon, J. C. Bardin, R. Barends, R. Biswas, S. Boixo, F. G. Brandao, D. A. Buell *et al.*, “Quantum supremacy using a programmable superconducting processor,” *Nature*, vol. 574, no. 7779, pp. 505–510, 2019.



Improving Quantum Computation by Optimized Qubit Routing

Friedrich Wagner¹ · Andreas Bärmann² · Frauke Liers² · Markus Weissenböck¹

Received: 3 June 2022 / Accepted: 13 April 2023 / Published online: 4 May 2023
© The Author(s) 2023

Abstract

In this work we propose a high-quality decomposition approach for qubit routing by swap insertion. This optimization problem arises in the context of compiling quantum algorithms formulated in the circuit model of computation onto specific quantum hardware. Our approach decomposes the routing problem into an allocation subproblem and a set of token swapping problems. This allows us to tackle the allocation part and the token swapping part separately. Extracting the allocation part from the qubit routing model of Nannicini et al. (Optimal qubit assignment and routing via integer programming, 2021, <http://arxiv.org/abs/2106.06446>), we formulate the allocation subproblem as a binary linear program. Herein, we employ a cost function that is a lower bound on the overall routing problem objective. We strengthen the linear relaxation by novel valid inequalities. For the token swapping part we develop an exact branch-and-bound algorithm. In this context, we improve upon known lower bounds on the token swapping problem. Furthermore, we enhance an existing approximation algorithm which runs much faster than the exact approach and typically is able to determine solutions close to the optimum. We present numerical results for the fully integrated allocation and token swapping problem. Obtained solutions may not be globally optimal due to the decomposition and the usage of an approximation algorithm. However, the solutions are obtained fast and are typically close to optimal. In addition, there is a significant reduction in the number of artificial gates and output circuit depth when compared to various state-of-the-art heuristics. Reducing these figures is crucial for minimizing noise when running quantum algorithms on near-term hardware. As a consequence, using the novel decomposition approach leads to compiled algorithms with improved quality. Indeed, when compiled with the novel routing procedure and executed on real hardware, our experimental results for quantum

Communicated by Giacomo Nannicini.

✉ Friedrich Wagner
friedrich.wagner@iis.fraunhofer.de

approximate optimization algorithms show an significant increase in solution quality in comparison to standard routing methods.

Keywords Combinatorial optimization · Integer programming · Approximation algorithms · Quantum compilation

Mathematics Subject Classification 90C27 · 90C10 · 81P68

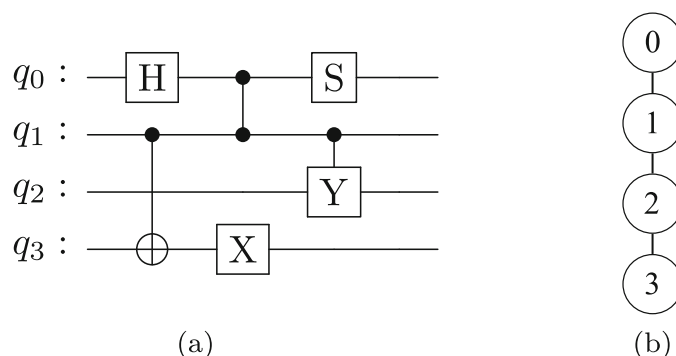
1 Introduction

Qubit routing by swap insertion is a subroutine in the process of compiling quantum algorithms onto specific hardware. Quantum algorithms are usually formulated in the circuit model of quantum computation, see e.g. [19] for an introduction. Such a circuit consists of wires representing qubits as well as gates representing operations applied to subsets of qubits. We refer to the qubits in the circuit as *logical* qubits, in contrast to the *physical* qubits in quantum hardware. Furthermore, we assume that the circuit only contains gates acting on at most two qubits, i.e. single and two-qubit gates (also known as “SU(4) circuits”). This might already be the case for the input circuit to be compiled; otherwise this can be achieved using the Solovay-Kitaev Theorem, see e.g. [19, pp. 188–202]. In the circuit model, gates may be applied on any pair of logical qubits. However, this is not the case in many currently available quantum processors. Here, two-qubit gates can only be applied on specific pairs of physical qubits defined by the *hardware connectivity graph*. Now the task is to choose an initial allocation of logical qubits in the quantum circuit to physical qubits in the hardware graph such that for each two-qubit gate the corresponding logical qubits are located at neighbouring physical qubits. The example in Fig. 1 shows that this is not always possible directly.

In such a case, it is necessary to insert additional *swap gates* into the algorithm. They effectively swap the positions of two logical qubits in the hardware graph. The result is an SU(4) circuit meeting the connectivity restrictions which is equivalent to the original one when taking into account the permutation resulting from initial allocation and swap gates. A simple example for this procedure is shown in Fig. 2.

When solving the routing by swap insertion problem, two possible objective functions arise naturally: either one aims to minimize the execution time on the quantum processor, which can be achieved by minimizing the output circuit depth, or one can

Fig. 1 Example for a quantum circuit (a) consisting of four logical qubits as well as single and two-qubit gates. In the context of this work, the meaning of the gates is irrelevant. Only the subset of qubits involved in a gate matters. The circuit is not compatible with the hardware connectivity graph shown in (b)



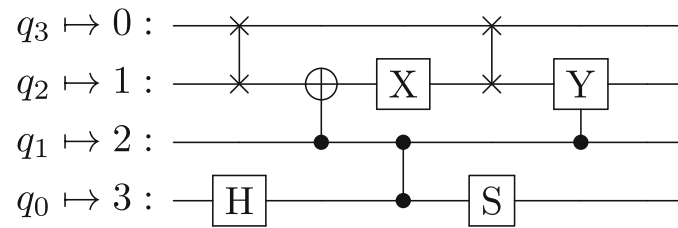


Fig. 2 Example for routing by swap insertion. The given circuit is equivalent to the quantum circuit from Fig. 1a and fulfills the hardware connectivity from Fig. 1b. The equivalence holds under the initial allocation of logical qubits $\{q_0, q_1, q_2, q_3\}$ to physical qubits $\{0, 1, 2, 3\}$ given by $q_0 \mapsto 3, q_1 \mapsto 2, q_2 \mapsto 1, q_3 \mapsto 0$. After the first swap the allocation changes to $q_0 \mapsto 3, q_1 \mapsto 2, q_2 \mapsto 0, q_3 \mapsto 1$, and after the second swap to $q_0 \mapsto 3, q_1 \mapsto 2, q_2 \mapsto 1, q_3 \mapsto 0$

minimize the total gate number to be executed, which amounts to minimizing the number of swap gates added. Intuitively, both objectives are correlated. However, it is easy to construct examples where optimal-depth solutions are suboptimal in terms of gate count and vice versa. For an experimental study of the influence of depth and gate count on the performance of quantum algorithms, we refer to [11, 18].

1.1 Existing Methods for Qubit Routing

Many compilation procedures based on routing by swap insertion have been developed so far. Some of them may be classified by their local working principle: they define an initial mapping of logical to physical qubits, iterate through the circuit in temporal order and change the allocation by adding swaps when gates are encountered which cannot be applied in the current allocation. Of course, this general principle allows for many different sophisticated procedures, in particular when it comes to choosing the initial allocation. Cowtan et al.'s *Tket compiler* [5, 23] belongs to this class. It uses a heuristic cost function to choose the next allocation. Zuhleiner et al. [30] employ an A*-search algorithm to define the next allocation. Li et al. *Sabre compiler* [14] uses the reversibility of quantum circuits to perform a bidirectional search for a good initial mapping. The default compiler in IBM's SDK *Qiskit*, *Stochastic swap*, based on work by Bravi [20], as well as the default compiler in Google's SDK *Circ*, *Greedy router* [2] also belong to this class.

A connection from routing by swap insertion to the problems of token swapping and subgraph isomorphism has already been established and employed by Siraichi et al. in [21, 22] and by Childs et al. in [1]. In their work, the subgraph isomorphism problem arises in the context of searching for high-quality qubit-mappings, whereas routing between mappings is translated to a token swapping problem.

Two approaches to routing by swap insertion that do not rely on local swap insertion but consider the whole circuit instead are the SAT-based approach by Wille et al. [27] and the approach by Nannicini et al. [18] based on an integer programming model. These approaches suffer from exponential growth of problem size and NP-hard problem complexity. In practice, the running times exceed reasonable limits already for moderate circuit sizes.

Furthermore, there are quantum compilation methods in the literature which do not rely on swap insertion: Kissinger and Meijer-Van De Griend consider circuits only

built from CNOT gates and solve the compiling problem by finding an equivalent hardware compatible CNOT circuit [13]. Moro et al. [17] use reinforcement learning for quantum compilation.

Kim [12] compares some of the aforementioned routing heuristics on benchmarks with known optimal solution. His results reveal a significant margin for improvement.

1.2 Our Contribution

We provide a high-quality solution method that efficiently exploits the capabilities of integer programming to solve the routing by swap insertion problem. The key is to decompose the problem into two separate subproblems, of which the first one is solved via integer programming. It determines an allocation sequence that is compatible with the hardware constraints. Herein, we employ a cost function that is a lower bound on the total number of swaps required for routing. This subproblem is called *token allocation problem*. For its solution, we develop a simplification of the binary model introduced by Nannicini et al. in [18], which results in a significant reduction in problem size. Once an optimal allocation sequence has been found, each pair of subsequent allocations defines a token swapping instance. These token swapping problems are solved by an approximation algorithm based on the work of Miltzow et al. in [15]. Also, we develop an exact algorithm for token swapping. By comparison of both solution techniques, we conclude that the approximation algorithm typically delivers high-quality solutions in drastically reduced solution time.

Re-targeting the gates according to the allocation sequence and inserting the swap gates from the token swapping solutions finally yields the compiled circuit. In [21], a similar decomposition approach, called *Bounded mapping tree* (BMT), is proposed which, however, is based on dynamic programming. In contrast to other heuristic methods, we are able to give bounds on the quality of the obtained solution. We perform extensive numerical experiments on several benchmark instances from the literature, covering a broad range of different hardware graphs. The results show that the proposed approach finds close-to-optimal solutions and outperforms well-established heuristics. In comparison to the exact approach from [18], on which the present work is based, our method takes much less time. This makes the proposed routing procedure applicable for practically relevant circuit sizes. Furthermore, our experiments on actual quantum hardware show that quantum computation benefits from the proposed routing method.

1.3 Structure

This work is structured as follows. Section 2 introduces the token allocation problem and the token swapping problem. In Sect. 3, we further analyse the token allocation problem and motivate the use of integer programming for its solution. We show the NP-hardness of the problem at hand, introduce the binary model and derive valid inequalities for its linear relaxation. In Sect. 4 we study solution methods for the token swapping problem. The efficient approximation algorithm employed in our routing procedure as well as the exact branch and bound algorithm used for benchmarking the

former are developed. Section 5 discusses numerical results for the two subproblems of token allocation and token swapping as well as for the entire routing problem. Additionally, we present experimental results for example quantum algorithms executed on actual hardware when routed with different methods. Finally, in Sect. 6 we give a conclusion and indicate directions of further research.

2 Preliminaries and Definitions

We start by formally defining the token allocation problem as well as the token swapping problem. While the latter is well known, the former has not been extensively studied yet, to the authors' best knowledge. It already occurred in [18, 21, 22] as part of a larger problem. In the context of qubit routing, the token allocation problem can be informally described as follows. Its input is a hardware graph representing the connectivity of the physical qubits, together with a quantum algorithm acting on a set of logical qubits ("tokens"). The algorithm is described as a sequence of layers. In each layer, a set of two-qubit gates needs to be performed in parallel. Now the task is to find an allocation for each layer such that the gates can be executed, i.e. logical qubits involved in gates are located at neighbouring physical qubits.

We remark, that grouping gates into layers needs to be performed by the routing procedure, since an algorithm formulated as a circuit is not by itself divided into layers but simply modeled as a sequence of individual gates. Grouping gates into layers containing more than a single gate restricts the set of feasible solutions, since changing allocation between gates in the same layer is not allowed. However, grouping significantly reduces the number of layers and thus the size of the token allocation problem.

For a given allocation sequence, we define its cost as the sum of all distances logical qubits move on the hardware graph between subsequent allocations. Herein, the distance a logical qubit moves between two allocations is defined as the length of a shortest path connecting the vertices in the hardware graph, to which the qubit is allocated. The motivation for this choice of objective is that the number of swaps needed for routing between subsequent allocations $a, a': Q \xrightarrow{1:1} V$ is bounded from below by

$$\frac{1}{2} \sum_{q \in Q} d_H(a(q), a'(q)),$$

where $d_H: V \times V \rightarrow \mathbb{Z}_0^+$ denotes the *edge distance* in H , i.e. the number of edges in a path of minimal length connecting two nodes. A proof of this result on the token swapping problem is found e.g. in [15]. This means, that instead of minimizing the total number of swaps required, we minimize a lower bound on this value which can be computed much more easily.

We now define the token allocation problem formally as a combinatorial optimization problem.

Definition 2.1 (*Token allocation problem*) An instance of the *token allocation problem* (TAP) is given by a triple (H, Q, Γ) , where

- $H = (V, E)$ is a connected undirected graph,

- Q is a set of tokens of size $|Q| = |V|$, and
- $\Gamma = G^1, G^2, \dots, G^L$ is a finite sequence of sets of disjoint token pairs, i.e.
 - (i) $G^t = \{(p_t^1, q_t^1), (p_t^2, q_t^2), \dots, (p_t^{|G^t|}, q_t^{|G^t|})\} \subset Q \times Q$, $q_t^j \neq p_t^j \quad \forall j \in \{0, \dots, |G^t|\}$ and
 - (ii) $\{p_t^i, q_t^i\} \cap \{p_t^j, q_t^j\} = \emptyset \quad \forall i \neq j \in \{0, \dots, |G^t|\}, \forall t \in \{1, \dots, L\}$.

A feasible solution is given by a sequence $a_1, \dots, a_L$ of bijective mappings $a_t: Q \xrightarrow{1:1} V$ such that at each time step t the token pairs in G^t are allocated to neighbouring vertices in H :

$$\forall 1 \leq t \leq L : \forall (q_1, q_2) \in G^t : \{a_t(q_1), a_t(q_2)\} \in E.$$

The cost of a feasible solution $a_1, \dots, a_L$ is defined as

$$\frac{1}{2} \sum_{t=0}^{L-1} \sum_{q \in Q} d_H(a_t(q), a_{t+1}(q)).$$

The goal is to find a feasible allocation minimizing this cost function.

In the context of qubit routing, the vertex set V represents the physical qubits while the edge set E represents the set of physical qubit pairs between which two-qubit gates can be applied. The logical qubits are represented by the set Q , and the sets $G^t \in \Gamma$, $1 \leq t \leq L$, are the gates to be executed in parallel at time step t .

Next, we give an informal description of the token swapping problem. Its input is an undirected, connected graph and a set of tokens. Initially, each vertex holds a unique token. Further, each vertex is assigned a target token. Now the task is to move tokens along the edges of the graph such that each vertex holds its desired token. However, movement of tokens is restricted to swapping adjacent tokens. Formally, the token swapping problem is defined as follows.

Definition 2.2 (*Token swapping problem*) An instance of the *token swapping problem* is given by a tuple (H, Q, a, a') , where

- $H = (V, E)$ is a connected undirected graph,
- Q is a set of tokens with size $|Q| = |V|$,
- $a: Q \xrightarrow{1:1} V$ is an initial allocation, and
- $a': Q \xrightarrow{1:1} V$ is a final allocation.

For two nodes $i, j \in V, i \neq j$, the transposition (i, j) is the unique bijective mapping from V onto itself interchanging exactly the two elements i and j . Let $\mathcal{T}_H$ denote the set of transpositions on V restricted to E , i.e. $(i, j) \in \mathcal{T}_H \Leftrightarrow \{i, j\} \in E$. A feasible solution $\mathcal{S}$ is a finite sequence of transpositions (“swaps”) $\mathcal{S} = \tau_1, \tau_2, \dots, \tau_N$, where $\tau_i \in \mathcal{T}_H$ for $i \in \{1, \dots, N\}$, such that

$$a = \tau_N \circ \dots \circ \tau_2 \circ \tau_1 \circ a'.$$

The cost of a feasible solution is given by its length N . The goal of the token swapping problem is to find a feasible solution minimizing these costs.

There is a close link between Problems 2.1 and 2.2: every feasible solution to a TAP instance defines a sequence of token swapping instances in a canonical way. In the application to qubit routing, a solution to this sequence of problems together with the TAP solution results in a solution to the qubit routing by swap insertion problem. Note, that this solution might not be optimal in terms of used swaps.

3 An Analysis of the Token Allocation Problem

In the following, we analyse the TAP from Definition 2.1. After showing its NP-hardness, we derive an integer programming formulation based on edge flows. Additionally, we strengthen the model by introducing valid inequalities.

3.1 NP-Hardness

To motivate the use of integer programming for the solution of the TAP, we first show its NP-hardness. For this purpose, we construct a reduction of the *subgraph isomorphism problem* to the TAP. A similar reduction was already sketched by Siraichi et al. in [22] to show NP-completeness of a problem called *qubit assignment problem*. This problem is equivalent to the decision version of the TAP. Here, we transfer their main idea to the TAP and work out the reduction in detail.

Definition 3.1 (*Subgraph isomorphism problem*) Given two graphs H, H' , the *edge-induced (node-induced) subgraph isomorphism problem (SGI)* asks whether there is an edge-induced (node-induced) subgraph of H isomorphic to H' .

The complexity of the node-induced SGI is well known.

Theorem 3.2 ([26]) *The node-induced subgraph isomorphism problem is NP-complete.*

To transfer the NP-completeness to the edge-induced SGI, we need the notion of a *line graph*.

Definition 3.3 (*Line graph*) The *line graph* $\mathcal{L}(G)$ of a graph G possesses a node for every edge in G . Edges in $\mathcal{L}(G)$ connect two nodes if and only if the corresponding edges in G share a common node.

The proof of the following result is straight-forward and omitted here for brevity.

Lemma 3.4 *Two graphs H and H' are node-induced subgraph isomorphic if and only if the corresponding line graphs $\mathcal{L}(H)$ and $\mathcal{L}(H')$ are edge-induced subgraph isomorphic.*

Using this result, the node-induced SGI can be reduced to the edge-induced SGI.

Corollary 3.5 *The edge-induced subgraph isomorphism problem is NP-complete.*

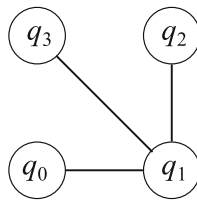


Fig. 3 Example for a connectivity graph. This example is constructed from the quantum circuit in Fig. 1a. Logical qubits in the circuit are interpreted as tokens, gates are interpreted as token pairs

For the reduction of edge induced SGI to TAP, we need

Definition 3.6 (*Connectivity graph*) For a finite set of tokens Q , let $G := \{(q_1, p_1), \dots, (q_n, p_n)\} \subset Q \times Q$, with $q_i \neq p_i$ for $i = 1, \dots, n$ be a set of token pairs. Then the *connectivity graph* of G is given by $C(G) = (\tilde{Q}, E_C)$ with

$$\tilde{Q} := \bigcup_{1 \leq k \leq n} \{q_k\} \cup \{p_k\} \quad \text{and} \quad E_C := \{\{q, p\} \mid (q, p) \in G\}.$$

An example of a connectivity graph is depicted in Fig. 3. The following Lemma establishes a connection between the connectivity graph and the TAP. The proof is not hard and omitted here.

Lemma 3.7 ([22]) *A TAP instance (H, Q, Γ) has optimal value of 0 if and only if there is an edge-induced subgraph of H isomorphic to the connectivity graph $C(\bigcup_{G^t \in \Gamma} G^t)$.*

With these preliminaries established, we now study the complexity of the TAP.

Theorem 3.8 *The token allocation problem is NP-hard.*

Proof To show NP-hardness, we construct a polynomial reduction of the edge-induced SGI to the decision version of the TAP. Let $H = (V, E)$ and $H' = (V', E')$ be two graphs. For the SGI instance (H, H') , we construct an TAP instance (H, Q, Γ) . Set $Q := V$ and construct Γ as follows: choose an arbitrary order $1, \dots, |E'|$ of the edges in E' , and choose an arbitrary order of the two vertices in each edge. This yields a sequence of edges $(p_1, q_1), (p_2, q_2), \dots, (p_{|E'|}, q_{|E'|})$, where $\{p_k, q_k\} \in E' \forall k \in \{1, \dots, |E'|\}$. Now, set $G^k := \{(p_k, q_k)\}$ for all $k \in \{1, \dots, |E'|\}$ and $\Gamma := G^1, \dots, G^{|E'|}$. Then we have $C(\Gamma) = H'$. Finally, Lemma 3.7 implies that the TAP instance (H, Q, Γ) has optimal value 0 if and only if there is an edge-induced subgraph isomorphism from H' to H . $\square$

For NP-hard discrete optimization problems, integer programming methods such as branch-and-bound and branch-and-cut are known to be particularly well suited. We will thus follow this route in the following.

3.2 Network Flow Model

In [18], Nannicini et al. formulate the entire routing by swap insertion problem as a network flow problem. We adjust their model to the TAP. This results in a

model significantly reduced in size which, of course, is due to the fact that only a subproblem of routing by swap insertion is modeled. Let $H = (V, E)$ be a connected graph, and let (H, Q, Γ) be a TAP instance. We introduce the directed arc set $A_H := \bigcup_{\{u,v\} \in E} \{(u, v)\} \cup \{(v, u)\}$ as well as variables with the following interpretations. The binary variables $x_{q,i,j}^t \in \{0, 1\}$ take a value of 1 if qubit $q \in Q$ moves from node $i \in V$ to node $j \in V$ between time steps t and $t + 1$ from $\{1, \dots, L\}$, and 0 otherwise. The auxiliary binary variables $w_{q,i}^t \in \{0, 1\}$ indicate by a value of 1 whether qubit $q \in Q$ is located at node $i \in V$ in time step $t \in \{1, \dots, L\}$, or 0 if not. Further binary auxiliary variables $y_{(p,q),(i,j)}^t \in \{0, 1\}$ express whether gate $(p, q) \in G^t$ is performed along edge (i, j) , value 1, or not, value 0. With these notions, the TAP can be modeled by the following quadratic binary program:

$$\min_x \sum_{t=1}^{L-1} \sum_{q \in Q} \sum_{i,j \in V \times V} d_H(i, j) x_{q,i,j}^t \quad (1)$$

$$\text{s.t. } w_{q,i}^t = \sum_{j \in V} x_{q,i,j}^t \quad \forall 1 \leq t < L, \forall i \in V, \forall q \in Q \quad (2)$$

$$w_{q,i}^t = \sum_{j \in V} x_{q,j,i}^{t-1} \quad \forall 1 < t \leq L, \forall i \in V, \forall q \in Q \quad (3)$$

$$\sum_{(i,j) \in A_H} y_{(p,q),(i,j)}^t = 1 \quad \forall 1 \leq t \leq L, \forall (p, q) \in G^t \quad (4)$$

$$y_{(p,q),(i,j)}^t = w_{p,i}^t \cdot w_{q,j}^t \quad \forall 1 \leq t \leq L, \forall (p, q) \in G^t, \forall (i, j) \in A_H \quad (5)$$

$$\sum_{i \in V} w_{q,i}^t = 1 \quad \forall 1 \leq t \leq L, \forall q \in Q \quad (6)$$

$$\sum_{q \in Q} w_{q,i}^t = 1 \quad \forall 1 \leq t \leq L, \forall i \in V \quad (7)$$

$$w_{q,i}^t \in \{0, 1\} \quad \forall 1 \leq t \leq L, \forall q \in Q, \forall i \in V \quad (8)$$

$$x_{q,i,j}^t \in \{0, 1\} \quad \forall 1 \leq t \leq L - 1, \forall q \in Q, \forall (i, j) \in V \times V \quad (9)$$

$$y_{(p,q),(i,j)}^t \in \{0, 1\} \quad \forall 1 \leq t \leq L, \forall (p, q) \in G^t, \forall (i, j) \in A_H. \quad (10)$$

Constraints (2) and (3) ensure logical qubit conservation. Via Constraints (4), we enforce that every gate is implemented. Constraints (5) demand that a gate be implemented along an arc if and only if logical qubits are located at the physical qubits of the arc. Finally, Constraints (6) and (7) establish bijective mappings between logical and physical qubits while Constraints (8),(9),(10) define the domains of the variables.

Model (1)–(10) can be illustrated by a time-expanded hardware graph G , which contains a copy of the nodes in H for every time step as well as edges connecting subsequent time steps. An example is shown in Fig. 4. A feasible solution is represented by a collection of $|Q|$ many vertex-disjoint paths in G , where each path starts at time step $t = 1$ and ends at time step $t = L$. Furthermore, in every time step t , neighboring restrictions implied by the gate group G^t need to be satisfied by the paths.

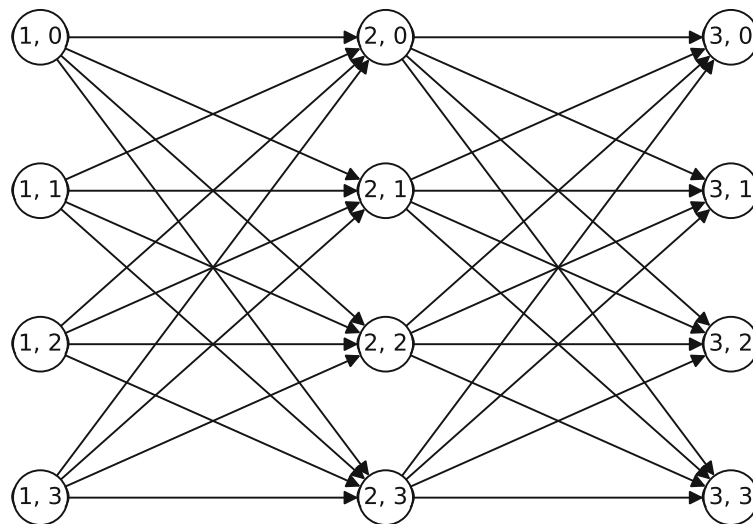


Fig. 4 Example for the time-expanded hardware graph G on which the network flow formulation of the TAP is based. The instance is taken from the example in Fig. 1. Accordingly, there are $|Q| = 4$ logical qubits and $L = 3$ time steps. The first number in a node labels the time step while the second number labels the physical qubit

For our further analysis of the model, we linearize the quadratic Constraints (5) by replacing them with

$$\begin{aligned} y_{(p,q),(i,j)}^t &\leq w_{p,i}^t \\ y_{(p,q),(i,j)}^t &\leq w_{q,j}^t \\ y_{(p,q),(i,j)}^t &\geq w_{p,i}^t + w_{q,j}^t - 1. \end{aligned} \quad (11)$$

From now on, we only consider the linearized version of (1)-(10) and refer to it as Model (1).

3.2.1 Optimal solution to the linear relaxation.

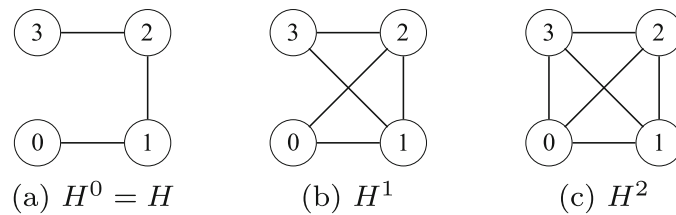
To illustrate the need for the introduction of strengthening inequalities, we show that the linear programming (LP) relaxation of Model (1) is very weak in the sense that for *any* TAP instance there is an optimal LP solution with value 0. To see this, we assign the flow value $1/|Q|$ to all paths in G with zero costs. More formally, we set

$$\begin{aligned} x_{q,i,j}^t &= \begin{cases} 1/|Q|, & i = j \\ 0, & i \neq j \end{cases} \quad \forall i, j \in V, \forall q \in Q, \forall 1 \leq t \leq L-1, \\ w_{q,i}^t &= 1/|Q| \quad \forall i \in V, \forall q \in Q, \forall 1 \leq t \leq L \end{aligned}$$

and

$$y_{(p,q),(i,j)}^t = 1/|A_H| \quad \forall 1 \leq t \leq L, \forall (p,q) \in G^t, \forall (i,j) \in A_H.$$

Fig. 5 Example for the d -th relaxed graphs H^d from Definition 3.9. The graph H is the four-qubit line graph from the example in Fig. 1b



It can be checked that this fully symmetrical, fractional solution is valid for the LP relaxation of Model (1). Concerning Constraints (11), note that $|A_H| \geq 2(|Q| - 1) \geq |Q|$ and $2/|Q| - 1 \leq 0$ hold, since H is connected and we have $|Q| \geq 2$.

3.3 Subgraph Isomorphism Inequalities

We will now derive valid inequalities for Model (1) to strengthen its LP relaxation. To this end, we extend the result of Lemma 3.7, which we also used in the proof of Theorem 3.8. For a given TAP instance (H, Q, Γ) , this will enable us to identify a qubit subset $\tilde{Q} \subseteq Q$, an integer distance $d \geq 0$ in H as well as time steps t_0 and t_1 , between which at least one qubit pair in $\tilde{Q}$ has to move a total distance of at least $d + 1$. For the derivation we need

Definition 3.9 (d -th relaxed graph) For an undirected, connected graph $H = (V, E)$ and an integer $d \geq 0$, we define the d -th relaxed graph $H^d = (V, E^d)$ as the graph defined by

$$\{i, j\} \in E^d \iff d_H(i, j) \leq d + 1.$$

Note that $H^0 = H$. An example for Definition 3.9 is shown in Fig. 5.

We are now able to sketch the main idea for deriving valid inequalities. Choose a starting time t_0 and an end time t_1 with $1 \leq t_0 < t_1 \leq L$ as well as a subset $\tilde{G}$ of the gates between t_0 and t_1 . Now, consider the connectivity graph $C(\tilde{G}) = (\tilde{Q}, E_C)$. We will relate it to the d -th relaxed graph H^d of H for some integer $d \geq 0$. Namely, if there is no isomorphism from $C(\tilde{G})$ to some edge-induced subgraph of H^d , we know that there is no allocation such that all qubits involved in the gates of $\tilde{G}$ are at most an edge distance of $d + 1$ apart. Consequently, there has to be at least one gate in $\tilde{G}$ such that the involved qubits are more than $d + 1$ apart at t_0 . Between time steps t_0 and t_1 , this qubit pair will move a total distance of at least $d + 1$. This leads us to the following

Lemma 3.10 (Subgraph isomorphism inequalities) *Let (H, Q, Γ) be a TAP instance with $H = (V, E)$ and $\Gamma = G^1, \dots, G^L$. Further, let $\tilde{G} := \bigcup_{t=1}^L \tilde{G}^t$, $\tilde{G}^t \subseteq G^t$, be a set of gates and let $C := C(\tilde{G}) = (\tilde{Q}, E_C)$ be the corresponding connectivity graph. Finally, let $d \geq 0$ be an integer and let H^d be the d -th relaxed graph of H . Now consider the time steps $t_0 := \min\{t \mid \tilde{G}^t \neq \emptyset\}$ and $t_1 := \max\{t \mid \tilde{G}^t \neq \emptyset\}$. If there is no edge-induced subgraph of H^d isomorphic to C , then the inequalities*

$$\sum_{t=t_0}^{t_1-1} \sum_{q \in \tilde{Q}} \sum_{(i,j) \in V \times V} d_H(i, j) x_{q,i,j}^t \geq d + 1 \quad (12)$$

and

$$\sum_{t=t_0}^{t_1-1} \sum_{q \in \tilde{Q}} \sum_{(i,j) \in V \times V} d_H(i, j) x_{q,i,j}^t \geq 2(d + 1) \quad (13)$$

hold.

Proof First, we show the validity of (12). For a feasible binary solution to Model (1), consider the allocation a_{t_0} of qubits at time step t_0 . From a_{t_0} , construct the edge-induced subgraph $H' = (V', E')$ of $H^d = (V, E^d)$ defined by the edge subset

$$E' := \left\{ \{i, j\} \in E^d \mid a_{t_0}^{-1}(i), a_{t_0}^{-1}(j) \in \tilde{Q} \wedge \{a_{t_0}^{-1}(i), a_{t_0}^{-1}(j)\} \in E_C \right\}. \quad (14)$$

The mapping defined by the allocation a_{t_0} is not an isomorphism from H' to C , since C is not subgraph isomorphic to H^d by assumption. Furthermore, the definition (14) of E' implies, that for every edge $\{i, j\}$ in H' there is a corresponding edge $\{a_{t_0}^{-1}(i), a_{t_0}^{-1}(j)\}$ in C . Thus, there is an edge $\{q_1, q_2\}$ in C such that $\{a_{t_0}(q_1), a_{t_0}(q_2)\}$ is not an edge in H' . Now, Eq. (14) implies that $\{a_{t_0}(q_1), a_{t_0}(q_2)\} \notin E^d$, since $q_1, q_2 \in \tilde{Q}$ and $\{q_1, q_2\} \in E_C$ but $\{a_{t_0}(q_1), a_{t_0}(q_2)\} \notin E'$. This edge corresponds to a gate $(q_1, q_2) \in G^{t'}$ for some t' with $t_0 < t' \leq t_1$. For this gate, the logical qubits q_1 and q_2 are located at physical qubits more than $d + 1$ apart at t_0 , i.e.

$$d_H(a_{t_0}(q_1), a_{t_0}(q_2)) > d + 1, \quad (15)$$

but need to be at neighbouring physical qubits at t' . Thus, together they need to move a distance of at least $d + 1$:

$$d_H(a_{t_0}(q_1), a_{t'}(q_1)) + d_H(a_{t_0}(q_2), a_{t'}(q_2)) \geq d + 1. \quad (16)$$

This implies (12). Now, we show the validity of (13). For a subset of vertices $\tilde{Q} \subseteq Q$ let

$$L(\tilde{Q}) := \sum_{q \in \tilde{Q}} d_H(a_{t_0}(q), a_{t'}(q))$$

denote the sum of distances between qubit allocations at t_0 and qubit allocations at t' for logical qubits in $\tilde{Q}$. Then, for the left-hand side of (12) it holds

$$\sum_{t=t_0}^{t_1-1} \sum_{q \in \tilde{Q}} \sum_{(i,j) \in V \times V} d_H(i, j) x_{q,i,j}^t \geq L(\tilde{Q}),$$

since $t' \leq t_1$. Consider the Q -permutation $\pi := a_{t_0}^{-1} \circ a_{t'} : Q \rightarrow Q$. Let $K_1, K_2 \subseteq Q$ be the cycles in π containing q_1 and q_2 , respectively. First, consider the case $K_1 \neq K_2$.

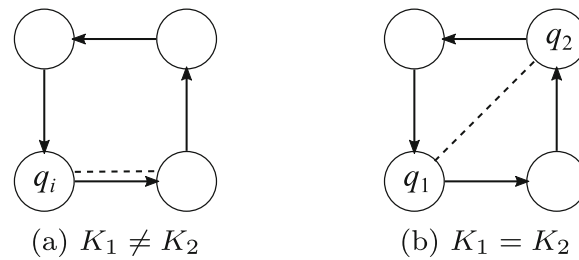


Fig. 6 Sketch for the two cases in the proof of Lemma 3.10. The cycle in **a** contains exactly one of the qubits q_1 and q_2 and its length is greater or equal than twice the distance this qubit moves (dashed line). The cycle in **b** contains both qubits q_1 and q_2 , and its length is greater or equal than twice the initial distance of q_1 and q_2 (dashed line)

Here it holds

$$L(\tilde{Q}) \geq L(K_1) + L(K_2),$$

since $K_1 \cap K_2 = \emptyset$. Furthermore, we know $L(K_i) \geq 2d_H(a_{t_0}(q_i), a_{t'}(q_i))$, $i \in \{1, 2\}$, since K_1 and K_2 are cycles and $d_H(\cdot, \cdot)$ satisfies the triangle inequality, see Fig. 6a for an illustration.

Thus, we have

$$L(K_1) + L(K_2) \geq 2d_H(a_{t_0}(q_1), a_{t'}(q_1)) + 2d_H(a_{t_0}(q_2), a_{t'}(q_2)) \stackrel{(7)}{\geq} 2(d+1).$$

On the other hand, if $K_1 = K_2 =: K$, we find

$$L(\tilde{Q}) \geq L(K).$$

Thus, again using the triangle inequality and the fact that K is a cycle, we have

$$L(K) \geq 2d_H(a_{t_0}(q_1), a_{t_0}(q_2)) \stackrel{(6)}{\geq} 2(d+1)$$

as illustrated in Fig. 6b. This implies (13). $\square$

Clearly, any valid subgraph isomorphism inequality cuts off all solutions of the LP relaxation with zero cost. However, generating these inequalities can be computationally expensive, since subgraph isomorphism is NP-complete.

Having shown its NP-hardness and analysed its binary model, we conclude this section about the TAP by introducing methods to reduce the model size for larger instances.

3.4 Algorithmic Enhancements for Large Instances

To keep the size of Model (1) tractable for larger hardware graphs and quantum circuits, we describe two heuristic methods for eliminating variables, which means that both methods may in principle lead to suboptimal solutions. The number of flow variables is

decreased by either removing edges from the time-expanded graph G or by removing logical qubits not participating in two-qubit gates.

3.4.1 Limiting Distance

The first method eliminates edges from G . We only consider edges which connect physical qubits in H that are within a given distance limit. Thus, we limit the distance that a logical qubit may move between subsequent allocations. A method to find a reasonable distance limit is to iteratively increase the limit until the TAP becomes feasible. Once the model is feasible, the distance is again increased by one and the TAP is solved a last time.

3.4.2 Limiting the Number of Qubits

In practice, the number of logical qubits is usually smaller than the number of physical qubits. In principle, this can be reduced to the case of an equal number of logical and physical qubits by adding inactive logical qubits not participating in any two-qubit gates. However, the number of variables in the TAP model grows linearly with $|Q|$. Therefore, we only consider flow variables associated with active logical qubits. This bears the problem that costs associated with the flow of inactive qubits are not counted. To circumvent this issue, we do not allow active logical qubits to change positions with inactive qubits. Thus, the subgraph of H at which active logical qubits are located is fixed for all time steps. However, we stress that the particular choice of this subgraph remains subject to optimization.

4 Solving the Token Swapping Problems

We now turn to the solution of the token swapping problems which need to be solved as the second step in the proposed routing procedure, once an optimal TAP solution has been found. Token swapping is known to be NP-hard (see e.g. [15]). We will develop two solution methods for the token swapping problem: an efficient approximation algorithm as well as an exact branch-and-bound approach.

4.1 Approximate Solution

The proposed qubit routing procedure employs a modified version of the efficient token swapping algorithm given by Miltzow et al. in [15]. This approximation algorithm has running time that is bounded by a polynomial in the number of tokens, also in the worst case.

It has a guaranteed approximation factor of four in terms of swap count. However, we note that on all instances we considered, the algorithm exhibits an approximation factor better than 1.5. Indeed, the analysis in [15] shows that the approximation factor is strictly less than four.

We briefly review the original approximation algorithm. For a detailed description and analysis, we refer the interested reader to [15]. Given a token swapping instance

(H, Q, a, a') , the algorithm iteratively performs swaps by gradually changing the allocation from a to a' . We adapt the notion of [15] for an arbitrary token allocation $\tilde{a}$: an *unsatisfied vertex* is a vertex $i \in H$ which does not hold its target token, i.e. $\tilde{a}^{-1}(i) \neq (a')^{-1}(i)$. A *distance-decreasing neighbour* of a vertex $i \in H$ is a neighbour of i from which the distance to the target vertex of the token sitting at i is strictly less than the distance from i . A *happy swap chain* of length k is a sequence of k swaps $(i_1, i_2), (i_2, i_3), \dots, (i_{k-1}, i_k), (i_k, i_{k+1}) \subseteq \mathcal{T}_H$ such that every token involved has its distance decreased after the swaps in the chain have been performed sequentially. An *unhappy swap* is a swap that moves a token already positioned at its target vertex, while the other token moves closer to its target. Given these notions, the algorithm works as follows:

1. Choose an unsatisfied vertex.
2. Perform a walk along distance-decreasing neighbours until a cycle is closed or a dead end encountered, i.e. no distance-decreasing neighbour exists.
3. Perform the happy swap chain or unhappy swap.
4. Go to 1.

The key insight why this algorithm terminates is, that vertices involved in an unhappy swap will be part of a future happy swap chain. Furthermore, the number of happy swap chains is finite since they necessarily move tokens further towards their targets. The original algorithm, as described in [15], chooses random vertices in steps 1 and 2. We modify the algorithm in two ways aiming to further reduce both resulting swap count and depth. First, in step 1, we choose an unsatisfied vertex that was not part of the previously performed swap chain or unhappy swap. With this choice, we try to reduce depth: only swaps on disjoint vertices can be performed in parallel. Second, we modify step 2. In each step of the walk we try to choose cleverly among all distance-decreasing neighbours by exploring them first. If possible, we choose the one closing the smallest cycle. The motivation for this is, that all swaps in a happy swap chain cannot be performed in parallel since they share common vertices. If no cycle can be closed, we try to avoid dead ends, i.e. unhappy swaps. Intuitively, a typical optimal solution does not use many unhappy swaps.

Having introduced an efficient but approximate method, we now derive an exact branch and bound algorithm.

4.2 Exact Solution

The exact approach will be used for benchmarking the approximation algorithm presented in the previous section. The exact method finds a shortest path from the initial allocation to the final allocation in the associated *Cayleigh graph*. The latter possesses a node for each of the $|Q|!$ possible allocations. Two nodes are adjacent if and only if there is a swap, i.e. a transposition $\tau \in \mathcal{T}_H$, which maps between the associated allocations. It follows that an optimal solution to a token swapping problem is given by a shortest path from the initial allocation to the final allocation in the Cayleigh graph. The search algorithm works through the nodes of the Cayleigh graph starting at the initial allocation. At each node, we give upper and lower bounds on the shortest path length to the goal node. Upper bounding is achieved by running the modified

4-approximation algorithm from Sect. 4.1, yielding a path from the current node to the goal node. This path is augmented by the path from the start node to the current node, giving a feasible solution to the token swapping problem.

We use two different types of lower bounds, described in the following sections.

4.2.1 Improved Distance Lower Bound

A well-known lower bound on token swapping, already introduced as objective of the TAP in Sect. 3, is given by

Lemma 4.1 ([15, 28] Distance lower bound) *For any token swapping instance $T = (H, Q, a, a')$ it holds*

$$\text{OPT}(T) \geq \left\lceil \frac{1}{2} \sum_{q \in Q} d_H(a(q), a'(q)) \right\rceil. \quad (17)$$

The following Lemma describes a class of instances for which this is bound sharp, i.e. satisfied with equality.

Lemma 4.2 *The distance lower bound (17) is sharp on all token swapping instances that require two or less swaps.*

Proof It is easy to see that the bound is sharp for an instance that requires one or less swaps. Now, consider an instance that requires two swaps. Here, exactly a two cases can occur. Either the two swaps act on disjoint vertices, or they share a common vertex. In the first case, the sum in (17) amounts to 4, where in the second case it amounts to at least 3. $\square$

A simple example for a token swapping instance requiring two swaps is given by a 3-vertex line graph where initially no vertex holds its desired token. On the contrary, for every optimal objective value greater or equal to 3, one can construct token swapping instances, for which the distance bound is not sharp. On a complete (sub-)graph, place n tokens such that the underlying permutation has exactly one cycle. Then, an optimal solution has $n - 1$ swaps as we will see in the following section. However, the distance bound is $\left\lceil n/2 \right\rceil < n - 1$ for ≥ 3 .

Lemma 4.2 implies that the decomposition of the qubit routing problem into TAP and token swapping is guaranteed to return optimal solutions for all instances that have a solution with two or less swaps. Our aim is now to improve upon lower bound (17). To this end, we introduce the notion of a *blocking vertex*.

Definition 4.3 (*Blocking vertex*) Let (H, Q, a, a') be a token swapping instance. Let $q \in Q$ be a token not yet sitting at its target vertex, i.e. $a(q) \neq a'(q)$. A vertex $v \in H$ is called *q-blocking vertex* if it is contained in a shortest path in H from $a(q)$ to $a'(q)$, and if it holds its desired token i.e. $a^{-1}(v) = (a')^{-1}(v)$.

Intuitively, the presence of blocking vertices increases the number of required swaps. This intuition is confirmed by

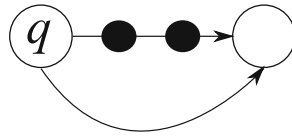


Fig. 7 Sketch for the proof of Lemma 4.4. The shortest path from the vertex holding token q to its target vertex (horizontal arrow) is blocked by vertices already holding their desired tokens (black circles). In any solution, q takes a detour (curved arrow) or satisfied tokens at blocking vertices are moved. Either alternative results in additional swaps

Lemma 4.4 (Improved distance lower bound) *Let $T = (H, Q, a, a')$ be a token swapping instance. Let $Q^* \subseteq Q$ denote the set of tokens not yet sitting at their target vertices. For every $q \in Q^*$ let $B_q \subset V$ be a set of q -blocking vertices (see Definition 4.3) of size k_q such that*

- (i) *Any path in H from q to its target vertex avoiding any of the blocking vertices in B_q has at least length $d_H(a(q), a'(q)) + 2k_q$ and*
- (ii) *The sets B_q are pairwise disjoint.*

Then it holds

$$\text{OPT}(T) \geq \left\lceil \sum_{q \in Q} \frac{d_H(a(q), a'(q))}{2} \right\rceil + \sum_{q \in Q^*} k_q.$$

Proof Consider a solution $\mathcal{S} = \tau_1, \tau_2, \dots, \tau_N$ and denote $[N] := \{1, \dots, N\}$. We partition $Q^* = Q_1^* \dot{\cup} Q_2^*$ according to the following scheme: for $q \in Q^*$ let $q \in Q_1^*$ if the path of q in $\mathcal{S}$ contains all associated blocking vertices B_q ; otherwise, let $q \in Q_2^*$.

For $q \in Q_2^*$ and $v, v' \in V$ let $\mathcal{P}_q(v, v')$ denote the set of v - v' -paths in H which do not contain all of the blocking vertices B_q . For a path $p \in \mathcal{P}_q(v, v')$, define its length as $l(p) := |p| - 1$. Moreover, define

$$d_{H,q}(v, v') := \min\{l(p) \mid p \in \mathcal{P}_q(v, v')\}$$

as the minimum edge-distance between v and v' if only paths not containing all q -blocking vertices are allowed. Note, that $\mathcal{P}_q(a(q), a'(q)) \neq \emptyset$: from the definition of Q_2^* , it follows that for all $q \in Q_2^*$ there is a path not containing B_q . Furthermore, by property (i) of B_q it holds

$$d_{H,q}(a(q), a'(q)) \geq d_H(a(q), a'(q)) + 2k_q, \quad (18)$$

for all $q \in Q_2^*$, see Fig. 7 for an illustration.

For $i \in [N]$, consider the sum d_i of all edge distances from tokens in Q^* to their targets after swap τ_i has been performed, accounting for the detours taken by tokens in Q_2^* ,

$$d_i := \sum_{q \in Q_1^*} d_H(\tau_i \circ \tau_{i-1} \circ \dots \circ \tau_1 \circ a(q), a'(q)) +$$

$$\sum_{q \in Q_2^*} d_{H,q}(\tau_i \circ \tau_{i-1} \circ \cdots \circ \tau_1 \circ a(q), a'(q)).$$

Accordingly, set

$$d_0 := \sum_{q \in Q_1^*} d_H(a(q), a'(q)) + \sum_{q \in Q_2^*} d_{H,q}(a(q), a'(q)).$$

Using (18) yields

$$d_0 \geq \sum_{q \in Q^*} d_H(a(q), a'(q)) + \sum_{q \in Q_2^*} 2k_q = \underbrace{\sum_{q \in Q} d_H(a(q), a'(q))}_{=: D_0} + \sum_{q \in Q_2^*} 2k_q, \quad (19)$$

where we used that $d_H(a(q), a'(q)) = 0$ for all $q \in Q \setminus Q^*$. Furthermore, for $i \in [N]$ let Δd_i denote the difference

$$\Delta d_i := d_i - d_{i-1}.$$

Clearly, $d_N = 0$, since $\tau_N \circ \cdots \circ \tau_2 \circ \tau_1 \circ a = a'$. Thus,

$$\sum_{i \in [N]} \Delta d_i = d_N - d_0 = -d_0. \quad (20)$$

Furthermore, for any $i \in [N]$ it holds $-2 \leq \Delta d_i \leq 2$, since a swap moves two tokens each for a distance of 1.

Now, we turn to Q_1^* . By definition of Q_1^* , a vertex $v \in \bigcup_{q \in Q_1^*} B_q$ is necessarily involved in a swap. For $v \in \bigcup_{q \in Q_1^*} B_q$, denote by $i_v := \min\{i \in [N] \mid v \in \tau_i\}$ the first swap that involves i . Let $I := \{i_v \mid v \in \bigcup_{q \in Q_1^*} B_q\}$. Partition $I = I_0 \dot{\cup} I_2$ in the following way. For $i_v \in I$, if there is an $v' \in \bigcup_{q \in Q_1^*} B_q$ such that $v' \neq v$ but $i_v = i_{v'}$, we let i_v belong to the set I_2 . Otherwise, we let i_v belong to the set I_0 . Then, for $i \in I_0$ we have $\Delta d_i \geq 0$, since at least one token leaves its target vertex by swap τ_i , namely the token $a^{-1}(v)$. Furthermore, for $i \in I_2$ we have $\Delta d_i = 2$, since both tokens leave their target vertices by swap τ_i .

Let $n_0 := |I_0|$ and $n_2 := |I_2|$. Using property (ii) of B_q and the definition of I_0 and I_2 , it holds

$$\left| \bigcup_{q \in Q_1^*} B_q \right| = \sum_{q \in Q_1^*} k_q = 2n_2 + n_0.$$

Let $\bar{I} := [N] \setminus I = [N] \setminus (I_0 \cup I_2)$. Then we can use (20) to obtain

$$-d_0 = \sum_{i \in [N]} \Delta d_i = \underbrace{\sum_{i \in I_0} \Delta d_i}_{\geq 0} + \underbrace{\sum_{i \in I_2} \Delta d_i}_{=2n_2} + \sum_{i \in \bar{I}} \Delta d_i \geq 2n_2 + \sum_{i \in \bar{I}} \Delta d_i.$$

Thus, with (19), we have

$$\sum_{i \in \bar{I}} \Delta d_i \leq -D_0 - \sum_{q \in Q_2^*} 2k_q - 2n_2.$$

Since $\Delta d_i \geq -2$, it follows

$$|\bar{I}| \geq \left\lceil \frac{D_0 + \sum_{q \in Q_2^*} 2k_q + 2n_2}{2} \right\rceil = \left\lceil \frac{D_0}{2} \right\rceil + \sum_{q \in Q_2^*} k_q + n_2.$$

Altogether, this means

$$\begin{aligned} N &= |I_0| + |I_2| + |\bar{I}| \geq n_0 + n_2 + \left\lceil \frac{D_0}{2} \right\rceil + \sum_{q \in Q_2^*} k_q + n_2 \\ &= \left\lceil \frac{D_0}{2} \right\rceil + \sum_{q \in Q_2^*} k_q + \sum_{q \in Q_1^*} k_q = \left\lceil \frac{D_0}{2} \right\rceil + \sum_{q \in Q^*} k_q. \end{aligned}$$

□

The question remains, how the sets of blocking vertices B_q can be computed in practise. We employ a simple greedy procedure that iterates over all tokens and takes as many blocking vertices as possible in each iteration. As it turns out computationally, this lower bound is particularly strong on graphs with low connectivity. On the contrary, the second lower bound employed by the branch-and-bound algorithm is stronger on dense graphs.

4.2.2 Complete Split Graph Lower Bound

While the bound from Lemma 4.4 is based on relaxing the restriction, that tokens need to be moved by swaps, we now derive a lower bound that is based on relaxing the set of allowed transpositions $\mathcal{T}_H$. This set is increased by adding edges missing in H . We use a result of Yasui et al. [29], who give an efficient algorithm for solving token swapping problems on *complete split graphs*.

Definition 4.5 (*Complete split graph*) A vertex subset of a graph is called *independent* if no two vertices in the subset are connected by an edge. A vertex subset is called a *clique* if any two vertices in the subset are connected by an edge. A graph is called a *complete split graph* if it can be partitioned into an independent set and a clique such that every vertex in the independent set is connected to all vertices in the clique.

Lemma 4.6 (Complete split graph lower bound [29]) *Let $T = (H, Q, a, a')$ be a token swapping instance. Furthermore, let $V' \subset V$ be an independent set in H . Let $n = |V|$ be the number of vertices, let r be the number of cycles in the permutation $a \circ (a')^{-1}$ including fix points, i.e. cycles of length 1. Let q be the number of cycles in the permutation $a \circ (a')^{-1}$ excluding fix points that only involve vertices in V' . Then it holds*

$$\text{OPT}(T) \geq n - r + 2q.$$

Proof Consider the graph $H' = (V, E')$ which is derived from H and V' in the following way: connect every vertex in V' with all vertices in $V \setminus V'$. Additionally, add all missing edges in $V \setminus V'$. Then H' is a complete split graph. Furthermore, $E \subseteq E'$ holds, and thus also $\mathcal{T}_H \subseteq \mathcal{T}_{H'}$. Consider a solution $\mathcal{S}$ of size N to T . This is also a solution to the token swapping instance $T' := (H', Q, a \circ (a')^{-1}, \text{id})$. Thus, we have

$$|\mathcal{S}| \geq \text{OPT}(T').$$

According to [29], the optimal value for T' is given by

$$\text{OPT}(T') = n - r + 2q.$$

Thus,

$$|\mathcal{S}| \geq n - r + 2q.$$

□

We remark that Lemma 4.6 is a generalization of the well-known fact that token swapping on complete graphs has optimal value $n - r$ (cf. e.g. [12]). When applying Lemma 4.6 in practise, an independent set needs to be constructed first. In our implementation, we employ a simple greedy procedure that iterates over the vertices increasing in degree, adding it to the independent set if possible.

As already mentioned, both lower bounds complement each other: while the distance bound is strong on sparse graphs, the complete-split-graph bound is strong on dense graphs. The following lemma allows to increase the lower bounds from Lemmas 4.4 and 4.6 by one in some cases.

Lemma 4.7 (Parity of solutions) *Let $T = (H, Q, a, a')$ be a token swapping instance. Let $\text{sgn}(\pi)$ denote the sign of a permutation π . For any solution $\mathcal{S} = \tau_1, \tau_2, \dots, \tau_N$ to T , it holds*

$$N \equiv \left[1 - \text{sgn}(a' \circ a^{-1}) \right] / 2 \pmod{2}.$$

Proof For any solution $\mathcal{S}$, it holds

$$a' = \tau_N \circ \dots \circ \tau_2 \circ \tau_1 \circ a$$

or, equivalently

$$a' \circ a^{-1} = \tau_N \circ \dots \circ \tau_2 \circ \tau_1.$$

Using the fact that $\text{sgn}(\sigma \circ \pi) = \text{sgn}(\sigma) \cdot \text{sgn}(\pi)$ for any two permutations σ, π , we have

$$\text{sgn}(a' \circ a^{-1}) = \text{sgn}(\tau_N) \cdots \text{sgn}(\tau_2) \cdot \text{sgn}(\tau_1) = (-1)^N.$$

Thus, N is either even or odd, independent of S :

$$N \equiv \left[1 - \text{sgn}(a' \circ a^{-1}) \right] / 2 \pmod{2}.$$

□

With these theoretical properties of our solution method to the qubit routing by swap insertion problem established, we now present various experimental results.

5 Experimental Results

In the following, we benchmark the proposed routing algorithm. We perform experiments covering four aspects:

1. The computation time and solution quality when solving the TAP model with strengthening inequalities.
2. The quality and speed of the token swapping approximation algorithm.
3. The quality of the routing solution produced by the combined algorithm compared to other routing methods.
4. The quality of solutions produced by quantum algorithms compiled with our routing method, executed on actual quantum hardware.

The datasets generated and analyzed during the current study are available from the corresponding author on reasonable request.

5.1 Benchmarking the Strengthened TAP Model

In the first step of the proposed decomposition approach, a TAP needs to be solved. Here, we give numerical results showing how the introduction of subgraph isomorphism cutting planes (Sect. 3.3) can accelerate the solution of the TAP binary linear program. The linearized flow model (1) is implemented and solved via the Python API of *Gurobi* [8]. Before parsing the model to the solver, we generate a set of valid inequalities introduced in Sect. 3.3. These are added to a cut pool. The solver can add cuts from this pool to the model in order to cut off relaxed solutions.

The set of valid inequalities is generated as follows: first, the graphs H^d for $0 \leq d \leq \max_{i,j \in H} d_H(i, j)$ are created and transformed into their respective line graphs. To generate a subset of gates for which a valid cut may be derived, we first choose time steps t_0 and t_1 such that $1 \leq t_0 < t_1 \leq L$. Then, all gates in gate groups between t_0 and t_1 are considered. The corresponding connectivity graph is generated and transformed into its line graph. To perform the node-induced-subgraph isomorphism check, we employ an implementation of the VF2 algorithm (see [3, 4]) from

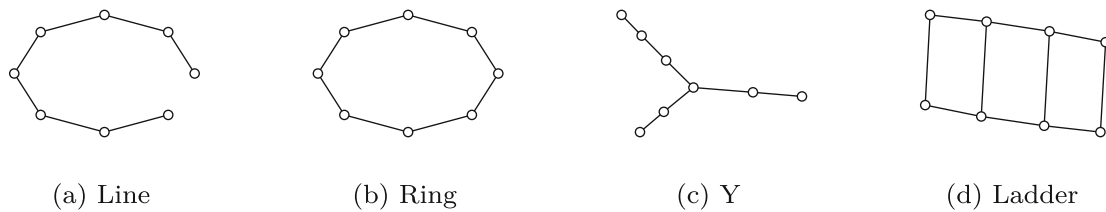


Fig. 8 Hardware graphs used in the benchmarks of Sects. 5.1 and 5.3

the NetworkX python package [9]. This procedure is repeated by iterating over time steps t_0, t_1 until all possible time step pairs have been checked.

In the first benchmark, we measure the computation time required for solving TAP instances to optimality. We compare the total time taken with strengthening inequalities to the time taken for the bare model (1). We measure the ratio $r_t = t_{\text{SGI}}/t_{\text{bare}}$, where t_{SGI} and t_{bare} are the runtimes of the strengthened model and the bare model, respectively. The time consumed by cutting plane generation lies between 0.1 s and 20 s and is included in the total runtime t_{SGI} .

In all instances (H, Q, Γ) , the hardware graph H has 8 vertices and $Q := \{q_1, \dots, q_8\}$. Tests are performed on four hardware graphs with different sparsity, namely the line, ring, Y and ladder graph, illustrated in Fig 8. Each of the gate groups in $\Gamma = G^1, G^2, \dots, G^L$ consists of $\lfloor L/2 \rfloor$ randomly chosen pairs of qubits among the set $\{q_1, \dots, q_L\}$. This results in instances with as many gates per time step as possible, making the instances computationally hard. The depth L is varied from 4 to 8, where 10 instances are generated per value of L . The instance sizes resemble the capabilities of currently available quantum hardware. All instances are solved to optimality. The absolute solutions times lie between 0.6 s and 1000 s. Results are given in Table 1 a. Shorter average runtimes are achieved for $L = 5$ on ring, Y, and ladder. A maximum saving of 72% is observed for the $L = 4$ line. Furthermore, we observe rather high differences between minimum and maximum runtime ratio, indicating a strong instance dependence. Summarizing, the separation of valid subgraph isomorphism inequalities can improve the TAP-model runtime. Thus, also the algorithm for the full routing problem benefits from generating valid inequalities for the TAP subproblem. However, the strong instance dependence suggests that more research is necessary in order to fully exploit the potential of subgraph isomorphism inequalities.

For the same benchmark set, we measure the tightness of the lower bound on the number of required swaps in an optimal routing solution, given by the optimal TAP solution. To this end, the objective of the full routing model (BIP) from [18] is adjusted to minimize swap count instead of depth. Solving this model yields the optimal swap number. We measure the relative bound $r_b = \frac{\text{OPT}_{\text{TAP}}}{\text{OPT}_{\text{BIP}}}$, with OPT_{BIP} and OPT_{TAP} denoting the optimal BIP solution and TAP solution, respectively. These results are shown in Table 1 b. We observe tight bounds with a relative difference of at most 23% ($L = 8$ line), together with small deviations from the average value. This is a key feature of our decomposition approach: optimal TAP solution values typically give a good estimate for the number of required swaps.

Table 1 TAP-model benchmark results

H	L														
	4			5			6			7			8		
	Min	Max	Avg	Min	Max	Avg	Min	Max	Avg	Min	Max	Avg	Min	Max	Avg
<i>(a) Runtime comparison when solving to optimality</i>															
Line	0.28	3.9	1.31	0.8	2.56	1.28	1.27	7.76	2.96	0.31	4.19	2.49	0.47	2.35	1.35
Ring	0.5	0.98	1.02	0.64	1.9	0.8	0.76	3.72	1.56	0.74	2.78	1.34	0.48	1.47	1.01
Y	0.48	1.58	1.16	0.49	2.76	0.87	0.75	2.54	1.49	0.77	3.39	1.89	0.56	2.87	1.74
Ladder	0.36	1.16	1.2	0.48	2.47	0.9	0.78	9.72	2.62	0.63	4.58	2.4	0.93	7.26	2.53
<i>(b) Bound quality when solving to optimality</i>															
Line	1.0	1.0	1.0	0.8	1.0	0.98	0.9	1.0	0.98	0.85	1.0	0.95	0.77	1.0	0.86
Ring	1.0	1.0	1.0	0.8	1.0	0.98	1.0	1.0	1.0	1.0	1.0	1.0	0.92	1.0	0.97
Y	1.0	1.0	1.0	0.8	1.0	0.94	0.89	1.0	0.96	0.85	1.0	0.93	0.81	1.0	0.89
Ladder	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	0.8	1.0	0.94	0.83	1.0	0.97

The different hardware graphs, denoted H , are depicted in Fig. 8. The qubit set is $Q := \{q_1, \dots, q_8\}$. Each instance contains L many gate groups $G^1, G^2, \dots, G^L$, where each gate group consists of $\lfloor L/2 \rfloor$ randomly chosen pairs of qubits among the set $\{q_1, \dots, q_L\}$. 10 instances are generated per depth L and graph H . Each instance is solved with and without additional strengthening inequalities. We compare the ratio r_t of runtimes in (a). Additionally, in (b) we give the relative bound quality r_b for the optimal TAP value as a lower bound on the number of required swaps in an optimal routing solution

5.2 Benchmarking the Token Swapping Approximation

The second step in the decomposition approach consists of several consecutive token swapping instances. Here, we evaluate the performance of the employed token swapping approximation. All algorithms described in this section have been implemented in Python from scratch. First, we compare the original 4-approximation algorithm given in [15] to our modified version. For this purpose, we generate random token swapping instances, i.e. initial and final allocations¹ on three types of planar graphs with different sparsity. As graph types we choose ring graphs, ladders and square meshes with node numbers of 16, 36 and 64 for each type. As in the previous section, this choice of graph types and sizes aims at resembling current realizations of quantum computer hardware connectivity graphs. We generate 100 instances per graph. In Table 2, we give the average ratio $r_{\#}$ of swap numbers between the modified version and the original version as well as the average ratio of depths r_d .

Firstly, note that these ratios are less than or equal to 1 across all instances. This means that our modification always performs as least as good as the original version, or better (if the factor is smaller than 1). Furthermore, both swap and depth ratio are roughly constant across graph sizes for a given graph type. However, the advantage of the modified version increases with growing connectivity of the graph type. An intuitive explanation of this behaviour is the increase of swapping possibilities with growing edge number. This allows the modified version to employ its potential of deciding for “good” swaps instead of choosing them randomly.

¹ Due to symmetry, it is sufficient to only vary initial allocations while fixing the final allocation.

Table 2 Comparison of original and modified token swapping approximation algorithms. We give the average ratio of inserted swaps $r_{\#} = \#_{\text{mod}}/\#_{\text{orig}}$ and resulting depths $r_d = d_{\text{mod}}/d_{\text{orig}}$

N	Ring		Ladder		Mesh	
	$r_{\#}$	r_d	$r_{\#}$	r_d	$r_{\#}$	r_d
16	0.96	0.91	0.88	0.82	0.81	0.78
36	1.0	1.0	0.91	0.83	0.83	0.78
64	1.0	0.99	0.93	0.79	0.84	0.81

Comparison are made on rings, ladders and meshes with increasing vertex number N

Table 3 Comparison of token swapping approximation and exact algorithm

N	Line			Ring			Ladder		
	$r_{\#}$	r_t	$\bar{t}_{\text{appr}}$	$r_{\#}$	r_t	$\bar{t}_{\text{appr}}$	$r_{\#}$	r_t	$\bar{t}_{\text{appr}}$
5	1.0	0.2	0.0	1.25	0.3	0.0	1.04	0.6	0.0
10	1.0	0.05	0.0	1.19	0.2	0.0	1.19	0.2	0.0
12	1.0	–	0.0	1.23	0.08	0.0	1.13	0.035	0.0
50	1.0	–	0.8	1.17	–	0.4			0.4

We give the average approximation ratio $r_{\#} = \#_{\text{appr}}/\#_{\text{opt}}$ and runtime ratio $r_t = t_{\text{appr}}/t_{\text{bnb}}$ as well as the average absolute runtime of the approximation algorithm $\bar{t}_{\text{appr}}$ in seconds. Instance graphs are lines, rings and ladders of increasing vertex number N . Token swapping can be solved efficiently on lines and rings, cf. [12]. This allows us to compare approximation ratios even when the runtime is intractable for the exact algorithm (indicated by –). For ladders, no efficient exact algorithm exists. Therefore, no results are presented for large ladder graphs (empty cells). Averages are taken over 100 random instances. Exceptions due to excessive runtime are: 54 instances for $N = 10$ lines, 22 for $N = 12$ rings and 18 for $N = 12$ ladders

Secondly, we compare the modified 4-approximation algorithm to the exact branch-and-bound algorithm in terms of runtime and solution quality. We measure the approximation ratio $r_{\#} = \#_{\text{approx}}/\#_{\text{opt}}$ as well as the runtime ratio $r_t = t_{\text{approx}}/t_{\text{bnb}}$ on various instances. As graph types, we choose lines, rings and ladders. Again, this choice tries to resemble real hardware graphs while we also want to make use of the fact that token swapping can be solved efficiently on lines and rings.² This allows use to measure the approximation quality for instances which are intractable for the exact algorithm. Experimental results are summarized in Table 3.

First, we observe that for lines the approximation ratio is unity across all instances. As it might be conjectured from these numerical results, one can also convince oneself theoretically that the approximation algorithm always finds an optimal solution on lines.

Furthermore, Table 3 shows that for ring and ladder graphs the average approximation ratio is less than or equal to 1.25, which is well below the theoretical value of 4. This indicates that the theoretical approximation factor of 4 underestimates the approximation quality observed in practice. As already mentioned, the proof in [15] delivers an approximation ratio of strictly less than four.

² In fact, token swapping can be solved efficiently on lines, rings, cliques, stars and brooms, see [12].

Comparing runtimes, we note a drastic improvement with growing graph size on all graph types. For the largest instances the exact algorithm was able to solve in reasonable time, an average reduction of 96.5% is achieved. Moreover, all absolute runtimes for the approximation algorithm are well below 1 s. Altogether, the approximation algorithm typically delivers close-to-optimal solutions with a significant time saving on all instances considered.

5.3 Benchmarking the Routing Algorithm

In the following, the proposed routing method (abbreviated TAP+TS) is compared to various routing algorithms integrated in different open source quantum compilers: The binary program (BIP) by Nannicini et al. [18], solved via *Gurobi*, Li et al. Sabre heuristic [14], Qiskit's default compiler Stochastic swap [20], Cambridge Quantum Computing's Tket [5, 23], Google's Cirq [2], as well as the Bounded mapping tree (BMT) by Siraichi et al. [21] which is based on a decomposition approach similar to ours. BMT employs dynamic programming to solve a problem closely related to the TAP.

We briefly summarize the Python implementation of our routing algorithm: first, the two-qubit gates of the input circuit are grouped in layers containing as much gates as possible. These gate groups together with the hardware graph form a TAP instance. Next, the network flow model of the TAP instance is solved by *Gurobi*. The resulting TAP solution defines a set of token swapping problems, which are then solved by the approximation algorithm described in Sect. 4. Finally, the routed quantum circuit is constructed by changing the target qubits in the gates of the original circuit according to the allocation sequence and by adding the computed swaps between subsequent allocations.

As performance metrics, we measure the resulting gate count and circuit depth. Benchmarks are performed on two different classes of circuits. The first class is taken from a publicly available benchmark library while the second is generated according to a protocol used for benchmarking quantum hardware.

5.3.1 QUEKO Circuits

First, we use the publicly available benchmarks circuits generated by Tan and Cong in [24]. They propose a method for generating random circuits on a given hardware graph with given depth as well as given single and two-qubit gate numbers. Their construction delivers instances of the routing by swap insertion problem with optimal value 0, i.e. there exists an allocation of logical to physical qubits such that all gates can be performed without inserting swaps.

We use the “benchmark for near-term feasibility” (BNTF) circuit set from [24]. It is subdivided into circuits on a 16-qubit hardware graph (Rigetti's Apsen-4, Fig. 9a) with few gates, and circuits on a 54 qubit graph (Google's Sycamore, Fig. 9b) with many gates. For each hardware graph, 90 circuits are available 10 for each depth in $\{5, 10, \dots, 45\}$. On each instance the time limit for BIP is the set to 3600 s. The results are visualized in Fig. 10. By construction, our algorithm will find a zero-swap

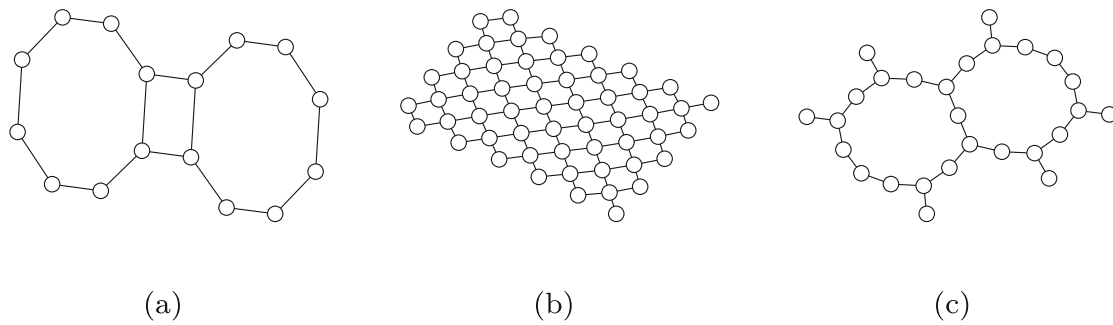


Fig. 9 Hardware graphs of *Aspen-4* (a), *Sycamore* (b) and *ibmq_ehningen* (c)

solution if the TAP is solved to optimality. This can be observed in Fig. 10a, b. For this benchmark set, we achieve improvements in resulting depth of up to a factor of 2 on small circuits (Fig. 10c) and up to factor of 5 on larger circuits (Fig. 10d), compared to the best heuristic. This improvement comes at the cost of increased runtime. Indeed, the runtimes of all other heuristics applied in this benchmark are in the order of a few seconds (not shown in Fig. 10), while the proposed algorithm takes up to 12 s on average for small circuits (Fig. 10e) and up to 5 min on average for larger circuits (Fig. 10f). However, we note a drastic reduction of runtime compared to the exact approach BIP (Fig. 10e) which is unable to find optimal solutions on larger instances in less than one hour.

5.3.2 QV Circuits

For a second benchmark, we consider quantum volume (QV) circuits. According to [16], a QV circuit of depth L and width m is a circuit on m qubits with L layers where each layer consists of $\lfloor m/2 \rfloor$ random two-qubit gates. Thus, the resulting circuit is maximally dense in terms of gates per layer. QV circuits of equal depth and width are used to benchmark quantum computers (see [6]). We consider QV circuits of equal depth and width. Circuits are constructed from the gate groups of the TAP instances generated in Sect. 5.1 resulting in QV circuits with depths between $L = 4$ and $L = 8$. As in Sect. 5.1, tests are performed on a 8-vertex-line, ring, Y and ladder graph. Thus, the TAP instances associated with the routing instances generated here are exactly those of Sect. 5.1.

Optimal-depth solutions cannot be calculated in reasonable time for all instances. However, BIP would find optimal solutions in terms of depth when solved to optimality. Thus, we measure resulting depth and gate count relative to the BIP solution with a runtime limit of 7200 s. Our results are visualized in Figs. 11 and 12. In Figs. 11c, d and 12c, d we observe that the proposed algorithm finds solutions which are close-to-optimal in terms of depth, assuming BIP returned an optimal-depth solution after a runtime of 7200 s. The average ratio of TAP+TS solution to BIP solution is at most 1.25%. Furthermore, it performs best among all considered routing heuristics, in particular better than the conceptually similar method BMT. This also holds true when comparing gate numbers, as can be seen in Figs. 11a, b and 12a, b. Interestingly, we observe that the proposed method delivers smaller gate numbers than BIP for some instances on all topologies, as indicated by values less than 1 in Figs. 11a, b and 12a,

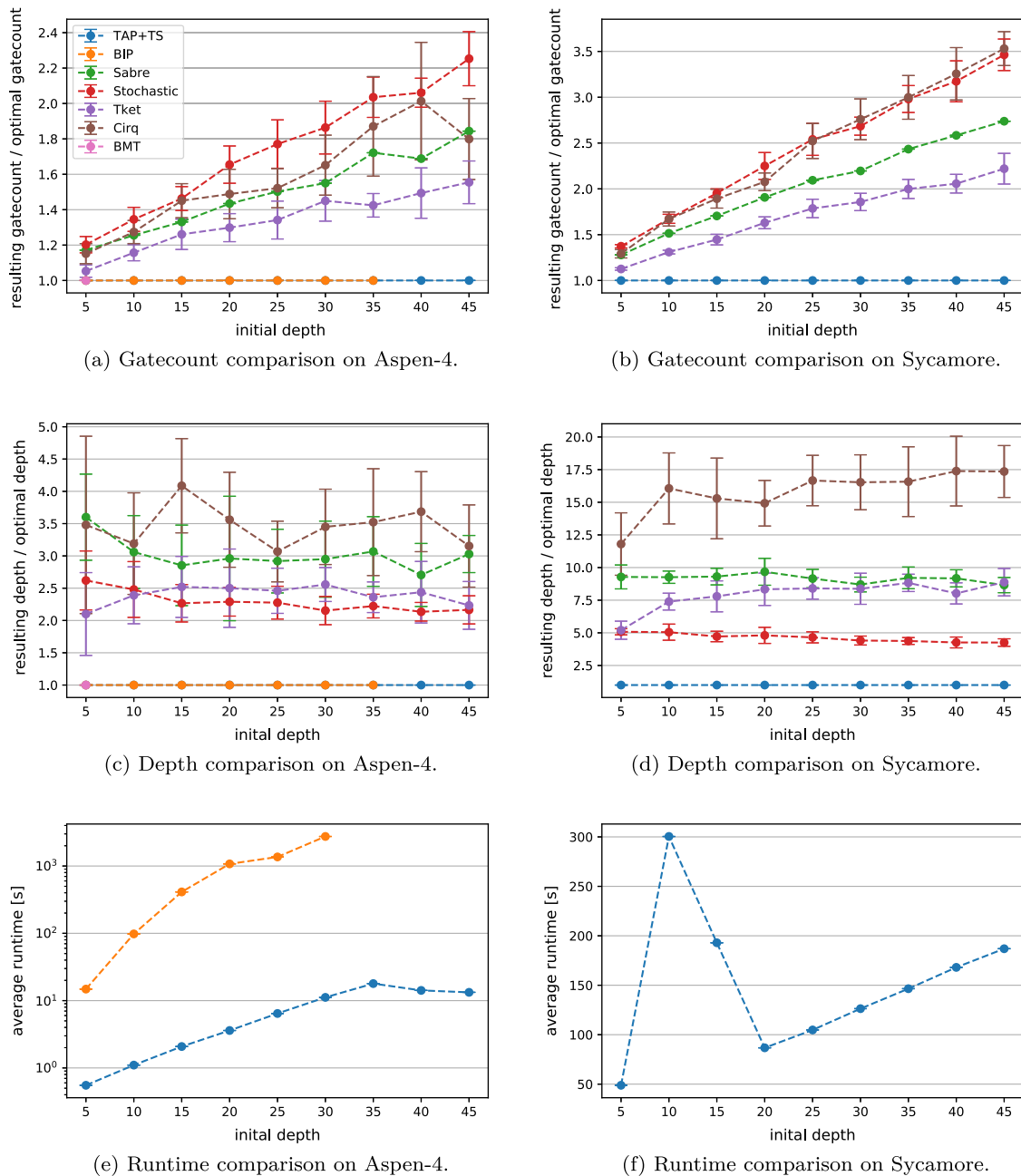


Fig. 10 Comparison of different routing algorithms on the QUEKO benchmark set. Data points are averaged over 10 instances. Error bars indicate the empirical standard deviation. The legend in **a** holds for all plots. Instances are the BNTF circuits from [24] on Rigetti's *Aspen-4* 16-qubit hardware graph (left plots) as well as Google's *Sycamore* 54-qubit hardware graph (right plots). For every BNTF instance, a zero-swap solution exists. Missing data points for BIP and BMT indicate that no solution was returned after 3600s runtime. **a**, **b**: Gatecount relative to optimum. **c**, **d**: Depth relative to optimum. **e**, **f**: Average runtime for BIP and BMT. Other heuristics are not shown since all runtimes are in the order of 1 s

b. Again, the improvement comes at the cost of increased runtime. While the other heuristics require several seconds (not shown in Figs. 11, 12), the proposed method takes up to 900s on average (Fig. 11f). Still, the runtime is significantly shorter than for the exact method BIP which runs into the time limit of 7200s on large instances (Figs. 11f, 12f).

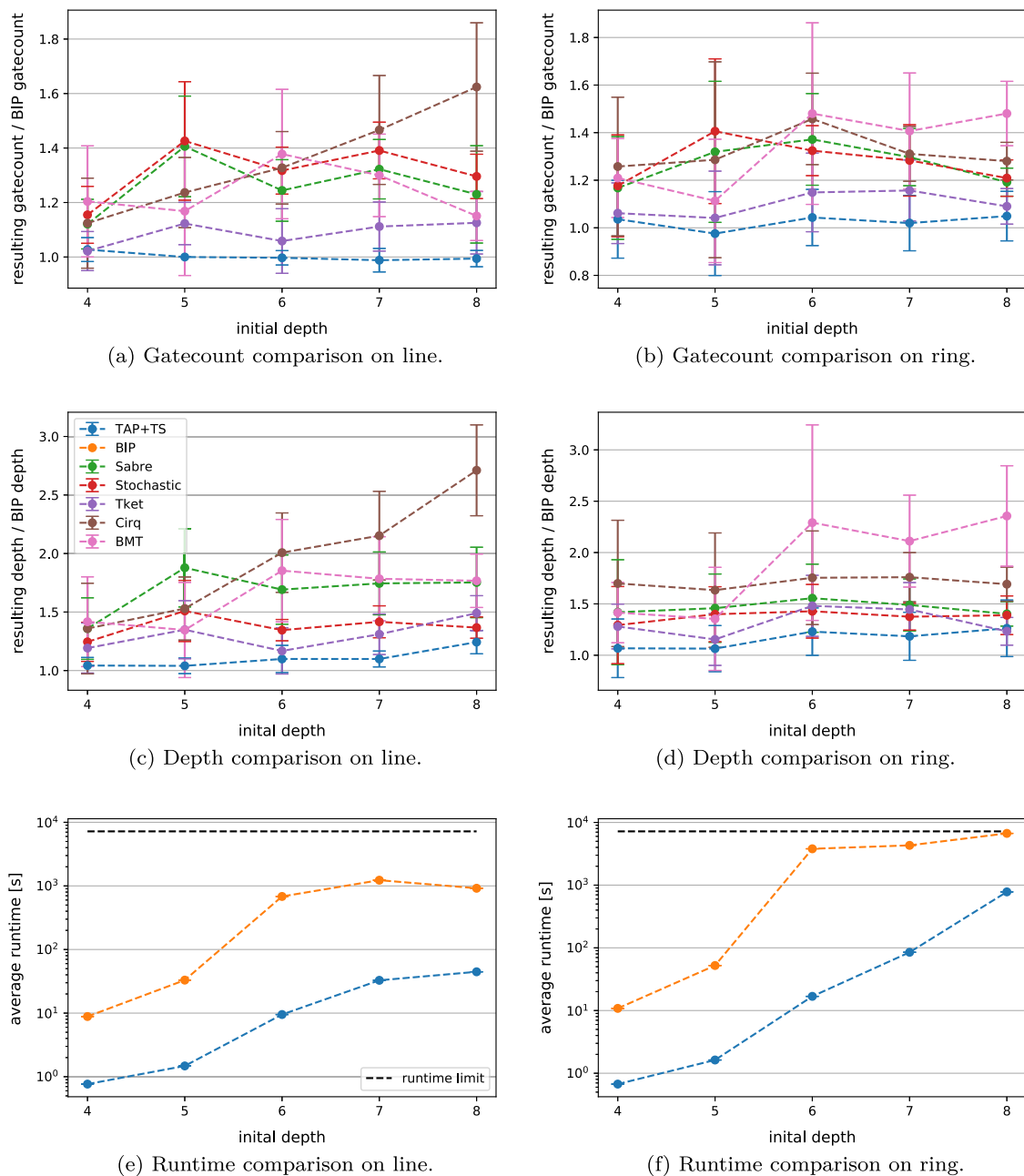


Fig. 11 Comparison of different routing algorithms on the QV benchmark set. Instances are QV circuits of equal width and depth [6, 16] on a 8-vertex line (left plots) and a 8-vertex ring (right plots). Data points are averaged over 10 instances. Error bars indicate the empirical standard deviation. The legend in **c** holds for all plots. **a, b**: Gate count relative to BIP. **c, d**: Depth relative to BIP. **e, f**: Average runtime. The dashed line indicates the runtime limit of 7200s. Other heuristics are not shown since their runtimes are in the order of 1 s

We discuss three reasons why the improved routing solutions can be worth the additional time investment. First, access to currently available quantum hardware is often organized by a waiting queue, such that the time for compilation is not a critical factor. Second, different quantum circuits with equal connectivity graphs only need to be routed once. A popular example are parameterized quantum circuits which are often considered as promising candidates for first quantum applications. The connectivity

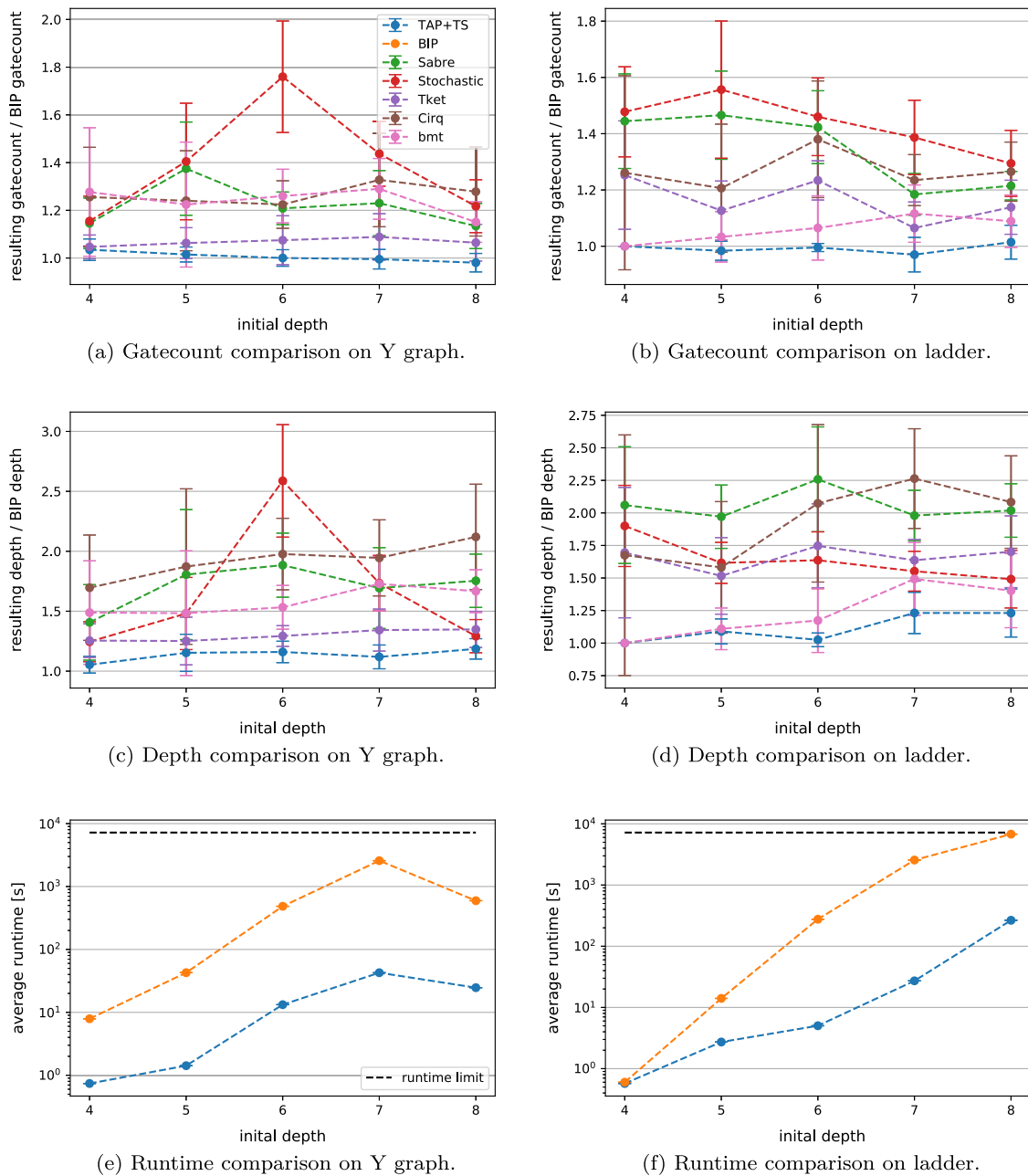


Fig. 12 Comparison of different routing algorithms on the QV benchmark set. Instances are QV circuits of equal width and depth [6, 16] on a 8-vertex Y-graph (left plots) and a 8-vertex ladder-graph (right plots). Data points are averaged over 10 instances. Error bars indicate the empirical standard deviation. The legend in **a** holds for all plots. **a, b**: Gate count relative to BIP. **c, d**: Depth relative to BIP. **e, f**: average runtime. The dashed line indicates the runtime limit of 7200s. Other heuristics are not shown since their runtimes are in the order of 1 s

graph of such a circuit is not parameter dependent. Third, current quantum hardware benefits strongly from good routing solutions. This is demonstrated in the next section.

5.4 Performance on Real Hardware

Lastly, we demonstrate the influence of routing quality on the performance of current quantum hardware. For this purpose, we compile example quantum algorithms to a

Table 4 Results for compiling quantum algorithms to real hardware with different routing algorithms

N	$\#_{2q}^{\text{TAP+TS}}$	$\#_{2q}^{\text{Tket}}$	$\#_{2q}^{\text{Sto}}$	$d^{\text{TAP+TS}}$	d^{Tket}	d^{Sto}	r_{ideal}	$r^{\text{TAP+TS}}$	r^{Tket}	r^{St}
7	21	31	54	22	28	41	0.875	0.77	0.61	0.56
8	25	71	80	21	53	65	0.75	0.64	0.52	0.51
9	30	54	63	21	39	45	0.84	0.65	0.58	0.55

Compiled quantum circuits are QAOA for MaxCut on ring graphs with N vertices. The target quantum device is *ibmq_ehningen*. Given are the total two-qubit gate counts $\#_{2q}^{\text{TAP+TS}}$, $\#_{2q}^{\text{Tket}}$, $\#_{2q}^{\text{Sto}}$ for the circuit routed with TAP+TS, Tket and Stochastic swap, respectively, as well as the corresponding depths $d^{\text{TAP+TS}}$, d^{Tket} , d^{Sto} . Furthermore, we give the theoretical approximation ratio r_{ideal} as well as the actually achieved approximation ratios $r^{\text{TAP+TS}}$, r^{Tket} , r^{St} .

real hardware device using different routing methods and compare the results after execution.

The example algorithms are the Quantum approximate optimization algorithm (QAOA) with depth parameter $p = 1$ applied to MaxCut on ring graphs with 7, 8 and 9 vertices. QAOA is a well-known hybrid quantum-classical algorithm. A detailed description is given by Farhi et al. in [7]. We choose QAOA for three reasons: First, it is often considered as a promising candidate for near-term quantum applications because of its universality and adaptive complexity. Second, routing is an essential requirement when running QAOA on real hardware, since QAOA employs two-qubit gates along the edges of an underlying problem graph which usually is not isomorphic to the hardware connectivity graph. This is also the case in our experiment. Third, the approximation ratio of QAOA is an easily accessible, single-metric performance measure. The approximation ratio is defined as $r = \mathbb{E}(c)/c^*$. Here $\mathbb{E}(c)$ denotes the expectation value of the cut size c produced by QAOA, and c^* is the size of an optimal cut. QAOA is a parameterized quantum algorithm. For QAOA with depth $p = 1$ applied to MaxCut on ring graphs, the optimal parameters can be calculated analytically, cf. [25].

The routing algorithms tested are the proposed method TAP+TS, Tket and Stochastic swap. For circuit construction, compilation and backend communication, we use IBM's SDK Qiskit [20]. The quantum backend in our experiment is *ibmq_ehnigen* [10], see Fig. 9c. We compile each of the QAOA circuits with all three routing methods while other compiler options remain unchanged.

Table 4 compares the gate counts and depths of the compiled circuits as well as the achieved approximation ratios after execution on *ibmq_ehnigen*. The proposed routing procedure TAP+TS performs best both in terms of swaps added and depth increase on all instances, followed by Tket. For the instance with $N = 8$, we note large reductions in gate count of 65% and 69% when compared to Tket and Stochastic swap, respectively. Similarly, the circuit depth reductions amount to 60% and 68%, respectively. We observe that small gate count and depth correlate with an increase of approximation ratio on all instances. For the $N = 8$ instance, we observe an increase of 23% compared to Tket. The influence of depth and gate count on noise in quantum computers are currently investigated in depth by different groups. Although

the relations are not yet fully understood, this experiment shows a clear correlation between routing quality and quantum algorithm performance.

Of course, this experiment is by no means a comprehensive benchmark of our routing method. However, it stresses the need for good compiling methods in quantum computation on currently available hardware and shows clear benefits of our approach.

6 Conclusion

In this article, we have studied the problem of qubit routing by swap insertion. This problem needs to be solved in the context of quantum compilation but is practically intractable even for moderate instance sizes when solved by an exact approach. We have proposed a new decomposition approach that exploits the capabilities of exact integer programming while reducing complexity significantly, at the expense of possibly returning suboptimal solutions. Improvement in performance is achieved by solving only a subproblem by integer programming. The objective of this subproblem, called token allocation problem (TAP), is a lower bound on the overall routing problem. We strengthened the model of the TAP by deriving valid inequalities. The solution of the TAP defines the second subproblem, which is the well-known problem of token swapping. Its solution yields the actual objective value of the overall routing problem. For the token swapping problem, we have enhanced an existing approximation algorithm and have developed an exact branch-and-bound algorithm. Combining the solutions of both subproblems produces a solution to the overall routing problem.

Finally, we have given experimental results showing the applicability of the proposed algorithm. Although it being a heuristic approach, we observed close-to-optimal solutions outperforming other heuristics. This improvement comes at the expense of increased solution time; however, the solution time is still much smaller than for exact approaches. Moreover, the lower bounds computed in the TAP turn out to be tight, which we consider as a reason for the good performance of the proposed decomposition method.

Our experiments on actual quantum hardware show the impact of high-quality solutions to qubit routing for quantum computation on current hardware. For the example algorithm QAOA, we observed an increase in approximation ratio when compiled with the proposed routing method compared to standard heuristics.

A possible field of future research is the structural analysis of the TAP in order to further strengthen its integer programming formulation. Additionally, practical experience with current quantum hardware suggests more complicated cost functions to be considered: specific qubits and gates suffer from higher error rates than others. Even more complicated, executing gates on specific qubit pairs simultaneously results in high error rates—a phenomenon known as *crossstalk*. Incorporating these effects in the routing algorithm is an additional direction of future research.

Acknowledgements This research has been supported by the Bavarian Ministry of Economic Affairs, Regional Development and Energy with funds from the Hightech Agenda Bayern as well as the Center for Analytics—Data—Applications (ADA-Center) within the framework of “BAYERN DIGITAL II” (20-3410-2-9-8). The research is part of the Munich Quantum Valley, which is supported by the Bavarian state



Efficient compiler optimization by modeling passes dependence

Jianfeng Liu¹ · Jianbin Fang¹ · Ting Wang¹ · Jing Xie¹ · Chun Huang¹ · Zheng Wang²

Received: 7 July 2024 / Accepted: 30 September 2024 / Published online: 18 December 2024
© The Author(s) 2024

Abstract

Selecting the optimal combination of compiler passes is a significant challenge to enhance performance and reduce the code size of compiled binaries. While a well-selected sequence of compiler passes can yield considerable benefits, the large number of potential combinations and the scarcity of effective ones make this task prohibitively complex. To tackle this problem, we propose a novel approach to group compiler passes into a small set of sub-sequences. This approach translates the task of identifying the right compiler passes combination into determining the appropriate combination of these sub-sequences. We apply our approach to CBench and PolyBench, demonstrating remarkable performance improvements. Our approach enhances runtime performance by 22% compared to the default LLVM ‘O3’ option, and achieves a code size reduction of 24% compared to the ‘Oz’ option. Our approach also outperforms state-of-the-art across various optimization tasks and hardware platforms.

Keywords Compiler phase ordering · Passes clustering · Iterative compilation

1 Introduction

Modern compilers offer a wide range of optimization passes to enhance the performance of compiled code. To facilitate their usage, compilers provide tuning flags to customize individual optimization passes, such as turning on or off a pass, changing the threshold for vectorization and loop unrolling. While the compiler’s default options like ‘O3’ for performance optimization and ‘Oz’ for code size reduction provide a bundle of default compiler passes, prior studies have demonstrated that customizing compiler flag combinations can lead to significant performance improvements compared to the default settings (Ansel et al. 2014; Fursin et al. 2011).

However, the search space for potential compiler pass combinations is vast, making it challenging to identify the optimal pass sequence. Modern compilers such as LLVM offer over 100 general-purpose optimization passes,¹ which can be applied iteratively. This results in an exponentially growing search space with regard to the sequence length. The vast optimization space makes it a highly daunting task for discovering the right compiler pass sequence that enhances performance beyond the default settings. Despite the potential for substantial performance gains, the execution time required to explore this space based on searching algorithms can be prohibitively long, limiting their practical adoption. This problem is also known as *phase-ordering* (Vegdahl 1982).

The prior approaches to tackling the phase-ordering problem in compiler optimization include search-based algorithms and iterative compilation techniques. The search-based algorithms, such as genetic algorithms and simulated annealing, have been used to explore the optimization space (Fursin et al. 2011; Cooper et al. 1999). However, these methods can be computationally expensive and time-consuming due to the large number of possible pass combinations. Iterative compilation frameworks, like OpenTuner (Ansel et al. 2014), aim to reduce the search space by iteratively refining the optimization passes based on

✉ Jianfeng Liu
liujianfeng14@nudt.edu.cn

✉ Jianbin Fang
j.fang@nudt.edu.cn

✉ Ting Wang
tingwang@nudt.edu.cn

Jing Xie
xiejing@nudt.edu.cn

¹ National University of Defense Technology, Changsha, China

² Northwest University, Xi’an, China

¹ <https://llvm.org/docs/Passes.html>.

performance feedback. While these frameworks have shown promise, they still face challenges in efficiently navigating the vast search space to find optimal pass sequences.

This paper presents Dependence-based Compiler Optimization framework (DCO), a new approach for reducing the search space of phase ordering. Our key insight is that some compilation passes exhibit mutual benefits or dependencies, making it beneficial to consider and use them together, e.g., incorporating loop unrolling and instruction scheduling is likely to benefit a vectorization pass. By identifying and organizing these related passes into *sub-sequences* to leverage their interdependencies, we can translate the problem of finding a sequence of individual compiler passes into seeking a combination of subsequences. Since the number of sub-sequences is significantly smaller than the number of individual compiler passes, doing so can substantially reduce the overall search space and greatly diminish the search overhead.

The core of our work is modeling two fundamental types of relations to capture compiler pass dependence: *source code dependence* and *pairwise performance dependence*. These dependences are determined by analyzing the order in which compilation passes are invoked and by evaluating the performance of pairwise combinations of optimization passes. By effectively capturing these dependences, we construct an optimization dependence graph (ODG) to further generate sub-sequences. Our approach generates sub-sequences by clustering this optimization dependence graph, which are then specifically designed to be applied within two iterative compilation frameworks [i.e., OpenTuner (Ansel et al. 2014) and LaMCTS (Wang et al. 2020)]. Our approach facilitates more targeted and efficient reduction of the optimization space, leading to improved performance within the iterative compilation frameworks.

Our experimental results demonstrate the performance advantages of DCO over state-of-the-art approaches. In the two iterative compilation frameworks, DCO achieves an up to 1.22x speedup compared to the ‘O3’ optimization sequence, and a 1.24x improvement in code size reduction compared to the ‘Oz’ sequence. These results confirm the effectiveness of our approach in enhancing both running-time performance and code size optimization.

The key contributions of our paper are summarized as follows.

- We introduce a novel approach to model compiler pass dependence through source code dependence and pairwise performance dependence, and construct an optimization dependence graph (ODG) to serve as the basis for generating optimization sub-sequences (Sects. 3.1–3.4).
- We implement our approach within two iterative compilation frameworks, OpenTuner and LaMCTS,

demonstrating its applicability and effectiveness (Sect. 3.5).

- Our experimental results show substantial improvements in both runtime performance and code size optimization tasks, validating the superiority of our method over state-of-the-arts (Sect. 4).

2 Background and motivation

2.1 Problem definition

Our work targets the classical compiler optimization problem of *phase-ordering* (Ashouri et al. 2018; Wang and O’Boyle 2018; Leather et al. 2020). This problem considers the selection and ordering of compiler passes to be applied to the target problem, typically observed in LLVM optimizations. This differs from compiler pass selection, which focuses on selecting the optimal set of compiler passes without considering their order, as seen in GCC.

The designed space of compiler phase ordering can be vast. In this work, we let Ω denote the whole optimization space, N denote the number of available compiler passes, and M denote the maximum length of compiler optimizations. We use these notations to define the problem size for *pass selecting* in Eq. 1.

$$\Omega_{select} = 2^M \quad (1)$$

Note that some optimization passes in compilers exist in intermediate states beyond a simple on or off setting. For instance, configurable parameters like loop tiling factors or loop vectorization factors can take on multiple values. Thus, Eq. 1 only offers a lower-bound estimate of the optimization space, while the actual space can be significantly larger. GCC currently incorporates over 200 optimization passes (flags), and attempting to encompass the entire optimization space would result in an exceedingly vast search space.

Meanwhile, determining a suitable optimization sequence for LLVM will introduce an even larger search space. Note that merely selecting a set of optimization passes for LLVM is inadequate because how to arrange the order of these passes can also impact performance. The problem size for phase ordering can be quantified with Eq. 2.

$$S|\Omega_{Phase}| = \sum_{i=1}^M N^i \quad (2)$$

Optimizing codes in LLVM is particularly challenging, as it offers more than 100 compiler passes for code optimization. Considering the length of the predefined ‘O3’ sequence in LLVM, which is approximately 300 (version 12.0.1), the size of the optimization space becomes overwhelmingly large, exceeding 10^{600} according to the Eq. 2. This immense

search space makes it infeasible to exhaustively explore and find the optimal solution to the phase ordering problem.

Our work mainly focuses on the phase-ordering problem and tries to reduce the optimization space by clustering optimization passes into several sub-sequences.

2.2 Motivation

Previous studies have investigated using *sub-sequences* to address the extensive search space linked with compiler optimization passes. A sub-sequence, which comprises one or multiple optimization passes arranged in a specific order, acts as the basic unit. Categorizing compiler passes into k groups leads to k larger passes, effectively reducing the overall search space. Several compiler optimization works are based on sub-sequences.

MiCOMP (Ashouri et al. 2017) generates optimization sub-sequences from the ‘O3’ sequence. It models the ordering relationship within the sequence as pass dependence and clusters all passes occurring in ‘O3’ into several sub-sequences. By analyzing each pass’s dependence, MiCOMP effectively groups passes to create sub-sequences that maintain the optimization benefits of ‘O3’.

CORL (Mammadli et al. 2020) also derives optimization sub-sequences from the ‘O3’ sequence. It follows the observation that the optimization sequence consists of smaller logical sub-sequences ending with a ‘simplifycfg’ pass. This generation is empirical and requires human effort to identify and segment these logical sub-sequences.

POSET-RL (Jain et al. 2022) constructs optimization sub-sequences using IR2Vec embeddings and the ‘Oz’ optimization sequence. The framework identifies critical paths and nodes with high degrees in the Oz dependence graph and creates paths representing sub-sequences of optimization passes. These sub-sequences are designed to efficiently optimize both binary size and execution time.

Table 1 Comparison among sub-sequence based methods

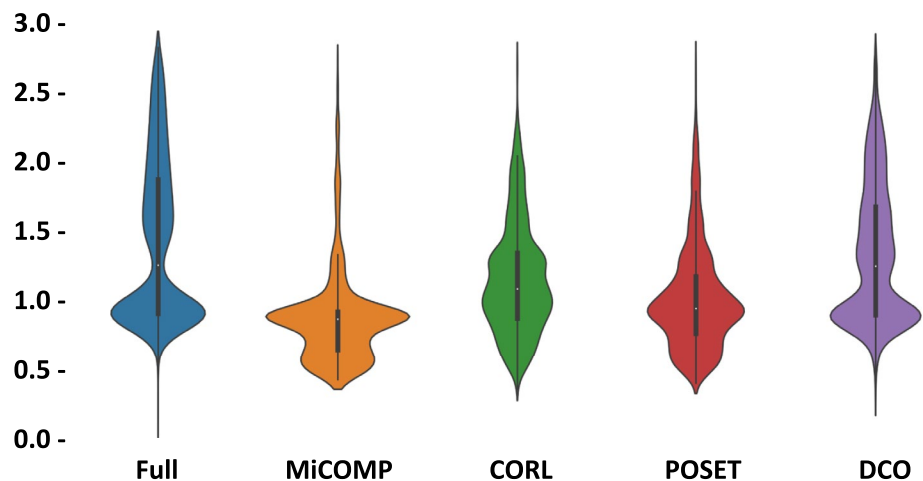
Method	Representative	Evenness	Automation
MiCOMP	★	★	★★
CORL	★★	★★★	★
POSET-RL	★★	★★★	★
DCO	★★★	★★★	★★★

However, the prior works based on sub-sequence have drawbacks in *representatives*, *evenness* and *automation*, which are summarized in Table 1.

The generated sub-sequences of the prior approaches are not sufficiently representative In this paper, we use the term *representatives* to describe the optimization potential of compiler sub-sequences. Figure 1 presents the performance results of pairwise combinations of units from different optimization spaces, where ‘Full’ represents the entire optimization space without applying any sub-sequence method. We evaluate the differences using two metrics: the mean value of the combination results and the threshold of cover range. The figure clearly shows that the ‘Full’ space outperforms the current sub-sequence methods on both metrics.

Not all combinations of sub-sequences produce favourable results. For instance, most combinations of sub-sequences in MiCOMP do not perform better than the non-optimized code. This implies that the sub-sequences in MiCOMP may lack the required diversity or effectiveness to achieve significant optimizations. Conversely, our approach, named DCO, achieves performance comparable to the ‘Full’ space by effectively combining different sub-sequences and covering a wider optimization space. This suggests that DCO can generate more representative sub-sequences within the optimization space, thus striking a better balance between optimization potential and computational efficiency.

Fig. 1 Pairwise combination of sub-sequences from different search spaces, where the ‘Full’ space means the entire space formed by all optimization passes



The generated sub-sequences show a notable issue of unevenness We found that the *unevenness* of sub-sequence construction not only affects its effectiveness but also influences the compilation time and tuning process of iterative compilation. We will discuss the details in Sect. 4. The reason behind this could be that when sub-sequences are unevenly constructed, there tends to be one longer sub-sequence compared to the others. A longer sub-sequence has a more significant chance of being chosen during iterative compilation, leading to increased compilation times.

The sequences of optimization passes in MiCOMP show significant unevenness in their distribution. Most passes are concentrated in one long sub-sequence, while other sub-sequences usually contain only one or two passes. This disparity primarily comes from the usage of the standard ‘O3’ optimization sequence, which typically includes most passes only once. Additionally, the lack of clear distinctions among compiler passes may introduce inconsistencies in the analysis of pass dependencies. For example, some passes with no direct relationship might be sequenced together in ‘O3’, which could result in erroneous information, misleading the analysis into suggesting that these passes are highly related. Moreover, the frequent co-occurrence of passes leads to extensive chains within the compiler, creating a clustering phenomenon where numerous passes accumulate in a single long sub-sequence. This disproportionate distribution inadvertently extends the overall length of the optimization sequence, potentially resulting in prolonged compilation times and sub-optimal performance.

The sub-sequences are handcrafted and have limited applicability Sub-sequences can be generated either manually or automatically. Manual construction of sub-sequences has been explored in CORL and POSET-RL. These predefined sub-sequences have shown promising performance on specific optimization tasks but require human effort for generation, limiting their applicability across different scenarios. Another limitation is the restricted set of optimization passes used in these frameworks. For instance, CORL and POSET-RL both rely on

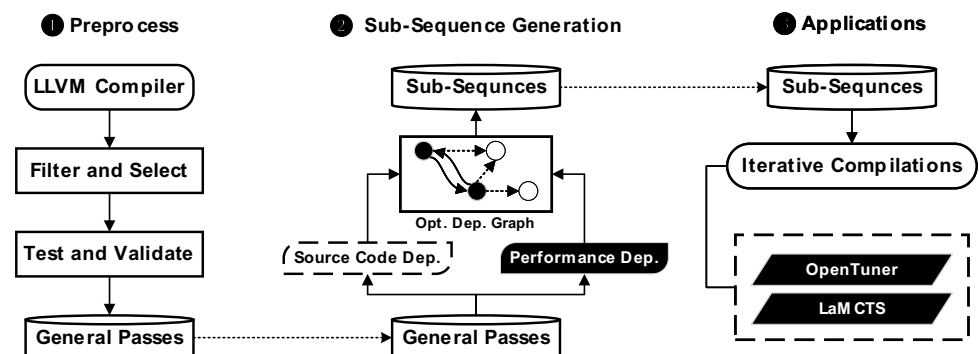
predefined sequences like ‘O3’ or ‘Oz’ in the compiler, which represent only a subset of the available optimization passes. This constraint might lead to a loss in optimization potential as some useful passes may not be included in the optimization space. On the other hand, MiCOMP demonstrates the automatic construction of optimization sub-sequences. However, these sub-sequences are typically designed for specific optimization tasks and may not be suitable for other optimization tasks, e.g., MiCOMP is only applicable to the running-time optimization task. Consequently, when encountering new optimization situations or even the same problem on different architectures, the effectiveness of handcrafted sub-sequences may diminish. Therefore, the generality of handcrafted sub-sequences is a significant constraint, necessitating more adaptive and versatile approaches.

Our analysis indicates that current approaches have limitations in these areas, often resulting in unreliable optimizations. These shortcomings are primarily due to the lack of well-defined boundaries between compiler passes and ambiguity in defining sub-sequences, hindering the effectiveness of optimization strategies. To address these limitations, our work introduces a new approach that uses in-depth dependence analysis and constrained clustering to automatically generate sub-sequences. By focusing on these key features, our method aims to overcome the drawbacks of existing methods and provide more robust and effective optimization strategies. The in-depth dependence analysis ensures that our sub-sequences are highly representative and evenly distributed, while the constrained clustering enhances adaptability to various optimization contexts and tasks.

3 Our DCO approach

The large number of available compiler optimization passes can result in an exceedingly large search space, making it difficult to find the optimal solution. To address this challenge, we propose a novel method that generates groups of

Fig. 2 The overall framework of our DCO approach



reliable sub-sequences through constraint clustering, shown in Fig. 2. Our method is based on a comprehensive dependence analysis that considers both source code dependence and performance dependence, allowing us to identify and cluster related passes into meaningful sub-sequences. DCO has three main steps: passes preprocessing (❶), sub-sequence generation (❷), and sub-sequence application (❸).

3.1 Preprocess compiler optimization passes

3.1.1 Compiler passes analysis

Compiler optimization passes are used to control the behavior of the compiler and influence the quality of generated codes. LLVM has a diverse set of optimization passes, encompassing various aspects of code optimization. These passes can be classified into three categories: *analysis passes*, *transform passes*, and *utility passes*.

Analysis passes They serve the purpose of gathering information that can be used for subsequent optimization tasks. These passes do not directly transform the code but provide insights into the program's structure, characteristics, and dependencies. They lay the foundation for informed decision-making during the optimization process.

Transform passes They are responsible for code transformation. These passes apply various code transformations, such as loop unrolling, function inlining, or vectorization, to change the code structure. Applying transform passes has a significant impact on the performance of the resulting executable, in terms of execution time, code size, power consumption, and many others.

Utility passes They are designed to provide auxiliary information or debugging messages to programmers. While these passes do not directly influence programs' performance, they serve as useful tools for developers in understanding the behavior of the compiler and aiding the debugging process.

Being unrelated to performance enhancements, the utility passes are excluded from our consideration. Our work focuses primarily on the *analysis passes* and *transform passes*, because they directly contribute to performance optimization.

3.1.2 Automated pass filtering

Figure 3 shows that we have developed an automatic tool to extract the relevant optimization passes from the LLVM command line utility 'opt'. Our tool uses a three-step filtering

process to identify the general-purpose passes that are applicable for compiler optimization.

❶ **Filtering unrelated passes** Initially, we obtain a comprehensive list of all available passes provided by LLVM by invoking the 'opt -help' command. To distinguish compiler optimization-related passes from the broader set, we employ pattern recognition techniques. Specifically, we target passes with descriptions that contain particular keywords or phrases indicative of their purpose. First, we filter out utility passes with keywords such as 'Print', 'Annotation', 'Configure', or 'Prepare', which are typically used for auxiliary functions like debugging, logging, and setting up the compilation environment. Next, we separate analysis passes from the remaining pool by identifying keywords like 'Analysis', 'Information', 'Profile', or 'Tracker' in their descriptions. These passes are primarily concerned with gathering and reporting data about the program's structure and behavior, which can be useful for further optimization but do not directly contribute to the transformation of the code itself. By applying these filters, we can focus on the actual optimization passes that directly alter the code to improve performance or reduce size, thereby refining our selection process and enhancing the efficiency of our optimization strategies.

❷ **Filtering architecture-specific passes** In addition to the initial filtering process, we undertake an essential step to pinpoint and exclude passes tailored to specific hardware architectures. Since LLVM supports a wide range of architecture-dependent optimizations, it is crucial to identify and exclude passes that are specific to architectures such as 'aarch64', 'arm', 'mips', 'x86', and others. The procedure for this architectural filtering step involves employing string pattern matching techniques to detect architecture-specific keywords in the pass descriptions. By carefully analyzing the descriptions and identifying these keywords, we can effectively filter out passes that are not universally applicable. This ensures that our optimization strategy remains architecture-agnostic and can be applied across a broad spectrum of target platforms. This step not only simplifies the optimization process but also enhances the portability and flexibility of the compiled code, making it suitable for diverse computing environments.

❸ **Validation and filtering of application-specific passes** We will rigorously test the remaining passes across all available benchmarks to determine their practical suitability. Our goal is to identify any passes that are ineffective, inapplicable, or cause compilation errors for any of the benchmark programs. If a pass shows no noticeable benefits or causes compilation errors across all benchmarks, it will be systematically removed. This validation process guarantees that the selected



Fig. 3 Procedure of the passes filtering tool

passes positively contribute to the optimization and maintain the stability of the compilation process.

The generated passes are shown in Appendix 1, which are regarded as general-purpose and conducive to compiler optimization. Once we have obtained all the valid general purpose optimization passes, we construct an optimization dependency graph based on two kinds of dependence: source code dependence (Sect. 3.2) and performance dependence (Sect. 3.3).

3.2 Identifying source code dependence

Many compiler optimization passes are designed to function in concert with one another, and their isolated application often fails to achieve the optimal results. This interdependence arises because certain transform passes rely on the output of other analysis or transform passes. For example, effectively utilizing the ‘-loop-unroll’ transform pass relies on the insights provided by the ‘-loops’ analysis pass, which provides crucial information about the structure of loops in the code. These relationships among compiler tuning passes, characterized by their dependence on each other’s output, are called *source code dependencies*. They represent the direct relations between various compiler passes and are pivotal in guiding the optimization process. By accounting for these source code dependencies, the compiler can make informed decisions regarding the order and application of different passes, ensuring that optimizations are executed in a manner that maximizes their performance.

In this work, we harness the LLVM APIs to systematically construct the graph of source code dependence relationships among compiler passes, shedding light on their intricate interactions and interdependencies within the optimization process. To achieve this, we employ the LLVM command line parameter ‘-debug-pass=Arguments’ while applying each pass individually. This will generate the printing of dependent passes in an ordered sequence. Typically, this sequence culminates with commands such as ‘-verify’ and ‘-write-bitcode’, which serve validation and data-writing purposes. Any pass that appears before the target optimization pass in this sequence is considered the source code-dependent pass for that particular target. These APIs help us understand how various passes within LLVM’s optimization pipeline depend on one another.

Figure 4 illustrates an example of the ‘-loop-unroll’ pass. By using the ‘-debug-pass=Arguments’ pass, LLVM generates a comprehensive report that discovers the dependence information associated with the ‘-loop-unroll’ pass. This report effectively shows the pass’s dependencies on its preceding and subsequent passes, offering insights into the intricate inter-dependencies that govern the optimization process. The resultant source code dependence graph is depicted in Fig. 6, where the rounded rectangles represent analysis passes, and the others denote transform passes. For simplicity, all dependent connections are assigned a weight of 1. This graphical representation overviews the intricate dependencies influencing the optimization process. We show the extracted source code dependence in Fig. 5.

3.3 Building performance dependence

Section 3.2 has successfully constructed the initial portion of the overall Optimization Dependence Graph (ODG) by establishing the source code dependence graph among compiler passes. But it is important to note that the source code dependence could solely capture the code-level dependencies among the optimization passes. Recognizing the substantial impact of the order in which transform passes are applied on the performance of the target code, we extend the ODG by incorporating the performance dependence relationship between transform passes. This will allow for a more accurate representation of the interdependence among the optimization passes.

In this work, the term ‘performance dependence’ refers to how the collective application of multiple compiler passes influences performance. We define performance improvement as the ratio of the observed performance difference when combining two compiler passes compared to applying them individually. To quantify this performance dependence, we introduce the metric $e(p, a)$ to represent the performance of the target program p when compiled with pass a . Similarly, we use the metric $e(p, a, b)$ to denote the performance of p when compiled with pass a followed by pass b . Note that in both performance optimization and code size compression tasks, a smaller e indicates better performance. The performance dependence can be formulated in Eq. 3. By comparing the metric values, we can assess the performance impact of different combinations of compiler

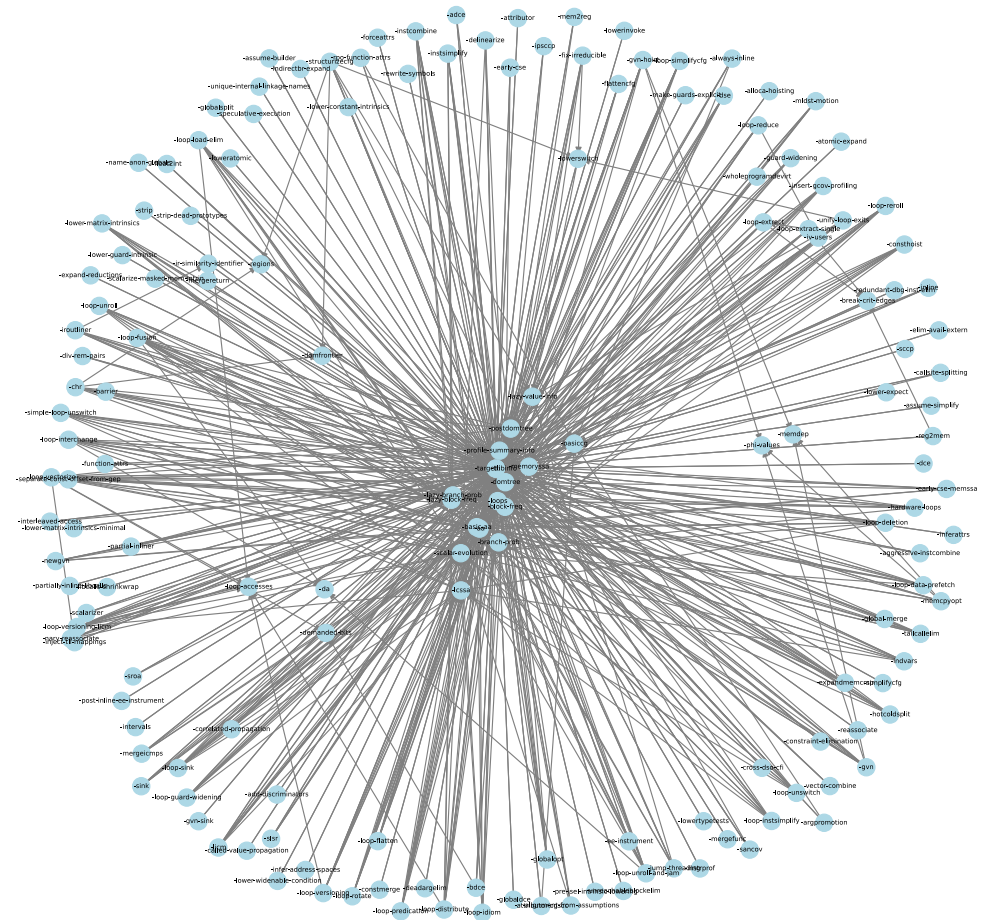
Fig. 4 Emitting the dependent passes

```

1  > opt -debug-pass=Arguments -loop-unroll test.ll -o
    ↪ test_opt.ll
    Pass Arguments: -targetlibinfo -tti -targetpassconfig
3  -assumption-cache-tracker -domtree -loops -loop-simplify
    ↪ evolution
5  -lcssa-verification -lcssa -basic-aa -aa -scalar-
    -loop-unroll -verify -write-bitcode

```

Fig. 5 Source code dependences extracted from LLVM compiler. Each directed edge means the back pass depends on the result of the front pass



passes. This analysis helps us understand how various passes work together to affect the overall performance of the compiled program.

$$dep_{per}(p, a, b) = \theta \cdot \left[\frac{e(p, a)}{e(p, a, b)} \right]^2 \quad (3)$$

where θ is used to control the effect of $\frac{e(p, a)}{e(p, a, b)}$ coming from the combination of passes a and b instead of an individual pass a or b . Supposing $e(p, a) < e(p, b)$, we consider θ under three cases shown as follows.

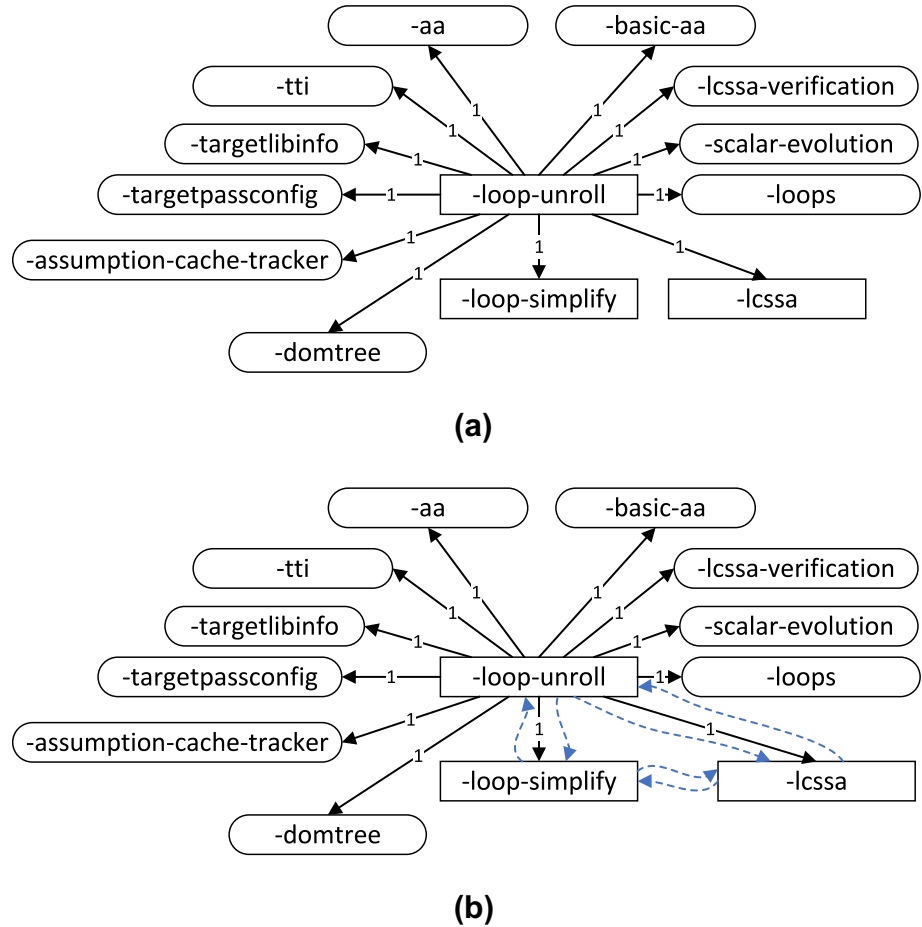
- When $e(p, a, b) < e(p, a) < e(p, b)$ indicating that the combined application of passes outperforms the individual application of each pass, we assign θ to be 1 to represent a positive performance impact.
- When $e(p, a) < e(p, b) < e(p, a, b)$, conversely, suggesting that using the combined passes results in a negative performance impact, we set θ to be -1.
- When $e(p, a) < e(p, a, b) < e(p, b)$, making it challenging to determine whether the combined pass yields a positive performance impact, we assign θ as 0 to denote the uncertainty.

This equation enables us to quantitatively measure the performance increase or decrease of applying the combination of passes (a, b) compared to applying pass a alone. By incorporating performance dependence into DCO, we obtain valuable insights into how compiler passes interact with each other and collectively impact performance. To illustrate this concept, we add dashed lines in Fig. 6 to indicate performance dependence among passes. These dashed lines represent the relationships between passes and depict how the combination of passes affects overall performance.

3.4 Clustering dependence graphs

Once the source code dependence and performance dependence are extracted from both Intel and ARM platforms, we will construct a dependence graph for all compiler optimization passes. This graph serves as a visual representation of the dependence relationships among the passes and aids developers in understanding the intricate interdependencies. We refer to this graph as the Optimization Dependence Graph (ODG).

Fig. 6 Construct optimization dependence graph for a sample optimization pass *-loop-unroll*, including constructing source code dependence graph (a) and adding performance dependence (b)



In the ODG, each node represents a compiler pass, while the edges denote the dependence relationships between pass pairs. Each edge of ODG carries information about the performance dependence or source code dependence between the related passes. The weight assigned to each edge quantifies the degree of dependence between two passes. By leveraging the ODG, developers can gain a deeper understanding of how different compiler passes interact and impact the overall optimization process.

Based on the constructed ODG, we employ a clustering algorithm to generate sub-sequences by grouping the graph nodes. We use the well-established K-means clustering algorithm. The standard K-means clustering algorithm has inherent limitations, particularly in its inability to ensure that clusters adhere to specific constraints, such as a maximum size or balanced distribution of elements. This shortcoming can lead to uneven sub-sequences where certain clusters become disproportionately large, while others remain underpopulated. Such imbalances may adversely affect the performance of subsequent processes that rely on these clusters. To address this issue, we introduce a constrained version of the K-means algorithm,

referred to as *Constraint K-means*. This modified algorithm (Algorithm 1) imposes a maximum threshold on the size of each cluster, ensuring that no single cluster exceeds this limit. The constraint is integrated into the clustering process by iteratively adjusting the assignment of points to clusters, particularly when a cluster reaches its maximum allowable size. By doing so, the algorithm not only maintains balanced clusters but also improves the overall effectiveness of the optimization process by avoiding the pitfalls of excessively large or small sub-sequences. The introduction of these constraints is crucial for applications that require a more uniform distribution of elements across clusters, as it enhances the stability and predictability of the resulting clustered sub-sequences. This constraint ensures that the resulting sub-sequences exhibit a balanced compiler pass distribution. We use the average link approach to measure affinity calculated in Eq. 4.

$$\phi_{x_i, \zeta_j} = \frac{1}{n_j} \sum_{x_k \in \zeta_j} e_{ik} \quad (4)$$

Algorithm 1 Constraint K-means Clustering

Require: A set of n sample points $X = \{x_1, x_2, \dots, x_n\}$, the target number of clusters K , and the maximum threshold of each cluster T .

- 1: **function** AddToNearestCluster(x_i):
- 2: Compute the affinity measure ϕ_{x_i, c_j} between x_i and each cluster c_j , forming a set $\Phi_{x_i} = \{\phi_{x_i, c_1}, \phi_{x_i, c_2}, \dots, \phi_{x_i, c_K}\}$ with descending order.
- 3: **for** $j = 1, 2, \dots, K$ **do**
- 4: **if** $\phi_{x_i, c_j} > T$ **then**
- 5: **if** $\phi_{x_i, c_j} > \max \phi_x$ **then**
- 6: Continue;
- 7: **else**
- 8: Delete the boundary point z from c_j ;
- 9: Add z in c_j ;
- 10: AddToNearestCluster(z);
- 11: **end if**
- 12: **else**
- 13: Add sample x_i to c_j ;
- 14: **end if**
- 15: **end for**
- 16: **end function**
- 17: Randomly form K initial cluster centers $C = \{c_1, c_2, \dots, c_K\}$.
- 18: **for** $i = 1, 2, \dots, n$ **do**
- 19: Select current sample point x_i ;
- 20: AddToNearestCluster(x_i);
- 21: **end for**

3.5 Sub-sequence application

This section applies the DCO sub-sequences within iterative compilation frameworks, highlighting the ability of these frameworks to explore and identify the extreme performance potentials of the generated sub-sequences. Iterative compilation frameworks, such as OpenTuner and LaMCTS, excel at systematically refining optimization sequences through repeated iterations, allowing them to uncover the highest-performing combinations of sub-sequences. By leveraging the iterative nature of these frameworks, DCO can achieve substantial performance improvements by dynamically adjusting and optimizing the sequence of compiler passes based on real-time feedback.

3.5.1 Enabling DCO in OpenTuner

OpenTuner is an advanced open-source tool renowned for its efficacy in general-purpose compiler auto-tuning [1]. It incorporates a wide range of search heuristics (e.g., Nelder-Mean, dynamic programming, evolution, and gradient ascent algorithms), making it a versatile and powerful solution. But it was originally designed specifically for GCC

compiler optimization, primarily focusing on the pass selection problem.

To apply DCO to OpenTuner with the LLVM compiler, several modifications are necessary. First, we define the search space as the collection of all possible optimization sequences, considering the unique characteristics and flexibility of the LLVM compiler. Unlike GCC, which adheres to a fixed optimization sequence, LLVM allows for a more adaptable and dynamic optimization sequence. To harness the capabilities of OpenTuner within the LLVM framework, we leverage the open APIs provided by OpenTuner. We use a sequence of *IntegerParameter* to identify a specific optimization sequence. By integrating OpenTuner into our workflow, we can systematically explore the vast search space of optimization sequences and identify the most optimal sequence based on the defined objectives.

3.5.2 Enabling DCO in LaMCTS

LaMCTS (Wang et al. 2020) operates in an iterative manner, managing a ‘good’ space and a ‘bad’ space based on observations made during the optimization process. The distinction between the ‘good’ and ‘bad’ spaces is determined by the performance outcomes achieved in different

optimization passes. This separation allows for targeted exploration in subsequent iterations to improve the optimization process. LaMCTS uses the Upper Confidence Bound for Trees (UCT) algorithm to intelligently select the next optimization sequence based on accumulated knowledge. The boundaries of the ‘good’ and ‘bad’ spaces are adaptively updated based on the outcomes of the current exploration. This dynamic adjustment process effectively refines the search regions, ultimately aiding in converging towards a stable state and identifying an optimal sequence of optimization passes.

By incorporating LaMCTS into the compiler optimization framework, we can effectively navigate the complex search space. Through a process of iterative exploration and refinement, LaMCTS efficiently identifies and takes advantage of promising areas within the search space, resulting in the discovery of the best sequence of optimization passes. This approach significantly enhances the overall performance and efficiency of the compiler optimization process, leading to improved outcomes for various optimization tasks. By utilizing its iterative and adaptive exploration capabilities, LaMCTS provides a promising solution for optimizing the selection of optimization passes, ultimately enhancing the performance of the compiler optimization process.

4 Performance evaluation

This section presents our experimental methodology and the evaluation setting used to gauge the performance of our DCO method. We outline the steps and considerations to ensure a rigorous and comprehensive evaluation. We first

outline the experimental settings, including the benchmark programs, target platforms, and compiler suite used in our study. Following that, we introduce the comparison baselines, which consist of current sub-sequence generation methods. These baselines are considered the most advanced sub-sequence-based methods in compiler optimization and will be used as a point of comparison to highlight the improvements achieved by our DCO method. Finally, we present the experimental details and analysis of the results. This involves a step-by-step explanation of our experimental procedure, from data collection to analysis. We then discuss the metrics used to evaluate performance, such as execution time, compilation time, and code size. The results will then be analyzed to identify trends, strengths, and weaknesses of our DCO method in comparison to the baselines.

4.1 Experimental setup

We use two benchmark suites, CBench and PolyBench, to evaluate the efficacy of DCO. These benchmarks are selected due to their widespread acceptance in compiler optimization. Table 2 details the programs used in our experiments.

- **CBench**, part of the Collective Knowledge framework, is a comprehensive collection of open-source programs for compiler research and auto-tuning. It has diverse real-world applications and computational kernels from different domains such as signal processing and numerical methods. This diversity in CBench ensures that our evaluation covers a wide spectrum of computational patterns, making it an excellent benchmark for assessing the adaptability and performance improvements offered by compiler optimizations across varied real-world scenarios.
- **PolyBench** is a benchmark suite that primarily focuses on evaluating the effectiveness of polyhedral model optimizations in compilers. It consists of programs characterized by intensive use of nested loops and numerical computations, which are typical in scientific and engineering workloads. PolyBench is mainly used for examining loop transformations and data locality optimizations in a controlled setting, providing insights into the optimization of computational kernels.

The experiments are run on two distinct platforms: (1) an Intel CoreX i9-10900X CPU operating at 3.70 GHz with 64 GB memory, and (2) a Phytium 2000+ ARM CPU running at 2.3 GHz with 64 GB memory. We use Ubuntu 20.04 and LLVM 12.0.0.

Table 2 Benchmark programs used in evaluation

CBench	CBench	PolyBench	PolyBench
bitcount	dijkstra	correlation	cholesky
qsort1	particia	covariance	durbin
susan_c	bzip2d	gemm	gramschmidt
susan_e	bzip2e	gemver	lu
susan_s	blowfish_d	gesummv	ludcmp
ghostscript	blowfish_e	symm	trisolv
rsyth	pgp_d	syr2k	deriche
stringsearch	pgp_e	syrk	floyd-warshall
jpeg_c	rijndael_d	trmm	nussinov
jpeg_d	rijndael_e	2 mm	adi
lame	sha	3 mm	fdtd-2d
tiff2bw	adpcm_c	atax	heat-3d
tiff2rgba	adpcm_d	bicg	jacobi-1d
tiffdither	CRC32	doitgen	jacobi-2d
tiffmedian	gsm	mvt	seidel-2d

4.2 State-of-the-art baselines

We compare DCO to several state-of-the-art methods based on sub-sequences.

- MiCOMP uses weak interactive relationships in generating sub-sequences. We evaluate its effectiveness in producing reliable subsequences for further optimization tasks (Ashouri et al. 2017).
- CORL uses a cooperative approach for the sub-sequence generation. CORL considers the inter-dependencies among tuning passes and uses reinforcement learning techniques (Mammadli et al. 2020).
- POSET-RL uses partially ordered sets to guide the sub-sequence generation process. It exploits the inherent order and dependencies among tuning passes to achieve improved optimization results (Jain et al. 2022).
- Full space. We compare the results obtained from these methods with the full optimization space, representing the complete set of tuning passes in LLVM.

4.3 Evaluation on representatives

Exploration and auto-tuning become easier and faster with sub-sequences, even though they cover only a limited part of the optimization space. However, simplifying the process could affect overall performance because some effective combinations of optimization passes may be left out for specific programs. Therefore, it is crucial to check the *representatives* of the generated sub-sequences.

4.3.1 Pairwise performance

To tackle this problem, we conducted a practical assessment to determine the effectiveness of combining different optimization units in sub-sequences, which we refer to as 'Pairwise Performance'. We treat each sub-sequence as a single comprehensive optimization pass and explore the connections between these sub-sequences. We think that analyzing the pairwise relationships between different sub-sequences can demonstrate the potential of these optimizations.

Table 3 Pairwise performance of different sub-sequences

Method	Performance coverage	Mean performance
Full	[0.11–2.95]	1.17
MiCOMP	[0.36–2.73]	0.94
CORL	[0.29–2.82]	1.08
POSET-RL	[0.33–2.83]	1.05
DCO	[0.16–2.91]	1.16

The bold values highlight wider coverage and higher mean performance. Among the sub-sequence methods, DCO demonstrates the best performance, achieving results close to those of the full optimization space

This evaluation aims to strike a balance between minimal exploration and optimal performance. In our tests, we closely examine the interconnection of various optimization units. We use two primary metrics to gauge performance: the range of covered performance and the average optimization performance. Through analyzing these metrics, we can assess the effectiveness of the sub-sequences compared to a more comprehensive exploration of all possible optimizations.

Figure 1 and Table 3 shows the performance results. We see that the different methods exhibit varying degrees of performance coverage and mean performance. The *Full* method entails exhaustive exploration of the optimization space and demonstrates the widest performance range, from 0.11 to 2.95, with a mean performance of 1.17. On the other hand, *MiCOMP* shows a narrower performance range, from 0.36 to 2.73, with a mean performance of 0.94. This suggests that while *MiCOMP* may simplify the search process, it might miss some high-performing combinations identified by the Full method. In comparison, the *CORL* and *POSET-RL* methods have similar performance coverages, ranging from 0.29 to 2.82 and 0.33 to 2.83, respectively, with mean performances of 1.08 and 1.05. These methods strike a balance between exploration efficiency and performance gain, making them effective in identifying good optimization sequences without exhaustive searching. Finally, our DCO demonstrates a performance coverage from 0.16 to 2.91 and a mean performance close to that of the Full method at 1.16. This indicates that DCO can achieve near-optimal performance while reducing the complexity of the exploration process.

4.3.2 Iterative performance

Another important aspect to showcase the potential of sub-sequences is the actual improvement they bring to applications. We compare results in two different scenarios:

- **Constrained Resource Scenario:** This scenario represents situations where only a limited number of iterative compilation steps are allowed, such as in embedded sys-

Table 4 Iterative performance on OpenTuner under different platforms (Intel/ARM)

Method	Execution Time		Code Size	
	iter=50	iter=1000	iter=50	iter=1000
MiCOMP	0.97 1.03	1.02 1.10	0.98 1.04	1.02 1.07
CORL	1.03 1.06	1.05 1.17	1.08 1.07	1.12 1.18
POSET-RL	1.05 1.06	1.07 1.18	1.07 1.06	1.13 1.21
DCO	1.07 1.08	1.11 1.19	1.08 1.09	1.14 1.23

We compare iterative performance across two frameworks to evaluate the optimization potential of different sub-sequence methods. The bold values represent the best results under the same conditions

Table 5 Iterative performance on LaMCTS under different platforms (Intel/ARM)

Method	Execution Time		Code Size	
	iter=50	iter=1000	iter=50	iter=1000
MiCOMP	0.84 1.01	0.99 1.08	0.97 1.03	1.02 1.11
CORL	0.97 1.05	1.03 1.18	1.01 1.06	1.09 1.15
POSET-RL	1.05 1.03	1.07 1.17	1.04 1.07	1.11 1.22
DCO	1.04 1.08	1.08 1.22	1.05 1.07	1.13 1.24

We compare iterative performance across two frameworks to evaluate the optimization potential of different sub-sequence methods. The bold values represent the best results under the same conditions

tems or mobile applications. These environments often have restricted computational resources and need to optimize performance within tight constraints.

- **Exhaustive Exploration Scenario:** This scenario pertains to programs that are compiled once and run continuously, such as high-performance computing applications or server-side software. These applications can allocate more time to extensive optimization in order to achieve the best performance possible. For example, optimizing the code for a scientific simulation running on a super-computer, where the initial compilation time is less critical compared to the runtime performance.

We examine the outcomes in two different scenarios. The first scenario involves allowing only minimal effort in the iterative compilation steps, to illustrate that these efforts do not have unlimited compiler optimization resources. The second scenario involves representing programs that compile once and run indefinitely. In these cases, the applications can withstand extensive exploration of the optimization space to achieve the best performance. For these two scenarios, we restrict the tuning steps of two iterative compilation frameworks, OpenTuner and LaMCTS, to 50 steps for the constrained resource scenario and 1000 steps for the exhaustive exploration scenario.

Tables 4 and 5 shows the performance results on both Intel and ARM platforms. In a scenario with limited resources and 50 iterations, we observe several trends. Among the methods, MiCOMP shows the least favorable performance, indicating that its sub-sequences are less effective in compiler optimization. POSET-RL shows strong performance, with execution times of 1.05 in OpenTuner and 1.05 in LaMCTS, along with code size values of 1.07 and 1.04 respectively. While not as high as DCO, POSET-RL is still effective in optimizing performance. CORL shows

improvements of 1.03 in execution time in OpenTuner and 0.97 in LaMCTS, with code size reduction improvements of 1.08 and 1.01 respectively on the two frameworks. While it may be less effective than POSET-RL and DCO, CORL still shows good potential in performance optimization. DCO consistently outperforms other sub-sequence methods in both execution time and code size optimization. With an improvement of 1.07 in OpenTuner and 1.04 in LaMCTS on execution time, and an improvement of 1.08 and 1.05 on code size respectively, DCO demonstrates better efficiency even with minimal tuning steps. This indicates that DCO is highly effective at quickly finding near-optimal solutions, making it an excellent choice when resources are limited. The ARM platform shows similar trends and our DCO outperforms other sub-sequence methods on the all scenarios.

In the exhaustive exploration scenario with 1000 iterations, DCO demonstrates even more significant performance improvement. Although MiCOMP shows improvement with increased iterations, its relative performance remains the lowest among the methods. It achieves execution times of 1.02 in OpenTuner and 0.99 in LaMCTS, with code size reduction of 1.02 and 1.02 respectively. POSET-RL maintains strong performance with execution time improvement of 1.07 in OpenTuner and 1.07 in LaMCTS, and code size reduction of 1.13 and 1.11 respectively. CORL also improves with more iterations, achieving an execution improvement of 1.05 in OpenTuner and 1.03 in LaMCTS, with code size reductions of 1.12 and 1.09 respectively. However, it still performs worse than DCO. DCO continues to excel with extensive tuning, achieving the largest execution time improvement of 1.11 in OpenTuner and 1.08 in LaMCTS. Our code size reductions are also the most significant, with an improvement of 1.14 and 1.13 respectively. This robust performance underlines DCO's capability to effectively explore the optimization space, making it the most powerful method for achieving optimal performance. Our DCO gets even better results on the ARM platform for achieving the optimal performance improvement on all scenarios.

4.4 Evaluation on Evenness

While the evenness of optimization is less related to optimization performance, we found that it significantly influences compilation time, which is crucial in the tuning process of iterative compilations.

Assuming that,

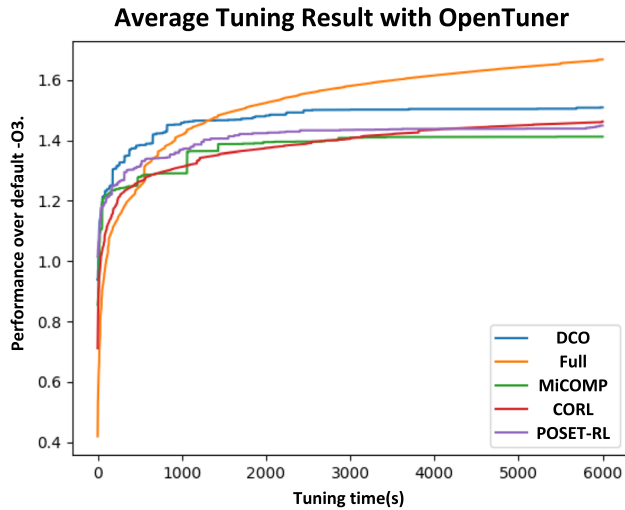


Fig. 7 Tuning performance of OpenTuner on different optimization space with same time cost

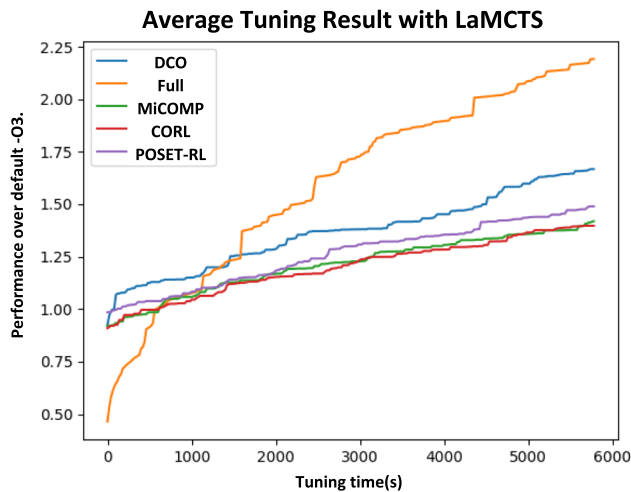


Fig. 8 Tuning performance of LaMCTS on different optimization space with same time cost

1. Each optimization pass O_i (where $i = 1, 2, \dots, n$) has the same compilation time t .
2. Each optimization pass O_i has an equal probability $p = \frac{1}{n}$ of being selected during auto-tuning process.

Now, we can calculate some interesting numbers that indicate the expected compilation time.

Let S_j be a sub-sequence consisting of k_j optimization passes.

Expected Compilation Time for a Single Optimization Pass:

$$E(T) = t \quad (5)$$

Expected Compilation Time for a Sub-sequence S_j :

$$T_{S_j} = k_j \cdot t \quad (6)$$

Probability of Selecting a Sub-sequence S_j :

$$P(S_j) = \frac{k_j}{\sum_{i=1}^m k_i} \quad (7)$$

where m is the total number of sub-sequences.

Expected Compilation Time for the Auto-tuning Process:

$$E(T_{auto}) = \sum_{j=1}^m P(S_j) \cdot T_{S_j} = t \cdot \sum_{j=1}^m \frac{k_j^2}{\sum_{i=1}^m k_i} \quad (8)$$

Hence, the anticipated compilation time is determined by the squared lengths of the sub-sequences. Greater lengths of sub-sequences lead to a larger contribution to the overall expected compilation time, showing that the uniformity of sub-sequence lengths directly affects the total compilation time. Although this is a simplified estimate (as actual compilation times and selection probabilities may vary), it demonstrates the influence of sub-sequence lengths on the observed overall compilation time in real applications.

During our testing, we assessed the performance of various sub-sequences using iterative compilation frameworks while maintaining a consistent time limit, rather than a fixed number of iteration steps. This method takes into consideration the compilation time, as shorter times facilitate a more thorough exploration of the optimization space. The results are shown in Figs. 7 and 8. Our method, referred to as DCO, exhibited the most significant performance enhancement among all sub-sequence-based techniques. Specifically, DCO demonstrated a notable performance improvement compared to other methods, including DCO, MiCOMP, CORL, and POSET-RL. This improvement was evident across both the OpenTuner and LaMCTS frameworks. For instance, in the OpenTuner framework (Fig. 7), DCO achieved superior performance early in the process and

Table 6 Compiler version iteration and changes to optimization passes and the default ‘O3’ and ‘Oz’

Compiler Version	O3 Length	O3 Number	Oz Length	Oz Number	# Passes
v10.0.0	276	88	261	83	147
v11.0.0	288	91	265	85	147
v12.0.0	294	93	271	87	147

maintained this advantage as tuning progressed. Likewise, in the LaMCTS framework (Fig. 8), DCO consistently outperformed the other methods.

We observe that exhaustive exploration of the ‘Full’ space surpasses our approach after approximately 1500 s of tuning. However, it is important to note that our method achieves competitive performance with significantly less tuning time, highlighting its efficiency and practicality for real-world applications.

4.5 Evaluation on automation

Our DCO can automatically extract available optimization passes from the compiler’s API, rather than relying on predefined optimization sequences such as ‘O3’ and ‘Oz’. One notable difference between these two approaches is the stability of the generated sub-sequences.

Since its initial release in 2003, the LLVM compiler has undergone several significant updates. For instance, version 10.0.0 was released in March 2020, followed by version 11.0.0 in October 2020, and version 12.0.0 in April 2021. Table 6 illustrates the default settings of optimization sequences for the LLVM compiler over time. The ‘O3’ and ‘Oz’ optimization sequences have shown rapid improvement across different versions. Surprisingly, the optimization passes have remained unchanged despite three major compiler version changes. This indicates that our method remains relatively stable over time, unlike approaches reliant on predefined optimization sequences such as ‘O3’ and ‘Oz’.

Frequent software releases can lead to inconsistencies in predefined optimization sequences. These sequences may vary in effectiveness with each new version of the compiler. In contrast, DCO extracts and uses the current set of optimization passes in the compiler’s API dynamically. This approach ensures that the generated optimization sub-sequences are always aligned with the latest capabilities of the compiler, resulting in a more stable and adaptable optimization process.

Furthermore, DCO’s method enables the discovery of new optimization paths that predefined sequences might overlook, potentially resulting in greater performance improvements. This flexibility is especially advantageous in quickly evolving software environments, where keeping up with the latest optimization techniques is essential for maintaining high performance.

5 Related work

This section introduces the work on iterative compilation and optimization space.

Iterative compilation Iterative compilation is one approach of releasing programmers and has shown inspiring

results in the past few decades. Programs are auto-tuned with a sequence of iterations to improve performance iteratively. As each iteration requires an online compilation and an execution, iterative compilation is very time-consuming. Thus there are many works proposed to improve the efficiency of iterative compilation by customizing heuristics.

Almagor et al. use a genetic algorithm to select efficient optimization sequences for an adaptive compiler (Almagor et al. 2004). They construct self-steering and self-tuning compilers, which use multiple compilations in a feedback loop to discover good configurations for each combination of input program and target machine. To lower the cost of iterative compilation, Leather et al. provide a method to filter out those ‘bad’ samples through statistic analysis (Leather et al. 2009). This approach can reduce the number of runs of generating statistically rigorous results on compiler performance optimizations. OpenTuner [1] is one of the state-of-the-art models for general-purposed compiler optimization. It is a framework for building domain-specific multi-objective program auto-tuners. OpenTuner achieves to obtain an optimal solution by allowing ensembles of various search techniques on programming auto-tuning. It integrates exhaustive searching, dynamic programming and several evolutionary algorithms. Recently, Chen et al. treat the problem of compiler auto-tuning as a black-box and apply Bayesian Optimization to solve it (Chen et al. 2021). Moreover, auto-tuning methods have been employed across various domains and backend architectures (Zheng et al. 2019; Zhang et al. 2018). Compared to traditional iterative compilation techniques applying black-box optimization techniques in compiler, auto-tuning has shown a significant improvement in terms of search efficiency and overall performance.

Optimization space One major solution of the size problem is to specify a subset of the optimization space. For example, NeuroVectorizer (Haj-Ali et al. 2020) focuses on a loop optimization pass ‘-loop-vectorize’ in LLVM compiler and the whole optimization space can be formed by the combination of two integers.

A promising approach is to use sub-sequences of compiler passes, which are combinations of compiler optimizations that are known to be effective in certain scenarios. Ashouri et al. (2017) propose a technique called MiCOMP that uses optimization sub-sequences and machine learning to mitigate the compiler phase-ordering problem. They show that by selecting the optimization sub-sequence based on the ‘O3’ sequence and pairwise occurrence relationship, they could improve the performance for executing a program and outperform the standard LLVM ‘O3’. Similarly, Mammadli et al. (2020) use a deep reinforcement learning-based approach for static neural compiler optimization, which uses a sub-sequence of optimization passes to optimize LLVM IR code. Jain et al. present POSET-RL (Jain et al. 2022), which optimizes the size

and execution time of programs by learning a policy for selecting a sub-sequence of optimization passes. Their approach involves selecting the right combination of compiler optimizations through LLVM ‘Oz’ sequence, and using reinforcement learning to learn the optimal order of these optimizations.

Compared to these approaches, DCO is a purely automatic method that effectively harnesses the in-depth dependence of compiler passes to deliver reliable sub-sequences.

6 Conclusion and future work

Our work addresses the limitations of existing methods by capturing in-depth pass dependence relationships and then using the constraint clustering. By harnessing the power of these techniques, we have successfully constructed reliable and efficient sub-sequences that significantly reduce the optimization space while maintaining performance gains.

We have showcased the efficacy of our DCO approach in the optimization tasks of running-time performance and code size reduction. The results have shown that DCO can outperform the standard optimization sequences and the state-of-the-art sub-sequence generation methods.

Our work contributes to the broader understanding of the intricacies in compiler optimization. By demonstrating the potential of sub-sequences, we provide valuable insights for improving iterative compilation processes and enhancing software performance. Our findings indicate that the intricate relationships between compiler passes is key to unlocking more optimization potentials.

For future work, we will investigate the adaptability of our method to diverse software contexts and hardware architectures. Ultimately, our work serves as a stepping stone towards more efficient and intelligent compiler

optimizations that can accommodate the ever-moving landscape of software and hardware technologies.

Appendix 1. LLVM optimization passes

Our experiment is carried on LLVM version 12.0.1, we collect all analysis passes and general purpose transform passes and show them in Table 7 and 8.

Table 7 Analysis passes

Analysis passes		
-aa	-aa-eval	-basic-aa
-block-freq	-branch-prob	-cfl-anders-aa
-cfl-steens-aa	-cg-profile	-cost-model
-da	-demanded-bits	-divergence
-domfrontier	-domtree	-external-aa
-globals-aa	-instcount	-lazy-block-freq
-lazy-branch-prob	-lazy-value-info	-loop-accesses
-loops	-machine-branch-prob	-memdep
-module-summary-analysis	-module-summary-info	-pa-eval
-phi-values	-polyhedral-info	-postdomtree
-profile-summary-info	-prune-eh	-regions
-scalar-evolution	-scev-aa	-scoped-noalias-aa
-stack-safety	-stack-safety-local	-targetlibinfo
-tbaa	-tti	

Table 8 Transform passes

Transform passes		
-adce	-add-discriminators	-aggressive-instcombine
-alignment-from-assumptions	-alloca-hoisting	-always-inline
-argpromotion	-assume-builder	-assume-simplify
-atomic-expand	-attributor	-attributor-cgsc
-barrier	-basiccg	-bdce
-break-crit-edges	-called-value-propagation	-callsite-splitting
-chr	-consthoist	-constmerge
-constraint-elimination	-correlated-propagation	-cross-dso-cfi
-dce	-deadargelim	-delinearize
-div-rem-pairs	-dse	-early-cse
-early-cse-memssa	-ee-instrument	-elim-avail-extern
-expand-reductions	-expandmemcmp	-fix-irreducible
-flattencfg	-float2int	-forceattrs
-function-attrs	-global-merge	-globaldce
-globalopt	-globalsplit	-guard-widening
-gvn	-gvn-hoist	-gvn-sink
-hardware-loops	-hotcoldsplit	-indirectbr-expand
-indvars	-infer-address-spaces	-inferattrs
-inject-tli-mappings	-inline	-insert-gcov-profiling
-instcombine	-instrprof	-instsimplify
-interleaved-access	-intervals	-ipscpp
-ir-similarity-identifier	-iroutliner	-iv-users
-jump-threading	-lcssa	-libcalls-shrinkwrap
-licm	-loop-data-prefetch	-loop-deletion
-loop-distribute	-loop-extract	-loop-extract-single
-loop-flatten	-loop-fusion	-loop-guard-widening
-loop-idiom	-loop-instsimplify	-loop-interchange
-loop-load-elim	-loop-predication	-loop-reduce
-loop-reroll	-loop-rotate	-loop-simplifycfg
-loop-sink	-loop-unroll	-loop-unroll-and-jam
-loop-unswitch	-loop-vectorize	-loop-versioning
-loop-versioning-licm	-lower-constant-intrinsics	-lower-expect
-lower-guard-intrinsic	-lower-widenable-condition	-lower-matrix-intrinsics
-lower-matrix-intrinsics-minimal	-loweratomic	-lowerinvoke
-lowerswitch	-lowertypetests	-make-guards-explicit
-mem2reg	-memcpyopt	-memoryssa
-mergefunc	-mergeicmps	-mergereturn
-mldst-motion	-name-anon-globals	-nary-reassociate
-newgvn	-partial-inliner	-partially-inline-libcalls
-post-inline-ee-instrument	-pre-isel-intrinsic-lowering	-reassociate
-redundant-dbg-inst-elim	-reg2mem	-rewrite-symbols
-rpo-function-attrs	-safe-stack	-sancov
-scalarize-masked-mem-intrin	-scalarizer	-sccp
-separate-const-offset-from-gep	-simple-loop-unswitch	-simplifycfg
-sink	-slsr	-speculative-execution
-sroa	-stack-protector	-strip
-strip-dead-prototypes	-structurizecfg	-tailcallem
-type-promotion	-unify-loop-exits	-unreachableblockelim
-unique-internal-linkage-names	-vector-combine	-wholeprogramdevirt

Appendix B. Sub-sequences for compiler optimization

Here we provide current sub-sequence-based methods designed for compiler optimization. CORL and POSET-RL's sub-sequences are extracted from their paper. As

MiCOMP uses an early version of LLVM compiler, we reconstruct the sub-sequences follow all steps in its paper based on LLVM version 12.0.1.

These sub-sequences are shown in Table 9, Table 10 and Table 11 (see Table 12).

Table 9 MiCOMP's sub-sequences

MiCOMP Sub-Sequences ¹	
1	-tti -tbaa -scoped-noalias-aa -assumption-cache-tracker -verify -ee-instrument -sroa -early-cse -lower-expect -profile-summary-info -annotation2metadata -forceattrs -inferattrs -callsite-splitting -ipsccp -called-value-propagation -globalopt -mem2reg -deadargelim -instcombine -basiccg -globals-aa -prune-eh -inline -openmpopt -function-attrs -argpromotion -memoryssa -early-cse-memssa -speculative-execution -lazy-value-info -jump-threading -correlated-propagation -aggressive-instcombine -libcalls-shrinkwrap -postdomtree -branch-prob -block-freq -pgo-memop-opt -tailcallelim -reassociate -lcssa -scalar-evolution -loop-rotate -licm -loop-unswitch -loop-idiom -indvars -loop-deletion -loop-unroll -mldst-motion -phi-values -memdep -gvn -memcpyopt -sccp -demanded-bits -bdce -adce -dse -barrier -elim-avail-extern -rpo-function-attrs -globaldce -float2int -lower-constant-intrinsics -loop-accesses -loop-distribute -inject-tli-mappings -loop-vectorize -loop-load-elim -slp-vectorizer -vector-combine -transform-warning -alignment-from-assumptions -strip-dead-prototypes -constmerge -cg-profile -loop-sink -instsimplify -div-rem-pairs -annotation-remarks
2	-targetlibinfo -simplifycfg
3	-domtree -aa
4	-basic-aa
5	-loops
6	-lazy-branch-prob
7	-lazy-block-freq
8	-opt-remark-emitter
9	-loop-simplify
10	-lcssa-verification

¹ This is our reconstruction of MiCOMP sub-sequence based on LLVM version 12.0.1

Table 10 CORL's sub-sequences

CORL Sub-Sequences	
1	-tti -verify -tbaa -scoped-noalias-aa -simplifycfg -sroa -early-cse -lower-expect
2	-targetlibinfo -tti -forceattrs -tbaa -scoped-noalias-aa -inferattrs -ipsccp -globalopt -mem2reg -deadargelim -instcombine -simplifycfg
3	-globals-aa -prune-eh -inline -function-attrs -argpromotion -sroa -early-cse -jump-threading -correlated-propagation -simplifycfg
4	-instcombine -tailcallelim -simplifycfg
5	-reassociate -loop-rotate -licm -loop-unswitch -simplifycfg
6	-instcombine -indvars -loop-idiom -loop-deletion -loop-unroll -mldst-motion -gvn -memcpyopt -sccp -bdce -instcombine -jump-threading -correlated-propagation -dse -licm -adce -simplifycfg
7	-instcombine -barrier -rpo-function-attrs -elim-avail-extern -globals-aa -float2int -loop-rotate -loop-vectorize -instcombine -slp-vectorizer -simplifycfg
8	-instcombine -loop-unroll -instcombine -licm -alignment-from-assumptions -strip-dead-prototypes -globaldce -constmerge

Table 11 POSET-RL's sub-sequences

POSET-RL's Sub-Sequences	
1	-ee-instrument -simplifycfg -sroa -early-cse -lower-expect -forceattrs -inferattrs -mem2reg
2	-ipsccp -called-value-propagation -attributor -globalopt
3	-deadargelim -instcombine -simplifycfg
4	-prune-eh -inline -function-attrs -barrier
5	-sroa -early-cse-memssa -speculative-execution -jump-threading -correlated-propagation
6	-simplifycfg -instcombine -tailcallelim -simplifycfg -reassociate
7	-loop-simplify -lcssa -loop-rotate -licm -loop-unswitch -simplifycfg -instcombine
8	-loop-simplify -lcssa -indvars -loop-idiom -loop-deletion -loop-unroll
9	-mldst-motion -gvn -memcpyopt -sccp -bdce -instcombine -jump-threading -correlated-propagation -dse
10	-loop-simplify -lcssa -licm -adce -simplifycfg -instcombine
11	-barrier -elim-avail-extern -rpo-function-attrs -globalopt -globaldce -float2int -lower-constant-intrinsics
12	-loop-simplify -lcssa -loop-rotate -loop-distribute -loop-vectorize
13	-loop-simplify -loop-load-elim -instcombine -simplifycfg -instcombine
14	-loop-simplify -lcssa -loop-unroll -instcombine -loop-simplify -lcssa -licm -alignment-from-assumptions
15	-strip-dead-prototypes -globaldce -constmerge -loop-simplify -lcssa -loop-sink -instsimplify -div-rem-pairs -simplifycfg

Table 12 DCO's sub-sequences

DCO's Sub-Sequences	
1	-instrorderfile -loop-extract -loop-extract-single -called-value-propagation -lower-guard-intrinsic -lower-matrix-intrinsics -lower-expect -lowerswitch -interleaved-load-combine -inject-tli-mappings
2	-lcssa -loop-flatten -loop-simplify -memoryssa -instcombine -newgvn -gvn -licm -sroa -mem2reg
3	-hotcoldsplit -extract-blocks -deadargelim -globalsplit -div-rem-pairs -chr -delinearize -inline -instrprof -expand-reductions -constraint-elimination -cross-dso-cfi -dce -mergeicmps -function-attrs -expandmemcmp -forceattrs -interleaved-access -make-guards-explicit -callsite-splitting -lower-constant-intrinsics -global-merge -guard-widening -barrier -ee-instrument
4	-loop-reroll -loop-sink -add-discriminators -argpromotion -loop-versioning -loop-simplifycfg -globalopt -loop-unswitch -assume-simplify -loop-load-elim -tailcallelim -loop-unroll-and-jam -aggressive-instcombine -loop-rotate -jump-threading
5	-atomic-expand -always-inline -sccp -loop-instsimplify -strip-dead-prototypes -ipsccp -speculative-execution -loop-unroll -rpo-function-attrs -loop-predication -scalarize-masked-mem-intrin -mergefunc -rewrite-symbols -attributor-cgsc -loop-guard-widening -mldst-motion -vector-combine -alignment-from-assumptions -loop-vectorize -partially-inline-libcalls -loop-reduce -alloca-hoisting -flattencfg -whole-programdevirt -assume-builder
6	-indirectbr-expand -loop-data-prefetch -reg2mem -constmerge -loop-distribute -float2int -instsimplify -correlated-propagation -consthoist -lower-widenable-condition -irce -libcalls-shrinkwrap -safe-stack -lower-matrix-intrinsics-minimal -globaldce -fix-irreducible -lower-atomic -loop-deletion -load-store-vectorizer -lint -attributor -loop-versioning-licm -lowerinvoke -indvars -bdce
7	-memcpyopt -scalarizer -sancov -instcount -reassociate -rewrite-statepoints-for-gc -stack-protector -partial-inliner -loop-interchange -loop-fusion -redundant-dbg-inst-elim -loop-idiom -slsr -type-promotion -mergereturn -strip -adce -nary-reassociate -post-inline-ee-instrument -lowertypetests -unique-internal-linkage-names -strip-gc-relocates -name-anon-globals -elim-avail-extern -tti
8	-dse -break-crit-edges -aa -assumption-cache-tracker -basic-aa -domtree -lazy-block-freq -lazy-branch-prob -loops -scalar-evolution
9	-aa-eval -cg-profile -cost-model -divergence -external-aa -globals-aa -intervals -liveintervals -liveregmatrix -memprof -memprof-module -module-summary-analysis -pa-eval -prune-eh -reaching-deps-analysis -regions -sample-profile -scev-aa -scoped-noalias-aa -stack-safety -stack-safety-local -tbaa
10	-basiccg -block-freq -branch-prob -da -demanded-bits -domfrontier -loop-accesses -memdep -phi-values -postdomtree

Acknowledgements We thank anonymous reviewers for their constructive comments. This work was partially supported by the National Key Research and Development Program of China under Grant No.2023YFB3001503, and the National Natural Science Foundation of China under Grant No. 61972408.

Data Availability The datasets used and analyzed during the current study are available from the corresponding author upon reasonable request.

Declarations

Conflict of interest On behalf of all authors, the corresponding author states that there is no Conflict of interest.

Received November 14, 2020; revised March 7, 2021; accepted March 19, 2021; date of publication March 23, 2021; date of current version May 24, 2021.

Digital Object Identifier 10.1109/TQE.2021.3068355

Survey on Quantum Circuit Compilation for Noisy Intermediate-Scale Quantum Computers: Artificial Intelligence to Heuristics

JANUSZ KUSYK¹ , SAMAH M. SAEED²  (Member, IEEE),
AND MUHARREM UMIT UYAR²  (Fellow, IEEE)

¹ Computer Systems Technology, New York City College of Technology, Brooklyn, NY 11201 USA

² Department of Electrical Engineering, City College of the City University of New York, New York, NY 10031 USA

Corresponding author: Muharrem Umit Uyar.

ABSTRACT Computationally expensive applications, including machine learning, chemical simulations, and financial modeling, are promising candidates for noisy intermediate scale quantum (NISQ) computers. In these problems, one important challenge is mapping a quantum circuit onto NISQ hardware while satisfying physical constraints of an underlying quantum architecture. Quantum circuit compilation (QCC) aims to generate feasible mappings such that a quantum circuit can be executed in a given hardware platform with acceptable confidence in outcomes. Physical constraints of a NISQ computer change frequently, requiring QCC process to be repeated often. When a circuit cannot directly be executed on a quantum hardware due to its physical limitations, it is necessary to modify the circuit by adding new quantum gates and auxiliary qubits, increasing its space and time complexity. An inefficient QCC may significantly increase error rate and circuit latency for even the simplest algorithms. In this article, we present artificial intelligence (AI)-based and heuristic-based methods recently reported in the literature that attempt to address these QCC challenges. We group them based on underlying techniques that they implement, such as AI algorithms including genetic algorithms, genetic programming, ant colony optimization and AI planning, and heuristics methods employing greedy algorithms, satisfiability problem solvers, dynamic, and graph optimization techniques. We discuss performance of each QCC technique and evaluate its potential limitations.

INDEX TERMS Artificial intelligence (AI), noisy intermediate scale quantum (NISQ), quantum algorithms, quantum circuit compilation (QCC), quantum circuit mapping, quantum computing.

I. INTRODUCTION

Quantum computing is an emerging computing paradigm that possesses the potential of bringing efficient solutions to many modern problems that are otherwise computationally overwhelming for classical computers. Noisy intermediate-scale quantum (NISQ) technology is increasingly accessible to the scientific community, bringing advantages of quantum hardware to a range of important tasks. However, reliability of current quantum computers is a serious challenge that stems from the impact of environmental hardware noise on quantum operations, and hence may result in loss of information.

Major global industry players, such as IBM [1], Google [2], Intel [3], and Rigetti [4], have implemented NISQ architectures, each with geometric limitations (e.g.,

lack of full connectivity among qubits), quantum gate constraints (e.g., insufficient number of qubits), *decoherence* limitations (e.g., qubits losing their state over time), and error rates varying in time.

Computationally expensive applications, such as machine learning, chemical simulations, and financial modeling, are only some of the promising candidates for NISQ computers. One important challenge is mapping a quantum algorithm onto a specific NISQ hardware, while satisfying physical constraints of its underlying quantum architecture, a process referred to as quantum circuit compilation (QCC) or *quantum mapping*. A quantum algorithm (i.e., an algorithm designed to run in quantum hardware) specified in an intermediate representation (IR) must be compiled into a set of elementary quantum gates, which are then applied in sequence

at specific times when run on a particular NISQ quantum hardware. A quantum compilation technique should synthesize, decompose, optimize, and map a quantum circuit into the elementary quantum gates supported by a target hardware. To generate a feasible quantum circuit via QCC, it is often necessary to modify the original circuit IR by adding new quantum gates and auxiliary qubits so that the final circuit adheres to the constraints of the target hardware.

A successful quantum mapping, which is known to be an NP-complete problem [5], involves judicious allocation of qubits in a timely manner while adding quantum gates as needed to reduce errors. A nonefficient QCC will significantly increase error rate and latency of the final quantum computation, resulting in low fidelity of the generated outputs. *Fidelity* quantifies the difference between the output state of a quantum circuit in the absence and presence of noise when the circuit is executed in a quantum computer. Since quantum hardware constraints change frequently for a given architecture, recalculation of feasible mappings is an on-going effort.

In this article, we consider QCC approaches for synthesis and mapping of quantum circuits based on artificial intelligence (AI) and heuristic methods. Basic quantum computing concepts are introduced in Section II. Challenges in QCC process are discussed by illustrative examples together with motivations for developing effective QCC methods in Section III. Recently reported AI-based techniques for QCC are in Section IV. Heuristics to generate feasible quantum circuits using greedy algorithms, satisfiability problem solvers, dynamic, and graph optimization techniques are in Section V. Evaluation of methods and future directions are in Section VI.

II. QUANTUM CIRCUIT DESIGN

Quantum gates operate on quantum bits (qubits), where a qubit can be in a state 0, 1, or superposition of both. A qubit state is described as a state vector represented in a *ket* notation $|\cdot\rangle$. The state vector of a single-qubit is expressed as $|\varphi\rangle = \alpha_0|0\rangle + \alpha_1|1\rangle$, where $|0\rangle$ and $|1\rangle$ are base states and α_0 and α_1 are complex numbers such that $|\alpha_1|^2 + |\alpha_0|^2 = 1$. The probability that a qubit is in state 0 or 1 is $|\alpha_0|^2$ and $|\alpha_1|^2$, respectively. For a quantum system with n qubits, the state vector is represented as $|\varphi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle$, where $\sum_{x \in \{0,1\}^n} |\alpha_x|^2 = 1$. The state of the quantum system can also be represented using a density matrix (ρ) defined as $\rho = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle \langle x|$, where α_x is the probability that the system is in a pure state of $|x\rangle$. Due to decoherence, a quantum system may spontaneously change its state from pure to mixed.

A quantum gate can be represented by a unitary matrix U , whose conjugate transpose U^* is its inverse ($UU^* = U^*U = I$). For n qubits, U is a $(2^n \times 2^n)$ matrix. Quantum gates change qubit state on which they operate. A gate can be a single- or multiqubit gate. Examples of single-qubit gates include bit-flip (X), Hadamard (H) for superposition, phase shift by $\pi/4$, (T), conjugate transpose of T ($T^\dagger$), and generic

single-qubit rotation with three Euler angles ($U3(\theta, \phi, \lambda)$) (see Fig. 1). A controlled NOT (CNOT) gate operates on two qubits with target and control qubits. When the state of control the qubit is 1, target qubit is inverted; otherwise, it remains the same. Output of a quantum circuit is determined by a quantum measurement. Fig. 1 lists the basic quantum gates and their matrix representations.

Among different technologies used to build computers are ion trapped qubits (e.g., in IonQ) and superconducting qubits (e.g., in IBM, Google, and Rigetti). Table 1 illustrates single- (1-Q) and two-qubit (2-Q) native gates implemented by various quantum technology vendors [6], [7]. While all quantum computers support single-qubit rotation around the z -axis [i.e., $R_z(\lambda)$], some vendors also implement single-qubit rotation of angle $\pi/2$ around the x -axis [i.e., $R_x(\pi/2)$, $R_x(\pm\pi/2)$], and yet others enable single-qubit rotation around both x - and y -axes [i.e., $R_{xy}(\theta, \Phi)$]. Superconducting quantum computers such as IBM Q use cross-resonance (CR) gate as two-qubit entangling gate for building the CNOT gate. In IonQ, two-qubit gate ($XX(x)$) causes xx -Ising interaction between qubits by driving a spin-dependent force [7], [8]. Two-qubit native gates implemented by vendors include cross resonance and controlled z gates, which are shown as CR and CZ, respectively, in Table 1.

The goal of a QCC system is to map an algorithm onto a quantum circuit that consists of operations supported by a given underlying NISQ architecture. A high-level view of the QCC process for mapping an IR of the quantum algorithm (IR) to an appropriate circuit for a target NISQ hardware is shown in Fig. 2.

Physical constraints of a quantum architecture specify not only the qubits that can directly interact with each other (through CNOT gates), but also the error rates associated with the qubits and their gates. Fig. 3 shows the coupling constraints of the target quantum hardware of IBM Q16 Melbourne v1.0.0 architecture, where there are a total of 14 qubits and each node represents one of them [1]. In Fig. 3, the top number inside each node indicates its error rate for a single-qubit $u3$ gate operation, while the bottom one is its readout error. Note that the error rates shown in Fig. 3 are as of August 21, 2019, and they must be multiplied by 10^{-10} for IBM Q16 architecture. Directional edges show entanglement capabilities. For example, a CNOT operation between q_0 and q_1 can only be defined as $\text{CNOT}(q_1, q_0)$ since there is an edge from q_1 to q_0 , whereas $\text{CNOT}(q_0, q_1)$ is prohibited. The edge labels indicate the CNOT gate error rate. Similarly, $\text{CNOT}(q_1, q_2)$ is allowed, but $\text{CNOT}(q_2, q_1)$ is not. Errors in qubit operations may spawn from leakage of microwave pulses during computation and vary from one qubit to another and change over time.

Mapping nonquantum operations to quantum circuits is restricted by the number of qubits in a NISQ hardware. Since most systems have less than 100 qubits [9]–[11], it is often necessary to reuse qubits for different operations, increasing circuit latency. Another challenge is that IR may require gates that are not supported by a target hardware, where QCC must

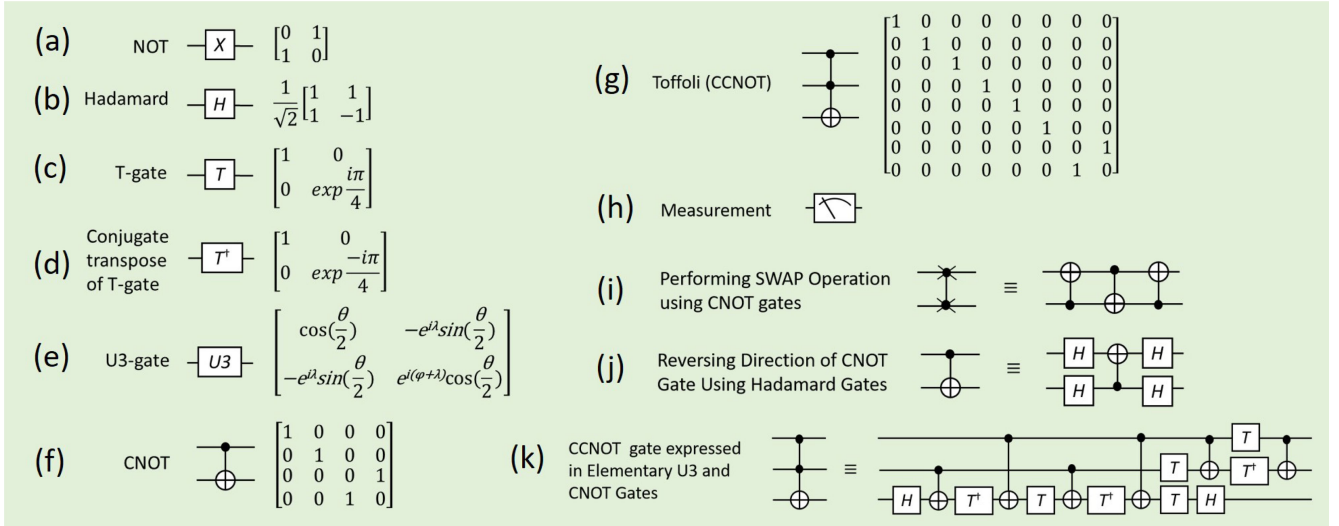


FIGURE 1. Basic quantum gates and operations used for building a quantum circuit.

TABLE 1. Quantum Hardware Technologies and the Native Gates They Support

	IBM		Rigetti		IonQ		Google	
	1-Q gates	2-Q gates	1-Q gates	2-Q gates	1-Q gates	2-Q gates	1-Q gates	2-Q gates
Native gates	$R_x(\pi/2)$, $R_z(\lambda)$	CR	$R_x(\pm\pi/2)$, $R_z(\lambda)$	CZ	$R_{xy}(\theta, \Phi)$, $R_z(\lambda)$	XX(x)	$R_{xy}(\theta, \Phi)$, $R_z(\lambda)$	CZ, CNOT
Qubit technology	Superconducting qubits		Superconducting qubits		Ion trapped qubits		Superconducting qubits	

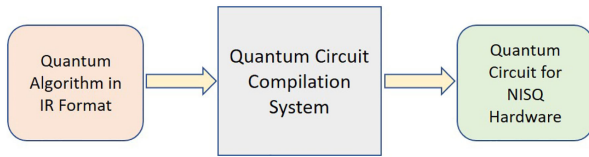
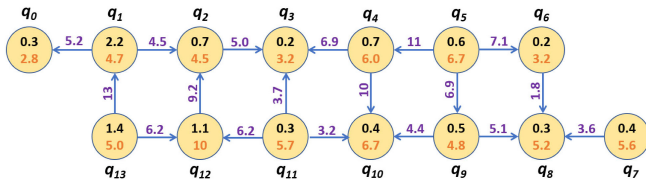
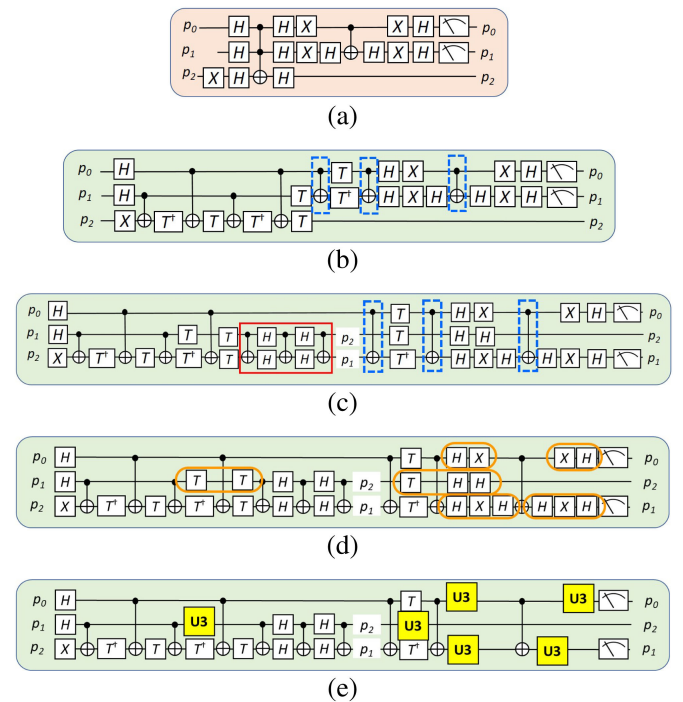


FIGURE 2. QCC process overview.

FIGURE 3. IBM Q16 Melbourne architecture and its hardware constraints [1] (calibration parameters shown here are obtained on August 21, 2019 and must be multiplied by 10^{-10} to obtain the actual error rates at that date).

transform quantum algorithm operations (including unsupported ones) into the gates available at a target hardware. For example, IBM Q14 Melbourne architecture supports only U1, U2, U3, CNOT, and identity gates [1]. Hence, Grover search [12] [see Fig. 4(a)] cannot be directly implemented on IBM Q16 without gate conversions. Fig. 1 demonstrates reversing direction of CNOT using Hadamard gates, SWAP operation using CNOT gates, and Toffoli gate transformations to cope with constraints of the target quantum computer.

Qubits suffer from *decoherence* caused by natural *relaxation* time T1 and *dephasing* time T2 due to environmental effects. A feasible quantum circuit must satisfy physical

FIGURE 4. Grover search: (a) IR format, (b) NISQ circuit [blue boxes are $\text{cnot}(p_0, p_1)$], (c) circuit satisfying IBM Q16 constraints for M_1 (swap is in red box), (d) combining consecutive single-qubits (orange boxes), (e) final circuit.

constraints of an underlying quantum architecture. Although inefficient, a typical remedy is to run a quantum circuit repeatedly and select the best performing output.

TABLE 2. QCC Methods and Their Target Quantum Circuits

		Synthesized Quantum Circuit							
		1-bit gates	CNOT	Toffoli	RC	QFT	Grover	QAOA	Other circuits [†]
AI	GA	[13]	[14]	[13] [14]	[15]	[16] [17]	[16]	[18]	[14] [19] [13]
	GP		[20]	[21] [22]	[21] [23]				[22] [24] [20]
	ACO	[25]	[25]	[25] [26] [27] [28]		[25] [26] [27]	[25]		[29] [28]
	AI planning							[30] [31]	[32]
Heuristics	Greedy		[33] [34]	[33] [34]	[33] [34]	[34] [35]	[35]		[36] [33] [35] [37] [38]
	SAT and SMT		[39]	[39] [40]	[39] [40]	[39] [40]			[40] [41] [42]
	Dynamic				[43] [44]	[45]	[44]	[43]	[43] [46] [47] [45] [44]
	Graph		[33] [48] [49] [50]	[33] [48] [49] [51]	[33] [48] [49] [51] [52]	[51] [52]		[53]	[54]

III. MOTIVATIONS AND EXISTING METHODS FOR QCC

A. NEED FOR EFFECTIVE QCC

The need for effective QCC algorithms can be seen by converting Grover search IR [see Fig. 4(b)] into a quantum circuit supported by IBM Q16 architecture. QCC aims to assign logical qubits (p_0, p_1, p_2) to physical ones such that the total number of gates in the resulting circuit is minimized. A possible qubit mapping is $M_1 = \{q_{11} \leftarrow p_0; q_9 \leftarrow p_1; q_{10} \leftarrow p_2\}$, which requires three CNOT(p_0, p_1) gates [blue boxes in Fig. 4(b)]. M_1 implies that IBM Q16 hardware should support a link between q_{11} (representing p_0) and q_9 (representing p_1). However, since there is no link between them (see Fig. 3), q_9 and q_{10} are switched to connect q_9 and q_{11} . Hence, to generate a feasible circuit using M_1 , position of q_9 and q_{10} have to be switched by adding SWAP (q_9, q_{10}). With this addition [see red box in Fig. 4(c)], p_0 and p_1 share a link and can be used in CNOT(p_0, p_1). Consecutive single-qubit gates can be combined into a single U3 gate to reduce circuit delay. For example, in Fig. 4(d), consecutive T gates on p_1 can be replaced by a U3 gate. Fig. 4(d) shows all such necessary substitutions in orange boxes and Fig. 4(e) the final circuit.

This example illustrates the motivation for designing a QCC system that will take into account many possibilities in qubit selections for mapping and assigning SWAP operations. There are a large number of mappings to be considered for implementing the circuit shown in Fig. 4(a). In addition, many possible selections of qubits for swap operations must be considered in mapping since they can generate many different outcomes. For example, instead of swapping q_9 and q_{10} , one could have swapped q_{10} with q_{11} , which would result in a different circuit.

To construct a quantum circuit that yields a high success probability, physical constraints of a given hardware platform and error rates of quantum gates should be considered when defining mapping policies. Resulting circuits under different physical qubit assignments need to be evaluated with respect to their number of gates, circuit latency, error rates, and, therefore, the success probability for the output.

B. QCC METHODS REPORTED IN THE LITERATURE

Table 2 presents the methods included in this survey. We group these methods based on underlying techniques that they implement: AI algorithms include genetic algorithms, genetic programming (GP), ant colony optimization (ACO), and AI planning, whereas heuristic-based QCC methods are greedy algorithms, satisfiability problem solvers, dynamic, and graph optimization techniques. After briefly presenting each technique, we evaluate its application to QCC with respect to its adaptability to different classes of problems and its potential limitations. As shown in Table 2, these techniques use target circuits of CNOT, Toffoli, reversible circuit (RC), (other than Toffoli), quantum Fourier transform (QFT), Grover, quantum approximate optimization algorithm (QAOA), and others (e.g., Franklin, cascade of RC, entangled two-, three-, and four-qubit circuits).

IV. ARTIFICIAL INTELLIGENCE TECHNIQUES FOR QCC

AI-based computation techniques imitate learning, reasoning, and self-correction processes of intelligent beings to provide adequate solutions to problems that otherwise may require closed-form formulations or computationally expensive algorithms. Among AI methods, biologically inspired algorithms have been successfully applied to a diverse set of problems including image processing [55], robotics [56], finance [57], cybersecurity [58], data processing [59], and autonomous movement control [60]. Genetic algorithms (GAs), GP, swarm intelligence, and AI planning are among the most popular AI methods employed for QCC.

A. GENETIC ALGORITHMS

Background: GAs emulate the process in nature, where better adapted individuals have greater chance of survival in a given environmental niche and, therefore, of passing their genetic materials to their offspring. A candidate solution is a *chromosome*, and a set of solutions is a *population*. Depending on a problem to be solved, chromosomes can be encoded using either binary strings, direct value representations, tree

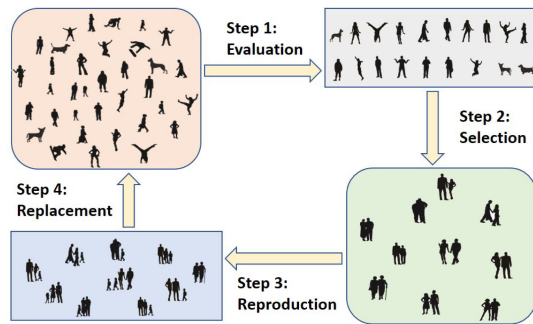


FIGURE 5. Typical operations in a single generation of a genetic algorithm: evaluation, selection, reproduction, and replacement.

Algorithm 1: A Simplified Implementation of GA.

```

Initialize population randomly;
Evaluate fitness for each individual in population;
while Stopping Criteria is not attained do
    Select parents for reproduction;
    // Parents are selected from fittest individuals
    Generate offspring by crossover and mutation;
    Evaluate offspring and form a new generation;
    // Offspring replaces less fit individuals
end while
  
```

structures, or any other format that suits the target problem. Fig. 5 presents key operations of a GA that are often recognized as the canonical example for bioinspired computational techniques [61]. A pseudocode for a simple GA given in Algorithm 1, where first an initial population of candidate solutions is evaluated and then parents are selected to form offspring to be considered for future generations.

In Algorithm 1, an initial population of candidate solutions is formed and evaluated using a mathematical function, called *fitness*, to determine their suitability for solving a problem at hand (see Step 1 in Fig. 5). An appropriate fitness function is crucial in all bioinspired algorithms as it must accurately indicate how close a candidate solution is to meet an overall goal, while facilitating speedy convergence on an appropriate solution. Then, parents are selected based on a predetermined criteria (see Step 2 in Fig. 5). For example, in the selection process called *elitism*, the best candidate solutions are chosen as parents. Offspring are formed by crossover and mutation steps (see Step 3 in Fig. 5), where the crossover operation exchanges parts of candidate solutions and a small probability mutation prevents algorithm from getting stuck at local maxima/minima outcomes. Finally, offspring is evaluated for inclusion in the next generation (see Step 4 in Fig. 5). Steps 1–4 are repeated for a predetermined number of iterations or until a satisfactory solution is found.

GA for QCC: A GA can be used for an automated synthesis of factorized reversible exclusive-or–sum-of-products circuits, where the GA evolves population of circuit candidates by searching the space of all possible circuits defined by a set of available quantum gates for a given problem and a hardware platform [19]. Quantum circuit parameters can

be encoded into chromosomes, and fitness can be defined to be inversely proportional to its error rate and its length. This type of GA-based QCC performs better compared to GAs employing group theory, function decomposition, gate composition, regular and symmetric structures, decision diagrams, wave cascades, and backtracking methods [19], [62].

In [14], quantum gates are synthesized using a GA with chromosomes encoding candidate solutions in qubits and *qutrits* (quantum information described by a superposition of three mutually orthogonal states). Fitness of a quantum gate is evaluated for its performance and accuracy. The final quantum circuit is obtained by a probabilistic selection of synthesized gates. Since genetic operations need to be simplified to adequately reflect quantum operations, slow convergence to an acceptable solution is expected.

Ternary reversible circuits using M-S gates [63] require an exponentially large search space if one wishes to design a multivalued reversible logic circuit with minimal errors, quantum computation cost, and auxiliary qubits. However, they can be successfully synthesized with the help of GAs [15]. In other applications of GA, chromosomes can be encoded as three-tuples satisfying operational requirements with a fitness based on the number of M-S gates and their delay, where permutations of qubits and available gates can be indexed as binary codons of a chromosome to evolve a quantum circuit [17]. Half- and full-adder circuits generated using GA show promising performance characteristics. The fitness function assesses a total cost of all gates in a candidate circuit using different cost values for single- and multiqubit gates and their correctness with respect to the design goals. The GA-based approach in [17] is tested by synthesizing QFT circuits.

Qubits and gates can be evolved for each individual in a population with a fitness defined as their coverage of output function as in the GA application given in [13]. A circuit with a fitness value smaller than a given threshold is eliminated from the population and replaced by a new individual. This GA implements a one-point crossover to swap gates between randomly selected parents.

In [18], a GA maps a quantum circuit onto a hardware platform while satisfying hardware couplings using SWAP gates, where genes control quantum gates by considering their fitness. Based on defined chromosomes, quantum gates are added to a circuit. In fitness calculations, a lexicographic double-key quantum gate ranking is used to improve circuit selection. Multiobjective GAs can be used to build quantum circuits with relatively lower error rates and reduced number of quantum gates [16], where fitness functions consider the number of hardware qubits, types of gates in a circuit, and the target hardware restrictions. Output error rates of a candidate circuit are evaluated with respect to a manually constructed ideal solution.

Computational complexity of implementing QCC using a GA-based method with g number of generations and population size of p is $O(g \cdot p \cdot \log(p))$, where $p \cdot \log(p)$ is the typical computational cost of a sorting at every generation of

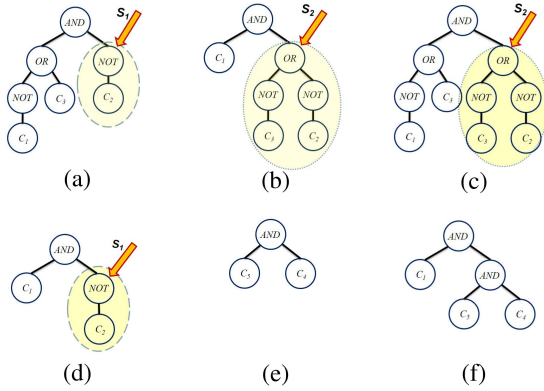


FIGURE 6. GP crossover: programs (a) $\mathcal{P}_1$ and (b) $\mathcal{P}_2$, and final programs (c) $\mathcal{P}_3$ and (d) $\mathcal{P}_4$. GP mutation: (e) program $\mathcal{P}_4$ at point S_1 , (f) a random tree, and (g) final program $\mathcal{P}_5$.

the GA. This low complexity of GA-based GCC can result in satisfactory outcomes if the fitness function is properly designed for a given application.

B. GENETIC PROGRAMMING

Background: If a solution to a given problem can be modeled or represented as a computer program, the AI technique called GP can evolve candidate programs toward an acceptable solution [64]. In GP, candidate programs are generated by employing evolutionary methods. Chromosomes are used to generate syntactically correct candidate programs (i.e., the individuals in a population). Each candidate is executed in its environment to determine its fitness. As GP progresses, better programs, obtained by reproduction, crossover, and mutation, are evolved to next generations. GP is independent of the types of evolutionary methods used to generate programs (e.g., GA can be employed for program generation).

To illustrate its operation, let us consider two candidate programs generated by GP, namely $\mathcal{P}_1$ and $\mathcal{P}_2$ shown in Fig. 6(a) and (b), respectively. Suppose the functions for these programs are defined as {AND, OR, NOT} and operands as $\{c_1, c_2, c_3\}$. Each candidate program can be represented as a tree, where the root and the intermediate nodes are functions and the leaves are operands. Let these example candidate programs be defined as $\mathcal{P}_1 = ((\bar{c}_1 \text{ OR } c_3) \text{ AND } \bar{c}_2)$ and $\mathcal{P}_2 = (c_1 \text{ AND } (\bar{c}_3 \text{ OR } \bar{c}_2))$ (an overbar means negation). For a crossover operation, we can use, for example, the right subtree of each program, namely $S_1 \in \mathcal{P}_1$ and $S_2 \in \mathcal{P}_2$. After swapping S_1 with S_2 in $\mathcal{P}_1$, we obtain program $\mathcal{P}_3 = ((\bar{c}_1 \text{ OR } c_3) \text{ AND } (\bar{c}_3 \text{ OR } \bar{c}_2))$, as shown in Fig. 6(c). Similarly, a new program $\mathcal{P}_4$ is formed by swapping $S_2 \in \mathcal{P}_2$ with $S_1 \in \mathcal{P}_1$ [see Fig. 6(d)]. To illustrate a mutation operation in GP, a subtree of a candidate is randomly replaced. For example, S_1 in Fig. 6(d) is replaced by a randomly generated tree shown in Fig. 6(e), resulting in a new program $\mathcal{P}_5 = (\bar{c}_1 \text{ AND } (c_4 \text{ AND } c_5))$ [see Fig. 6(f)]. Repetitively performed crossover operations may lead to a complex tree structure with an uncontrolled growth, which is referred to as *bloating*.

Algorithm 2: A Simplified Implementation of GP.

```

Initialize a population represented as parse trees, each
individual with randomly selected basic quantum gates;
while Stopping Criteria is not attained do
    Evaluate fitness of each candidate solution;
    Select candidates for genetic operations: do
        Perform crossover on pairs of candidate solutions;
        Mutate some of the candidate solution;
    Form a new population of best fitted candidate solutions;
end while

```

It should be noted that occasionally bloating can be helpful to develop variants for cyberattacks to uncover hidden malicious actions. As is true for all evolutionary methods, mutation operation in GP can substantially alter programs, and hence, in practice, it should be performed with a small probability.

A pseudocode implementing a simple GP is presented in Algorithm 2, where chromosomes representing program trees are generated and their fitness is evaluated. Any invalid programs obtained by crossover and mutation operations are discarded. Fitter candidate programs are advanced to the next generations.

GP for QCC: Quantum circuits can be generated and evolved using GP methods by designing a chromosome representing various features for QCC process, including gate types, qubits that they control, and rotation angle for qubits. The GP implementation in [20] defines a fitness function as the sum of distances between amplitudes of the desired output of qubit states and the ones of a candidate circuit. If a gate in a candidate circuit is found to be incorrect (e.g., CNOT gate without a control or target qubit), it is ignored by being replaced by an identity transformation. With the circuit information captured in chromosomes encoded as binary strings representing sequence of gates, each of which operating on qubits with different parameters, GP can successfully generate entangled three-, four-, and five-qubit circuits.

Automatic construction of reversible logic for quantum circuits can be built using GP, as reported in [21], where candidate circuits are modeled as strings representing binary trees. In this application, internal tree nodes correspond to multiplication matrices, whereas leaves are the individual gates. Initially, leaves are randomly selected from a library of basic single- and two-qubit gates. A single-point crossover exchanges parts of two trees, and a low-probability mutation evolves feasible solutions over multiple iterations of the GP. The size of each tree is typically limited by a maximum depth to prevent bloating from uncontrolled tree growth that would result in a low-fidelity circuit. Fitness of a candidate tree is computed as its *difference* [65] with a reference circuit.

In another GP-based QCC method [23], proposed for multiobjective synthesis of quantum circuits, a two-step fitness function evaluates accuracy and cost of candidate circuits. The fitness is defined as the candidate solution's similarity to a unitary matrix to be synthesized. To measure fitness, the

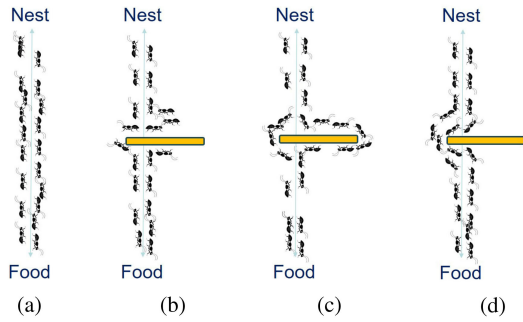


FIGURE 7. Ants finding the shortest path between a nest and food source in a dynamic environment.

target unitary matrix and the expected outcome of a candidate circuit are compared. To alleviate bloating throughout generations, a dynamically defined limit is enforced on tree depth.

In [24], GP generates quantum circuits encoded as symbolic expressions (called s-expressions [66]) for tree-shaped data structures that represent qubits and gates. A candidate solution is generated using NOT, H, U3, CNOT, and SWAP basic gates. A fitness of each circuit is evaluated by finding its quantum computation error rate, while favoring candidate solutions that use lower number of elementary gates.

Efficient quantum circuits for a three-qubit nuclear magnetic resonance (NMR) quantum processor [67] can be obtained by GP algorithms, as introduced in [22]. By using GP generated circuits for Toffoli and Fredkin gates, solutions that are robust against decoherence errors can be obtained. Selection mechanism puts proportional higher weights on individuals with better fitness values to determine the best solutions without compromising fidelity of an evolved circuit.

Computational complexity of a GP algorithm is $O(g \cdot w^d)$, where g is the number of generations in an underlying evolutionary algorithm used by GP, w is the average number of choices for selecting options in Backus–Naur form (BNF) rules, and d is the average depth of recursion allowable during parsing a chromosome with respect to a given set of BNF rules. Although w tends to be relatively small in GCC applications, d can be large due to the structure of a given quantum algorithm is complex, in which case the number of generations g required for convergence can also be large.

C. ANT COLONY OPTIMIZATION

Background: ACO is an AI technique which mimics ants depositing an ephemeral hormone, the *pheromone*, on the ground in order to mark a path while roaming for food [68], [69]. When the path is traveled more frequently or is shorter, the pheromone trace on the ground remains stronger, attracting more ants, and hence serving as *stigmergic* reinforcement of a good path. In the example illustrated in Fig. 7(a), while ants collect food from a source, if an obstacle blocks their path [see Fig. 7(b)], they may at first randomly explore the area around the obstacle to consider alternative paths to the

Algorithm 3: A Simple ACO Algorithm.

```

Initialize pheromone trails;
while Stopping criteria is not attained do
    Obtain ant solution;
    Apply local search for a path;
    Update pheromones;
end while

```

food source [see Fig. 7(c)]. Since a shorter path accumulates more pheromone, it attracts more ants leading to an optimal solution [see Fig. 7(d)]. Finding an optimal path in a graph can be intuitively mapped into an ant biological system (e.g., finding the shortest route to a food source in Fig. 7). Pheromone variables on each node can indicate quality of links. Contrary to real ants that deposit pheromone wherever they move, artificial ants often would first discover a path and then update the respective pheromone variables, and hence, mimic deposition of pheromone only on their way back. All pheromone values should be decreased over time to imitate pheromone evaporation process. Similarly, pheromone values associated with good solutions, if discovered, should be increased. A pseudocode for the ACO algorithm is presented in Algorithm 3.

ACO for QCC: In quantum ant programming (QAP) [25], directed graphs are typical tools to represent a search space. Internal nodes of a directed graph are the basic quantum logical gates, whereas the edges are the connections among gates. The graph depth corresponds to the maximum length of a quantum circuit to be designed. A team of ants travel through the graph choosing edges between nodes, which are then used in defining a quantum circuit. Multiple iterations of ants (i.e., qubits) traveling through the directed graph (i.e., the quantum architecture) generate different circuits. When an ant passes through an n -qubit gate, states of the qubits are altered accordingly. When a colony of ants completes its pass through a graph, corresponding candidate circuit is evaluated and the pheromone trail on the visited edges are updated. After QAP completes, the most suitable circuit is selected as the solution for QCC mapping. QAP generates better results compared to random search based methods for a set of benchmark problems (e.g., implementation of quantum Fourier transformation, Grover's diffusion operator, and Toffoli gate) [25].

In [27], ACO is used to synthesize reversible circuits, with ants making bit assignments based on pheromone values accumulated through the iterations, where the gates with lower quantum cost accumulate more pheromones. In another application [26], ACO is combined with simulated annealing and Quine–McCluskey (SA-QM) methods. In this stochastic approach, each time an ant adds a gate to the circuit, SA-QM finds possible minimum circuits, which are further used by other ants. In [29], reversible logic based ALU design is proposed, where ants find a shortest path in the form of a synthesized reversible function based on accumulated pheromone deposited by most ants.

ACO can synthesize reversible gates by ants traversing a given search space, modeled as a graph with nodes as stages of synthesis and edges as possible gates to be included in the final circuit [28]. So-called *Walsh–Hadamard transforms* [70] can partition a search space into small areas with well-defined transition operations to obtain a circuit with minimized *quantum cost*, as defined in [71]. In [28], a Walsh spectral representation of a Boolean function is used, which allows dividing the search space into smaller *promising* areas with well-defined transition operations between them. As a result, it is possible to minimize a search space and generate better solutions than obtained by ACO synthesis with classical reversible function representation. This process starts with an initial function to be synthesized. Each ant tries to evolve the function spectrum by applying reversible gates. The solution is a sequence of gates that represents the original function. In the proposed ACO process, a state (i.e., a node in the graph) is connected to the spectrum of a reversible function, while the actions (i.e., the edges in the graph) are the gates that can be added to the solution. Each edge is associated with a value that represents the quantum cost of the gate to which it is connected.

Computational cost of a typical ACO algorithm depends on n number of qubits on the physical quantum computer and c direct connections among them. Finding a solution for a QCC problem requires that all connections are traversed by each of the u artificial ants at every iteration of ACO for t iterations. In this case, the computational complexity of ACO-based QCC is $O(u \cdot t \cdot n^c)$.

D. AI PLANNING

Background: AI planning techniques include autonomous reasoning for finding the effects of a given set of actions by searching the space of possible operations to find a procedural sequence of stages needed to reach predefined goals [72]. In AI planners, domain knowledge can be learned over time to ensure satisfactory solutions in the presence of dynamically changing goals. Inputs to AI planners include initial state of a problem, sets of desired goals, and allowable operations to be used in a sequence of actions leading to a desired outcome. Planning domain definition language (PDDL) [73] and Stanford Research Institute problem solver [74] are examples of environments designed for AI planning.

An AI planner is expected to determine the most appropriate operations and a proper sequence of subgoals to optimize a generated solution. A pseudocode for implementing an AI planner is presented in Algorithm 4. First, the algorithm populates a stack with the goals to be solved, with the first one at the top of the stack. Each goal is then evaluated to determine if a given operation can accomplish it. If no such operation exists, a set of subgoals is determined to replace the goal at the top of the stack.

Temporal planning (TP) algorithms can be considered among the tools for solving AI planning problems. TP can account for multiple actions with different durations and complex interdependencies to be evaluated simultaneously.

Algorithm 4: A Simple Algorithm for AI Planner.

```

Set stack = list of goals;
while All goals are not accomplished do
     $t$  = top of stack;
    if Operation to solve  $t$  are known then
        Add the operation to the sequence of operations;
    else
        Represent  $t$  as subgoals; Push subgoals into stack;
    end if
end while

```

TP methods are suitable for many scheduling problems, including quantum gate assignments in QCC.

AI Planning for QCC: Temporal techniques can employ PDDL to identify parallelizable conflict-free machine instructions suitable for QCC for high-level quantum algorithms [30], [32]. It is shown in [30] that AI TP combined with constraint programming is a viable approach for compiling circuits for quantum approximate optimization algorithms [75] that are designed to solve max-cut combinatorial optimization problems [76]. In [30], quantum algorithm qubits are mapped to physical qubits by iteratively inserting gates during computation as needed while obeying physical constraints of a target NISQ architecture. An optimization method called *quantum alternating operator ansatz* [77] can be employed to map graph coloring algorithms on NISQ computers [32], where hybrid SWAP/P-S and SWAP/MIX gates are modeled in PDDL.

In another example of TP application for QCC, satisfactory mappings on target hardware are explored by solving problems with a high degree of options to minimize the depth of final quantum circuits [31]. In this approach, QCC is modeled by TP algorithms utilizing standard PDDL such that the predicates represent the location of qubit states and the goal requirements. For example, SWAP and phase-shift gates can be modeled as temporal actions whereas conditions indicate execution of phase-shift gates and validate correctness of qubit placements in SWAP operations.

Given that a typical AI planning algorithm requires $O(f^2)$ states, where each state is the number of different intermediate assignments for n qubits to be computed by algorithm, the cost of evaluating each state is a binary search with cost $O(\log f)$ plus some constant time operations. Therefore, the overall computation complexity of QCC based on AI planning will be $O(f^2 \cdot n \cdot \log f)$.

V. HEURISTIC METHODS FOR QCC

Many heuristic methods are proposed for QCC to generate quantum circuits that are feasible in terms of their execution in a given hardware architecture. Typically in a heuristic approach, a solution is constructed by making a locally optimal choice at each decision point, which may not lead to a globally optimum solution at the end [78]. Boolean satisfiability (SAT) and satisfiability modulo theory (SMT) solvers [79],

Algorithm 5: A Simple Greedy Algorithm.

```

Divide the problem into  $P$  sub-problems;
while  $P \neq \emptyset$  do
  Select a subset  $P_i \subset P$ ;
  Obtain the best solution  $s_{P_i}$  for  $P_i$ ;
  Store  $s_{P_i}$  as a candidate solution:  $S^{best} = S^{best} \cup \{s_{P_i}\}$ ;
  Update problem domain:  $P = P / \{P_i\}$ ;
end while
Select the best solution from  $S^{best}$  as the solution for  $P$ ;

```

dynamic methods, and graph algorithms are among the popular heuristics employed for quantum circuit compilation.

A. GREEDY ALGORITHMS

Background: Greedy algorithms often find solutions that may be only locally optimal, with the expectation that such local results will eventually lead to acceptable, but not necessarily optimum, solutions. Algorithm 5 presents a high-level view for the steps of a typical greedy algorithm. First a problem P is partitioned into smaller and manageable subsets P_i where $P = \bigcup_i P_i$, each of which is typically explored by exhaustive searches. As a satisfactory solution for each P_i is found, it is added to a set of candidate solutions S^{best} . At the end, when all subproblems are handled, the best candidate from S^{best} is selected as the solution for P . There are several limitations of greedy-based solutions. First, even if each solution $s_{P_i} \in S^{best}$ may be optimal, the final solution is not guaranteed to be optimal for P since the subproblem solutions do not capture the entire problem domain. Second limitation is the high computational cost of greedy algorithms since they have to perform exhaustive searches in a search space of each subproblem $P_i \subset P$.

Greedy Algorithms for QCC: Recent greedy-based heuristics reported in the literature for QCC aim to reduce the circuit depth and the number of gates as much as possible. For example, the greedy method presented in [37] identifies the best qubit topology in a grid-based layout for a quantum circuit, whereas the method reported in [38] attempts to map a quantum circuit to any arbitrary quantum architecture. Several heuristic approaches have also targeted existing NISQ architectures with limited connectivity, where some provide local suboptimal solutions for each CNOT gate [80] and others obtain solutions by attempting to minimize the total number of quantum gates [33].

While these greedy heuristics may find acceptable mappings for target quantum circuits, they often do not consider variations in qubit behavior. A mapping that reduces the number of gates and the quantum cost, and hence, produce a workable but imperfect solution, may not necessarily be acceptable with respect to circuit errors. This is due to the fact that qubits and their gates typically do not manifest a uniform behavior over time. Recently, a new set of heuristic mapping techniques have been proposed to address the variation in physical characteristic of quantum hardware. As an attempt

in reducing errors due to decoherence, greedy randomized algorithms using nearest-neighbor quantum architectures with short durations of quantum circuits are proposed [36], where the selected SWAP operations do not result in deadlock cycles caused by the same qubits being part of multiple CNOT gates at the same time.

Integer linear programming (ILP), which is considered as a greedy-based approach, brings solutions to problems that can be modeled with integer variables used in linear objective functions. The ILP-based QCC technique proposed in [35] reduces quantum circuit depth by solving a set of nearest neighbor constraints [81]. In an attempt to reduce computational complexity, circuits are divided into nonoverlapping windows that are evaluated separately. Physical qubit reassignments are based on qubit and gate error rates to maximize quantum circuit fidelity.

By considering only the gate costs of qubits that are active at the same time, one can avoid exhaustive searches needed to identify the minimum number of swaps for CNOT gates [34] by transforming the circuit forward and backward in a greedy way to determine the final qubit assignments. This approach also explores the tradeoff between reducing the number of quantum gates and the quantum circuit depth.

Overall, computational complexity of greedy-based QCC methods are polynomial. For example, the method proposed in [33] has a complexity of $O(n^2)$, where n is the number of qubits. This apparent advantage stemming from the low computational cost of greedy methods is offset by their lack of ability to generate optimum solutions.

B. SAT AND SMT SOLVERS

SAT solvers are designed to find assignments to a set of variables in a given propositional formula with Boolean variables and basic logic gates. Since finding an optimum solution with a SAT solver is NP-complete, several heuristics are proposed based on searching qubit assignments by using the Davis–Putnam–Logemann–Loveland algorithm [82].

For example, conflict analysis can reduce the search space using a conflict-driven clause learning algorithm [83]. MiniSAT [84], PicoSAT, and CryptoMiniSAT [85] are among the popular SAT solvers reported in the literature. For SMT solvers, a problem is posed using general propositional structures, including arithmetic expressions, arrays, and classical data type operations [86]. An SMT solver attempts to satisfy an objective function based on inputs that are formulated as a set of linear constraints.

A simple SAT solver is presented in Algorithm 6, where a problem with logical qubits $q_I \in I$ is formulated as a Boolean expression B using variables $v_I \in V$. The variables in V must be mapped into hardware qubits $q_H \in H$ such that B is satisfied. If B cannot be satisfied with the current mapping, a new mapping of V to H is tried until either a solution is found or a predetermined limit is reached.

SAT and SMT Solvers for QCC: SAT and SMT solvers have been proposed for QCC to reduce the gate count and enhance the quantum circuit reliability. For example, in [39],

Algorithm 6: A Simple SAT Solver.

```

Set of logical qubits to be assigned is  $I = \bigcup \{q_I\}$ ;
Set of hardware qubits to be mapped is  $H = \bigcup \{q_H\}$ ;
Map logical qubits to variables  $I \Rightarrow V$ ;
Generate Boolean expression  $B$ ;
Set of variables used in  $B$  is  $V = \bigcup \{v_I\}$ ;
Map hardware qubits to expression  $B$ :  $v_I \rightarrow q_H, \forall v_I \in V$ ;
while  $B$  evaluates to FALSE do
    Assign hardware qubits to expression  $B$ :  $v_I \rightarrow q_H, \forall v_I \in V$ ;
end while
Boolean expression  $B$  with assignments is a solution;

```

the QCC mapping process is set as a SAT problem. This method generates hardware qubit mappings with reduced quantum cost by using Boolean satisfiability SMT solver called Z3 [87]. Additional constraints are added based on the target quantum architecture to restrict the search space and, hence, reduce computational complexity of the solver. These constraints limit each physical qubit assignment to a single logical qubit and apply gates to physical qubits that are supported by the coupling constraints of a target NISQ computer.

A SAT solver-based approach is proposed to reduce the number of SWAP operations [41], where the order of quantum gates are changed so that each subcircuit has only gates performing on adjacent qubits. SWAP operations are inserted between pairs of subcircuits to adjust the qubit placements as needed. Another SAT solver application aims to design a circuit using the minimum of CNOT and T gates since they show lower fidelity and higher error rate when compared to single-qubit operations [42].

Although prone to scalability limitations, SMT solvers have also been proposed for optimization of reducing number of gates and stratification of reliability constraints for gates, measurements, and decoherence errors when mapping a quantum circuit to an underlying quantum hardware [40].

Computational complexity for SAT solvers in attempting to find an optimal circuit grows exponentially with the number of single-qubit gates plus the number of CNOT operations. The complexity for the satisfiability problem has been proven to be NP-complete [88]. Given n variables, the number of decisions may require as many as $O(2^n)$ steps. Although several restrictions are added to reduce the search space and the execution time at the cost of additional gate count (e.g., [39]), overall complexity is still exponential.

C. DYNAMIC METHODS FOR QCC

Background: Due to the presence of high and varying error rates of qubits and gate operations, it has been proposed that QCC methods may rely on dynamic information to improve quantum circuit design. For example, dynamic compilation methods can reduce execution time of a compiled program [89], and, hence, can be used in quantum computing to enhance the reliability of a quantum circuit. Algorithm 7 outlines the basic steps of a dynamic method for physical

Algorithm 7: A Simple Dynamic Algorithm.

```

Generate a set of qubit assignments  $A = \bigcup \{\alpha_i\}$ ;
for each trial  $\alpha_i \in A$  do
    Generate a circuit compilation;
    Set  $\theta_i$  as the number of trials for allocation  $\alpha_i$ ;
    for  $count \in \theta_i$  do
        Run circuit determined by  $\alpha_i$ ;
        Record performance  $\rho_i$  for trial  $\theta_i$ :  $P = P \cup \{\rho_i\}$ ;
    end for
end for
Select assignment  $\alpha_i$  with the best circuit output  $P$ ;

```

qubit assignment. First, multiple feasible assignments are determined, each of which is then run multiple times over a given NISQ computer. The best performing assignment is selected as the circuit to be implemented on that hardware.

Dynamic Methods for QCC: A diverse group of dynamic quantum compilation techniques based on run time interaction with quantum computers have been proposed in [43]. These techniques make their decisions by considering the impact of correlated errors from multiple runs of a quantum circuit. To improve success probability in outputs, various allocations of physical qubits are explored in multiple runs. Weighted probability distribution for each different allocation gathered at run time is evaluated to detect possible similarities among outcomes to determine the correct outputs so that selection of preferred mappings can be determined. Since measuring an all-zero state has a higher fidelity compared to an all-one state, inverting quantum states prior to a measurement has been proposed to reduce measurement errors [46]. Instead of indiscriminately inverting all outputs, the circuits are run several times to predict the appropriate qubit inversions. Similarly, QCC can benefit from efficient utilization of quantum hardware resources to increase system throughput by running multiple quantum algorithms concurrently on a NISQ computer [47].

Ancillary qubits have been used to identify violations of superposition and entanglement through quantum circuit assertions inserted at different circuit locations [45]. Quantum gates can be assigned to ancillary qubits to detect errors in a circuit at measurement time. In the absence of an error, the measurement operation applied to an ancillary qubit does not affect the functionality (or entanglement) of a circuit.

In the absence of accurate failure models, online monitoring of quantum circuit error rates can detect any unexpected changes in qubit behavior at runtime [44] to evaluate qubit selections that enhance the success probabilities of outputs.

Dynamic methods for QCC rely on mapping quantum circuit using other QCC methods followed by the several additional quantum circuit executions to collect runtime information of the error rates. Dynamic QCC methods dominated by the computational cost of their underlying heuristic methods employed to generate physical quantum circuits with n qubits, ranging from $O(n^2)$ for greedy methods to $O(2^n)$ for SAT solvers. Some dynamic approaches require

Algorithm 8: A Simple Graph Optimization Algorithm.

```

Set of qubits to be assigned is  $I = \bigcup \{q_i\}$ ;
Construct a graph  $G_H = (V, E)$  for target NISQ hardware;
Select initial qubit assignments:  $q_I \rightarrow q_H \in V$ ;
Update qubit sets:  $H = H \cup \{q_H\}$  and  $I = I/\{q_I\}$ ;
for each  $q_i \in I$  do
    Select qubit  $q_H \in V$  with shortest path to  $H$ ;
    Assign selected qubits into hardware:  $q_I \rightarrow q_H$ ;
    Update qubit sets:  $H = H \cup \{q_H\}$  and  $I = I/\{q_I\}$ ;
end for
 $H$  is the final mapping of qubits to target hardware;

```

searches for isomorphic subgraphs to their initial qubit allocations to provide diversity in mappings. They typically use graph matching algorithms (e.g., VF2 algorithm [43]), with computational complexity of $O(n!)$ for n nodes representing qubits in a coupling graph of a quantum hardware [90].

D. GRAPH OPTIMIZATION FOR QCC

Background: In graph optimization techniques for QCC, connectivity of a quantum hardware architecture is modeled as a graph, where vertices represent qubits and edges are the connections between them. Graph-based optimization algorithms, including permutation and extraction of subgraphs with high connectivity, are proposed to determine allocation of qubits into physical ones.

Algorithm 8 presents a simple qubit mapping algorithm using graph optimization, where the target hardware is represented as a graph G with V vertices denoting the qubits and E edges the connections among them. First, an initial qubit is selected in target hardware by a given criteria (e.g., lowest error rate, and highest number of neighbors). Then, for each remaining qubit, an assignment is determined by its distance to already allocated hardware qubits.

Graph Optimization for QCC: Several graph optimization based heuristics have been proposed for mapping quantum circuits onto given target hardware. In [48] and [49], quantum gate costs are reduced by an A^* graph searching algorithm [91]. Using a directed acyclic graph, a quantum circuit is divided into layers of quantum gates that can be executed in parallel. For each layer, logical qubits are mapped to physical ones by means of an exhaustive search while satisfying the CNOT constraints. This solution is analogous to solving a shortest path problem, where the objective is to minimize the number of quantum gates.

Error models of quantum circuits can be used to represent physical characteristics of quantum architectures. For example, a *beam search* for QCC, where a graph is explored by traversing a most promising node at a given point, was proposed for QCC [50]. This method generates successful results for small quantum circuits; however, its scalability to larger circuits is not straightforward. Another graph optimization based QCC [33] first selects physical qubits with the lowest CNOT error rates for mapping. With Dijkstra's algorithm [92], it then maps remaining qubits to physical qubits by selecting paths with the lowest probability of error. A

longer path with more quantum gates can only be selected if its overall CNOT error rate is low. This approach may enhance the fidelity of a quantum circuit, but it does not address decoherence issues, where qubits can retain data only for a limited time. Therefore, a mapping that considers only quantum error gates may not satisfy the retention time requirements of qubits, resulting in target circuits with low fidelity.

Decoherence errors can be incorporated into quantum circuit mapping by considering both the depth of a quantum circuit and the cumulative decoherence errors of all qubits [52]. Isomorphic subgraphs of coupling constraints of a quantum hardware are found to generate possible unique qubit allocations within a subgraph that represents multiqubit gate dependencies. While finding isomorphic subgraphs is an NP-complete problem [93], its complexity can be reduced in special cases, for example, in grid-like quantum architectures. An approximation of all unique isomorphic subgraphs can be generated by extracting rectangle grids from its coupling graph, where each grid contains qubits to implement a quantum circuit. Within each grid, different permutations of qubit allocations can be generated. Estimated error rate of a quantum circuit under each permutation can, then, be used to select better allocations of physical qubits.

The QCC techniques based on token swapping to reduce quantum circuit depth and the number of qubits of a target quantum circuit are examples of graph optimization based methods [54]. In another method [51], nearest neighbor architecture of a quantum circuit is explored using eigenvector centrality concept of graph theory [94], which ranks qubits in a coupling graph based on their connectivity with other qubits. First, the qubit with maximum centrality is selected and placed at the center of a grid, to which, neighbor qubits are added at each iteration by prioritizing the ones with higher centrality.

Edge coloring and maximum subgraph isomorphism techniques are other examples of graph optimization used in QCC [53]. In these methods, dependency among quantum operations is represented as a logical data precedence graph, whereas temporal order of qubits is determined by solving an edge-coloring problem for the graph representing qubit interactions. Hardware connectivity constraints are modeled as a *color-pairing* problem to determine the qubit assignments to reduce the number of SWAP operations in a target circuit.

Similar to greedy-based heuristic approaches, the time complexity of the graph-based QCC approaches is polynomial. For example the time complexity of the beam search is $O(r^2 \cdot m + r^3 \cdot b)$, where r is the number of quantum gates, m is the number of random initial mappings with which the search starts, and b is a fixed number that restricts the search by limiting the width of the graph [50].

VI. EVALUATION OF METHODS AND FUTURE DIRECTIONS

A quantum algorithm must be compiled into a set of elementary quantum gates, which are then executed in sequence at specific times on a NISQ quantum hardware. Current NISQ

systems have formidable hardware-related challenges including limitations on qubit interconnectivity, restrictions on number of supported basic quantum gates, loss of state information in qubits over time due to decoherence, and fluctuations of computation error rates due to noise and latency. During quantum circuit compilation, mapping a quantum circuit onto a NISQ hardware must satisfy these constraints of an underlying target quantum architecture. QCC often requires to modify an original circuit by adding new quantum gates and auxiliary qubits to generate a feasible quantum circuit while attempting to keep low error rate and circuit latency.

In this article, we present a survey of AI- and heuristic-based QCC methods recently reported in the literature. Genetic algorithms, GP, swarm intelligence, and AI planning are among the popular AI methods employed in QCC. Among heuristics methods for QCC to generate feasible quantum circuits are greedy algorithms, satisfiability problem solvers, dynamic, and graph optimization techniques.

In genetic algorithm based QCC, GA generates candidate solutions of quantum circuits and evolves these circuit candidates within the search space of possible circuits for a given specific hardware platform. As chromosomes encoding quantum circuit parameters are evolved by crossover and mutation operators, fitness function is defined to favor candidates with low error rate and latency in generating outputs.

Quantum circuits can be generated and evolved using GP methods, where chromosomes are program trees representing circuits. Each candidate program is evaluated with respect to its error rate and latency to determine its fitness. Reproduction, crossover, and mutation operations evolve the trees modeling quantum circuits such that, as GP progresses, better and syntactically correct programs are advanced to next generations.

Ant colony optimization techniques mimic ants depositing hormones on ground to mark their paths while roaming for food. In ACO applications to QCC, ants explore a quantum hardware architecture modeled as a directed graph to choose appropriate sequence of quantum gates for mapping. The most traveled paths by the ants are the fittest and the most desirable assignments for a given quantum circuit.

AI planning techniques find procedural sequences of steps to reach predefined goals. AI planning for QCC can identify conflict-free machine instructions that can be executed in parallel, where qubits are iteratively inserted into a target quantum circuit while adhering to physical constraints of a target NISQ architecture.

Greedy-based heuristics for QCC aim to reduce the depth and total number of gates in the final circuit by partitioning a quantum algorithm into smaller and manageable subsets that can be handled using exhaustive search methods. Most greedy methods reported for QCC provide locally optimal solutions and typically cannot handle multiple objectives simultaneously (e.g., reducing circuit depth, error rate, and latency).

Satisfiability problem solvers for QCC generate assignments for a set of variables (e.g., qubits and gates) in a given

propositional formula that represents a quantum circuit to be compiled. The SAT and SMT solvers attempt to increase quantum circuit reliability by reducing the number of gates and quantum operations that a target circuit performs. Using satisfiability problem solvers may result in solutions that are computationally too expensive for handling large quantum circuits that are often seen in practical applications.

Dynamic techniques adopted for QCC evaluate outputs of quantum circuits at runtime. They generate a set of qubit assignments and gate operations, each of which is then executed multiple times on the target NISQ computer. Distribution of the generated outputs determine the selection of a particular assignment. First, a set of promising qubit assignments for a target NISQ architecture is generated. Each assignment is then run multiple times at the target NISQ computer. By merging the output probabilities of different assignments, probability of generating incorrect outputs can be reduced. The best performing qubit assignment (i.e., the mapping that provides the highest fidelity of its outputs compared to the expected results) is then selected to perform quantum computations. Since dynamic techniques require repetitive execution of multiple candidate circuits on a given target NISQ hardware, their usage is limited to those users who have easy access to quantum computers.

By expressing physical qubits as vertices and connections between qubits as edges, graph theory based algorithms can be used to compute preferable qubit assignments for a given NISQ platform. All qubits can be mapped to graph vertices either simultaneously or sequentially while considering hardware connectivity constraints. Graph optimization methods for QCC usually consider only a subset of circuit characteristics at a time (e.g., either depth, number of gates, or qubit decoherence), as opposed to evaluating all of them as ought to be for obtaining an optimum solution.

Table 3 outlines the computational complexities of the QCC methods analyzed in this survey, pointing to a typical application that implements each one. We can observe that the cost of computation for mapping n qubits varies for different methods from linear to factorial. At the fastest end of the AI spectrum is the GA-based QCC, whose complexity is bounded by its population size p and the number of generations g , both of which are selected by the algorithm designer and stay constant throughout run time. Note that the impact of fitness function on computational cost, which requires evaluating a candidate solution on hardware or a simulator, is not typically significant for all AI methods and, hence, is not included in boundaries given in Table 3. For GP, the width w and depth d of BNF rules dominate the computation cost, both of which are determined by the QCC designer to improve efficiency of mapping. In ACO, u ants need to travel t times over c connections of a given hardware, which increases its cost exponentially. Number of possible qubit assignments f for each gate in the quantum algorithm dominates the cost of AI planning methods for QCC.

Among heuristics based QCC in Table 3, the greedy method is the fastest running one with a cost of $O(n^2)$, where

TABLE 3. Computation Complexity of AI- and Heuristics-Based QCC Methods

	AI techniques				Heuristics			
	GA	GP	ACO	AI planning	Greedy	SAT / SMT	Dynamic	Graph
Computational	$O(g \cdot p \cdot \log(p))$	$O(g \cdot w^d)$	$O(u \cdot t \cdot n^c)$	$O(f^2 \cdot n \cdot \log f)$	$O(n^2)$	$O(2^n)$	$O(n^2) - O(n!)$	$O(r^2 \cdot m + r^3 \cdot b)$
Complexity [†]	[16]	[20]	[25]	[31]	[33]	[39]	[33], [43]	[50]

[†] n is the number of qubits and gates in a given quantum circuit to be mapped by a QCC method. g and p are the number of generations and population size, respectively, in a GA. w and d are the width and recursion depth of BNF, respectively, used in a GP. u , t and c are the number of artificial ants, the number of traversals and the number of direct connections among qubits, respectively, used in an ACO algorithm. f is the number of qubit assignments in AI planning based QCC. r , m and b are the number of gates, initial qubit mappings and the search width in a graph optimization algorithm for QCC, respectively.

the number of qubits n tends to be relatively small for all currently available hardware platforms. Satisfiability solvers suffer from exponential growth with respect to n qubits due to their trail and error nature. Dynamic methods for QCC depend on other heuristics for their decision making mechanisms, and, hence, their complexities are dictated by the cost incurred by those underlying heuristics. Graph optimization based QCC is heavily influenced by not only the depth of the search algorithm but also its width b and the number of random mappings m considered for finding an initial assignment.

Selection of a QCC method for a given application depends on multiple factors, including size of the quantum algorithm, topology of the target quantum architecture, frequency of recompilations, and desired accuracy of the final circuit. For a quantum algorithm with relatively large number of qubits, one should avoid selecting methods that grow exponentially with respect to n (e.g., satisfiability solvers). Similarly, ACO and graph optimization methods should not be considered for a target hardware topology with a high degree of connectivity among its physical nodes. It is necessary to recompile quantum circuits when physical constraints of the target NISQ computer change. If these changes occur frequently, it is wise to select a computationally inexpensive QCC technique so that adequate qubit mappings reflecting the latest hardware restrictions can be generated promptly. For such cases, GA and greedy-based QCC methods, which are among the fastest algorithms listed in Table 3, are the best candidates. However, it needs to be noted that greedy-based QCC techniques typically lack the ability to generate optimal solutions and should be avoided in applications where high fidelity quantum circuit outputs are essential.

We observe that several AI- and heuristic-based QCC methods reported in the literature are specifically designed to work on a particular quantum hardware architecture, including grid-based layouts (e.g., [37] and [32]) and nearest neighbor architectures (e.g., [36], [51], and [30]). Although they perform well for the target architectures that they were intended for, such as methods that rely on searching for isomorphic subgraphs (e.g., [43], [52], and [44]), it is not clear how easily they can be adapted to different NISQ platforms and various types of quantum algorithms. Another major concern is the scalability of existing QCC techniques. Most studies use orders-of-magnitude smaller number of gates than those needed in real-life algorithm implementations. It is difficult to predict the performance of these techniques

in the presence of hundreds of qubits with their physical characteristics changing relatively frequently.

It is noteworthy to point out that an additional deficiency stems from the lack of comprehensive failure models in evaluating quantum circuit performance. Current QCC methods typically rely on simplistic failure models to represent gate, measurement, and coherence errors, but may not capture the impact of complicated noise sources on circuit output fidelity (e.g., crosstalk errors). Incoming QCC methods will benefit from incorporating recent advances in modeling noise sources (e.g., [95]–[97]) into their failure models.

Based on these observations, we believe that there is a compelling need for mapping quantum circuits to NISQ platforms such that the mapping technique is: i) generic enough for a broad range of applications; ii) compatible with different NISQ hardware; iii) adaptive to frequent changes in architectural constraints; and iv) computationally inexpensive. Next-generation QCC techniques are expected to: i) provide solutions for different classes of problems; ii) rely on lightweight algorithms to be efficiently run as often as needed; iii) generate feasible mappings for a broad range of quantum architectures; iv) take into account even the most stringent hardware requirements; and v) deliver solutions satisfying timing and error requirements.

REFERENCES

- [1] IBM. IBM Q. Systems, 2019. [Online]. Available: <https://www.research.ibm.com/ibm-q/technology/devices/>
- [2] J. Kelly, "A Preview of Bristlecone, Google's New Quantum Processor," *Google Quantum AI Lab*. 2018. [Online]. Available: <https://ai.googleblog.com/2018/03/a-preview-of-bristlecone-googles-new.html/>
- [3] I. Corp, "The future of quantum computing is counted in qubits," 2018. [Online]. Available: <https://newsroom.intel.com/news/future-quantum-computing-counted-qubits/gw17jd/>
- [4] R. Computing, "Rigetti 16Q Aspen," 2018. [Online]. Available: <https://www.rigetti.com/qpu/>
- [5] A. Botea, A. Kishimoto, and R. Marinescu, "On the complexity of quantum circuit compilation," in *Proc. 11th Annu. Symp. Combinatorial Search*, 2018, pp. 138–142.
- [6] Quantum AI team and collaborators, "Cirq," Mar. 2021, doi: [10.5281/zenodo.4586899](https://doi.org/10.5281/zenodo.4586899).
- [7] P. Murali, N. M. Linke, M. Martonosi, A. J. Abhari, N. H. Nguyen, and C. H. Alderete, "Architecting noisy intermediate-scale quantum computers: A real-system study," *IEEE Micro*, vol. 40, no. 3, pp. 73–80, May/Jun. 2020, doi: [10.1109/MM.2020.2985683](https://doi.org/10.1109/MM.2020.2985683).
- [8] A. Sørensen and K. Mølmer, "Quantum computation with ions in thermal motion," *Phys. Rev. Lett.*, vol. 82, pp. 1971–1974, Mar. 1999, doi: [10.1103/PhysRevLett.82.1971](https://doi.org/10.1103/PhysRevLett.82.1971).
- [9] J. Hsu, "CES 2018: Intel's 49-Qubit chip shoots for quantum supremacy," 2018. [Online]. Available: <https://spectrum.ieee.org/tech-talk/computing/hardware/intels-49qubit-chip-aims-for-quantum-supremacy>